

ALGORITMI E STRUTTURE DATI – CAMASTRA

UNIVERSITÀ DEGLI STUDI DI NAPOLI
“PARTHENOPE”



Appunti a cura di
FIORENTINO MICHELE

INTRODUZIONE

I seguenti appunti sono stati creati usando diverse **fonti**:

- “Introduzione agli algoritmi e strutture dati” – Cormen;
- “Automati, linguaggi e calcolabilità” – Hopcroft;
- slide presentate a lezione durante il corso, da parte di Camastra e Ferone;
- appunti presi personalmente durante le lezioni;
- vari siti web (GeeksForGeeks, StackOverflow, ...);
- vari canali youtube (Abdul Bari, fcamuso, ...);
- appunti da parte di Liccardo Giuseppe.
- confronto con altri studenti del corso;

ne consegue che questi appunti presentano una forte componente personale e sono il risultato dell’amalgamazione di molte fonti diverse. Ho cercato di essere quanto più non ambiguo e fedele alle lezioni dei prof, tuttavia è probabile che ci siano alcuni errori o imprecisioni. Anche l’implementazione degli algoritmi è spesso personale e non è detto che sia la migliore.

Per questo, tali appunti sono da considerare un “sostegno”, e NON hanno l’obiettivo di sostituire le lezioni.

Indice

LEZ 1 – Introduzione	9
LEZ 2 – Notazioni asintotiche	10
Teorema: un polinomio è O del suo termine di grado più elevato	11
Proprietà di O	12
Proprietà di Ω	12
Legami tra O e Ω	12
Ordinamento tra funzioni	13
Metodo Generale per la scelta tra le notazioni O , Ω , Θ	15
Ulteriori notazioni asintotiche	16
LEZ 3 – Analisi di un algoritmo & Insertion Sort	17
ANALISI DI UN ALGORITMO	17
INSERTION SORT	19
Invariante di ciclo: Insertion Sort	20
Analisi della complessità dell'Insertion Sort	20
LEZ 4 – Paradigma Divide-et-Impera & Merge Sort	21
MERGE SORT	22
• MergeSort	23
• Merge	24
Invariante di ciclo: MergeSort	25
Analisi della complessità del MergeSort	26
LEZ 5/6 – Ricorrenze	27
SERIE NUMERICHE DA RICORDARE	28
METODO DI SOSTITUZIONE	29
METODO DELL'ALBERO DI RICORSIONE	33
Albero bilanciato e non bilanciato	40
METODO DELL'ESPERTO	42
Ricettario MT	43
Ricorrenze non risolvibili con il MT	45
Esempi	45
METODO ITERATIVO	48

LEZ 7 – QuickSort	49
Invariante di ciclo	51
Complessità del Quicksort.....	52
Randomized Quicksort.....	55
Complessità del Randomized Quicksort.....	56
Miglioramenti per il Quicksort.....	60
HYBRID SORT.....	60
LEZ 8 – Alberi binari & Struttura dati Heap	61
Albero (in generale).....	61
Alberi binari Heap	62
Max-Heapify.....	63
Complessità di Max-Heapify	64
Build-max-heap	65
Invariante di ciclo (Build-max-heap).....	66
Complessità (Build-Max-Heap)	67
LEZ 9 – HeapSort, Code di priorità & BucketSort	68
HeapSort	68
Code di priorità.....	69
Heap-maximum	69
Heap-extract-max.....	69
Heap-increase-key.....	70
Max-heap-insert	71
TEOREMA (del limite inferiore).....	72
Ordinamenti in tempo lineare.....	73
Bucket Sort.....	73
Complessità del BucketSort	74
LEZ 10 – Programmazione Dinamica (e Fibonacci e LCS)	75
TABULATION vs MEMOIZATION	76
Fibonacci	77
LCS – Longest Common Subsequence.....	79
LCS basato su programmazione dinamica.....	80
Metodi alternativi a LCS: Distanza di editing	84

LEZ 11 – Programmazione Dinamica (Moltiplicazione di matrici)	87
Prodotto di due matrici	87
Prodotto di una sequenza di matrici	88
Determinazione del numero delle parentesizzazioni	89
Caratterizzazione della sottostruttura ottima (prodotto matrici)	90
Problema della moltiplicazione di matrici (Ricorsivo Divide et Impera)	92
Problema della moltiplicazione di matrici (PD con Tabulation)	94
Problema della moltiplicazione di matrici (PD con Memoization)	95
LEZ 12 – Tecniche di programmazione Greedy (Intro e Scheduling)	98
Algoritmi Greedy	98
PROBLEMA DELLA SELEZIONE DI ATTIVITÀ (SCHEDULING)	100
Sottostruttura ottima	101
Equazione di ricorrenza (calcolo del valore della soluzione ottima)	102
Teorema della scelta golosa	102
Problema della selezione di attività (Greedy Activity Selector)	103
LEZ 13 – Tecniche di programmazione Greedy (Problema dello zaino)	104
Problema dello zaino (PD vs Greedy)	104
ZAINO FRAZIONARIO	105
ZAINO 0-1 (Programmazione dinamica)	106
LEZ 14 – Tecniche di programmazione Greedy (Codici di Huffman)	109
Codici di Huffman	109
LEMMA (SOTTOSTRUTTURA OTTIMA)	112
LEMMA (SCELTA GOLOSA)	113
LEZ 15 – GRAFI e ALGORITMI DI VISITA DI UN GRAFO	114
GRAFI	114
Visita in ampiezza – Algoritmo BFS	116
Complessità computazionale della BFS	118
PERCORSI MINIMI	119
Teorema della Correttezza della Visita in Ampiezza	121
Visita in profondità – Algoritmo DFS	122
complessità computazionale dalla DFS	124
Teorema della Parentesi	125
Teorema del Cammino bianco	126

LEZ 16 – ARCHI e CONNESSIONE DI UN GRAFO	127
Classificazione degli Archi	127
CONNESSIONE DI UN GRAFO	130
Calcolo delle componenti fortemente connesse	131
Insiemi Disgiunti	133
Rappresentazione di insiemi disgiunti mediante alberi	134
TEOREMA SULLE OPERAZIONI SU INSIEMI DISGIUNTI	137
LEZ 17 – ALBERI DI COPERTURA MINIMO, KRUSKAL e PRIM	139
Albero di copertura minimo	139
DEFINIZIONE FORMALE	140
Algoritmo generico	140
ARCO SICURO	141
Algoritmo di Kruskal	143
Algoritmo di Prim	145
Differenze fra l'algoritmo di Prim e di Kruskal	147
LEZ 18/19 – TABELLE HASH	148
Tabelle Hash - introduzione	148
Funzioni Hash	149
HASHING PER DIVISIONE	150
HASHING PER MOLTIPLICAZIONE	150
HASHING UNIVERSALE	151
Progettare una classe universale di funzioni hash	151
Teorema	152
Corollario (hashing universale)	152
RISOLUZIONE DELLE COLLISIONI PER CONCATENAZIONE	153
TEOREMA (ricerca senza successo)	154
TEOREMA (ricerca con successo)	154
RISOLUZIONE DELLE COLLISIONI CON INDIRIZZAMENTO APERTO	156
SEQUENZE DI ISPEZIONE	158
ISPEZIONE LINEARE	158
SCANSIONE QUADRATICA	159
HASHING DOPPIO	159
Teorema (sul numero atteso di ispezioni senza successo)	160
Teorema (sul numero atteso di ispezioni con successo)	161

LEZ 20 – ALGORITMI DI RICERCA ESAUSTIVA (FORZA BRUTA)	162
BACKTRACKING	162
Problema delle 8 regine	165
Branch and Bound	168
PROBLEMA DELLO ZAINO 0-1 (con Branch and Bound)	169
LEZ 21 – AUTOMI A STATI FINITI	171
TEORIA DEGLI AUTOMI	171
• ALFABETO	171
• STRINGA	172
• LINGUAGGIO	173
Automa a Stati finiti deterministico	175
Automa a Stati finiti non deterministico	178
LEZ 22 – PROBLEMI INDECIBILI	180
PROBLEMI CHE I CALCOLATORI NON POSSONO RISOLVERE	180
PROBLEMA DECIDIBILE	181
IPOTETICO VERIFICATORE CIAO_MONDO	181
PROBLEMA ASTRATTO	183
DIMOSTRARE CHE UN PROBLEMA È INDECIDIBILE	184
LEZ 23 – LA MACCHINA DI TURING	185
LA MACCHINA DI TURING	185
LE MACCHINE DI TURING E L'ARRESTO	187
DESCRIZIONE ISTANTANEA DELLA MACCHINA DI TURING	189
LEZ 24 – CLASSI P e NP (Polynomial e Nondeterministic Polynomial)	191
PROBLEMI ASTRATTI	191
PROBLEMI CONCRETI	192
PROBLEMI DI DECISIONE	192
INTRODUZIONE A P e NP	193
CLASSE P (continuo)	194
Funzione calcolabile in tempo polinomiale	194
Codifiche correlate polinomialmente	194
Lemma	194
RELAZIONI TRA P & NP	195
ALGORITMO DI VERIFICA	196
PROBLEMI DI DECISIONE E DI RICERCA	196
RIDUCIBILITÀ	197
LINGUAGGIO NP-COMPLETO	198

“GERARCHIA” DEI PROBLEMI NP COMPLETI.....	199
--	------------

LEZ 1 – Introduzione

Un **algoritmo** è una procedura computazionale che prende alcuni valori in input e produce valori desiderati in output. Pertanto un algoritmo è uno strumento per risolvere un dato problema computazionale.

Informalmente, possiamo dire che un'algoritmo sia la descrizione di una sequenza di azioni che un esecutore deve compiere per giungere alla soluzione di un problema.

Quando un algoritmo non utilizza risorse superiori al necessario, come *memoria* e *tempo computazionale*, è detto **efficiente**.

Le risorse usate da un algoritmo possono essere misurate attraverso l'**analisi degli algoritmi**, mentre la progettazione di un algoritmo può essere effettuata attraverso delle **metodologie generali**.

Analizzare un algoritmo ha il significato di prevedere le risorse che l'algoritmo richiede. Quest'analisi può avvenire per mezzo della *notazione asintotica* e delle *ricorrenze*.

Per lo sviluppo di un algoritmo possiamo distinguere diverse *tecniche di programmazione*:

- **divide-et-impera**: Mergesort, Quicksort;
- **greedy**: codici di Huffman, Activity Selector, Zaino Frazionario;
- **programmazione dinamica**: Zaino 0-1, Longest Common Sequence, Catena di Montaggio, Parentesizzazione Ottima;
- **backtracking**;
- **branch and bound**.

Al fine di evitare ambiguità, la descrizione da utilizzare per un algoritmo deve essere il più possibile **formale**, e deve essere indipendente dal linguaggio di programmazione (pertanto si utilizza lo **pseudo-codice**).

Verranno poi introdotti ulteriori concetti, quali: automi, grammatiche regolari, linguaggi, macchina di Turing, Problemi P e NP.

LEZ 2 – Notazioni asintotiche

Il **calcolo asintotico** è utilizzato per analizzare la complessità di un algoritmo. In particolare, per stimare quanto aumenta il tempo al crescere della dimensione n dell'input. Attraverso le notazioni asintotiche è possibile **confrontare algoritmi differenti** che risolvono lo stesso problema e selezionare, quindi, il più efficiente.

Esistono **tre notazioni asintotiche**:

- **O** : è il limite superiore asintotico;
- **Ω** : è il limite inferiore asintotico;
- **Θ** : è il limite asintotico stretto.

Notazione O grande

Date $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ diremo che $f(n) = O(g(n))$ se e solo se $\exists c > 0, \exists n_0 > 0$ tale che $f(n) \leq cg(n), \forall n \geq n_0$.

(date due funzioni f e g definite sull'insieme dei valori naturali a valori nei reali positivi, diremo che $f(n)$ è un o-grande di $g(n)$ se e solo se esiste un valore $c > 0$ ed esiste un valore n_0 tale che $f(n)$ sia **minore o uguale di $cg(n)$** per ogni $n \geq n_0$).

Notazione Ω

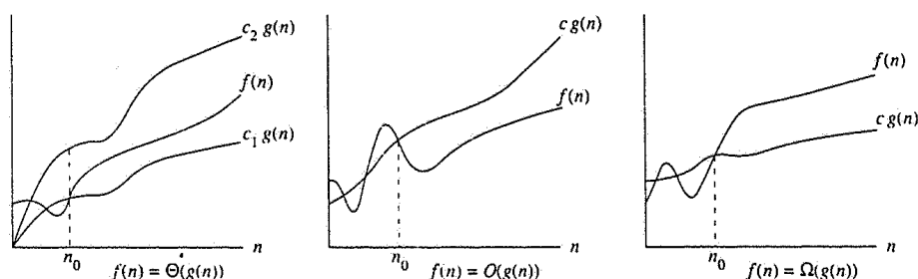
Date $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ diremo che $f(n) = \Omega(g(n))$ se e solo se $\exists c > 0, \exists n_0 > 0$ tale che $f(n) \geq cg(n), \forall n \geq n_0$.

(date due funzioni f e g definite sull'insieme dei valori naturali a valori nei reali positivi, diremo che $f(n)$ è un omega di $g(n)$ se e solo se esiste un valore $c > 0$ ed esiste un valore n_0 tale che $f(n)$ sia **maggiore o uguale di $cg(n)$** per ogni $n \geq n_0$).

Notazione Θ

Date $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ diremo che $f(n) = \Theta(g(n))$ se e solo se $\exists c_1, c_2 > 0, \exists n_0 > 0$ tale che $c_2g(n) \geq f(n) \geq c_1g(n), \forall n \geq n_0$.

(date due funzioni f e g definite sull'insieme dei valori naturali a valori nei reali positivi, diremo che $f(n)$ è un theta di $g(n)$ se e solo se esiste un valore $c_1, c_2 > 0$ ed esiste un valore n_0 tale che $f(n)$ sia **compreso fra $c_2g(n)$ e $c_1g(n)$** per ogni $n \geq n_0$).



Dunque:

- la notazione O descrive un limite superiore. Potremmo dire che stima la complessità *per eccesso*, e viene usato per lo studio del caso peggiore;
- la notazione Ω descrive un limite inferiore. Potremmo dire che stima la complessità *per difetto*, e viene usato per lo studio del caso migliore;
- la notazione Θ descrive un limite sia inferiore che superiore (*limite asintotico stretto*). Viene usato per lo studio del caso medio.

Teorema: un polinomio è O del suo termine di grado più elevato

Un polinomio è O del suo termine di grado più elevato, ovvero: dato un polinomio $P_k(n)$ di grado k allora $P_k(n) = O(n^k)$.

Dimostrazione:

$$\sum_{i=0}^k a_i n^i \leq \sum_{i=0}^k |a_i| n^i \leq \sum_{i=0}^k |a_i| n^k \leq \left(\sum_{i=0}^k |a_i| \right) n^k \leq c n^k$$

...se volessimo estendere le sommatorie:

$$\begin{aligned} a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0 &\leq \\ |a_k| n^k + |a_{k-1}| n^{k-1} + \dots + |a_1| n + |a_0| &\leq \\ |a_k| n^k + |a_{k-1}| n^k + \dots + |a_1| n^k + |a_0| n^k &\leq \\ (|a_k| + |a_{k-1}| + \dots + |a_1| + |a_0|) n^k &\leq c n^k \end{aligned}$$

Quindi, ad esempio:

$$\begin{aligned} - n^3 + 1000n^2 + 2000 &= O(n^3) \\ - 200n^3 + n^5 + 1000n^4 &= O(n^5) \\ - 100n^2 + n^{\frac{5}{2}} + 70n^{\frac{7}{3}} &= O(n^{\frac{5}{2}}) \end{aligned}$$

Proprietà di O

- Data una funzione $a(n) = O(c(n))$, allora $\lambda a(n) = O(c(n)) \forall \lambda > 0$.
- Date due funzioni a e b tali che $a(n) = O(c(n))$ e $b(n) = O(d(n))$, allora:
 1. $a(n) + b(n) = O(c(n) + d(n))$;
 2. $a(n)b(n) = O(c(n)d(n))$;
- Data una funzione $a(n) = O(c(n))$, se a sua volta $c(n) = O(d(n))$, allora $a(n) = O(d(n))$.

Proprietà di Ω

- Data una funzione $f(n) = \Omega(g(n))$, allora $af(n) = \Omega(g(n)) \forall a > 0$.
- Date due funzioni f e h tali che $f(n) = \Omega(g(n))$ e $h(n) = \Omega(i(n))$, allora:
 1. $f(n) + h(n) = \Omega(g(n) + i(n))$;
 2. $f(n)h(n) = \Omega(g(n)i(n))$;
- Data una funzione $f(n) = \Omega(g(n))$, se a sua volta $g(n) = \Omega(i(n))$, allora $f(n) = \Omega(i(n))$.

nb: queste proprietà sono analoghe alla notazione asintotica O.

Legami tra O e Ω

Tra O e Ω vale la **Regola della dualità**, ossia possiamo affermare che:

$$f(n) = \Omega(g(n)) \text{ se e solo se } g(n) = O(f(n))$$

Dimostriamolo:

$$\begin{aligned} f(n) = \Omega(g(n)) &\Leftrightarrow f(n) \geq cg(n) && \text{per costante } c \text{ opportuna ed } n \text{ sufficientemente grande} \\ &\Rightarrow g(n) \leq \left(\frac{1}{c}\right)f(n) && \text{per costante } c \text{ opportuna ed } n \text{ sufficientemente grande} \\ &\Rightarrow g(n) = O(f(n)) \end{aligned}$$

Ordinamento tra funzioni

Espressione O	Nome
$O(1)$	costante
$O(\log_a \log_b n)$	log log
$O(\log_a n)$	logaritmica
$O(\sqrt[c]{n}), c > 1$	sublineare
$O(n)$	lineare
$O(n \log_a n)$	$n \log n$
$O(n^k) (k > 1)$	polinomiale
$O(a^n) (a > 1)$	esponenziale
$O(n!)$	fattoriale
$O(n^n)$	n^n

Esempi

- $(\log n)^2 = O(n)$
- $3 \log n^2 = O(\sqrt{n})$
- $\sqrt{n} = O(n)$
- $n^9 = O(3^n)$
- $6^n = O(n!)$

nb: più scendiamo di livello, più velocemente cresce la funzione al crescere di n , dove: una funzione costante rappresenta il caso migliore, e n^n il caso peggiore. Se possibile, si cerca di utilizzare sempre algoritmi di complessità che non vadano oltre a quella polinomiale.

• Logaritmi contro lineari

Si ha che: $\log_2 n = O(n)$

Dalla definizione della notazione O , occorre **provare che** $\exists c > 0, \exists n_0$ tale che $f(n) \leq c g(n), \forall n \geq n_0$.

Si può **dimostrare per induzione**:

1. **caso base**: per $n = 1$ abbiamo che $\log_2 1 = 0$, il quale è ≤ 1 .
2. **passo induttivo**: supponiamo che sia vera $\log_2 n \leq n$ per ogni $n > 1$, dimostriamo che $\log_2(n+1) \leq n+1$.
 $\log_2(n+1)$ è più piccolo di $\log_2(n+n)$, che a sua volta può essere riscritto come $\log_2(2n)$, che per le proprietà dei logaritmi può essere riscritta ancora come $\log_2 2 + \log_2 n$, che può essere riscritto ancora come $1 + \log_2 n$, che è $\leq n+1$.

Questo secondo punto può essere riscritto in modo più compatto come:
 $\log_2(n+1) \leq \log_2(n+n) \leq \log_2(2n) \leq \log_2 2 + \log_2 n \leq 1 + \log_2 n \leq n+1$.

Se invece il **logaritmo non è in base 2**, ma è in una qualche base a , si ha che il $\log_a n = (\log_a 2)(\log_2 n) \leq (\log_a 2)n \Rightarrow \log_a n = O(n)$.

Se invece abbiamo che $\log_a n^b$ con $b > 1$, si ha che $\log_a n^b = b \log_a n \leq acn \Rightarrow \log_a n^b = O(n)$.

• Logaritmi contro Radici

Si ha che: $\forall a, b, c, k > 0, (\log_c n^b)^a = O(n^k)$

Possiamo verificarlo con alcuni esempi:

- $(\log_2 n^6)^{100000} = O(n^{\frac{1}{100000}})$
- $(\log_2 n^{6000})^{9000} = O(n^{\frac{1}{100000}})$

• Polinomi contro Esponenziali

Si ha che: $\forall k > 0, a > 1, n^k = O(a^n)$

Per dimostrarlo, ci basta osservare come $n^k \leq n^{\frac{n}{\log_a n}}, \forall n$ tale che $k \leq \frac{n}{\log_a n}$.

Quindi: $n^k \leq n^{\frac{n}{\log_a n}} \leq (a^{\log_a n})^{\frac{n}{\log_a n}} \leq a^{\frac{n \log_a n}{\log_a n}} \leq a^n, \forall n$ tale che $k \leq \frac{n}{\log_a n}$.

• Esponenziali e Fattoriali

Si ha che:

- $a^n = O(n^n)$, in quanto $a^n \leq n^n \forall a \leq n$;
- $n! = O(n^n)$, in quanto $n! \leq n^n$.
- $a^n = O(n!)$.

Il comportamento della funzione fattoriale quando n diventa molto grande ci viene suggerito dalla **formula di Stirling**:

$$n! \cong \left(\frac{n}{e}\right)^n \sqrt{2\pi n} \left(1 + O\left(\frac{1}{n}\right)\right)$$

nb Induzione:

Il principio di induzione è una tecnica di dimostrazione che consente di dimostrare la validità di una tesi della verifica di due condizioni: la validità del *passo zero* e la validità del *passo induttivo*.

Viene quasi sempre usata per tutti quegli enunciati del tipo “Dimostrare che per ogni $n \in \mathbb{N}$ vale *proprietà*”.

Tale principio stabilisce che, se valgono le seguenti condizioni:

1. *passo zero*: $P(n)$ è vera per $n = 0$, cioè $P(0)$ è vera;*
2. *passo induttivo*: supponendo che $P(n)$ sia vera, ne consegue che $P(n+1)$ è vera ($P(n) \Rightarrow P(n+1)$ vera);

Allora $P(n)$ è vera per tutti gli $n \in \mathbb{N}$.

Sull'*induzione* c'è da precisare una cosa: in questo contesto supponiamo sempre che n sia un intero > 0 , per questo non consideriamo $P(0)$ ma sempre $P(1)$. Anche per questo motivo sarebbe preferibile chiamarlo **caso base**.

Metodo Generale per la scelta tra le notazioni O , Ω , Θ

Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = p$, con $p \neq 0$, allora $f(n) = \Theta(g(n))$, questo perché $f(n) = O(g(n))$ ma anche $g(n) = O(f(n))$.

Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, allora $f(n) = O(g(n))$, questo perché $f(n) = O(g(n))$ tuttavia $g(n) \neq O(f(n))$.

Se $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, allora $f(n) = \Omega(g(n))$, questo perché $g(n) = O(f(n))$ tuttavia $f(n) \neq O(g(n))$.

Dimostriamolo:

Per semplicità proviamo la prima relazione: $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = p$, con $p \neq 0$. Inoltre, consideriamo due valori δ e c tali che $\delta = p$ e $c = 2p$.

Per la def. di limite abbiamo che $\left\| \frac{f(x)}{g(x)} - p \right\| \leq \delta$, ma questo implica che $\left\| \frac{f(x)}{g(x)} \right\| \leq \delta + p$.

Poiché $\delta + p = c$, allora $\left\| \frac{f(x)}{g(x)} \right\| \leq c$. Da qui, possiamo affermare che: $f(n) \leq cg(n)$, $\forall n \geq n_0$, ossia $f(n) = O(g(n))$.

Una volta verificato che $f(n) = O(g(n))$, ci resta solo da dimostrare che $g(n) = O(f(n))$, tuttavia si tratta di una conseguenza banale in quanto $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = p$ implica che $\lim_{x \rightarrow \infty} \frac{g(x)}{f(x)} = \frac{1}{p}$, e quindi si applica il ragionamento precedente scambiando semplicemente i ruoli tra $f(x)$ e $g(x)$.

Esempi:

- Se $f(n) = n^3 + n \log n + 8$, $g(n) = n^3 + 7$ allora

$$\lim_{n \rightarrow \infty} \frac{n^3 + n \log n + 8}{n^3 + 7} = 1 \neq 0 \Rightarrow f(n) = \Theta(g(n))$$

- Se $f(n) = \log n + 8$, $g(n) = 2n + 7$ allora

$$\lim_{n \rightarrow \infty} \frac{\log n + 8}{2n + 7} = 0 \Rightarrow f(n) = O(g(n))$$

Ulteriori notazioni asintotiche

Esistono notazioni asintotiche per parlare di limiti **non** asintoticamente stretti.

$$O(n) \rightarrow o(n)$$

$$\Omega(n) \rightarrow \omega(n)$$

Si usa la **notazione o** (o piccolo) per denotare un limite superiore che non è asintoticamente stretto. Formalmente scriveremo che:

$$f(n) = o(g(n)) \text{ se e solo se } \forall c > 0, \exists n_0 > 0 \text{ tale che } f(n) < cg(n), \forall n \geq n_0.$$

Si usa la **notazione ω** (omega piccolo) per denotare un limite inferiore che non è asintoticamente stretto. Formalmente scriveremo che:

$$f(n) = \omega(g(n)) \text{ se e solo se } \forall c > 0, \exists n_0 > 0 \text{ tale che } f(n) > cg(n), \forall n \geq n_0.$$

La **differenza fra le versioni “piccole”** rispetto a quelle “**grandi**”, è che nelle grandi volevamo che ci fosse almeno una $c > 0$ per cui esistesse $n_0 > 0$, mentre in questo caso vogliamo che per OGNI $c > 0$ esista $n_0 > 0$.

Inoltre, abbiamo che $f(n)$ è strettamente minore (o maggiore) di $cg(n)$.

LEZ 3 – Analisi di un algoritmo & Insertion Sort

ANALISI DI UN ALGORITMO

Per **analisi di un algoritmo** si intende la valutazione delle risorse richieste da un algoritmo per risolvere un problema dato.

Per risorse intendiamo essenzialmente:

- **tempo** (di esecuzione) necessario all'algoritmo per produrre l'output richiesto;
- **memoria** utilizzata dall'algoritmo durante la sua esecuzione.

Il **tempo di esecuzione** dipende da molti fattori, come: l'abilità del programmatore nella stesura del programma, dalla dimensione dell'input, dal numero di operazioni elementari eseguite, ecc.

Le costanti moltiplicative non ci interessano in quanto possono essere difficili da quantificare (es. l'abilità del programmatore), per questo per misurare il tempo di esecuzione utilizzeremo le notazioni asintotiche O , Ω , Θ .

Con **operazioni elementari** si intendono operazioni come l'addizione o la moltiplicazione, il confronto fra numeri, e lo spostamento di un puntatore. Assumiamo che ogni operazione elementare richieda un *tempo costante di esecuzione*, che rappresenta la nostra unità elementare di tempo.

Il tempo di esecuzione non è solo influenzato dalla dimensione dell'input, ma spesso anche dalla **forma** di questo. Ad esempio, alcuni algoritmi di ordinamento sono in grado di ordinare un array più velocemente se questo risulta già parzialmente ordinato. Proprio perché la **forma dell'input** può incidere sul tempo di esecuzione, distinguiamo 3 casi:

- il **caso migliore**, ossia la forma più favorevole di input;
- il **caso peggiore**, ossia la forma più sfavorevole di input;
- il **caso medio**.

Con $T(n)$ si indica il tempo di esecuzione di un algoritmo A su un input di dimensione n . Se $T(n)$ è $O(n^2)$ allora l'algoritmo A non richiede mai un tempo superiore a cn^2 , mentre se $T(n)$ è $\Omega(n^2)$ allora l'algoritmo A non richiede mai un tempo inferiore a cn^2 .

Calcolo delle complessità di tempo

Per il calcolo della complessità di tempo si seguono alcune semplici regole:

- il costo delle **istruzioni semplici** è **$O(1)$** .
- Il costo di strutture del tipo **if then else** è pari al tempo per effettuare il confronto, solitamente **$O(1)$** più il **costo dell'alternativa più costosa**.
- Il costo dei cicli (**do, for, repeat, while**) è pari alla **somma del costo di ogni singola iterazione**.

Invariante di ciclo

L'invariante di ciclo è uno strumento che permette di dimostrare la correttezza dei cicli iterativi. Per un'invariante di ciclo, dobbiamo dimostrare 3 condizioni:

- **inizializzazione:** l'invariante di ciclo è vera prima della prima iterazione del ciclo;
- **conservazione:** se la l'invariante di ciclo è vera prima di un'iterazione del ciclo, rimane vera anche prima della successiva iterazione (ovvero rimane vera anche dopo aver eseguito il corpo del loop);
- **conclusione:** l'invariante di ciclo è vera dopo tutte le iterazioni.

In pratica, si determina una proprietà (che dipende dal ciclo) che è valida prima che il ciclo inizi (inizializzazione). Si va a verificare che tale proprietà sia valida anche dopo che sia stata eseguita una iterazione (conservazione). Se dopo aver eseguito tutte le iterazioni tale proprietà continua ad essere rispettata, allora l'invariante di ciclo è vera.

INSERTION SORT

L'**Insertion Sort** è un algoritmo di ordinamento molto semplice, e che risulta anche essere efficiente quando si deve ordinare un piccolo numeri di elementi.

L'algoritmo riproduce il modo con cui normalmente un essere umano ordina delle carte in una mano. Si inizia con una mano vuota e si prende una carta alla volta dal tavolo, inserendola nella *giusta posizione*. Per trovare la giusta posizione, si confronta la carta con ogni carta presente nella mano (es. leggendo da destra verso sinistra) e quando troviamo una carta il cui valore è minore della carta pescata, inseriamo la carta pescata subito dopo quella carta e facciamo slittare in avanti tutte le altre.

Come abbiamo già notato, la caratteristica di questo algoritmo è che l'insieme delle carte che avremo in mano in quel momento (sottoarray) sarà già ordinato, ed inoltre si tratta di un ordinamento *in loco* in quanto non vengono utilizzati array di appoggio.

Pseudocode

```
for j ← 2 to Length(A)
    key ← A[j]
    i ← j-1
    While (i > 0 and A[i] > key)
        Do    A[i+1] ← A[i]
              i ← i-1
    A[i+1] ← key
```

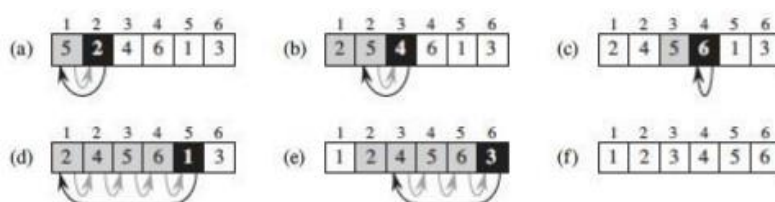
L'indice j indica l'elemento corrente. Siccome un array costituito da un solo elemento è già per forza ordinato, j avrà valore 2.

Il vettore $A[1 \dots j-1]$ indica tutti gli elementi già ordinati mentre il vettore $A[j \dots n]$ indica tutti gli elementi ancora da ordinare.

$A[j]$ rappresenta l'elemento che dovrà essere inserito nella giusta posizione, e per questo motivo gli diamo il nome di *key*.

Entriamo in un ciclo while interno. Con $A[i]$ indichiamo l' i -simo elemento che dovrà essere confrontato con la chiave. Se $A[i] > key$, significa che dobbiamo fare slittare il valore presente in $A[i]$ in una posizione in avanti (per questo $A[i+1] = A[i]$) e decrementiamo i in modo da effettuare poi il confronto fra la chiave con il nuovo $A[i]$, questo finché ci sono ancora elementi con cui effettuare il confronto ($i > 0$) oppure finché $A[i]$ non risulta essere $\leq$ della chiave.

Alla fine del ciclo while interno, l'elemento viene inserito nella posizione corretta.



Invariante di ciclo: Insertion Sort

L'**invariante di ciclo** che vogliamo essere sempre vera per l'insertion Sort è la seguente: "All'inizio di ogni iterazione del ciclo for, il sottoarray $A[1 \dots j-1]$ è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 \dots j-1]$ ".

Dimostriamo le 3 condizioni:

- **inizializzazione:** $j=2$, ciò significa che il sottoarray $A[1 \dots j-1]$ è costituito da un solo elemento, e quindi ovviamente è ordinato;
- **conservazione:** ogni iterazione ingrossa il sottoarray di un elemento. La posizione di questo elemento ($A[j]$) viene individuata dal corpo del ciclo for esterno, il quale opera spostando di una posizione verso destra $A[j-1]$, $A[j-2]$, ..., e così via finché non troverà la posizione appropriata per $A[j]$, dove lo inserirà. Il sottoarray $A[1 \dots j]$ è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 \dots j]$. Perciò, *l'incremento di j per la successiva iterazione del ciclo for preserva l'invariante di ciclo*;
- **terminazione:** j raggiunge n (la lunghezza di A). Poiché ogni iterazione del ciclo aumenta j di 1, alla fine dell'algoritmo avremo che $j=n+1$. Se sostituiamo j con $n+1$ nella formulazione dell'invariante di ciclo, avremo che "il sottoarray $A[1 \dots n]$ è ordinato ed è formato dagli stessi elementi che originariamente erano in $A[1 \dots n]$ ", tuttavia il sottoarray $A[1 \dots n]$ corrisponde all'intero array, e quindi tutto l'array è ordinato.

Pertanto, l'**algoritmo è corretto**.

Analisi della complessità dell'Insertion Sort

Dobbiamo considerare **3 casi**:

- **caso migliore:** si ha quando l'input è già ordinato. In tal caso il ciclo interno non viene mai eseguito e dunque il tempo di esecuzione $T(n)$ è lineare, ossia $T(n) = \Theta(n)$.
- **caso peggiore:** si ha quando l'input è ordinato in modo decrescente. Ciò significa che ogni volta che vogliamo inserire un nuovo elemento dovremo effettuare $j-1$ confronti e $j-1$ assegnazioni. Se teniamo conto che saltiamo la prima iterazione perché un array di size 1 è già ordinato, il numero di operazioni necessarie per calcolare l'IS è: $\sum_{j=2}^n (j-1) = \sum_{j=2}^n j - \sum_{j=2}^n 1 = \frac{n(n+1)}{2} - 1 - (n-1) = \frac{n(n-1)}{2}$. Dunque, $T(n)$ è un polinomio di grado 2 di n , ossia $T(n) = \Theta(n^2)$.
- **caso medio:** il caso medio è spesso cattivo quanto il peggiore. Supponendo di aver scelto a caso gli elementi di A , in media metà degli elementi saranno più piccoli di $A[j]$ mentre gli altri saranno più grandi, per questo motivo consideriamo $(j-1)/2$ per calcolare il caso medio, ma la sostanza non cambia e $T(n)$ risulterebbe comunque un polinomio di grado 2 di n , ossia $T(n) = \Theta(n^2)$.

LEZ 4 – Paradigma Divide-et-Impera & Merge Sort

Il **Divide-et-Impera** è un approccio *top-down* che consiste nel suddividere un problema in sottoproblemi simili al problema originale, ma di dimensioni più piccole, che poi vengono risolti separatamente. Infine, le soluzioni dei sottoproblemi vengono combinate per ottenere la soluzione del problema originale.

Dunque, il **divide et impera** si compone di 3 passi:

1. **Divide**: il problema viene suddiviso in vari sottoproblemi, simili al problema originale ma di dimensioni più piccole;
2. **Impera**: i sottoproblemi vengono risolti in modo ricorsivo;
3. **Combina**: Combinano le soluzioni dei sottoproblemi per costruire una soluzione del problema originale.

Questo approccio è utilizzato soprattutto da algoritmi con struttura ricorsiva (es. *MergeSort* e *QuickSort*). Quando i problemi sono abbastanza grandi da essere risolti ricorsivamente si ha il **caso ricorsivo**, mentre se sono sufficientemente piccoli da non richiedere ulteriori ricorsioni si è raggiunto il **caso base**.

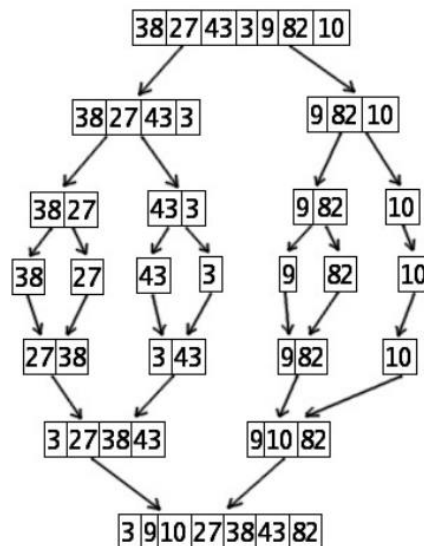
Il principale vantaggio del Divide-et-Impera è che consente di avere tempi di esecuzione nel caso peggiore assai inferiori rispetto a quelli, ad esempio, degli *approcci incrementali* (es. Insertion Sort).

MERGE SORT

Il Merge-Sort è un algoritmo di ordinamento di complessità $O(n \log(n))$, **stabile** (cioè, mantiene l'ordine iniziale (di input) di elementi uguali nell'array ordinato output), e che adotta l'approccio **divide-et-impera**. Inoltre, è un esempio di **comparison sort**. Fu inventato da John von Neumann nel 1945.

Possiamo dividere il funzionamento del Merge-Sort in tre passi:

- **divide**: l'array in input viene diviso in due sottoliste di metà lunghezza. Nel caso l'array fosse dispari, la prima sottolista avrà un elemento in più rispetto alla seconda.
- **impera**: viene ordinato, mediante chiamate ricorsive alla funzione stessa, ciascuno dei due sottoarray.
- **combina**: viene eseguito il Merge dei due sottoarray ordinati.

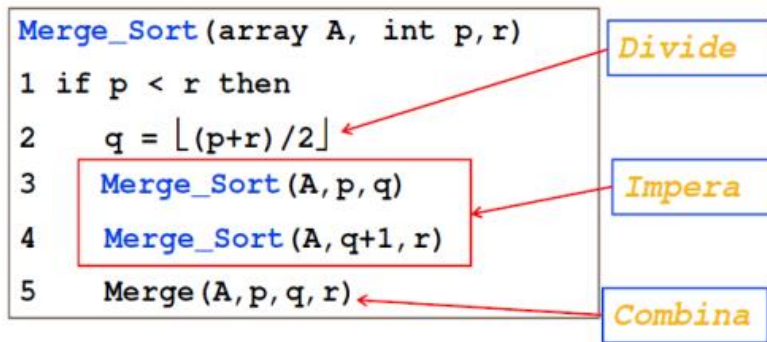


Dunque l'**idea** è chiara: l'algoritmo suddivide l'array originario in porzioni di dimensioni via via più piccole fino ad arrivare a porzioni di dimensioni 1. Al primo passo verrà applicato il *merge* su queste porzioni così da ottenere porzioni **ordinate** di dimensioni 2. Al passo successivo verrà applicato il merge a ciascuna coppia di porzioni di dimensioni 2 così da ottenere porzioni ordinate di dimensioni 4, e così via finché l'intero array non è ordinato. Dunque, il merge verrà applicato su sottosequenze di dimensioni via via crescenti.

L'**algoritmo di merging** può essere realizzato in due modi: *merge astratto sul posto* o *merge a due vie*.

Il Cormen propone un Merge sul posto con *sentinelle*.

Pseudocodice MergeSort



Pseudocodice Merge

1. $n_1 \leftarrow q-p+1$
2. $n_2 \leftarrow r-q$
3. Crea due nuovi array $L[1 \dots n_1+1]$ e $R[1 \dots n_2+1]$
4. for $i \leftarrow 1$ to n_1
5. $L[i] \leftarrow A[p+i-1]$
6. for $j \leftarrow 1$ to n_2
7. $R[j] \leftarrow A[q+j]$
8. $L[n_1+1] \leftarrow +\infty$
9. $R[n_2+1] \leftarrow +\infty$
10. $i \leftarrow 1$
11. $j \leftarrow 1$
12. for $k \leftarrow p$ to r
13. if $L[i] \leq R[j]$
14. $A[k] \leftarrow L[i]$
15. $i \leftarrow i+1$
16. else $A[k] \leftarrow R[j]$
17. $j \leftarrow j+1$

Spiegazione

• MergeSort

Il MergeSort calcola prima l'indice centrale in modo tale da suddividere la porzione di array considerata in due parti di dimensioni $n/2$, quindi richiama ricorsivamente se stesso prima sulla porzione sinistra, poi sulla porzione destra, infine richiama il MergeSort per combinare i risultati ottenuti.

p rappresenta l'indice di inizio, r rappresenta l'indice di fine, q rappresenta l'indice centrale.

Il MergeSort verrà eseguito solo se l'indice di inizio è minore dell'indice di fine ($p < r$), questo perché se tale condizione non è rispettata ci troviamo di fronte ad un array (o sottoarray) di dimensione 1, che non può essere ulteriormente suddiviso e dunque ci troviamo nel caso base.

• Merge

Il Merge calcola prima le dimensioni delle due porzioni di array da fondere: n_1 è la dimensione della porzione di sinistra, mentre n_2 è la dimensione della porzione di destra. Quindi crea i due array L (porzione sinistra) e R (porzione destra), i quali conterranno una determinata porzione degli elementi di A.

Dopodichè, grazie a un ciclo For che va dall'indice di inizio p all'indice di fine r, controlla ad ogni passo il minore tra l'i-esimo elemento di L e il j-esimo elemento di R. Il minore dei due viene inserito nell'array di output A ed, inoltre, viene incrementato l'indice dell'array che ha "fornito" l'elemento ad A.

Da notare come gli array L ed R in realtà siano sovradimensionati di 1 elemento, che conterrà il valore +inf, che corrispondono alle *sentinelle*. Il motivo per cui abbiamo bisogno di queste sentinelle è perché l'ultimo elemento rimasto dei due array non ancora inserito, per com'è fatto l'algoritmo, dovrà necessariamente fare il confronto con qualcosa, ma visto che è l'ultimo elemento non ha nessuno con cui fare confronti (in realtà dovrebbe essere inserito e basta). Per una questione di semplicità gli facciamo fare direttamente il confronto con +inf (la sentinella), da cui risulterà necessariamente più piccolo e verrà inserito nell'array A.

Es.

Supponiamo di avere l'array [7|5], da cui otteniamo i sottoarray [7] e [5].

Procediamo alla fase di *merge*: entrambi gli array hanno size $n = 1$, ma abbiamo anche detto che dobbiamo inserire uno spazio per ∞ , dunque al momento della creazione il loro vero size sarà $n+1$. I due sottoarray saranno dunque [7| ∞] e [5| ∞].

A questo punto dobbiamo stabilire il loro ordine nell'array A. Usiamo un ciclo for, dove k deve andare da p a r, supponiamo che $p=1$ e $r=2$, dunque effettueremo 2 iterazioni.

Alla prima iterazione avremo che 7 non è < 5 , dunque 5 sarà inserito nell'array A. Alla successiva iterazione avremo 7 dovrà essere confrontato con l'elemento successivo a 5, che è ∞ , ed essendo $7 < \infty$ allora anche 7 verrà inserito nell'array A.

Il ciclo termina e quella porzione dell'array A risulterà ordinata.

Invariante di ciclo: MergeSort

L'**invariante di ciclo** che vogliamo essere sempre vera per il MergeSort è la seguente:
"All'inizio di ogni iterazione del ciclo for, il sottoarray $A[p \dots k-1]$ contiene, ordinati, i $k-p$ elementi più piccoli di $L[1 \dots n_1+1]$, e $R[1 \dots n_2+1]$.
Inoltre, $L[i]$ e $R[j]$ sono i più piccoli elementi dei loro array che non sono copiati in A ".

Dimostriamo le 3 condizioni:

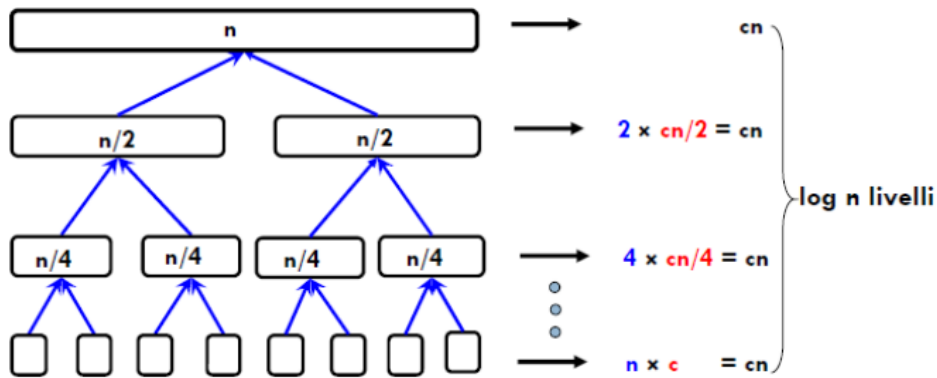
- **inizializzazione:** prima dell'iterazione di ogni ciclo, abbiamo $k = p$, quindi il sottoarray $A[p \dots k-1]$ è vuoto e contiene i $k - p = 0$ elementi più piccoli di L e R . Inoltre, poiché $i = j = 1$, $L[i]$ e $R[j]$ sono i più piccoli elementi degli array che non sono stati ancora copiati in A .
- **conservazione:** all'iterazione successiva, supponiamo che $L[i] \leq R[j]$, quindi $L[i]$ è l'elemento più piccolo che non è stato ancora copiato in A . Poiché $A[p \dots k-1]$ contiene i $k - p$ elementi più piccoli, dopo aver copiato $L[i]$ in $A[k]$ il sottoarray $A[p \dots k]$ conterrà i $k - p + 1$ elementi più piccoli. Incrementando k e i si ristabilisce l'invariante di ciclo per la prossima iterazione. Se è $L[i] \geq R[j]$ il ragionamento è simile al precedente, ma in questo caso si incrementano k e j per conservare l'invarianza di ciclo di L e R .
- **terminazione:** alla fine del ciclo, $k = r+1$. Per l'invariante di ciclo il sottoarray $A[p \dots k-1]$ contiene i $k - p = r - p + 1$ elementi ordinati che sono più piccoli di L e R . Insieme alle sentinelle, L e R contengono $r - p + 3$ elementi. Tutti gli elementi, tranne le sentinelle che sono i due più grandi, sono stati copiati in A .

Pertanto, l'**algoritmo è corretto**.

Analisi della complessità del MergeSort

L'operazione chiave dell'algoritmo è la fusione di due sottosequenze ordinate. Tale procedura, denominata **Merge**, impiega un tempo $\Theta(n)$.

Visto che l'idea è quella di suddividere l'array in sottoarray sempre più piccoli ($n/2$, $n/4$, ecc.), si crea una struttura ad albero composta da $\log n$ livelli. Quindi, il numero di passi è pari a $\Theta(\log n)$.



Per questi motivi, la **complessità di tempo** del MergeSort è data da $\Theta(n \log n)$.

Per quanto riguarda la **complessità di spazio**, essa risulta essere $\Theta(n \log n)$ perché quest'algoritmo non opera in place, ma utilizza un vettore n , due vettori $n/2$, quattro vettori $n/4$, ecc... quindi proprio $n \log n$.

Quando un algoritmo contiene una chiamata ricorsiva a se stesso, il suo tempo di esecuzione spesso può essere descritto con un'equazione di ricorrenza, che esprime il tempo di esecuzione totale di un problema di dimensione n in funzione del tempo di esecuzione per input più piccoli.

Per stabilire la ricorrenza per $T(n)$ possiamo fare il seguente ragionamento:

- **Divide:** questo passo calcola semplicemente il centro del sottoarray, richiedendo un tempo $D(n) = \Theta(1)$;
- **Impera:** risolviamo in modo ricorsivo i due sottoproblemi di dimensione $n/2$. Ciò contribuisce con $2T(n/2)$ al tempo di esecuzione;
- **Combina:** la procedura MERGE richiede un tempo $C(n) = \Theta(n)$.

Avremo dunque, per $n > 1$: $2T(n/2) + D(n) + C(n)$, ovvero $2T(n/2) + \Theta(1) + \Theta(n)$, ma sommare una funzione che è $\Theta(1)$ con una che è $\Theta(n)$ dà ancora $\Theta(n)$.

Per questo ragionamento, l'equazione di ricorrenza del MergeSort risulta essere:

$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & \text{se } n > 1 \end{cases} \quad \text{nb: } c = \text{costante}$$

Infatti, il pezzo sopra rappresenta il caso base mentre il pezzo sotto il caso ricorsivo. $T(n)$ è dato dal costo di due chiamate ricorsive che hanno una dimensione pari alla metà della dimensione originale ($2T(N/2)$) + il costo all'interno della funzione stessa (n).

LEZ 5/6 – Ricorrenze

Una **ricorrenza** è un'equazione o disequazione che descrive una funzione in termini del suo valore su input sempre più piccoli.

Risolvere una ricorrenza consiste nell'ottenere i limiti asintotici "O" e "Ω" della soluzione.

Per derivare le **relazioni di ricorrenza** per la complessità di un algoritmo $T(n)$ è necessario:

- determinare le dimensioni di un input;
- determinare quale valore di n , indicato con n_0 , viene utilizzato per il caso base della ricorsione;
- determinare la complessità di $T(n_0)$ del caso base della ricorsione (solitamente è una costante $O(1)$).

Ad esempio:

$$T(n) = \begin{cases} c & \text{se } n = n_0 \\ aT(f(n)) + g(n) & \text{se } n > n_0 \end{cases}$$

- n_0 è la base della ricorsione;
- c è la complessità per la base della ricorsione;
- a è il numero di chiamate ricorsive effettuate;
- $f(n)$ è la dimensione dell'input dei problemi risolti con le chiamate ricorsive;
- $g(n)$ è la complessità di calcolo dell'algoritmo non generata dalle chiamate ricorsive.

Possiamo distinguere **4 metodi** di risoluzione delle ricorrenze:

- Metodo di sostituzione;
- Metodo dell'albero di ricorsione;
- Metodo del teorema dell'esperto (Master Theorem);
- Metodo iterativo.

SERIE NUMERICHE DA RICORDARE

Serie aritmetica:

$$\sum_{k=1}^n k = \frac{1}{2}n(n+1) = \theta(n^2)$$

Sommatoria dei quadrati:

$$\sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \theta(n^3)$$

Sommatoria dei cubi:

$$\sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4} = \theta(n^4)$$

Serie armonica:

$$\sum_{k=1}^n \frac{1}{k} = \log_e n + O(1)$$

Serie geometrica infinita:

$$\sum_{k=1}^{\infty} x^k = \frac{1}{1-x}$$

Serie geometrica:

$$\sum_{k=0}^n x^k = \frac{1-x^{n+1}}{1-x}$$

o anche equivalentemente:

$$\frac{x^{n+1}-1}{x-1}$$

Serie geometrica (2):

$$\sum_{k=1}^n kx^k = \frac{x}{(1-x)^2}$$

METODO DI SOSTITUZIONE

Nel metodo di sostituzione si ipotizza un limite e poi si usa l'induzione matematica per dimostrare che l'ipotesi è corretta.

Tale metodo è composto da due passi:

1. ipotizzare la soluzione;
2. utilizzare l'induzione matematica per dimostrare che l'ipotesi è corretta.

Si tratta di un metodo potente, ma può essere usato solo quando è facile indovinare la soluzione. Infatti non esiste una regola generale per indovinare la soluzione corretta per ogni ricorrenza, però esistono delle **euristiche** che possono aiutarci:

- se una ricorrenza è simile ad una che abbiamo già visto in passato, allora può essere una buona idea tentare una **soluzione simile**;
- possiamo provare i **limiti laschi** della ricorrenza, ovvero possiamo ipotizzare un limite inferiore e un limite superiore piuttosto larghi (es. $\Omega(n)$ e $O(n^2)$), e una volta verificati si può gradualmente alzare il limite inferiore e abbassare il limite superiore finché entrambi i limiti non convergono alla soluzione corretta e asintoticamente stretta (es. $\Theta(n \log n)$).

Esempio:

Determinare un limite superiore per la ricorrenza: $T(n) = 3T\left(\left\lceil \frac{n}{3} \right\rceil\right) + n$

Ipotizziamo una soluzione: siccome sembra simile al mergesort, supponiamo che questa sia **$O(n \log n)$** . (nb: la base del log è 2).

Ipotizziamo che sia vera per $n/3$, dunque abbiamo:

$$T\left(\left\lceil \frac{n}{3} \right\rceil\right) \leq c \left(\left\lceil \frac{n}{3} \right\rceil\right) \log \left(\left\lceil \frac{n}{3} \right\rceil\right)$$

Sostituendo abbiamo:

$$T(n) \leq 3c \left\lceil \frac{n}{3} \right\rceil \log \left(\left\lceil \frac{n}{3} \right\rceil\right) + n \quad // \text{Sostituiamo } T\left(\left\lceil \frac{n}{3} \right\rceil\right) \text{ con } c \left(\left\lceil \frac{n}{3} \right\rceil\right) \log \left(\left\lceil \frac{n}{3} \right\rceil\right).$$

$$T(n) \leq cn \log \left(\left\lceil \frac{n}{3} \right\rceil\right) + n \quad // \text{Il 3 va via per semplificazione}$$

$$T(n) \leq cn \log n - cn \log 3 + n \quad // \text{proprietà dei logaritmi}$$

$$T(n) \leq cn \log n - n(c \log 3 - 1) \quad // \text{raccoltiamo } n \text{ a destra}$$

$$T(n) \leq cn \log n \quad \left(\text{per } c \geq \frac{1}{\log 3}\right)$$

Ricordiamo che vogliamo verificare che $cn \log n - n(c \log 3 - 1) \leq cn \log n$, dunque dobbiamo trovare una c tale che l'equazione venga rispettata (ultimo passaggio).

Esempio 2

Determinare un limite superiore per la ricorrenza: $T(n) = 3T\left(\left\lfloor \frac{n}{3} \right\rfloor + 17\right) + n$

Tale ricorrenza può sembrare complicata per via della presenza del termine 17. In realtà, quando n è grande, $T\left(\left\lfloor \frac{n}{3} \right\rfloor + 17\right)$ si comporta allo stesso modo di $T\left(\left\lfloor \frac{n}{3} \right\rfloor\right)$. Si dice che “17 viene assorbito dalla notazione asintotica”. Dunque possiamo sostituire la ricorrenza di prima con $3T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + n$ e otterremo lo stesso risultato.

Finezze

Può capitare che si ipotizzi correttamente la soluzione di una ricorrenza, ma in qualche modo i calcoli matematici non tornano quando operiamo la sostituzione. Il motivo è che normalmente l'ipotesi induttiva non è abbastanza forte per dimostrare il limite esatto. In tal caso basta correggere l'ipotesi sottraendo un termine di ordine inferiore per verificare l'ipotesi.

Esempio:

data la ricorrenza: $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1$, supponiamo che la soluzione sia $T(n) = O(n)$. Dunque, proviamo a dimostrare che $T(n) \leq cn$.

Se proviamo a risolvere abbiamo: $T(n) \leq c\left\lfloor \frac{n}{2} \right\rfloor + c\left\lfloor \frac{n}{2} \right\rfloor + 1$,
che non implica che $T(n) \leq cn$, qualunque sia il valore di c .

La nostra ipotesi non funziona per la presenza della costante 1. Possiamo superare questa difficoltà sottraendo un termine di ordine inferiore alla precedente.

La nuova ipotesi è: $T(n) \leq cn - d$, dove $d \geq 0$.

Se proviamo a risolvere abbiamo:

$$T(n) \leq \left(c\left\lfloor \frac{n}{2} \right\rfloor - d\right) + \left(c\left\lfloor \frac{n}{2} \right\rfloor - d\right) + 1$$

$$T(n) \leq cn - 2d + 1$$

$$T(n) \leq cn - d, \quad \forall d \geq 1.$$

Tranelli

Quando vogliamo dimostrare un'ipotesi, dobbiamo stare attenti a dimostrare la **forma esatta** dell'ipotesi.

Esempio:

data la ricorrenza: $T(n) = 3T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + n$, supponiamo che la soluzione sia $T(n) = O(n)$.

Se proviamo a risolvere abbiamo:

$$T(n) \leq 3c \left\lfloor \frac{n}{3} \right\rfloor + n$$

$$T(n) \leq cn + n$$

Dunque potremmo affermare che $T(n) = O(n)$, tuttavia NON abbiamo dimostrato la forma esatta dell'ipotesi, che invece richiede che $T(n) \leq cn$.

Sostituzione di variabili

Consideriamo la ricorrenza: $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log_2 n$

Se operiamo la sostituzione $n = 2^m$, questa diventa:

$$T(2^m) = 2T(\sqrt{2^m}) + \log_2 2^m$$

$$T(2^m) = 2T\left(2^{\frac{m}{2}}\right) + m \quad (\text{nb: } \log_2 2^m = m \log_2 2 = m)$$

Poniamo $S(m) = T(2^m)$:

$$S(m) = 2S\left(\frac{m}{2}\right) + m$$

Che ha soluzione:

$$S(m) = O(m \log m)$$

Se andiamo a sostituire avremo: $T(n) = O(\log n \log \log n)$

Vademecum

1. ipotizzare una soluzione; (es. $T(n) = cn$)
2. effettuare la sostituzione;
3. dimostrare che la ricorsione sia Θ dell'ipotesi, (es. $\Theta(n)$)
cioè vuol dire dimostrarlo sia per O che per Ω .
Per calcolare O dovremo porre $T(n) \leq \dots$
Per calcolare Ω dovremo porre $T(n) \geq \dots$
4. la dimostrazione dell'ipotesi sarà andata a buon fine se, dopo i vari calcoli, ci ritroviamo nella situazione di partenza.

nb: in questo genere di calcolo spesso si gioca sul fatto che se, ad esempio, $T(n) \leq a - b$, allora $T(n) \leq a$. Questo perché se $T(n)$ è più piccolo di a a cui abbiamo sottratto qualcosa, allora ovviamente sarà anche più piccolo di a e basta.

allo stesso modo se, ad esempio, $T(n) \geq a + b$, allora $T(n) \geq a$. Questo perché se $T(n)$ è più grande di a a cui abbiamo aggiunto qualcosa, allora ovviamente sarà anche più grande di a e basta.

In entrambi i casi, **b deve essere positivo**.

Ovviamente in modo equivalente potremmo anche dire che " $T(n) \leq a + b$, allora $T(n) \leq a$ " nel caso b fosse *negativo*. Si tratta solo di "giocare" con i segni.

METODO DELL'ALBERO DI RICORSIONE

Nel metodo dell'**albero di ricorsione** si converte la ricorrenza in un albero i cui nodi rappresentano i costi ai vari livelli della ricorsione;

Disegnare un albero di ricorsione risulta essere una tecnica semplice per formulare una **buona ipotesi** per la soluzione, che dovrà poi essere verificata con il **metodo della sostituzione** (infatti formulare una buona ipotesi può essere complesso).

In un albero di ricorsione, ogni **nodo** rappresenta il costo di un singolo sottoproblema in base alla specifica chiamata ricorsiva. Attraverso la somma di tutti i costi per livello si può determinare il costo totale di tutti i livelli della ricorsione.

In sostanza si deve **rispondere a 5 domande**:

1. quanti nodi ci sono al livello i ?
2. qual è il costo di un nodo al livello i ?
3. qual è il costo complessivo dei nodi al livello i ?
4. qual è il numero dei livelli (k)?
5. Qual è il costo complessivo di $T(n)$?

Esempio 1

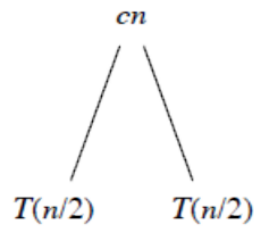
$$T(n) = \begin{cases} c & \text{se } n = 1 \\ 2T\left(\frac{n}{2}\right) + cn & \text{se } n > 1 \end{cases} \quad (\text{ricorrenza mergesort})$$

L'equazione di ricorrenza ci dice che il pezzo sopra rappresenta il caso base mentre il pezzo sotto il caso ricorsivo.

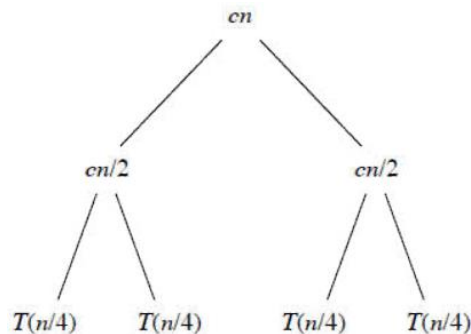
$T(n)$ è dato dal costo di due *chiamate* ricorsive che hanno una dimensione pari alla metà della dimensione originale di $T(n)$ (cioè $2T(n/2)$) + il costo all'interno della funzione ricorsiva (cioè n).

Dobbiamo **tradurre ciò nell'albero ricorsivo**:

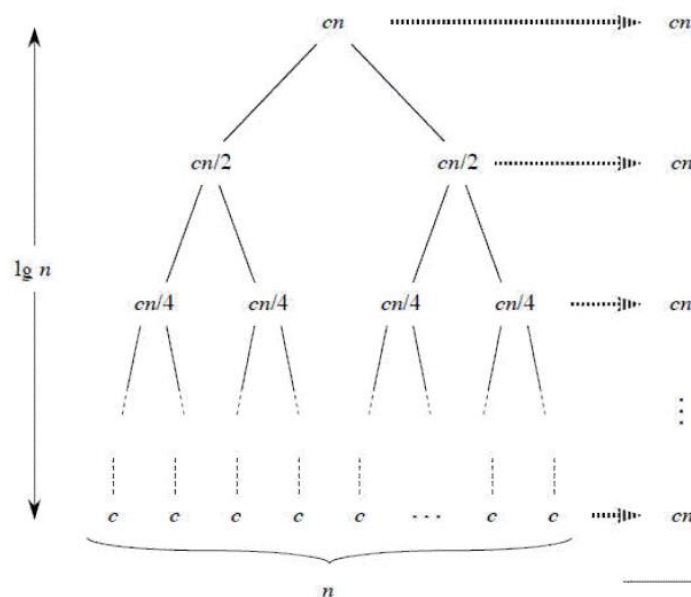
Al primo passo, $T(n)$ ha un costo n più 2 chiamate ricorsive che hanno una dimensione pari alla metà della dimensione originale di $T(n)$, cioè $2T(n/2)$. Ovvero avremo:



Al passo successivo, andiamo ad esplicitare a sua volta $T(n/2)$: $T(n/2)$ avrà un costo $n/2$ (data dal costo della funzione ricorsiva) + 2 sottochiamate la quale dimensione sarà la metà della precedente, dunque $T(n/4)$, ma ovviamente questo passo deve essere applicato ad entrambi i rami, per questo ci ritroviamo con $4T(n/4)$.



Se continuiamo ad andare avanti, finiremo per creare un **albero** (l'albero di ricorsione) che terminerà quando la dimensione del sottoproblema sarà 1 ($n=1$). Infatti in questo caso avremo raggiunto il caso base, e $T(n)$ avrà un costo costante ($T(n) = c$), dunque ogni nodo avrà un costo c .



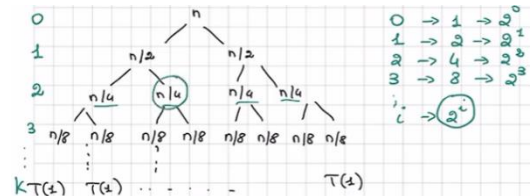
Rispondiamo alle 5 domande:

1. Quanti nodi ci sono al livello i ?

Si considerano i livelli partendo **dal livello 0 fino al livello k** .

Al livello 0 notiamo di avere 1 solo nodo (2^0), al livello 1 abbiamo 2 nodi (2^1), al livello 2 abbiamo 4 nodi (2^2), e così via...

Da qui capiremo che al livello generico i avremo 2^i nodi.



2. Qual è il costo di un nodo al livello i ?

Al livello generico i , un nodo avrà un costo di $n/2^i$.

(ad esempio, al livello 2 un nodo avrà costo $n/2^2$, ovvero $n/4$).

3. Qual è il costo complessivo dei nodi al livello i ?

Dobbiamo effettuare il prodotto fra: il numero di nodi presenti a quel livello i * il costo di un singolo nodo al livello i .

In questo caso: $2^i \cdot n/2^i = n$.

4. Qual è il numero dei livelli (k)?

Raggiungiamo il livello massimo quando ogni singolo nodo ha un costo costante 1, dunque dobbiamo arrivare ad un punto tale che il "costo di un nodo al livello k " sia uguale a 1, ovvero: $n/2^k = 1$.

Questo succede quando $2^k = n$, ossia quando $k = \log_2 n$.

Dunque l'albero ha $\log_2 n$ livelli.

5. Qual è il costo complessivo di $T(n)$?

Il costo complessivo sarà dato da: il numero di livelli k (+1 perché siamo partiti da 0) * il costo di ogni livello.

In questo caso: $(k+1) \cdot n = (\log_2 n + 1) \cdot n = n \log_2 n + n$.

Se ignoriamo il termine di ordine inferiore, abbiamo che la complessità di questa ricorrenza (e di conseguenza del Mergesort) sia $\Theta(n \log_2 n)$.

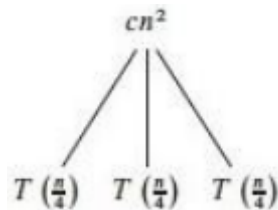
nb: tale risultato vale sia nel caso peggiore, che migliore, che in quello medio.

Esempio 2

$$T(n) = 3T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) + \Theta(n^2)$$

Dobbiamo **tradurre ciò nell'albero ricorsivo**:

Al primo passo, $T(n)$ ha un costo n^2 più 3 chiamate ricorsive che hanno una dimensione pari a un quarto della dimensione originale di $T(n)$, cioè $3T(n/4)$. Ovvero avremo:



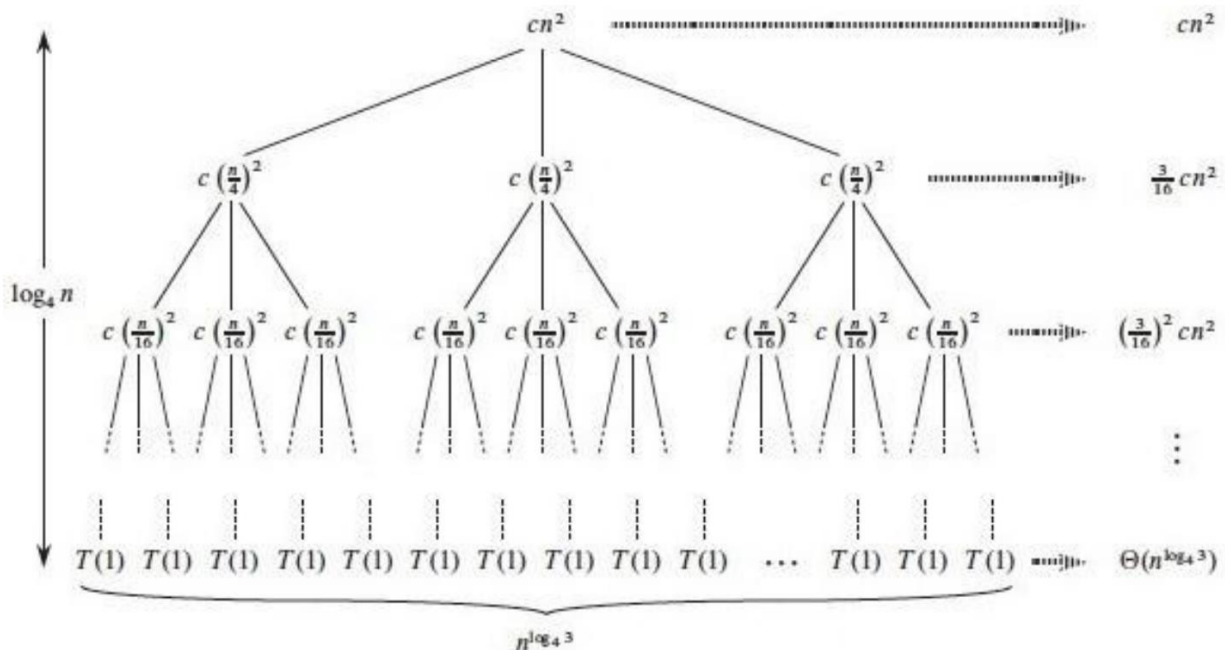
Nei passi successivi, andiamo ad esplicitare a sua volta $T(n/4)$ e quelli che seguono, così da andare a costruire man mano un albero. Dovremo continuare finché la dimensione del sottoproblema è 1:

$$T(n/4) = 3T(n/16) + (n/4)^2$$

$$T(n/16) = 3T(n/64) + (n/16)^2$$

$$T(n/64) = 3T(n/256) + (n/64)^2$$

...



Osservando come varia $T(n)$, possiamo convincerci come

Rispondiamo alle 5 domande:

1. Quanti nodi ci sono al livello i ?

Ogni chiamata ricorsiva genererà 3 nodi, dunque al livello generico i avremo: 3^i nodi.

2. Qual è il costo di un nodo al livello i ?

Al livello generico i , un nodo avrà un costo di $(n/4^i)^2$.

(ad esempio, al livello 2 un nodo avrà costo $(n/4^2)^2$, ovvero $n^2/256$).

Infatti osserviamo come varia $T(n)$:

$$T(n/4) = 3T(n/16) + (n/4)^2$$

$$T(n/16) = 3T(n/64) + (n/16)^2$$

$$T(n/64) = 3T(n/256) + (n/64)^2$$

...

3. Qual è il costo complessivo dei nodi al livello i ?

Possiamo effettuare il prodotto fra: il numero di nodi presenti a quel livello i * il costo di un singolo nodo al livello i , tuttavia il numero che esce fuori è complicato.

Invece conviene osservare come cresce il costo ad ogni livello:

- 0: n^2
- 1: $3/16 \cdot cn^2$
- 2: $9/16^2 \cdot cn^2$
- 3: $27/16^3 \cdot cn^2$
- ...

Da qui possiamo osservare come il costo complessivo dei nodi ad un generico livello i sia:

$$(3/16)^i \cdot cn^2$$

4. Qual è il numero dei livelli (k)?

Raggiungiamo il livello massimo quando ogni singolo nodo ha un costo costante 1, dunque dobbiamo arrivare ad un punto tale che il “costo di un nodo al livello k ” sia uguale a 1, ovvero: $(n/4^k)^2 = 1$.

Questo succede quando $4^{2k} = n^2$, ossia quando $k = \log_4 n$.

Dunque l'albero ha $\log_4 n$ livelli.

5. Qual è il costo complessivo di T(n)?

Invece di considerare il ragionamento fatto prima, dove escono numeri complicati, cerchiamo di ricostruire il costo complessivo di T(n) da una sommatoria.

Infatti il costo dell'intero albero sarà dato dalla **somma del costo di tutti i livelli**.

Ogni livello (a eccezione dell'ultimo) ha un costo pari a:

$$\left(\frac{3}{16}\right)^i cn^2$$

L'**ultimo livello**, che ha profondità $\log_4 n$, ha $3^{\log_4 n} = n^{\log_4 3}$ nodi con costo $\Theta(1)$.

Dunque avremo che:

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \theta(n^{\log_4 3})$$

Notiamo come la sommatoria sopra ricordi la **serie geometrica**:

$$\sum_{k=0}^N x^k = \frac{x^{N+1} - 1}{x - 1}$$

Poniamo $x = 3/16$ e $N = \log_4 n + 1$:

$$T(n) = \frac{\left(\frac{3}{16}\right)^{\log_4 n + 1} - 1}{\left(\frac{3}{16}\right) - 1} cn^2 + \theta(n^{\log_4 3})$$

Notiamo tuttavia essere troppo complessa per ottenere un'ipotesi per il nostro metodo di sostituzione. Per questo motivo non consideriamo N finito, ma *infinito*.

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \theta(n^{\log_4 3}) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \theta(n^{\log_4 3}) \end{aligned}$$

Dunque useremo la **serie geometrica infinita**, che ci fornirà un'approssimazione.

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

Considerato:

$$T(n) < \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \theta(n^{\log_4 3})$$

se poniamo $x = (3/16)$, avremo:

$$T(n) < \frac{1}{1 - \frac{3}{16}} cn^2 + \theta(n^{\log_4 3}) = \frac{16}{13} cn^2 + \theta(n^{\log_4 3})$$

Ma $\frac{16}{13}$ è una costante, dunque possiamo ometterla.

Dunque avremo che:

$$\mathbf{T(n) = O(n^2)}$$

Albero bilanciato e non bilanciato

Albero bilanciato

Un **albero** è **bilanciato** se il costo di ogni livello è lo stesso.

Considerata una ricorrenza:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

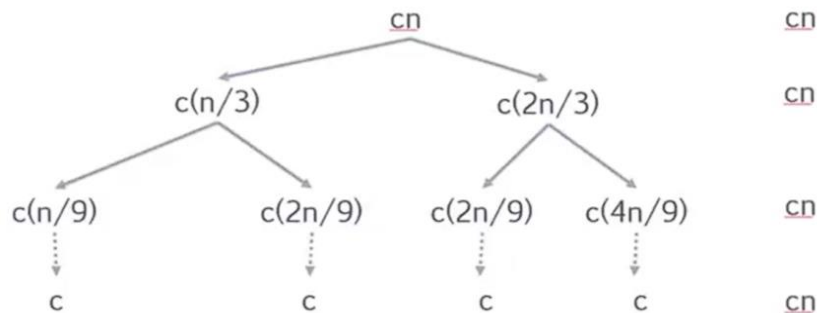
se $a = b$, si ipotizza una complessità: $\theta(f(n) \log n)$.

Esempi:

- 1) la ricorrenza del **mergesort** ($T(n) = 2T\left(\frac{n}{2}\right) + cn$) è bilanciata, in quanto possiamo subito vedere che il numero delle chiamate ricorsive è pari al numero dei sottoproblemi generati (2), e inoltre $f(n) = n$.
Dunque la complessità sarà $\theta(n \log n)$.

- 2) Consideriamo la ricorrenza: $T(n) = 2T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + cn$.

In questo caso per verificare che sia bilanciata dobbiamo sviluppare l'albero di ricorsione:



Notiamo che il costo di tutti i livelli è lo stesso, dunque si tratta di un albero bilanciato, ed essendo che $f(n) = n$, allora la complessità sarà $\theta(n \log n)$.

Infatti possiamo notare come la complessità totale sia " $cn \log n + n$ " (in quanto l'altezza è " $\log n + 1$ " (+1 perché consideriamo anche la radice), e questa viene moltiplicata n volte).

Però notiamo anche come l'albero non sia completo (alcuni rami raggiungono prima il nodo foglia), dunque più si scende di livello e minore sarà il contributo apportato a quel livello. Tuttavia accettiamo l'approssimazione che tutti i livelli diano un contributo pressoché uguale.

Ovviamente, fatta l'ipotesi, questa va sempre dimostrata.

Albero non bilanciato

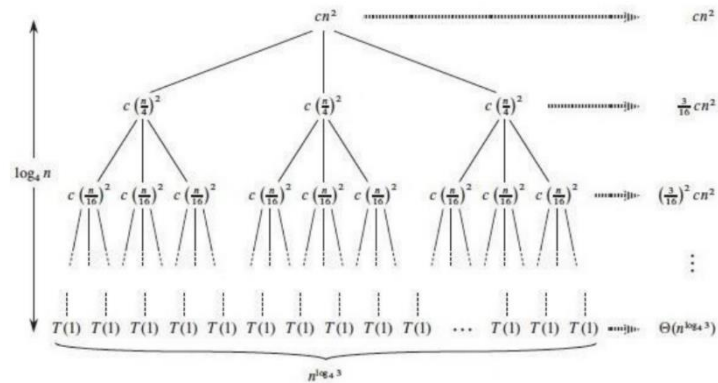
Un **albero** è **non bilanciato** se il costo di ogni livello NON è lo stesso.

Considerata una ricorrenza:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

se $a < b$, si ipotizza una complessità: $\theta(f(n))$.

Es. $T(n) = 3T\left(\frac{n}{4}\right) + \theta(n^2)$



trucco per alberi sbilanciati usando il metodo della sostituzione

$$T(n) = 3T\left(\frac{n}{4}\right) + \theta(n^2) \quad \text{è } O(n^2)?$$

sostituisco:

$$T(n) \leq 3d\left(\frac{n^2}{16}\right) + cn^2$$

$$T(n) \leq \frac{3dn^2}{16} + cn^2$$

Il nostro obiettivo è ovviamente trovarci che $T(n) \leq dn^2$.

Brutalmente quello che possiamo fare è aggiungere quanto ci manca per arrivare a dn^2 , e sottrarre la stessa quantità.

Ad esempio, per raggiungere dn^2 , dobbiamo aggiungere a $\frac{3dn^2}{16}$ la quantità $\frac{13dn^2}{16}$, in quanto $\frac{3}{16}dn^2 + \frac{13}{16}dn^2 = \frac{16}{16}dn^2 = dn^2$.

Ricordando che dobbiamo anche sottrarre la stessa quantità, avremo:

$$T(n) \leq dn^2 - \frac{13}{16}dn^2 + cn^2$$

$$T(n) \leq dn^2 - \left(\frac{13}{16}dn^2 - cn^2\right)$$

$$T(n) \leq dn^2 \quad \text{per } d \geq c\left(\frac{16}{13}\right)$$

METODO DELL'ESPERTO

Il metodo dell'esperto (aka Teorema principale, o Master Theorem) permette di risolvere la maggior parte delle ricorrenze generate dal divide et impera, che si presentano sotto una forma del tipo:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

dove $T(n)$ è una funzione definita sugli interi non negativi,
e dove: **$a \geq 1$** , **$b > 1$** e **$f(n)$** è una funzione data.

a rappresenta il numero di chiamate ricorsive da effettuare, e ogni sottoproblema avrà dimensioni $\frac{1}{b}$.

Infatti in tali casi, è spesso possibile ottenere una soluzione confrontando f con $n^{\log_b a}$. Possiamo distinguere **3 casi**.

nb:

b è "strettamente" maggiore di 1 perché queste non sono equazioni matematiche in sé per sé, ma equazioni matematiche frutto della risoluzione di un algoritmo.

Infatti, non avrebbe senso pensare che una ricorrenza, che viene dal divide et impera, generi un sottoproblema che la stessa grandezza del problema principale.

In realtà "**b**" potrebbe anche non essere un intero (quindi un valore frazionario). Otteniamo "**b**" facendo il reciproco dell'argomento (es. $2n/3$, $b = 3/2$).

Ricettario MT

1° Caso:

Se $f(n) = O(n^{\log_b a - \epsilon})$, per qualche costante $\epsilon > 0$

Allora: $T(n) = \theta(n^{\log_b a})$

2° Caso:

Se $f(n) = \theta(n^{\log_b a})$, (non c'è più ϵ)

Allora: $T(n) = \theta(n^{\log_b a} \cdot \log n)$

3° Caso:

Se $f(n) = \Omega(n^{\log_b a + \epsilon})$, per qualche costante $\epsilon > 0$

e $af\left(\frac{n}{b}\right) \leq cf(n)$ *, per qualche costante $c < 1$ e per ogni n sufficientemente grande.

Allora: $T(n) = \theta(f(n))$ *(condizione di regolarità).

• Significato del MT

In ciascuno dei 3 casi **confrontiamo** la funzione $f(n)$ con la funzione $n^{\log_b a}$.

- se, come nel caso 1, la funzione $n^{\log_b a}$ è la più grande, allora la soluzione è:
 $T(n) = \theta(n^{\log_b a})$.
- se, come nel caso 2, le due funzioni hanno la stessa dimensione, moltiplichiamo per un fattore logaritmico e la soluzione è:
 $T(n) = \theta(n^{\log_b a} \cdot \log n)$.
- se, come nel caso 3, la funzione $f(n)$ è più grande, allora la soluzione è:
 $T(n) = \theta(f(n))$.

Oltre ciò, ci sono alcuni **dettagli** a cui fare attenzione:

- nel primo caso, $f(n)$ non soltanto deve essere più piccola di $n^{\log_b a}$, ma deve essere polinomialmente più piccola, ossia più piccola di n^ϵ per qualche $\epsilon > 0$;
- nel terzo caso, $f(n)$ non soltanto deve essere più grande di $n^{\log_b a}$, ma deve essere polinomialmente più grande, ossia più grande di n^ϵ per qualche $\epsilon > 0$. Inoltre, deve soddisfare la condizione di regolarità (soddisfatta quasi sempre).

Estensione del Master Theorem

Tale estensione è considerabile come un 4° caso. La possiamo usare, ad esempio, quando ci ritroviamo nel 3° caso ma ϵ non è maggiore di 0.

Se $f(n) = \theta(n^{\log_b a} \cdot (\log n)^k)$, con $k > 0$ (ove k è arbitrario).

Allora: $T(n) = \theta(n^{\log_b a} \cdot (\log n)^{k+1})$

Situazioni in cui NON può essere applicato il MT

Il MT non copre tutte le possibili funzioni $f(n)$:

- c'è un intervallo fra il caso 1 e 2 in cui $f(n)$ è più piccola di $n^{\log_b a}$ ma non in modo polinomiale.
- analogamente, c'è un intervallo fra il caso 2 e 3 in cui $f(n)$ è più grande di $n^{\log_b a}$ ma non in modo polinomiale, oppure la condizione di regolarità nel caso 3 non è soddisfatta.

Vademecum

- individuare a e b , e di conseguenza individuare $\log_b a$.
nb: "b" non è altro che il reciproco dell'argomento (es. $T(2^{n/3}) \rightarrow b = 3/2$).
- Individuare $f(n)$ e $n^{\log_b a}$.
- Vedere chi è più grande fra queste due funzioni (se hanno la stessa dimensione, siamo nel 2° caso e abbiamo finito):
 - **1° caso:** se $n^{\log_b a} > f(n)$, dobbiamo verificare che $\epsilon > 0$.
Confermato ciò, applichiamo la soluzione.
 - **3° caso:** se $f(n) > n^{\log_b a}$, dobbiamo sia verificare che $\epsilon > 0$ e che la condizione di regolarità sia rispettata: fondamentalmente ci dobbiamo trovare due funzioni con lo stesso comportamento da entrambe le parti e da lì individuiamo c (che deve essere < 1).
 - **estensione:** se riusciamo a trovare una qualsiasi $k > 0$ per cui $f(n) = n^{\log_b a} \cdot (\log n)^k$, allora possiamo applicare direttamente la soluzione.

nb: possiamo calcolare ϵ come la differenza in valore assoluto fra l'esponente più grande di una funzione meno l'esponente più grande dell'altra funzione.

In $n \log n$, consideriamo l'esponente di n .

es. caso 3: $\frac{n}{2} \log n \leq cn \log n \rightarrow c \geq \frac{1}{2}$ ($c < 1$) verificata.

Ricorrenze non risolvibili con il MT

- $T(n) = 2^n T\left(\frac{n}{2}\right) + n^n$, “a” non è una costante.
- $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$, differenza non polinomiale fra $f(n)$ e $n^{\log_b a}$.
- $T(n) = \frac{1}{2}T\left(\frac{n}{2}\right) + n$, $a < 1$, infatti non ha senso avere “mezzo” sottoproblema (vale per i numeri frazionari in generali).
- $T(n) = 64T\left(\frac{n}{8}\right) - n^2 \log n$, $f(n)$ non è positiva, cosa che non ha senso in quanto $f(n)$ deve rappresentare un costo (divide + combina).
- $T(n) = T\left(\frac{n}{2}\right) + n(2 - \cos n)$, violazione della regolarità nel caso 3 (è molto comune quando compaiono funzioni trigonometriche).

nb: la 3 e la 5 non hanno alcun senso informatico.

Esempi

1. $T(n) = 4T\left(\frac{n}{2}\right) + n$

- $a = 4, b = 2 \rightarrow \log_b a = \log_2 4 = 2$

- $f(n) = n$, $n^{\log_b a} = n^2$

- $f(n) = O(n^{\log_b a})$ // $f(n)$ è più piccolo

o dunque **caso 1** (controlliamo ϵ):

$$\epsilon = |1 - 2| = 1, \quad \text{quindi } \epsilon > 0$$

confermato ciò, significa che: $T(n) = \theta(n^{\log_b a})$.

quindi la soluzione è: $T(n) = \theta(n^2)$.

2. $T(n) = 2T\left(\frac{n}{2}\right) + n$ (mergesort)

- $a = 2, b = 2 \rightarrow \log_b a = \log_2 2 = 1$

- $f(n) = n, n^{\log_b a} = n$

- $f(n) = \theta(n^{\log_b a})$ // $f(n)$ e $n^{\log_b a}$ hanno la stessa dim.

o dunque **caso 2** (siamo fortunati, non dobbiamo far nient'altro):

significa che: $T(n) = \theta(n^{\log_b a} \cdot \log n)$.

quindi la soluzione è: $T(n) = \theta(n \log n)$.

3. $T(n) = 2T\left(\frac{n}{4}\right) + n \log n$

- $a = 2, b = 4 \rightarrow \log_b a = \log_4 2 = \frac{1}{2}$

- $f(n) = n \log n, n^{\log_b a} = n^{\frac{1}{2}} = \sqrt{n}$

- $f(n) = \Omega(n^{\log_b a})$ // $f(n)$ è più grande

o dunque **caso 3** (controlliamo ϵ e la condizione di regolarità):

$\epsilon = \left|1 - \frac{1}{2}\right| = \frac{1}{2},$ quindi $\epsilon > 0$

confermato ciò, controlliamo la condizione di regolarità:

$af\left(\frac{n}{b}\right) \leq cf(n) ?$

$2\left(\frac{n}{4} \log\left(\frac{n}{4}\right)\right) \leq cn \log n$

dove $2\left(\frac{n}{4} \log\left(\frac{n}{4}\right)\right) = 2\left(\frac{n}{4} \log n - \frac{n}{4} \log 4\right) = \frac{n}{2} (\log n - 2) \leq \frac{n}{2} \log n.$

(leviamo il -2, perché comunque rimane $\leq$)

$\left(\frac{1}{2}\right) n \log n \leq cn \log n$

$cn \log n \geq \left(\frac{1}{2}\right) n \log n$ (ora mi basta dividere tutto per $n \log n$)

$c \geq \frac{1}{2}$, la condizione di regolarità è rispettata in quanto $c < 1$.

significa che: $T(n) = \theta(f(n))$. (nb: base log = 2)

quindi la soluzione è: $T(n) = \theta(n \log n)$.

$$4. T(n) = 3T\left(\frac{n}{3}\right) + n \log n$$

$$- a = 3, b = 3 \rightarrow \log_b a = \log_3 3 = 1$$

$$- f(n) = n \log n, \quad n^{\log_b a} = n$$

$$- f(n) = \Omega(n^{\log_b a}) \quad // f(n) \text{ è più grande}$$

○ dunque caso 3 (controlliamo ϵ e la condizione di regolarità):

$$\epsilon = |1 - 1| = 0, \quad \text{quindi } \epsilon \text{ NON È } > 0$$

Tuttavia, quando ci ritroviamo nel 3° caso ma ϵ non è maggiore di 0, è possibile utilizzare l'**estensione del Master Theorem**. Dobbiamo riuscire a trovare una qualsiasi $k > 0$ per cui $f(n) = n^{\log_b a} \cdot (\log n)^k$.

Proviamo a porre: $k = 1$.

$$n^{\log_b a} \cdot (\log n)^k$$

$$n^1 \cdot (\log n)^1$$

$$n \log n$$

//che è proprio $f(n)$! Dunque...

Notiamo come per $k = 1$, $f(n) = n^{\log_b a} \cdot (\log n)^k$.

Ciò significa che: $T(n) = \theta(n^{\log_b a} \cdot (\log n)^{k+1})$

quindi la soluzione è: $T(n) = \theta(n (\log n)^2)$.

METODO ITERATIVO

Nel metodo **iterativo** si itera la ricorrenza (senza costruire l'albero di ricorrenza) fino a calcolare la soluzione.

In pratica si "srotola" la ricorsione, ottenendo una sommatoria dipendente dalla sola dimensione n .

Esempio:

$$T(n) = \begin{cases} n + T(n/2) & n > 1 \\ 1 & \text{se } n = 1 \end{cases}$$

Il nostro obiettivo è srotolarla fino a raggiungere il caso base.

$$T(n) = n + T\left(\frac{n}{2}\right)$$

$$T(n) = n + \frac{n}{2} + T\left(\frac{n}{4}\right)$$

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + T\left(\frac{n}{8}\right)$$

...

In pratica avremo: $\frac{n}{2^0} + \frac{n}{2^1} + \frac{n}{2^2} + \frac{n}{2^3} + \dots + \frac{n}{2^k}$.

Che possiamo tradurre nella sommatoria:

$$\sum_{i=0}^{k-1} \frac{n}{2^i} + T\left(\frac{n}{2^k}\right)$$

Ci fermiamo quando $n/2^k = 1$.

Questo succede quando $2^k = n$, ossia quando $k = \log_2 n$.

Riscriviamo la sommatoria:

$$\sum_{i=0}^{\log_2 n - 1} \frac{n}{2^i} + T(1)$$

Somiglia alla serie geometrica ($\sum_{k=0}^n x^k = \frac{1-x^{n+1}}{1-x}$). Portiamo n fuori e avremo:

$$\begin{aligned} n \sum_{i=0}^{\log_2 n - 1} \frac{1}{2^i} + T(1) &= \frac{n \left(1 - \left(\frac{1}{2}\right)^{\log_2 n}\right)}{1 - \frac{1}{2}} + 1 = \frac{n \left(1 - \left(\frac{1}{2}\right)^{\log_2 n}\right)}{\frac{1}{2}} + 1 = 2n \left(1 - \left(\frac{1}{2}\right)^{\log_2 n}\right) = \\ &= 2n \left(1 - \frac{1}{2^{\log_2 n}}\right) + 1 = 2n \left(1 - \frac{1}{n}\right) + 1 = 2n \left(\frac{n-1}{n}\right) + 1 = 2n - 2 + 1 = 2n - 1 = \mathbf{O(n)} \end{aligned}$$

LEZ 7 – QuickSort

Il **QuickSort** è l'algoritmo di ordinamento con generalmente le prestazioni migliori fra quelli basati sul confronto. Si tratta di un algoritmo in place, facilmente implementabile, ed è general purpose, ovvero che ha un buon comportamento in un'ampia varietà di situazioni. Fu inventato da Hoare nel 1960.

Non è stabile.

Nonostante il tempo di esecuzione nel caso peggiore sia $\Theta(n^2)$ con un array di input di n numeri, il quicksort è comunque spesso la soluzione pratica migliore per effettuare un ordinamento perché mediamente è molto efficiente, con un tempo di esecuzione atteso di $\Theta(n \log n)$.

Il **Quick Sort** è un algoritmo di ordinamento che esiste in varie versioni: l'idea dell'algoritmo si basa sulla scelta di un elemento detto pivot, o anche elemento partizionatore.

Dopo aver scelto il pivot, si scorre il vettore disordinato in modo da spostare da una parte tutte le chiavi minori del partizionatore e dall'altra parte tutte quelle che sono maggiori. Una volta fatta questa separazione, l'elemento partizionatore deve stare in mezzo.

Il partizionatore si chiama così perché poi partiziona il vettore in 2 sottovettori disordinati caratterizzati dal fatto che la porzione di sinistra contiene le chiavi minori e la porzione di destra le chiavi maggiori (del pivot). Il Divide et Impera sta nel fatto che il problema iniziale è stato scisso in 2 sottoproblemi di ordinamento.

(Se non si presentano due sottoporzioni bilanciate vuol dire che non è stato scelto nel modo giusto il partizionatore).

Essendo un algoritmo di tipo **Divide et impera**, possiamo dividere il funzionamento del QuickSort in 3 parti. Consideriamo un array $A[p \dots r]$, ove p e r rappresentano l'indice del primo e dell'ultimo elemento, ed indichiamo con q l'indice dell'elemento partizionatore:

1. **Divide**: partizionare l'array $A[p \dots r]$ in due sottoarray $A[p \dots q-1]$ e $A[q+1 \dots r]$ (eventualmente vuoti) tali che ogni elemento di $A[p \dots q-1]$ sia minore o uguale ad $A[q]$ che, a sua volta, è minore o uguale a ogni elemento di $A[q+1 \dots r]$. Calcolare l'indice q come parte di questa procedura di partizionamento.
2. **Impera**: ordinare i due sottoarray $A[p \dots q-1]$ e $A[q+1 \dots r]$ chiamando ricorsivamente quicksort.
3. **Combina**: poiché i sottoarray sono ordinati sul posto, non occorre alcun lavoro per combinarli: l'intero array $A[p \dots r]$ è ordinato.

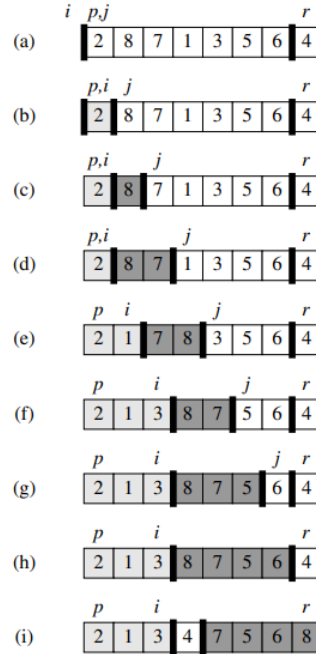
(nb: per eseguire il quicksort, p deve essere $< r$, altrimenti significa che l'array è costituito da un solo elemento, dunque è già ordinato, e dunque ci troviamo nel caso base).

```

QUICKSORT( $A, p, r$ )
1  if  $p < r$ 
2    then  $q \leftarrow \text{PARTITION}(A, p, r)$ 
3         QUICKSORT( $A, p, q - 1$ )
4         QUICKSORT( $A, q + 1, r$ )

```

Figura 7.1 L'operazione di PARTITION su un array campione. Gli elementi dell'array su sfondo grigio chiaro sono tutti nella prima partizione con valori non maggiori di x . Gli elementi su sfondo grigio scuro sono nella seconda partizione con valori maggiori di x . Gli elementi su sfondo bianco non sono stati ancora posti in una delle prime due partizioni; l'ultimo elemento su sfondo bianco è il *pivot* o *perno*. (a) L'array iniziale e le impostazioni delle variabili. Nessuno degli elementi è stato posto in una delle prime due partizioni. (b) Il valore 2 viene "scambiato con sé stesso" e posto nella partizione dei valori più piccoli. (c)–(d) I valori 8 e 7 vengono inseriti nella partizione dei valori più grandi. (e) I valori 1 e 8 vengono scambiati e la prima partizione cresce. (f) I valori 3 e 7 vengono scambiati e la prima partizione cresce. (g)–(h) La seconda partizione acquisisce i valori 5 e 6 e il ciclo termina. (i) Le righe 7–8 scambiano il pivot inserendolo fra le due partizioni.



Per ordinare un intero array A , la chiamata iniziale è
QUICKSORT($A, 1, \text{lunghezza}[A]$)

Partizionare l'array

L'elemento chiave dell'algoritmo è la procedura PARTITION, che riarrangia il sottoarray $A[p..r]$ sul posto.

```

PARTITION( $A, p, r$ )
1   $x \leftarrow A[r]$ 
2   $i \leftarrow p - 1$ 
3  for  $j \leftarrow p$  to  $r - 1$ 
4    do if  $A[j] \leq x$ 
5       then  $i \leftarrow i + 1$ 
6       scambia  $A[i] \leftrightarrow A[j]$ 
7  scambia  $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 

```

La Figura 7.1 illustra l'operazione di PARTITION su un array di 8 elementi. PARTITION seleziona sempre un elemento $x = A[r]$ come *pivot* intorno al quale partizionare il sottoarray $A[p..r]$. Durante l'esecuzione della procedura, l'array viene suddiviso in quattro regioni (eventualmente vuote). All'inizio di ogni iterazione del ciclo **for**, righe 3–6, ogni regione soddisfa alcune proprietà, che possiamo definire come invariante di ciclo:

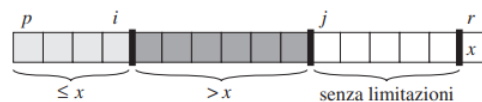


Figura 7.2 Le quattro regioni mantenute dalla procedura PARTITION in un sottoarray $A[p..r]$. I valori in $A[p..i]$ sono tutti minori o uguali a x , i valori in $A[i+1..j-1]$ sono tutti maggiori di x e $A[r] = x$. Gli elementi di $A[j..r-1]$ possono assumere qualsiasi valore.

Invariante di ciclo

All'inizio di ogni iterazione del ciclo, righe 3-6, per qualsiasi indice k dell'array:

1. Se $p \leq k \leq i$, allora $A[k] \leq x$.
2. Se $i + 1 \leq k \leq j - 1$, allora $A[k] > x$.
3. Se $k = r$, allora $A[k] = x$.

(in pratica, se dato un elemento che si trova in una generica posizione k , se l'indice k di questo elemento si trova fra gli indici p ed i , allora significa che l'elemento si trova nella porzione dell'array minore del pivot. Viceversa, se si trova fra gli indici $i+1$ e $j-1$ significa che si trova nella porzione dell'array maggiore del pivot.

Se $k=r$, significa che quell'elemento È il pivot).

Inizializzazione: prima della prima iterazione del ciclo, $i = p - 1$ e $j = p$. Non ci sono valori fra p e i né fra $i + 1$ e $j - 1$, quindi le prime due condizioni dell'invariante di ciclo sono soddisfatte. L'assegnazione nella riga 1 soddisfa la terza condizione.

Conservazione: ci sono due casi da considerare:

- Se $A[j] > x$, l'unica azione del ciclo sarà quella di incrementare j . Dopo l'incremento di j , la condizione 2 è soddisfatta per $A[j-1]$ e tutte le altre voci non cambiano.
- Se $A[j] \leq x$, viene incrementato l'indice i , vengono scambiati $A[i]$ e $A[j]$ e, poi, viene incrementato l'indice j . In seguito allo scambio, adesso abbiamo $A[i] \leq x$ e la condizione 1 è soddisfatta. Inoltre, abbiamo anche $A[j-1] > x$, in quanto l'elemento spostato è più grande di x .

Conclusione: Alla fine del ciclo, $j = r$. Pertanto, ogni posizione dell'array si trova in uno dei tre insieme descritti dall'invariante e noi abbiamo ripartito i valori dell'array in tre insiemi: quelli minori o uguali di x , quelli maggiori di x e un insieme di un solo elemento che contiene x .

L'output di PARTITION adesso soddisfa le specifiche del passo divide. Il tempo di esecuzione di PARTITION con il sottoarray $A[p..r]$ è $\Theta(n)$, dove $n = r - p + 1$.

Complessità del Quicksort

Il tempo di esecuzione di quicksort dipende dal fatto che il partizionamento sia bilanciato o sbilanciato.

Nel **caso peggiore**, i dati sono ordinati in modo **decrescente o crescente**. In tal caso, Partition produrrà sempre un vettore costituito da $n-1$ elementi e uno da nessun elemento.

Ergo, il comportamento nel caso peggiore di quicksort si verifica quando la routine di partizionamento produce un sottoproblema con $n-1$ elementi e uno con 0 elementi.

Poiché per una chiamata ricorsiva su un array vuoto $T(0) = \Theta(1)$, la ricorrenza per il tempo di esecuzione può essere espressa come:

$$\begin{aligned}T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n)\end{aligned}$$

Possiamo individuare la soluzione di tale ricorrenza attraverso il metodo di sostituzione.

Facciamo prima un'**ipotesi** sfruttando il **metodo iterativo**:

$$T(n) = T(n-1) + \theta(n)$$

$$T(n) = T(n-1) + n$$

Individuiamo $T(n-1)$...

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n) = [T(n-2) + (n-1)] + n$$

Individuiamo $T(n-2)$...

$$T(n-2) = T(n-3) + (n-2)$$

$$T(n) = [T(n-3) + (n-2)] + (n-1) + n$$

...continuando così, al passo k -simo avremo:

$$T(n-k) = T(n-k) + (n - (k-1)) , \text{ dunque:}$$

$$T(n) = T(n-k) + (n - (k-1)) + (n - (k-2)) + \dots + (n-1) + n$$

Supponiamo di aver raggiunto il caso base $n=0$, dunque $n-k=0$, cioè $n=k$. Avremo dunque:

$$T(n) = T(n-n) + (n - (n-1)) + (n - (n-2)) + \dots + (n-1) + n$$

$$T(n) = T(0) + (n - n + 1) + (n - n + 2) + \dots + (n-1) + n$$

$$T(n) = T(0) + \underbrace{1 + 2 + \dots + (n-1)}_{\text{Si tratta della somma dei primi } n \text{ numeri!}} + n$$

$$T(n) = T(0) + \sum_{i=1}^n i$$

$T(0)$ è il caso base, dunque $T(0) = 1$.

La somma dei primi n numeri è una sommatoria nota, dunque avremo:

$$T(n) = 1 + \frac{n(n+1)}{2} = \mathbf{O(n^2)}$$

Ora dobbiamo **dimostrare** che $T(n)$ sia $\mathbf{O(n^2)}$ e $\mathbf{\Omega(n^2)}$, dunque dovremo dimostrare che $T(n) \leq dn^2$ e $T(n) \geq en^2$.

nb: usiamo la costante “d” in quanto questa deve essere diversa da quella che usiamo per sostituire Θ (ovvero c). Insomma, serve per far capire che si trattano di due costanti diverse. Lo stesso discorso vale anche per “e”.

Dimostriamo attraverso il metodo della sostituzione che $T(n) = \Theta(n^2)$.

$$T(n) = T(n-1) + cn$$

Dimostriamo prima che è $O(n^2)$, verificando che $T(n) \leq dn^2$. Sostituendo avremo:

$$T(n) \leq d(n-1)^2 + cn$$

$$T(n) \leq dn^2 - 2dn + d + cn$$

$$T(n) \leq dn^2 - (2dn - d - cn)$$

mettiamo il “-” in evidenza. Infatti basta ricordare che se $T(n)$ è $\leq$ di una certa quantità a cui abbiamo sottratto una quantità **positiva**, allora sarà anche $\leq$ di quella quantità e basta.

Purché la disequazione sia vera, la quantità fra parentesi deve essere positiva.

$$2dn - d - cn \geq 0$$

$$d(2n - 1) \geq cn$$

$$d \geq \frac{cn}{2n - 1}$$

Dunque:

$$T(n) \leq \mathbf{dn^2} \quad \forall d \geq \frac{cn}{2n - 1}$$

Per quanto riguarda Ω , i conti sono identici, solo che adesso dovremo considerare “ $T(n) \geq$...”

Dimostriamo attraverso il metodo della sostituzione che $T(n) = \Omega(n^2)$.

$$T(n) = T(n-1) + cn$$

Dimostriamo prima che è $O(n^2)$, verificando che $T(n) \geq dn^2$. Sostituendo avremo:

$$T(n) \geq e(n-1)^2 + cn$$

$$T(n) \geq en^2 - 2en + e + cn$$

$$T(n) \geq en^2 - (2en - e - cn)$$

mettiamo il “-” in evidenza. Infatti basta ricordare che se $T(n)$ è $\geq$ di una certa quantità a cui abbiamo sottratto una quantità **negativa** (dunque in sostanza abbiamo aggiunto qualcosa), allora sarà anche $\geq$ di quella quantità e basta.

Purché la disequazione sia vera, la quantità fra parentesi deve essere negativa.

$$2en - e - cn \leq 0$$

$$e(2n-1) \leq cn$$

$$e \leq \frac{cn}{2n-1}$$

Più precisamente, abbiamo che: $0 \leq e \leq \frac{cn}{2n-1}$

Dunque:

$$T(n) \geq dn^2 \quad \forall 0 < e \leq \frac{cn}{2n-1}$$

Poiché abbiamo verificato sia che $T(n) = O(n^2)$, e sia che $T(n) = \Omega(n^2)$, allora **$T(n) = \Theta(n^2)$** .

Nel **caso migliore**, PARTITION produce due sottoproblemi, ciascuno di dimensione non maggiore di $n/2$, in quanto uno ha dimensione $\lceil n/2 \rceil$ e l'altro ha dimensione $\lceil n/2 \rceil - 1$. In questo caso, il quicksort viene eseguito molto più velocemente. La ricorrenza per il tempo di esecuzione è:

$$T(n) \leq 2T(n/2) + \Theta(n)$$

che può essere risolta con il 2 caso del Teorema dell'esperto, ed ha soluzione $T(n) = O(n \log n)$. Dunque, il perfetto bilanciamento dei due lati della partizione, ad ogni livello di ricorsione, produce un algoritmo asintoticamente più veloce.

Nel caso di un **partizionamento bilanciato**, la complessità sarà sempre di $O(n \log n)$, anche se la ripartizione è di 99 a 1.

La ragione è che qualsiasi ripartizione con proporzionalità costante produce un albero di ricorsione di profondità $\Theta(\log n)$, dove il **costo di ogni livello è $O(n)$** . Dunque, il tempo di esecuzione è $O(n \log n)$ quando la ripartizione ha proporzionalità costante.

Randomized Quicksort

Il **Randomized Quicksort** è fondamentalmente identico al Quicksort, con la differenza che il pivot non sarà più un elemento fisso (nb: ultimo elemento, $A[r]$) ma un elemento casuale ($A[p..r]$).

È preferibile utilizzare il randomized q. rispetto alla versione originale in quanto tale elemento di casualità rende più probabile avere prestazioni migliori nel caso medio. Infatti, essendo il pivot scelto a caso, prevediamo che in media la ripartizione dell'array di input potrà essere ben bilanciato.

RANDOMIZED-PARTITION(A, p, r)

```
1   $i \leftarrow \text{RANDOM}(p, r)$ 
2  scambia  $A[r] \leftrightarrow A[i]$ 
3  return PARTITION( $A, p, r$ )
```

Il nuovo quicksort chiama **RANDOMIZED-PARTITION**, anziché **PARTITION**:

RANDOMIZED-QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2      then  $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$ 
3          RANDOMIZED-QUICKSORT( $A, p, q - 1$ )
4          RANDOMIZED-QUICKSORT( $A, q + 1, r$ )
```

Per quanto riguarda la complessità del randomized q., è analoga a quella del quicksort, tuttavia il caso peggiore **atteso** è di $(n \log n)$, questo perché visto che andiamo a selezionare a caso il pivot di volta in volta è molto improbabile capitare nel caso peggiore.

Complessità del Randomized Quicksort

Abbiamo dimostrato che la complessità nel caso peggiore del quicksort è $\Theta(n^2)$. Anche il Randomized Quicksort ha una complessità nel caso peggiore di $\Theta(n^2)$, tuttavia la complessità nel caso peggiore “atteso” è $O(n \log n)$.

La ricorrenza nel caso peggiore del Randomized Quicksort è:

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \theta(n)$$

dove il parametro q varia tra 0 e $n-1$.

La differenza nel caso del Randomized Quicksort è che il caso peggiore dipende solo da quale q viene selezionata di volta in volta, e il caso peggiore si ha se viene sempre selezionata quella q che ci massimizza il costo della funzione.

Risolvere la ricorrenza significa quindi risolvere un problema di massimizzazione, dove per ogni chiamata ricorsiva si vede qual è quel partizionamento dell'array che ci dà il costo maggiore (cioè, il max fra: “ $T(0) + T(n-1)$ ”, “ $T(1) + T(n-2)$ ”, ecc...).

Supponiamo che $T(n) \leq cn^2$ per qualche costante c . Sostituendo avremo:

$$\begin{aligned} T(n) &\leq \max_{0 \leq q \leq n-1} (cq^2 + c(n - q - 1)^2) + \Theta(n) \\ &\leq c \max_{0 \leq q \leq n-1} (q^2 + (n - q - 1)^2) + \Theta(n) \end{aligned}$$

Ora l'espressione $q^2 + (n-q-1)^2$ è **massima nei due estremi dell'intervallo $0 \leq q \leq n-1$** , questo perché la derivata rispetto a q si annulla in un solo punto, che è un minimo.

Infatti se **$q=0$** , avremo “ $0 + (n-1)^2 = \mathbf{(n-1)^2}$ ”.

Se **$q = n-1$** , avremo “ $(n-1)^2 + (n-(n-1)-1)^2 = (n-1)^2 + 0 = \mathbf{(n-1)^2}$ ”.

Sostituendo $(n-1)$ al posto di q abbiamo:

$$\begin{aligned} T(n) &\leq c(n-1)^2 + c(n - (n-1) - 1)^2 + \Theta(n) \\ T(n) &\leq c(n-1)^2 + \Theta(n) \\ T(n) &\leq cn^2 - c(2n-1) + \Theta(n) \\ T(n) &\leq cn^2 \end{aligned}$$

A questo punto basta scegliere un c tale che “ $c(2n-1) - \Theta(n)$ ” sia positivo.

Quindi abbiamo dimostrato che: $T(n) = \Theta(n^2)$.

Il **tempo di esecuzione atteso** del Randomized Quicksort è $O(n \log n)$, ma per arrivare a questo dobbiamo fare prima alcuni ragionamenti e dobbiamo tener ben presente il concetto di **valore atteso** (2 pagina successiva).

Ovviamente il tempo di esecuzione del Quicksort è dominato dal tempo che impiega la funzione *partition*. Possono esserci al più n chiamate ricorsive di *partition*, ed ogni chiamata impiega un tempo $O(1)$ più una quantità di tempo che è proporzionale al numero di iterazioni del ciclo for delle righe 3-6.

Ogni iterazione di questo ciclo for effettua un confronto tra il pivot ed un elemento dell'array, pertanto vale il seguente lemma:

lemma: se X è il numero di confronti svolti nella riga di *partition* nell'intera esecuzione del Quicksort su un array di n elementi, allora il tempo di esecuzione del Quicksort è $O(n+X)$.

Il nostro obiettivo è quindi **calcolare X** .

Non calcoleremo esattamente X , ma deriveremo un limite superiore sul numero globale dei confronti. Abbiamo pertanto necessità di capire quando viene effettuato un confronto tra due elementi dell' array.

Rinominiamo $z_1, z_2, \dots, z_n$ gli elementi dell'array, dove z_i è l' i -simo elemento più piccolo. Definiamo $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$, ovvero come l'insieme degli elementi compresi fra z_i e z_j .

Utilizzando una variabile indicatrice, definiamo $X_{ij} = I\{z_i \text{ è confrontato con } z_j\}$.

Stiamo quindi considerando tutti i confronti fatti dall'algoritmo, e non solo un'iterazione o chiamata di *partition*.

Poiché una coppia viene confrontata al più una sola volta, il numero totale dei confronti è dato da:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

Prendendo i valori attesi da entrambi i lati, otteniamo:

$$E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

Dunque $E[X_{ij}]$ è dato dalla probabilità con cui z_i è confrontato con z_j .

In generale, una volta che viene scelto un pivot x con $z_i < x < z_j$, sappiamo che z_i e z_j non potranno essere confrontati nell'istante successivo. Se viene scelto z_i come pivot, prima di qualsiasi altro elemento di Z_{ij} , allora sarà confrontato con tutti gli elementi tranne che con se stesso. Stessa cosa per z_j .

Quindi z_i e z_j sono confrontati se e soltanto se il primo termine ad essere scelto come pivot sarà z_i o z_j .

Poiché l'insieme ha $j - i + 1$ elementi, la probabilità che qualsiasi elemento sia scelto come pivot è: $\frac{1}{j-i+1}$. Pertanto, la probabilità di confronto tra z_i e z_j è $E[X_{ij}] = \frac{2}{j-i+1}$.

Partendo da:

$$E[x] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

Sostituendo $E[X_{ij}]$ otteniamo:

$$E[x] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}$$

poniamo $k=j-i$. La sommatoria interna parte da $j=i+1$, ma ciò significa che $j-i = 1$, e se effettuiamo la sostituzione la sommatoria andrà da $k=1$ a $n-i$.

$$E[x] = \sum_{i=1}^{n-1} \sum_{k=i}^{n-i} \frac{2}{k+1}$$

Semplifichiamo la sommatoria interna approssimando “ $n-i$ ” a “ n ”. Ovviamente la sommatoria che va da $k=1$ a $n-i$ è più piccola della sommatoria che va da $k=1$ a n (non ci importa che sia un’approssimazione perché, come detto prima, non siamo interessati a calcolare precisamente X ma vogliamo trovare un suo limite superiore).

$$E[x] < \sum_{i=1}^{n-1} \sum_{k=i}^n \frac{2}{k+1}$$

Approssimiamo inoltre $\frac{2}{k+1}$ a $\frac{2}{k}$. Vogliamo essere in questa situazione così da trovarci nel caso della **serie armonica** per la sommatoria interna, il cui risultato sappiamo essere $\log n$. Approssimando avremo:

$$E[x] < \sum_{i=1}^{n-1} \sum_{k=i}^n \frac{2}{k}$$

$$E[x] < \sum_{i=1}^{n-1} \log n$$

In altre parole, possiamo dire che $E[x]$ è “limitata superiormente” da quella sommatoria.

$$E[x] = O\left(\sum_{i=1}^{n-1} \log n\right)$$

Questa sommatoria è banale da risolvere in quanto sommiamo quasi n volte $\log n$, ovvero:

$$E[x] = O(n \log n)$$

Pertanto, il Randomized Quicksort ha un **tempo di esecuzione atteso di $O(n \log n)$** (quando i valori degli elementi sono distinti).

Valore atteso

Il **valore atteso** (aka media o speranza matematica) **di una variabile aleatoria** (casuale), è un numero indicato con $E[x]$.

Il valore atteso formalizza il concetto di valore medio di un fenomeno aleatorio.

Il valore atteso di una variabile aleatoria è dato dalla **somma dei possibili valori di tale variabile**, ciascuno **moltiplicato** per la **probabilità di essere assunto**.

Cioè, si tratta di effettuare una **media ponderata** dei possibili risultati.

Nel caso di una **variabile aleatoria continua**, che ha funzione di probabilità $f(x)$, il valore atteso è:

$$E[x] = \int_{-\infty}^{+\infty} x f(x) dx$$

Nel caso di una **variabile aleatoria discreta**, che ha funzione di probabilità p_i , allora:

$$E[x] = \sum_{i=1}^{+\infty} x_i p_i$$

La **media** è un caso particolare del valore atteso, che si ottiene quando il numero dei valori è N e la probabilità per ogni valore $p_i = \frac{1}{N}$.

Possiamo ricordare due **proprietà** del valore atteso:

- **valore atteso di una costante**: è ancora la costante stessa: $E[c] = c$.
- **linearità**: $E[aX+bY] = aE[X] + bE[Y]$, dove a, b sono costanti e X, Y sono variabili aleatorie.

Miglioramenti per il Quicksort

Un miglioramento consiste nell'evitare che il Quicksort operi su un gran numero di array parziali di piccole dimensione.

Pertanto, invece di “terminare” il quicksort quando raggiungiamo il caso base (`if (r<=p) return;`), ovvero quando la dimensione dell'array è 1, potremmo dire che se la dimensione dell'array ha raggiunto un certo valore M , andremo a chiamare l'**insertionsort**, che sappiamo comportarsi molto bene per array di piccole dimensioni, con una complessità di tempo lineare nel caso migliore.

```
if r-p+1 > M
    q = partition(A,p,r)
    quicksort(A,p,-1)
    quicksort(A,q+1,r)
else
    INSERTIONSORT(A,r-p+1)
```

Utilizzando per M un valore compreso tra 5 e 25, si riesce ad ottenere circa un risparmio del 10% in esecuzione.

Un ulteriore miglioramento consiste nel **partizionamento col metodo della mediana fra tre elementi**.

Esso consiste nell'individuare il primo, l'ultimo e l'elemento centrale dell'array, e scegliere il mediano da questi tre elementi come pivot, diminuendo così la probabilità che possa presentarsi il caso peggiore.

Tale metodo riduce il tempo medio di esecuzione di circa il 5%(in media).

HYBRID SORT

Se combiniamo i due miglioramenti, ossia utilizziamo il quicksort per trattare gli array di piccole dimensioni, ed applichiamo il metodo della mediana, possiamo ottenere un miglioramento complessivo che si aggira tra il 25-30%.

nb: la ricerca del mediano ha una complessità $O(n)$, ma ciò ci assicura una complessità $\Theta(n \log n)$. Tuttavia questa opzione viene usata di rado e viene invece preferito selezionare il pivot a caso in quanto il tempo nel caso peggiore “atteso” è comunque $\Theta(n \log n)$, infatti il caso peggiore “assoluto” ($O(n^2)$) ha una probabilità estremamente bassa di verificarsi.

LEZ 8 – Alberi binari & Struttura dati Heap

Albero (in generale)

L'**albero** è una struttura dati gerarchica, in quanto è possibile accedere alle informazioni solo partendo dal nodo radice, cioè la radice dell'albero: dal nodo *radice* possiamo discendere ai livelli inferiori. I nodi finali del livello più basso sono dette nodi *foglie*.

Nell'albero, i nodi si raggruppano in livelli: nel livello 1 troviamo solo la radice. Il nodo che stiamo prendendo in considerazione si chiama nodo corrente, e i nodi che si possono raggiungere direttamente da esso sono i nodi figli. Il nodo padre del nodo corrente è il nodo grazie al quale siamo potuti arrivare al nodo corrente. Al livello più basso troviamo certamente nodi foglie. Quindi: la radice è l'unico nodo che non ha padre, e le foglie sono gli unici nodi che non hanno figli.

Per ogni nodo di un albero si definisce grado come il numero dei figli: quindi le foglie hanno tutte grado 0 perché non hanno figli.

Si dice sottoalbero un albero che ha come radice il nodo corrente che vogliamo considerare.

Gli algoritmi di visita di un albero sono descritti in maniera *ricorsiva*. Ricordiamo che in un albero non è possibile passare direttamente da un nodo al fratello o risalire da un nodo al nodo padre. Per poter attraversare i nodi dello stesso livello dobbiamo conservare il loro padre, così che attraverso di lui potremo riscendere da un suo figlio ad un altro figlio.

Alberi binari

Negli *alberi binari* si ha un nodo radice e due sottoalberi, dunque ogni nodo può avere al massimo 2 figli, rispetto agli alberi "generici" dove per ogni nodo potevamo avere un numero imprecisato di figli.

Questi due sottoalberi prendono il nome di: sottoalbero sinistro e sottoalbero destro.

La **profondità** di un nodo è la lunghezza del percorso dalla radice al nodo (cioè, il **numero di archi attraversati**).

L'**altezza** di un nodo è l'*inverso* della profondità. L'altezza dell'albero sarà data dall'altezza della radice.

Con **livello** si intende l'insieme dei nodi alla stessa altezza.

Ci sono *tre tipi* principali di **alberi binari**:

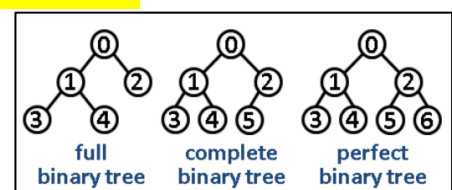
- **ALBERO BINARIO COMPLETO**: Si tratta di un albero nel quale tutte le foglie avranno profondità h o $h-1$. Il numero dei nodi al livello $(h-1)$ deve essere 2^{h-1} .

Tutti i nodi a livello h sono accatastati a sinistra.

Tutti i nodi interni hanno grado 2.

- **ALBERO BINARIO PIENO**: Si tratta di un albero dove ogni nodo ha 0 o 2 figli.

- **ALBERO BINARIO PERFETTO**: Si tratta di un albero nel quale tutti i nodi interni hanno due figli. L'**altezza** è $h = \log_2 n$. Un ABP di altezza h ha $2^{h+1}-1$ nodi.



Alberi binari Heap

Un **heap** (binario) è una struttura dati composta da un **array** che possiamo considerare come un **albero binario completo**. Ogni nodo dell'albero corrisponde a un elemento dell'array che memorizza il valore del nodo.

Un array A che rappresenta un heap è un *oggetto* con due attributi:

1. $lunghezza[A]$: indica il numero di elementi dell'array;
2. $heap-size[A]$: indica il numero di elementi dell'heap che sono registrati nell'array A .

nb: ciò significa che, anche se $A[1 \dots lunghezza[A]]$ contiene numeri validi, nessun elemento dopo $A[heap-size[A]]$, dove $heap-size[A] \leq lunghezza[A]$, è un elemento dell'heap.

La radice dell'albero è $A[1]$. Se i è l'indice di un nodo, gli indici di suo padre $PARENT(i)$, del figlio sinistro $LEFT(i)$ e del figlio destro $RIGHT(i)$ possono essere calcolati come:

(nb: tra parentesi le funzioni per array zero-based)

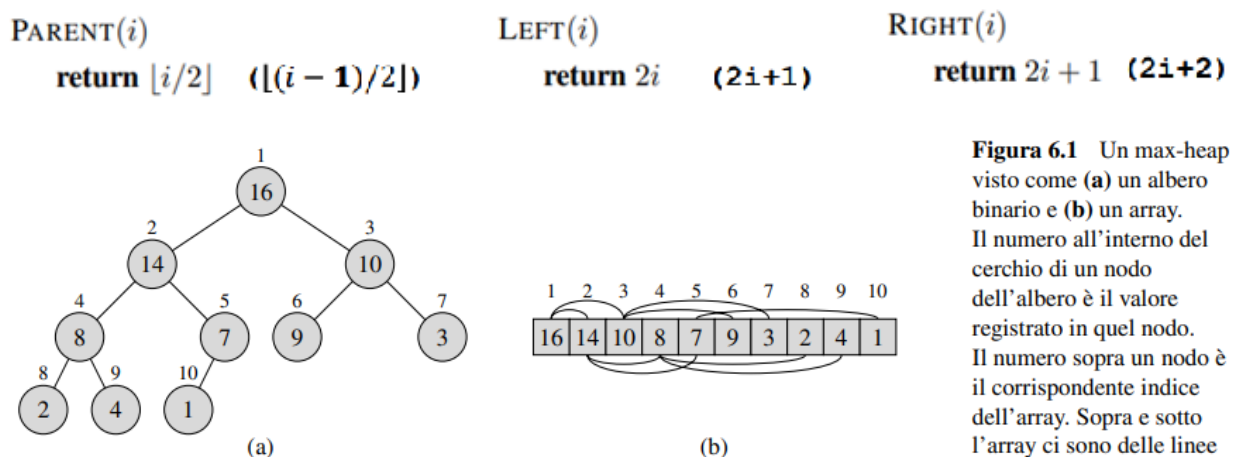


Figura 6.1 Un max-heap visto come (a) un albero binario e (b) un array. Il numero all'interno del cerchio di un nodo dell'albero è il valore registrato in quel nodo. Il numero sopra un nodo è il corrispondente indice dell'array. Sopra e sotto l'array ci sono delle linee che rappresentano le relazioni padre-figlio; i padri sono sempre a sinistra dei loro figli. L'albero ha altezza 3; il nodo con indice 4 e valore 8 ha altezza 1.

Ci sono due tipi di heap binari: max-heap e min-heap. In entrambi i tipi, i valori dei nodi soddisfano una **proprietà dell'heap**:

1. La proprietà del **max-heap** dice che il suo valore più grande è memorizzato alla radice, e il sottoalbero di un nodo contiene valori non maggiori di quello contenuto nel nodo stesso. $(A[PARENT(i)] \geq A[i])$
2. La proprietà del **min-heap** (al contrario) dice che il suo valore più piccolo è memorizzato alla radice, e il sottoalbero di un nodo contiene valori maggiori di quello contenuto nel nodo stesso. $(A[PARENT(i)] \leq A[i])$

Definiamo **altezza di un heap** l'altezza della sua radice, calcolata come **log n**.

Un albero heap implementa un **ordinamento parziale**.

Un ordinamento parziale è una nozione più debole rispetto a quello totale, ma è più facile da costruire. Un ordinamento è detto parziale se vengono rispettate **3 relazioni**:

- **riflessiva**: $a \leq a \quad \forall a \in B$; (in B , ogni elemento è $\leq$ di se stesso).
- **antisimmetrica**: se $a \leq b$ e $b \leq a \Rightarrow a=b \quad \forall a,b \in B$;
- **transitiva**: $a \leq b, b \leq c \Rightarrow a \leq c \quad \forall a,b,c \in B$.

Le **procedure per gestire un heap** sono tre:

1. La procedura **Max-Heapify**, di complessità $O(\log n)$, che è un algoritmo che serve per mantenere la proprietà dell'Heap.
2. La procedura **Build-Max-Heap**, di complessità $O(n)$, che è un algoritmo che costruisce un Heap da zero (partendo da un array).
3. La procedura **Heapsort**, di complessità $O(n \log n)$, che è un algoritmo che ordina sul posto un array.

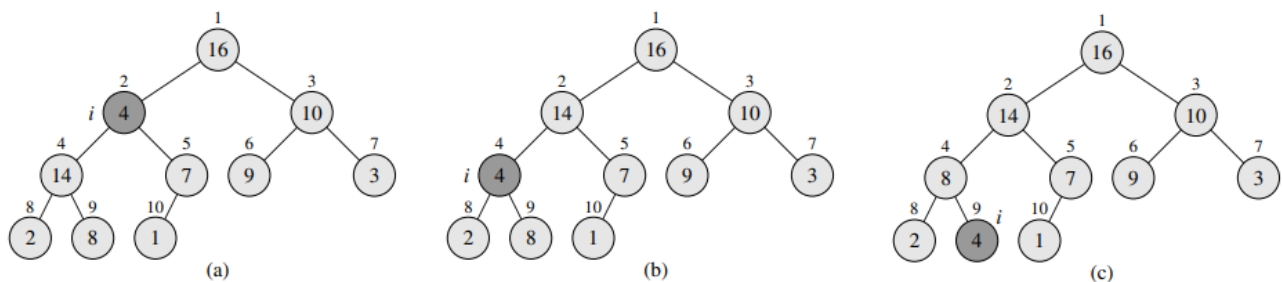
Max-Heapify

MAX-HEAPIFY è un'importante subroutine per manipolare i max-heap. I suoi input sono un array A e un indice i dell'array. Quando viene chiamata questa subroutine, si suppone che gli alberi binari con radici in $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ siano max-heap, ma che $A[i]$ possa essere più piccolo dei suoi figli, violando così la proprietà del max-heap. La funzione di MAX-HEAPIFY è quella di consentire al valore $A[i]$ di "scendere" nel max-heap in modo che il sottoalbero con radice di indice i diventi un max-heap.

In particolar modo, ad ogni passaggio, viene determinato il più grande degli elementi fra $A[i]$, $A[\text{LEFT}(i)]$ e $A[\text{RIGHT}(i)]$, e se ne salva l'indice in una variabile chiamata *massimo*.

- Se $A[i]$ è più grande, allora il sottoalbero con radice nel nodo i è un max-heap, e quindi la procedura termina.
- Altrimenti, uno dei due figli ha l'elemento più grande e $A[i]$ viene scambiato con $A[\text{massimo}]$.
Tuttavia, dopo questo scambio, il sottoalbero con indice *massimo* ha il valore originale $A[i]$, e ciò significa che potrebbe comunque esserci il rischio che il sottoalbero con radice in *massimo* possa violare la proprietà del max-heap. Per questo motivo, richiederemo nuovamente MAX-HEAPIFY per questo sottoalbero.

Inoltre, quando confrontiamo il valore massimo, dovremmo anche controllare che l'indice del figlio sinistro e destro di un determinato nodo non sia maggiore rispetto alla dimensione dell'heap. Infatti, se così fosse, vorrebbe dire che tale nodo figlio non esiste, e lo ignoriamo.



MAX-HEAPIFY(A, i)

```

1   $l \leftarrow \text{LEFT}(i)$ 
2   $r \leftarrow \text{RIGHT}(i)$ 
3  if  $l \leq \text{heap-size}[A]$  and  $A[l] > A[i]$ 
4      then  $\text{massimo} \leftarrow l$ 
5      else  $\text{massimo} \leftarrow i$ 
6  if  $r \leq \text{heap-size}[A]$  and  $A[r] > A[\text{massimo}]$ 
7      then  $\text{massimo} \leftarrow r$ 
8  if  $\text{massimo} \neq i$ 
9      then  $\text{scambia } A[i] \leftrightarrow A[\text{massimo}]$ 
10     MAX-HEAPIFY( $A, \text{massimo}$ )

```

Complessità di Max-Heapify

Il tempo di esecuzione MAX-HEAPIFY in un sottoalbero di dimensione n con radice in un nodo i è pari al tempo $\Theta(1)$ per le prime istruzioni più il tempo necessario per eseguire il MAX-HEAPIFY in un sottoalbero la quale radice è uno dei figli del nodo i .

I sottoalberi dei figli hanno ciascuno una dimensione (ossia il numero dei nodi) che non supera, nel caso peggiore, $2/3$ dell'albero.

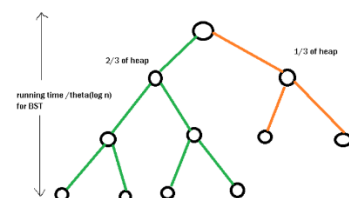
Infatti, il **caso peggiore**, si ha quando il sottoalbero sinistro è completo al livello h e il sottoalbero destro è completo al livello $h-1$.

Pertanto, il sottoalbero sinistro ha $2^{h+1}-1$ nodi, mentre il sottoalbero destro ha 2^h-1 nodi. Comprendendo anche la radice (1 nodo), il numero di nodi totali è dato da:

$$(2^{h+1}-1) + (2^h-1) + 1 = 3(2^h-1) \text{ nodi.}$$

Quindi il rapporto tra i nodi del sottoalbero sinistro e destro è:

$$\frac{2(2^h - 1)}{3(2^h - 1)} \leq \frac{2}{3}$$



E quindi sappiamo che il rapporto fra i nodi del sottoalbero sinistro e quelli totali è $\leq 2/3$.

Da qui ricaviamo la ricorrenza: $T[n] \leq T(2n/3) + \Theta(1)$

Che può essere facilmente risolta col Master Theorem (caso 2), risultando che: $T[n] = \Theta(\log n)$.

Build-max-heap

Per costruire un heap possiamo utilizzare la procedura MAX-HEAPIFY dal basso verso l'alto (bottom-up) per convertire un'array $A[1 \dots n]$, con $n = \text{lunghezza}[A]$, in un max-heap. Da notar bene che tutti gli elementi nel sottoarray $A[(\lfloor n/2 \rfloor + 1) \dots n]$ sono foglie dell'albero (le foglie in effetti compongono la metà dei nodi dell'albero) e quindi ciascuno è un heap di un elemento da cui iniziare.

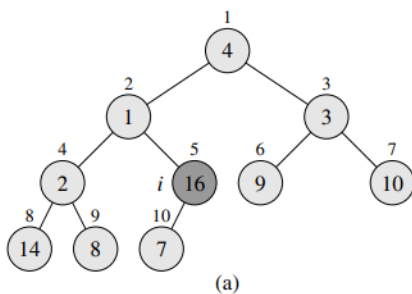
La procedura BUILD-MAX-HEAP attraversa i restanti nodi dell'albero ed esegue la procedura MAX-HEAPIFY in ciascuno di essi.

BUILD-MAX-HEAP(A)

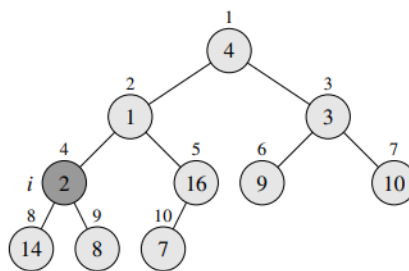
```
1   $\text{heap-size}[A] \leftarrow \text{lunghezza}[A]$ 
2  for  $i \leftarrow \lfloor \text{lunghezza}[A]/2 \rfloor$  downto 1
3      do MAX-HEAPIFY( $A, i$ )
```

A

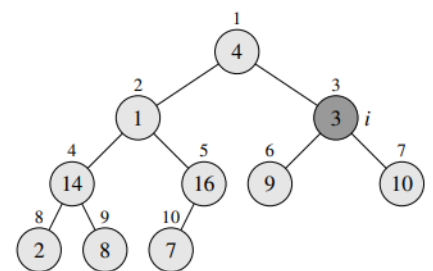
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



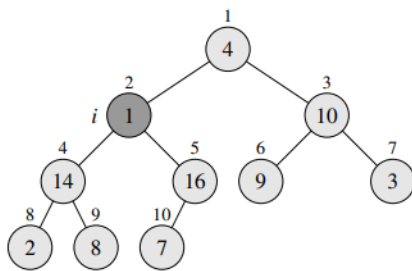
(a)



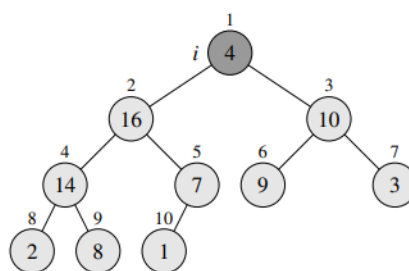
(b)



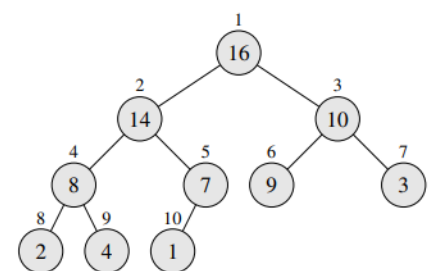
(c)



(d)



(e)



(f)

nb: Nota infatti come l'array abbia size 10, e infatti il primo elemento elemento che prendiamo in considerazione è al posto $i=5$. Da qui consideriamo $2i$ (ovvero 7) e $2i+1$ (che non esiste (infatti è $>$ del size di A), e non inseriamo alcun nodo).

Al passo successivo, consideriamo $i=4$, poi $i=3 \dots$ e così via.

Invariante di ciclo (Build-max-heap)

All'inizio di ogni iterazione del ciclo *for* (righe 2-3), per ogni nodo: $i+1, i+2, \dots, n$; avremo che questi nodi sono la radice di un max-heap.

Inizializzazione: prima della prima iterazione del ciclo, $i = \lfloor n/2 \rfloor$. Ogni nodo $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ è una foglia e, quindi, sono per forza radici di un max-heap in quanto tali max-heap sono costituiti da un solo elemento.

Conservazione: per verificare che ogni iterazione conserva l'invariante di ciclo, notiamo che i figli del nodo i hanno una numerazione più alta di i . Per l'invariante di ciclo, quindi, essi sono entrambi radici di max-heap. Questa è esattamente la condizione richiesta affinché la chiamata $\text{MAX-HEAPIFY}(A, i)$ renda il nodo i la radice di un max-heap. Inoltre, la chiamata MAX-HEAPIFY preserva la proprietà che tutti i nodi $i + 1, i + 2, \dots, n$ siano radici di max-heap.

La diminuzione di i nell'aggiornamento del ciclo *for* ristabilisce l'invariante di ciclo per la successiva iterazione.

Conclusione: alla fine del ciclo, $i = 0$. Per l'invariante di ciclo, ogni nodo $1, 2, \dots, n$ è la radice di un max-heap; in particolare, lo è il nodo 1.

Complessità (Build-Max-Heap)

Potremmo pensare di calcolare il limite superiore del BMH tenendo in considerazione che ogni chiamata MAX-HEAPIFY ha un tempo di esecuzione $O(\log n)$, e ci sono $O(n)$ di queste chiamate. Ergo, il tempo di esecuzione sarebbe $O(n \log n)$.

Tuttavia tale limite non è asintoticamente stretto. Infatti, possiamo notare come il tempo di esecuzione di MAX-HEAPIFY in un nodo varia in base all'altezza del nodo stesso, e le altezze della maggior parte dei nodi sono piccole.

Rircodando che un heap di n elementi:

1. ha un'altezza **$\log n$** .
2. Per ogni h , avrà **al massimo $[n/2^{h+1}]$ nodi di altezza h** .

Il tempo richiesto dalla procedura MaxHeapify, quando viene chiamata per un nodo di altezza h , è $O(h)$, quindi il costo totale del BMH è limitato superiormente da:

$$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} O(h)$$

Calcoliamo la sommatoria:

$$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} O(h) = O\left(n \sum_{h=0}^{\log n} \frac{h}{2^h}\right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right)$$

L'ultima sommatoria può essere calcolata ponendo $x = 1/2$ nella Serie geometrica (2)*:

$$O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O\left(n \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2}\right) = O(n)$$

Dunque, possiamo costruire un max-heap da un array non ordinato in tempo lineare.

Da notar bene, inoltre, che possiamo costruire un min-heap utilizzando la procedura Build-Min-Heap, che è uguale Build-Max-Heap, con la differenza che la chiamata Max-Heapify nella riga 3 è sostituita dalla chiamata Min-Heapify.

*Serie geometrica (2):

$$\sum_{k=1}^n kx^k = \frac{x}{(1-x)^2}$$

LEZ 9 – HeapSort, Code di priorità & BucketSort

HeapSort

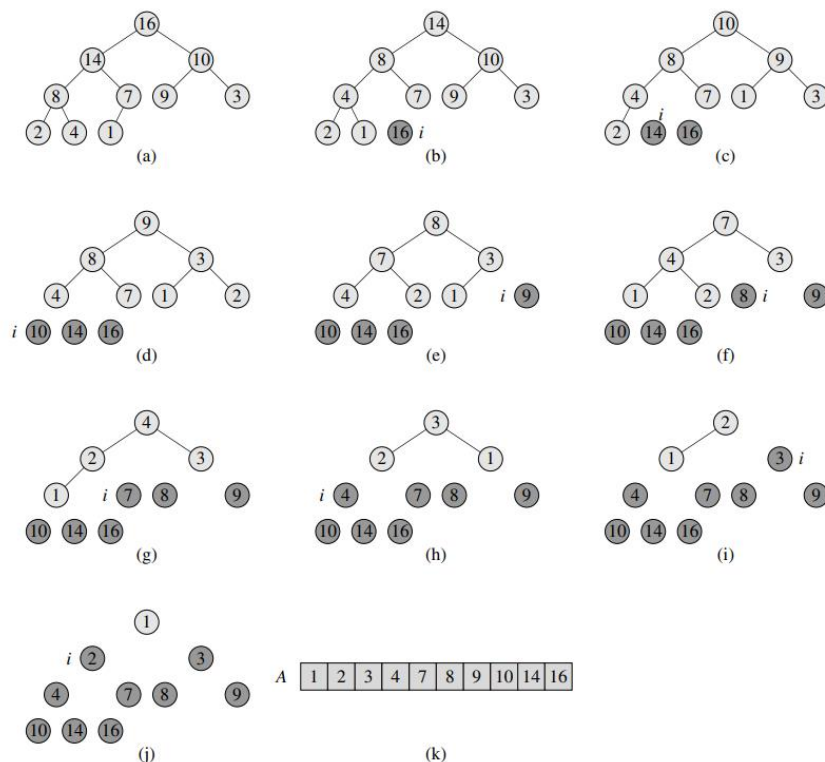
L'Heapsort è un eccellente algoritmo di ordinamento. Inizia utilizzando BUILD_MAX_HEAP per costruire un max-heap dell'array di input $A[1 \dots n]$, dove $n = \text{lunghezza}[A]$. Poiché l'elemento più grande dell'array è memorizzato nella radice $A[1]$, esso può essere inserito nella posizione finale corretta scambiandolo con $A[n]$. Se adesso "scartiamo" il nodo n dall'heap (diminuendo $\text{heap-size}[A]$), notiamo che $A[1 \dots (n-1)]$ può essere facilmente trasformato in un max-heap.

I figli della radice restano max-heap, ma la nuova radice potrebbe violare la proprietà del max-heap, che può tuttavia essere facilmente ripristinata richiamando MAXHEAPIFY($A, 1$).

L'algoritmo heapsort poi ripete questo processo per il max-heap di dimensione $n - 1$ fino a un heap di dimensione 2 (finché $i \geq 2$).

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i \leftarrow \text{lunghezza}[A]$  downto 2
3      do scambia  $A[1] \leftrightarrow A[i]$ 
4          $\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$ 
5         MAX-HEAPIFY( $A, 1$ )
```



La procedura HEAPSORT impiega un tempo $O(n \log n)$, in quanto la chiamata di BUILD_MAX_HEAP impiega $O(n)$ e ciascuna delle $(n - 1)$ chiamate di MAX_HEAPIFY impiega un tempo $O(\log n)$.

Code di priorità

Nonostante l'heapsort sia un eccellente algoritmo di ordinamento, di solito viene battuto da una buona implementazione del quicksort. Tuttavia, le strutture dati Heap risultano essere delle efficienti code di priorità.

Una **coda di priorità** è una struttura dati che serve a mantenere un insieme S di elementi, ciascuno con un valore associato detto **chiave**.

Analogamente all'heap, distinguiamo code di *max-priorità* e *min-priorità*.

Una **coda di max-priorità** supporta diverse operazioni:

1. **Insert(S, x)**: inserisce l'elemento x nella coda S .
2. **Maximum(S)**: restituisce l'elemento con chiave massima.
3. **Extract-Max(S)**: rimuove e restituisce l'elemento con chiave massima.
4. **Increase-priority(S, x, k)**: aumenta il valore della chiave di x al nuovo valore k (che si suppone essere grande almeno quanto il valore attuale della chiave di x).

Analogamente, una *coda di min-priorità* supporterà operazioni come: *insert*, *minimum*, *extract_minimum*, *decrease_key*.

Heap-maximum

Restituisce semplicemente il primo valore dell'array. Impiega un tempo $O(1)$.

HEAP-MAXIMUM(A)

```
1 return  $A[1]$ 
```

Heap-extract-max

Salva il primo elemento in una variabile *massimo*. Poi cambia il primo elemento (che è l'elemento massimo) con l'ultimo elemento, diminuisce *heap-size* di 1, e viene richiamata Max-Heapify per ripristinare la proprietà del Max-heap. Alla fine ritorneremo *massimo*.

Il tempo di esecuzione di HEAP-EXTRACT-MAX è $O(\log n)$, in quanto svolge soltanto una quantità costante di lavoro oltre al tempo $O(\log n)$ di MAX-HEAPIFY.

HEAP-EXTRACT-MAX(A)

```
1 if  $heap-size[A] < 1$ 
2   then error "underflow dell'heap"
3  $max \leftarrow A[1]$ 
4  $A[1] \leftarrow A[heap-size[A]]$ 
5  $heap-size[A] \leftarrow heap-size[A] - 1$ 
6 MAX-HEAPIFY( $A, 1$ )
7 return  $max$ 
```

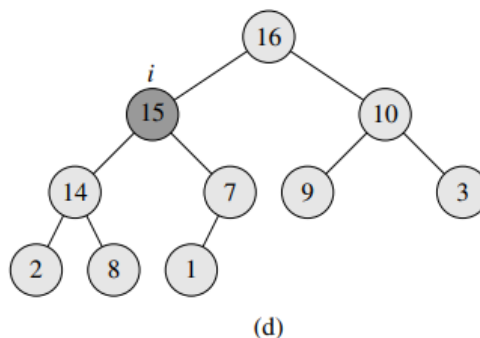
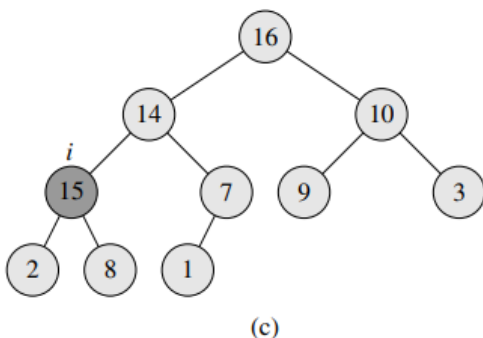
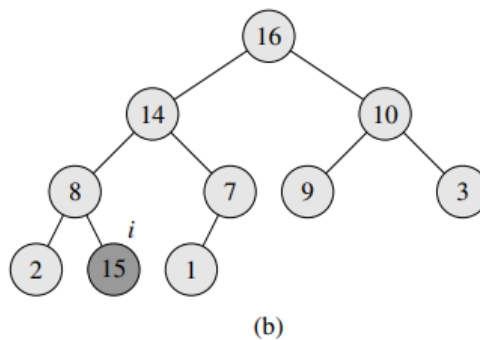
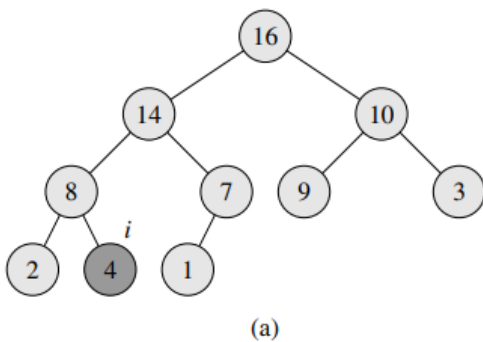
Heap-increase-key

L'elemento della coda di priorità la cui chiave deve essere aumentata è identificato da un indice i nell'array. Innanzitutto, la procedura aggiorna la chiave dell'elemento $A[i]$ con il suo nuovo valore. Successivamente, poiché l'aumento della chiave di $A[i]$ potrebbe violare la proprietà del max-heap, la procedura segue un percorso da questo nodo verso la radice per trovare un posto appropriato alla nuova chiave. Durante questo attraversamento, viene confrontata la chiave dell'elemento con quella del padre, ed esse vengono scambiate se la chiave dell'elemento risulta essere più grande; questa operazione termina se la chiave dell'elemento è più piccola, perché in questo caso la proprietà del max-heap è soddisfatta.

Il tempo di esecuzione di HEAP-INCREASE-KEY con un heap di n elementi è $O(\log n)$, in quanto il cammino seguito dal nodo aggiornato nella riga 3 alla radice ha lunghezza $O(\log n)$.

HEAP-INCREASE-KEY($A, i, chiave$)

```
1  if  $chiave < A[i]$ 
2    then error "la nuova chiave è più piccola di quella corrente"
3   $A[i] \leftarrow chiave$ 
4  while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5    do scambia  $A[i] \leftrightarrow A[PARENT(i)]$ 
6     $i \leftarrow PARENT(i)$ 
```



Max-heap-insert

La procedura MAX-HEAP-INSERT implementa l'operazione INSERT. Prende come input la chiave del nuovo elemento da inserire nel max-heap A . La procedura prima espande il max-heap aggiungendo all'albero una nuova foglia la cui chiave è $-\infty$; poi chiama HEAP-INCREASE-KEY per impostare la chiave di questo nuovo nodo al suo valore corretto e mantenere la proprietà del max-heap.

Il tempo di esecuzione della procedura MAX-HEAP-INSERT con un heap di n elementi è $O(\log n)$. In sintesi, un heap può svolgere qualsiasi operazione con le code di priorità su un insieme di dimensione n nel tempo $O(\log n)$.

MAX-HEAP-INSERT(A , *chiave*)

- 1 $heap-size[A] \leftarrow heap-size[A] + 1$
- 2 $A[heap-size[A]] \leftarrow -\infty$
- 3 **HEAP-INCREASE-KEY**(A , $heap-size[A]$, *chiave*)

TEOREMA (del limite inferiore)

Qualsiasi **algoritmo di ordinamento per confronti** richiede $\Omega(n \log n)$.

Infatti, gli ordinamenti per confronti possono essere visti astrattamente in termini di **alberi di decisione**. Quando raggiunge una foglia, l'albero ha trovato il suo ordinamento.

Poiché qualsiasi algoritmo di ordinamento corretto deve essere in grado di produrre ogni permutazione del suo input, **condizione necessaria è che ciascuna delle $n!$ permutazioni compaia come una foglia dell' albero**, e che ciascuna di queste foglie sia raggiungibile attraverso un percorso di nodi che porta alla radice.

La lunghezza del cammino semplice più lungo a una delle sue foglie raggiungibili rappresenta il numero dei confronti che svolgerà il corrispettivo algoritmo nel caso peggiore. Quindi **il numero dei confronti nel caso peggiore è dato dall' altezza del suo albero di decisione**.

Dimostrazione

L'altezza dell'albero di decisione, dove ogni permutazione appare come una foglia raggiungibile, fornisce il limite inferiore.

Supponiamo che l' albero abbia altezza h con l foglie raggiungibili che corrisponde ad un ordinamento di n elementi.

Ogni permutazione dell' input compare in una foglia, si ha $n! \leq l$. Ma ogni albero ha al più 2^h foglie. Quindi si ha: $n! \leq l \leq 2^h$.

Ergo, si ottiene che $\log(n!) \leq h$, e ricordando la formula di Stirling ($n! \sim n^n e^{-n}$) abbiamo che $h = \Omega(n \log n)$.

Ordinamenti in tempo lineare

Gli algoritmi di ordinamento visti fino ad ora effettuano un ordinamento basato solo su confronti fra gli elementi di input, e il quale limite inferiore è $n \log n$.

Gli algoritmi in tempo lineare usano operazioni diverse dai confronti e quindi non vale il precedente limite inferiore.

Esempi di tali algoritmi sono il: *counting sort*, *radix sort* e *bucket sort*.

Bucket Sort

Il Bucket Sort, o *bin sort*, è un algoritmo di ordinamento il cui funzionamento consiste nel suddividere un array in un **numero finito di buckets**.

Ciascun bucket è ordinato individualmente, o utilizzando un differente algoritmo di ordinamento, o applicando ricorsivamente il bucket sort.

Inizialmente si definisce un array di “buckets” inizialmente vuoti, poi si mette ogni elemento dell’array nel suo bucket e viene ordinato ogni bucket non vuoto.
Infine, vengono spostati gli elementi dai buckets all’array originale.

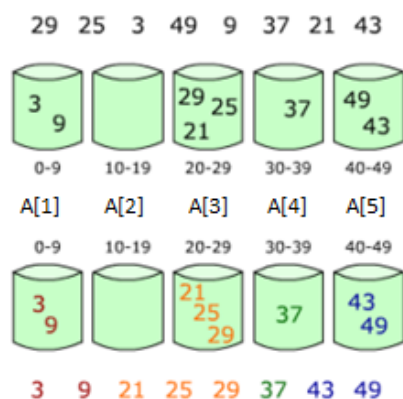
La sua complessità media è $\Theta(n)$.

Esempio:

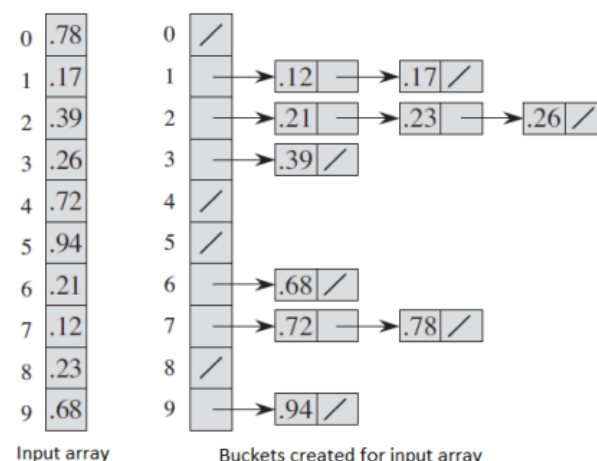
Consideriamo un array **A** di 8 elementi.

Una volta inseriti tutti gli elementi nei loro rispettivi bucket, ordineremo i singoli bucket, e infine sposteremo tali elementi nell’array originale.

(esempio Prof)



(esempio Cormen)



Il Bucket Sort (Cormen) presuppone che ogni elemento dell’array **A** sia compreso fra 0 ed 1 ($0 \leq A[i] < 1$). Il codice richiede un array ausiliario **B**[0 ... n-1] di liste concatenate (bucket) e assume che vi sia un meccanismo per mantenere tali liste.

bucketSort(A)

1. $n = A.length$
2. crea array $B[0 \dots n-1]$
3. **for** $i=0$ to $n-1$
4. crea lista vuota $B[i]$
5. **for** $i=1$ to n
6. inserisci $A[i]$ nella lista $B[\lfloor nA[i] \rfloor]$
7. **for** $i=0$ to $n-1$
8. insertionSort($B[i]$)
9. Concatena le liste $B[0], \dots, B[n-1]$ in quest'ordine

Un'accoppiata BucketSort/InsertionSort può risultare eccellente. Infatti sappiamo che l'InsertionSort si comporta molto bene con array di piccole dimensioni, in quanto sarà probabile che questi saranno già completamente o parzialmente ordinati, e l'InsertionSort è in grado di "rendersene conto". Quello che fa il BucketSort è prendere un array e suddividerlo in più gruppi, o "bucket", quindi creando una serie di array più piccoli (che verranno ordinati dall'InsertionSort).

Complessità del BucketSort

La suddivisione nei bucket richiede un tempo $\Theta(n)$, mentre la complessità dell'InsertionSort nel caso peggiore è $O(n^2)$. Quindi la complessità totale è:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

Valor medio del Bucket Sort

$$E[T(n)] = E[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)]$$

$$E[T(n)] = \Theta(n) + E[\sum_{i=0}^{n-1} O(n_i^2)]$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} E[O(n_i^2)]$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2])$$

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O\left(2 - \frac{1}{n}\right) = \Theta(n)$$

$$\text{Perché vale } E[n_i^2] = 2 - \frac{1}{n} \quad \text{per } 0 < i < n-1$$

nb: anche se sostituiamo l'InsertionSort con il MergeSort otteniamo $\Theta(n)$ perché vale:

$$\sum_{i=0}^{n-1} n_i \log n = O(n)$$

LEZ 10 – Programmazione Dinamica (e Fibonacci e LCS)

La programmazione dinamica è una tecnica di progettazione basata sulla divisione del problema in sottoproblemi e sull'utilizzo di sottostrutture ottimali.

Sottostruttura ottimale significa che la soluzione ottima del problema contiene la soluzione ottima dei sottoproblemi.

(Ricordando che: il Divide et Impera è un approccio ricorsivo e top-down che si applica quando i sottoproblemi sono **indipendenti**. In questo approccio, eventuali sottoproblemi comuni vengono risolti più volte).

La *programmazione dinamica* è una tecnica iterativa e bottom-up che si applica quando i sottoproblemi sono **dipendenti**. In questo approccio, eventuali sottoproblemi comuni vengono risolti una sola volta e si salva la soluzione del sottoproblema in una tabella, evitando così di ricalcolare la soluzione ogni volta che si presenta il sottoproblema.

L'idea dietro alla programmazione dinamica è: “evitare di fare cose che abbiamo già fatto”. Se un sottoproblema si ripresenta, lo andiamo a recuperare nella tabella ($O(1)$).

La programmazione dinamica viene generalmente adottata per risolvere problemi di ottimizzazione, ciascuno dei quali ammette solitamente più soluzioni ottime, e quando si presentano *sottoproblemi ripetuti* e lo *spazio dei sottoproblemi* è *polinomiale*.

I problemi di ottimizzazione sono quei problemi che hanno l'obiettivo **di trovare la soluzione** migliore associando un **punteggio** a tutte le soluzioni possibili.

Se bisogna **minimizzare** il punteggio, si parla di **problema di minimizzazione**, e il punteggio è detto costo.

Se bisogna **massimizzare** il punteggio, si parla di **problema di massimizzazione**.

Un algoritmo che adotta questo approccio prevede **quattro fasi di sviluppo**:

1. caratterizzazione della sottostruttura ottima del problema;
2. definizione ricorsiva del valore di una soluzione ottima;
3. calcolo del valore di una soluzione ottima secondo uno schema *bottom-up*.
4. costruzione di una (o la) soluzione ottima a partire dalle soluzioni calcolate.

TABULATION vs MEMOIZATION

La Programmazione Dinamica è solitamente implementata utilizzando la *tabulazione*¹, per questo PD e Tabulation vengono spesso usate come sinonimi, tuttavia la PD può essere implementata anche con la *memoizzazione*.

¹con Camastra è meglio non nominare la Tabulation in sé ma di trattarla come sinonimo della PD.

La **Tabulazione** è un metodo **bottom-up** e **iterativo**.

Parte dai casi base e trova la soluzione ottima ai sottoproblemi la cui soluzione è data proprio dai casi base. Poi, sale di livello e combina le soluzioni ottenute precedentemente per ottenere le soluzioni a sottoproblemi via via più complessi.

Alla fine, la tabulazione combinerà le soluzioni dei sottoproblemi del problema originale, e troverà la soluzione ottima.

La **Memoizzazione** è un metodo **top-down** e **ricorsivo**.

Parte direttamente dai sottoproblemi del problema principale, scendendo via via di livello fino a raggiungere i casi base. Poi, risale di livello risolvendo solo i problemi che risultano necessari.

Può essere vista come una **fusione del Divide et Impera con la PD**.

Alla fine, la memoizzazione combinerà le soluzioni dei sottoproblemi del problema originale, e troverà la soluzione ottima.

Usando un'analogia:

- **bottom-up**: voglio conquistare il mondo! Partirò prima da Napoli, poi conquisterò l'Italia, l'Europa e infine il mondo intero!
- **top-down**: voglio conquistare il mondo! Come farò? Conquistando prima l'Europa! Come farò? Conquistando prima l'Italia! Come farò? Conquistando prima Napoli!

Entrambe le versioni dicono la stessa cosa, cambia solo il mondo con cui si comunica il messaggio.

Sia la tabulazione che la memoizzazione permettono di risolvere problemi di PD, e ovviamente in entrambi i casi utilizziamo una tabella di appoggio nella quale salvare le soluzioni una volta calcolate la prima volta.

Dunque *concettualmente* sono la stessa cosa, ma **in base alla situazione** una tecnica può avere **prestazioni leggermente migliori all'altra**. Infatti:

- se **è necessario risolvere tutti i sottoproblemi** per ottenere la soluzione ottima, la PD con tabulation risulta essere la soluzione migliore in quanto non viene generato overhead per le chiamate ricorsive e si evita anche il rischio che queste possano "sfondare" lo stack.
- se **NON è necessario risolvere tutti i sottoproblemi** per ottenere la soluzione ottima, la PD con memoization risulta essere la scelta migliore in quanto risolve solo i sottoproblemi che sono richiesti.

Inoltre, la memoizzazione risulta generalmente più semplice da implementare.

Fibonacci

CONFRONTO: algoritmo ricorsivo puro VS tabulation VS memoization

I numeri di Fibonacci F sono definiti nel modo seguente:

$$\begin{aligned} - F[i] &= F[i-1] + F[i-2] \\ - F[1] &= F[0] = 1 \end{aligned}$$

• RICORSIVO SEMPLICE

```
fibonacci(i)
1.  if i=0 or i=1
2.      return 1
3.  return fibonacci(i-1) + fibonacci(i-2)
```

• TABULATION (bottom-up)

```
fibonacci(i)
1.  crea array KnFibonacci = [0...i+2] inizializzato a 0
2.  KnFibonacci[0] = KnFibonacci[1] = 1
3.  for j=2 to i
4.      KnFibonacci[j] = KnFibonacci[j-1] + KnFibonacci[j-2]
5.  return KnFibonacci[i]
```

• MEMOIZATION (top-down)

```
1.  crea array globale KnFibonacci = [0...MAXNUM]
```

```
num_fibonacci(i)
1.  for j=0 to i
2.      KnFibonacci[j] = 0 //known num fibonacci. Inizializzo array
3.  return fibonacci(i)
```

```
fibonacci(i)
1.  if KnFibonacci[i] ≠ 0
2.      return KnFibonacci[i]
3.  if i=0 or i=1
4.      KnFibonacci[i] = 1
5.  else
6.      KnFibonacci[i] = fibonacci(i-1) + fibonacci(i-2)
7.  return KnFibonacci[i]
```

Calcolo dei numeri di Fibonacci mediante Algoritmo ricorsivo

Il tempo necessario $T(n)$ è dato dall'equazione di ricorsione: $T(n) = 1.6T(n - 1)$. Ciò significa che la complessità è $\Theta(1.6^n)$, ovvero è di tipo esponenziale, il che rende arduo calcolare il numero di Fibonacci per valori moderatamente grandi.

Calcolo dei numeri di Fibonacci mediante PD con Tabulation

Creiamo un array locale KnFibonacci che va da 0 ad i , e ne inizializziamo tutti gli elementi a 0. Cominciamo già ad anticiparci i casi base ($i=0$, $i=1$) ponendo $\text{KnFibonacci}[0] = 1$ e $\text{KnFibonacci}[1] = 1$.

Con un ciclo for, calcoliamo i numeri di Fibonacci partendo dal 2 elemento fino a raggiungere l'elemento i -simo, ovvero il nostro obiettivo.

Infine ritorniamo $\text{KnFibonacci}[i]$, ovvero la soluzione ottima al problema.

Calcolo dei numeri di Fibonacci mediante PD con Memoization

Chiamiamo prima $\text{num_fibonacci}(i)$, poi in un array (globale) che va da $j=0$ fino ad i , poniamo tutti i valori a 0. Infine richiamiamo la vera funzione $\text{Fibonacci}(i)$.

Ad ogni chiamata ricorsiva andremo a controllare se $\text{KnFibonacci}[i] \neq 0$. Se così fosse, significa che il numero è già stato calcolato in precedenza, e quindi ritorneremo semplicemente il numero stesso.

Se $i = 1$ o $i = 0$, allora impostiamo $\text{KnFibonacci}[1]$ o $\text{KnFibonacci}[0]$ uguale a 1, e poi lo ritorniamo..

Altrimenti, se i è diverso sia da 1 che da 0, e $\text{KnFibonacci}[i] = 0$, allora calcoleremo l' i -esimo numero della sequenza di Fibonacci richiamando ricorsivamente la funzione.

Alla fine ritorneremo $\text{KnFibonacci}[i]$.

In questo caso è stato necessario calcolare la soluzione di tutti i sottoproblemi, dunque l'approccio più efficiente risulta essere quello con **Tabulation**.

LCS – Longest Common Subsequence

Il problema della **massima sottosequenza comune** (LCS – Longest Common Subsequence) consiste nel trovare la più lunga sottosequenza comune a tutte le sequenze considerate.

L'LCS considera sia i caratteri che compaiono nelle stringhe sia l'ordine con cui compaiono, quindi rappresenta un valido criterio per stabilire quanto due o più sequenze di caratteri differiscono tra loro.

Proprietà

- Una sequenza Y è una sottosequenza di X se è ottenuta da X rimuovendo uno o più elementi:

$$X = AABBBBC \quad Y = ABC$$

- La sequenza nulla è una sottosequenza di ogni sequenza.
- Z è una sottosequenza comune di X e Y se Z è una sottosequenza sia di X che di Y . Definiamo $C_s(X, Y)$ l'insieme di tutte le sottosequenze comuni di X e Y .
- Date due sequenze X e Y , Z è la LCS di X e Y se $Z \in C_s$ ed inoltre non esiste $W \in C_s$ tale che $|W| > |Z|$.

(giustamente, se Z è la LCS di X e Y , non può esserci un'altra sequenza più grande di Z . Appunto, con $|Z|$ intendiamo la *lunghezza di Z*).

ESEMPIO: Date due sequenze X e Y , calcolare la $LCS(X, Y)$:

$X = AAAATTGA, \quad Y = TAACGATA \quad \rightarrow \quad LCS = AAGA$

Teorema della dimostrazione ottima per l'LCS

Date due sequenze $X = (x_1, \dots, x_m)$ e $Y = (y_1, \dots, y_n)$, e sia $Z = (z_1, \dots, z_k)$:

- se $x_m = y_n$, allora $z_k = x_m = y_n$ e $Z_{k-1} = LCS(X_{m-1}, Y_{n-1})$;
- se $x_m \neq y_n$ e $z_k \neq x_m$, allora $Z_k = LCS(X_{m-1}, Y_n)$;
- se $x_m \neq y_n$ e $z_k \neq y_n$, allora $Z_k = LCS(X_m, Y_{n-1})$;

Sono quindi tre i possibili casi:

- Se l' m -simo elemento di X e l' n -simo elemento di Y sono uguali, allora anche il k -simo elemento di Z uguale, ed inoltre Z_{k-1} è una LCS sia di X_{m-1} e Y_{n-1} ;
- Altrimenti se sono diversi, possiamo avere 2 sottocasi:
 - Se il k -simo elemento di z è diverso dall' m -simo elemento di X , allora Z_k è una LCS di X_{m-1} e Y_n ;
 - Viceversa altrimenti.

DIMOSTRAZIONE

Si dimostra per assurdo. Consideriamo il primo caso, supponiamo per assurdo che $Z_{k-1} \neq LCS(X_{m-1}, Y_{n-1})$. Allora esiste necessariamente una Z_{k-1}^* che è più lunga di Z_{k-1} . Pertanto possiamo costruire una nuova Z_k^* , concatenando Z_{k-1}^* con il simbolo z_k , che avrà lunghezza maggiore di Z_k , che è un assurdo in quanto Z_k è una LCS.

Gli altri due casi sono simili.

LCS basato su programmazione dinamica

Definiamo una tabella per memorizzare la lunghezza dei vari sottoproblemi di LCS, dove $c[i, j]$ è la lunghezza della LCS di $X(i)$ e $Y(j)$, c tabella di mn elementi dove $|X| = m$ e $|Y| = n$.

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ or } j = 0 \\ c[i - 1, j - 1] + 1 & \text{se } x_i = y_j \\ \max(c[i, j - 1], c[i - 1, j]) & \text{altrimenti} \end{cases}$$

Nel *primo* caso, i o j sono uguali a zero, quindi non ci sono elementi in X o Y .

Nel *secondo* caso, i due elementi sono uguali e quindi inserisco quell'elemento nell'LCS.

Nel *terzo* caso, i due elementi sono diversi, e quindi mi calcolo l'LCS nei due casi visti in precedenza per poi considerare la più lunga come LCS.

LCS_lenght(X,m,Y,n)

```
1. for i=0 to m
2.   C[i,0] = 0
3. for j=1 to n    //la cella C[0,0] è già 0
4.   C[0,j] = 0
5. for i=1 to m
6.   for j=1 to n
7.     if X[i] == Y[j]
8.       C[i,j] = C[i-1,j-1] + 1
9.     else
10.      C[i,j] = max(C[i-1,j], C[i,j-1])
11. return C
```

L'algoritmo non dà la LCS, ma restituisce una tabella C il cui ultimo elemento corrisponde alla lunghezza della LCS. Passando questa tabella ad un algoritmo di backtracking possiamo ottenere la LCS.

L'algoritmo passa le due stringhe X e Y, e la loro lunghezza (rispettivamente m e n). Sia la matrice C che i due array X e Y sono 1 based, però per una questione di comodità supponiamo che C abbia una riga ed una colonna extra (riga 0 e colonna 0), e che queste abbiano tutti i valori a 0.

Se $x_i = y_i$ prendiamo il valore in alto a sx ed aggiungiamo 1, altrimenti prendiamo il massimo fra il valore che si trova in alto e fra quello che si trova a sx.

Alla fine dell'algoritmo restituiamo C e poi, attraverso un algoritmo di backtracking, possiamo risalire alla LCS.

LCS_lenght_withPath(X,m,Y,n)

```
1. for i=0 to m
2.   C[i,0] = 0
3. for j=1 to n      //la cella C[0,0] è già 0
4.   C[0,j] = 0
5. for i=1 to m
6.   for j=1 to n
7.     if X[i] == Y[j]
8.       C[i,j] = C[i-1,j-1] + 1
9.       P[i,j] = "\"
10.    else
11.      if C[i-1,j] ≥ C[i,j-1]
12.        C[i,j] = C[i-1,j]
13.        P[i,j] = "|"
14.      else
15.        C[i,j] = C[i,j-1]
16.        P[i,j] = "-"
17. return C
```

Si tratta dello stesso algoritmo di prima, con la differenza che ogni qualvolta prendiamo una scelta salviamo all'interno della matrice P (path) la "direzione" presa. Perché dobbiamo effettuare 2 assegnazione invece di una per ogni scelta, ho dovuto sostituire la funzione max con un if...else, ma la logica rimane la stessa.

Esempio di definizione di una tabella, di dimensione $m \times n$.

Notiamo come la prima colonna e la prima riga siano sempre 0, e questo perché " $i=0$ o $j=0$ ".

Quello che faremo è confrontare il carattere di ogni riga con i caratteri delle varie colonne. Quindi, nel 1 caso, confronteremo F con F, W, A, N, C... fino alla fine della riga, per poi passare a quella successiva, e così via.

Considerati gli indici (i,j) , nella posizione $(1,1)$ notiamo come i due caratteri siano uguali, e quindi applichiamo la seconda condizione.

Ciò significa che la casella di posizione $(1,1)$ avrà come valore " $c[i-1,j-1]+1$ ", ovvero il valore della casella che si trova in alto a sinistra + 1 (saliamo in diagonale).

Tale casella si troverà quindi nella posizione $(0,0)$, e il suo valore è 0. Quindi $c[i-1,j-1] = 0$. A tale valore dobbiamo aggiungere 1, e quindi $(1,1)$ avrà valore 1.

Poi continuiamo a spostarci verso destra (incrementiamo la j). In quest'altro caso i caratteri F e W sono diversi, e quindi dobbiamo applicare la terza condizione, ovvero dobbiamo inserire in tale casella il valore massimo fra $c[i-1,j]$ e $c[i,j-1]$, ovvero il valore massimo fra la casella che sta "sopra" e a "sinistra" della nostra. In questo caso, $c[i,j-1]$ ha valore maggiore e quindi in $c[i,j]$ riporteremo quel valore.

Ripeteremo tale processo fino ad aver completato tutta la tabella.

Da qui otteniamo l'LCS:

Per ottenere l'LCS, dobbiamo considerare l'ultimo passaggio in diagonale che abbiamo fatto, poi andiamo a ritroso fino a quando non raggiungiamo un altro passaggio in diagonale. Considereremo tutte le lettere in cui è avvenuto il passaggio.

(Più schematicamente...):

1. Innanzitutto, consideriamo l'ultimo passaggio in diagonale che abbiamo avuto. Quindi partiremo dall'angolo in basso a destra e verificheremo se qui è avvenuto un passaggio in diagonale. In caso affermativo, risaliremo *diagonalmente* di una posizione, altrimenti dovremo spostarci o verso l'alto, o verso sinistra, in base a quale dei due valori è più grande. Se hanno la stessa dimensione, andremo verso l'alto.
2. Ripeteremo tale processo fintanto che uno fra i o j sia uguale a 0. A questo punto ci basterà considerare solo tutti i caratteri che ci hanno permesso di spostarci in diagonale, in quanto andranno a costituire la nostra LCS.

In questo caso, l'LCS di Francesco è FANCCO.

nb: non è detto che esista una sola LCS (esempio prof).

	y	F	W	A	N	C	I	C	O
x	0	0	0	0	0	0	0	0	0
F	0	1	1	1	1	1	1	1	1
R	0	1	1	1	1	1	1	1	1
A	0	1	1	2	2	2	2	2	2
N	0	1	1	2	3	3	3	3	3
C	0	1	1	2	3	4	4	4	4
E	0	1	1	2	3	4	4	4	4
S	0	1	1	2	3	4	4	4	4
C	0	1	1	2	3	4	4	5	5
O	0	1	1	2	3	4	4	5	6

Metodi alternativi a LCS: Distanza di editing

L'LCS non è l'unico metodo per ottenere la stringa lessicograficamente più vicina rispetto a un'altra. Un ulteriore metodo è rappresentato dalla **distanza di Editing**, la quale può essere calcolata attraverso la *distanza di Hamming* o la *distanza di Levenshtein*:

1. La **distanza di Hamming** misura la distanza bit a bit tra due stringhe.
2. La **distanza di Levenshtein** è rappresentata dal numero di operazioni, fatte da tastiera, per trasformare una stringa in un'altra. Tale operazioni sono: *inserimento*, *cancellazione* e *sostituzione*.
(nb: in alcune versioni, vengono considerate solo inserimento e cancellazione come operazioni elementari, in quanto la sostituzione può essere visto come una combinazione di queste due).

Possiamo vederla come la duale dell'LCS. Infatti, quella che ha l'LCS più lunga ha contemporaneamente la distanza di editing più corta (in quanto abbiamo bisogno di meno operazioni per trasformarla nell'altra stringa).

es. Levenshtein

- FWANCICO
- FRANCICO (sostituzione di W con R)
- FRANCECO (sostituzione di I con E)
- FRANCESCO (inserimento di S)

Totale tre operazioni

Distanza di Levenshtein

Date due stringhe X (di lunghezza m) e Y (di lunghezza n), vogliamo conoscere il numero di operazioni necessarie per trasformare Y in X.

Definiamo $d[i, j]$ la *distanza di editing* tra $X[1...i]$ e $Y[1...j]$, dunque $d[i, j]$ rappresenta la distanza di editing tra una sottostringa di X (che va da 1 a i) e una sottostringa di Y (che va da 1 a j), e dunque rappresenta la soluzione ad *un sottoproblema*.

Di conseguenza $d[m, n]$ rappresenta la soluzione all'intero problema, in quanto stiamo considerando $X[1...m]$ e $Y[1...n]$, ovvero le stringhe per intero.

Questa soluzione verrà costruita individuando la soluzione ottima dei sottoproblemi.

Ogni qualvolta effettuiamo un'operazione (inserimento, cancellazione, sostituzione), aumenta la distanza di editing.

In generale:

- I **casi base** si hanno quando $i=0$ o $j=0$.
Infatti se $i=0$ significa che non ci sono elementi in X, dunque per poter trasformare $Y[1...j]$ in una stringa vuota, dovremo effettuare j cancellazioni.
Invece se $j=0$ significa che non ci sono elementi in Y, dunque per poter trasformare una stringa vuota in $X[1...i]$, dovremo effettuare i inserimenti.
- Se **i due elementi sono uguali** ($X[i] = Y[j]$) non dobbiamo effettuare modifica.
Di conseguenza, la distanza di editing rimane **invariata**.
- Se **i due elementi sono diversi** ($X[i] \neq Y[j]$) dobbiamo effettuare un'operazione.
Di conseguenza, la distanza di editing **aumenta**.

Siccome questo si tratta di un problema di **minimizzazione**, ad ogni passo il valore della cella $d[i, j]$ sarà dato dalla cella adiacente col costo minore, a cui dobbiamo aggiungere anche il costo dell'operazione.

Poniamo il costo di un'operazione di inserimento, cancellazione e sostituzione pari a 1. Nel caso della corrispondenza non effettuiamo alcuna operazione, dunque il costo è 0.

Se il **minimo** è:

- $d[i, j-1] + 1$, si tratta di una **cancellazione**;
- $d[i-1, j] + 1$, si tratta di un **inserimento**;
- $d[i-1, j-1] + \delta(X[i], Y[j])$, si tratta di una **sostituzione/corrispondenza**.

$\delta(X[i], Y[j])$ non è altro che una **variabile** che indica il **costo dell'operazione**.

Può assumere i valori **0** o **1**.

- Se i due elementi $X[i]$ e $Y[j]$ sono **uguali** significa che abbiamo una **corrispondenza**. Non dobbiamo effettuare alcuna operazione, perciò il valore della variabile è **0**.
- Se i due elementi $X[i]$ e $Y[j]$ sono **diversi** significa che NON abbiamo una corrispondenza. Dobbiamo effettuare un'operazione di **sostituzione**, perciò il valore della variabile è **1**.

Dunque, l'**equazione di ricorrenza** per la **distanza di Editing di Levenshtein** è:

$$d[i, j] = \begin{cases} i & \text{se } j = 0 \\ j & \text{se } i = 0 \\ \min \begin{cases} d[i, j-1] + 1 \\ d[i-1, j] + 1 \\ d[i-1, j-1] + \delta(X[i], Y[j]) \end{cases} & \text{altrimenti} \end{cases}$$

dove:

- $\delta(X[i], Y[j]) = 0$, se $X[i] = Y[j]$;
- $\delta(X[i], Y[j]) = 1$, se $X[i] \neq Y[j]$.

LevenshteinEditingDistance(X,m,Y,n)

```

1. for i=0 to m
2.   d[i,0] = i
3. for j=1 to n    //la cella d[0,0] è già 0
4.   d[0,j] = j
5. for i=1 to m
6.   for j=1 to n
7.     if X[i] == Y[j]
8.       delta = 1
9.     else
10.      delta = 0
11.      d[i,j] = min(d[i,j-1]+1, d[i-1,j]+1, d[i-1,j-1]+delta)
12. return d[m,n]
```

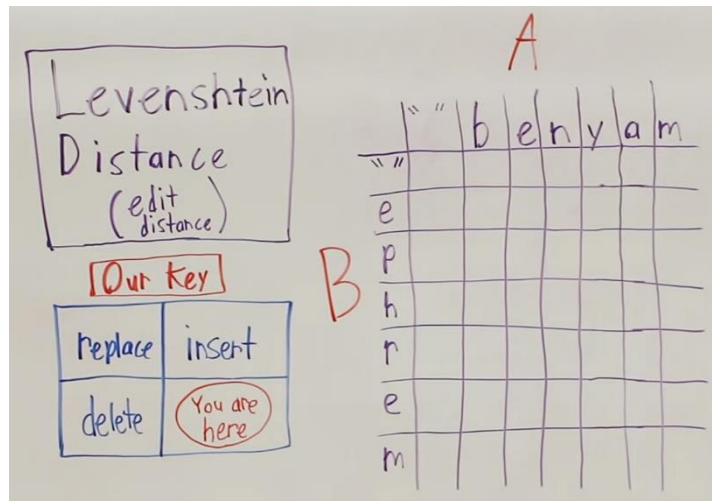
Esempio

Consideriamo due stringhe:

- A = "benyam"
- B = "ephrem"

Una qualsiasi cella di $d[i,j]$ rappresenta la soluzione ad un sottoproblema (ad esempio, la cella (2,3) rappresenta la distanza di editing fra le sottostringhe "be" e "eph").

La cella $d[n,m]$ rappresenta la soluzione all'intero problema (in quanto rappresenta la distanza di editing fra "benyam" e "ephrem").



Le righe $i=0$ e $j=0$ rappresentano le sottostringhe vuote di A e B. Trasformare una stringa vuota in una qualsiasi stringa str comporta effettuare un numero di inserimenti pari alla dimensione della stringa str .

(Ad esempio, trasformare "" in "e" comporta 1 inserimento, in "ep" 2 inserimenti, ... , in "ephram" 6 inserimenti).

	"	"	b	e	n	y	a	m
"	"	0	1	2	3	4	5	6
e	1							
p	2							
h	3							
r	4							
e	5							
m	6							

Nella cella $d[1,1]$, possiamo notare che "b" e "e" sono **diversi**, dunque dobbiamo effettuare un'operazione.

L'operazione da effettuare sarà quella col costo minore.

Relativamente ad una generica cella $d[i,j]$, se consideriamo la cella a:

- sinistra, effettuiamo un'operazione di cancellazione $d[i,j-1]$;
- in alto a sinistra, effettuiamo un'operazione di sostituzione $d[i-1,j-1]$;
- in alto, effettuiamo un'operazione di inserimento $d[i-1,j]$.

Il valore di quelle celle rappresenta la loro distanza di editing.

Il **valore della cella $d[i,j]$** sarà dato dal **minimo di queste 3 celle + 1**.

Nel caso di $d[1,1]$, la cella col valore minore sarà quella in alto a sinistra, dunque $d[0,0] = 0$.

La distanza di editing di $d[1,1]$ sarà data dal valore della cella + 1 = $0 + 1 = 1$.

Le celle $d[1,2]$ sono invece **uguali**, ciò significa che non è necessario effettuare alcuna operazione di editing.

Il **valore della cella $d[i,j]$** sarà data dalla **cella in alto a sinistra** senza **aggiungere nulla** (in quanto non abbiamo fatto alcuna operazione).

E così via, fino alla fine...

La cella $d[m,n]$, cioè $d[6,6]$, rappresenta la distanza di editing fra A e B.

	"	"	b	e	n	y	a	m
"	"	0	1	2	3	4	5	6
e	1	1	1	2	3	4	5	
p	2	2	2	2	3	4	5	
h	3	3	3	3	3	4	5	
r	4	4	4	4	4	4	5	
e	5	5	4	5	5	5	5	
m	6	6	5	5	6	6	5	

LEZ 11 – Programmazione Dinamica (Moltiplicazione di matrici)

Prodotto di due matrici

Possiamo moltiplicare due matrici soltanto se sono compatibili, cioè se il numero di colonne della prima matrice è uguale al numero di righe della seconda matrice.

Se A_{pq} e B_{qr} , allora C_{pr} .

Matrix_Multiply(A,B)

1. **if** $A.columns \neq B.rows$
2. **error** "dimensioni non compatibili"
3. **else** Sia C una nuova matrice $A.rows \times B.columns$
4. **for** $i \leftarrow 1$ **to** $A.rows$
5. **for** $j \leftarrow 1$ **to** $B.columns$
6. $c_{ij} \leftarrow 0$
7. **for** $k \leftarrow 1$ **to** $A.columns$
8. $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$
9. **return** C

Se A è una matrice $p \cdot q$, B è una matrice $q \cdot r$; allora C è una matrice $p \cdot r$. Il tempo per calcolare C dipende dal numero dei **prodotti scalari** pqr .

Il costo per moltiplicare le matrici dipende dalla parentesizzazione.

Prodotto di una sequenza di matrici

Nella moltiplicazione di matrici, data una sequenza di matrici $A_1, A_2, A_3, \dots, A_n$, compatibili due a due, si vuole calcolare il loro prodotto impiegando il numero minore possibile di prodotti scalari.

Infatti, la moltiplicazione di matrici si basa sulla **moltiplicazione scalare** come operazione elementare.

Proprietà delle matrici

Le matrici **godono** della **proprietà associativa**, ovvero in una moltiplicazione con più fattori è possibile associare alcuni termini sostituendoli con il loro prodotto.

Infatti date tre matrici A, B e C , fare $(A \cdot B) \cdot C$ restituisce lo stesso risultato di $A \cdot (B \cdot C)$.

Al contrario **NON godono** della **proprietà cumulativa**, ovvero se cambiamo l'ordine dei fattori non si ottiene lo stesso risultato (se non per casi banali).

Infatti date due matrici A e B , fare $(A \cdot B)$ NON restituisce lo stesso risultato di $(B \cdot A)$.

nb: un esempio di caso banale è moltiplicare una matrice quadrata A per una *matrice identità* I che ha lo stesso ordine di A . In tal caso $A \cdot I = I \cdot A$.

Infatti la matrice identità è l'elemento neutro rispetto al prodotto tra matrici.

NON CONFONDERE il prodotto scalare $(A \cdot B)$ con il prodotto vettoriale $(A \times B)$, che **non esiste** per le matrici.

Il problema della parentesizzazione

Il problema principale di quando si effettua il prodotto di una sequenza di matrici, è che il numero di prodotto da effettuare dipende dall'ordine con cui le matrici stesse sono moltiplicate. Questo problema prende il nome di **parentesizzazione**.

La parentesizzazione migliore è quella che richiede meno prodotti scalari. Occorre quindi determinare il *numero di prodotti scalari* necessari per ogni parentesizzazione, per poi scegliere quella che ne richiede il minor numero.

Un prodotto di matrici si dice **completamente parentesizzato** se consiste di una sola matrice o di un prodotto di matrici completamente parentesizzato.

ESEMPIO: Consideriamo tre matrici A, B, C , rispettivamente di 10×100 , 100×5 , 5×50 .

1. Se moltiplichiamo secondo lo schema $((AB)C)$,
abbiamo $(10 \cdot 100 \cdot 5) + (10 \cdot 5 \cdot 50) = 7500$ prodotti scalari.
2. Se moltiplichiamo secondo lo schema $(A(BC))$,
abbiamo $(100 \cdot 5 \cdot 50) + (10 \cdot 100 \cdot 50) = 75000$ prodotti scalari.

Determinazione del numero delle parentesizzazioni

Data una sequenza di matrici $A_1, A_2, \dots, A_n$, dove ogni matrice A_i ha dimensioni $p_{i-1} \cdot p_i$ per $i = 1, 2, \dots, n$, determinare lo schema di parentesizzazione completa del prodotto $A_1, A_2, \dots, A_n$, che minimizza il numero dei prodotti scalari (*parentesizzazione ottimale*).

(nb: “ $p_{i-1} \cdot p_i$ ” perché così mi accerto che il numero di colonne di ogni matrice sia uguale al numero di righe della successiva.

p è un array di sizes del tipo: $p[] = \{4, 2, 9, 6\}$. Ergo $A_1[4, 2], A_2[2, 9], A_3[9, 6]$.)

Indichiamo con $P(n)$ il numero di parentesizzazioni alternative di una sequenza di n matrici. Quando $n = 1$ abbiamo una sola matrice ed un solo schema di parentesizzazione, quando $n > 1$, un prodotto di matrici completamente parentesizzato è dato dal prodotto di due sottoprodotti di matrici parentesizzati, e la suddivisione tra i due prodotti può avvenire fra la k -esima e la $(k + 1)$ -esima matrice per $k = 1, 2, \dots, n - 1$.

Il numero di parentesizzazioni è dato dalla seguente ricorrenza:

$$P(n) = \begin{cases} 1 & \text{se } n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{se } n > 1 \end{cases}$$

La risoluzione della ricorrenza è data da $P(n) = C(n-1)$, dove $C(n)$ è detto **numero catalano** (dal matematico Catalan) ed è dato da:

$$C(n) = \frac{1}{n+1} \binom{2n}{n} = \Omega\left(\frac{4^n}{n^{\frac{3}{2}}}\right).$$

I primi 10 numeri di Catalan sono: **1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796**.

Ciò significa che se abbiamo 9 matrici, abbiamo 4862 differenti parentesizzazioni.

Per risolvere tale problema dobbiamo quindi avvalerci della Programmazione dinamica, ma per far ciò dobbiamo innanzitutto caratterizzare la sottostruttura ottima (senza la programmazione dinamica, il numero di soluzioni risulta esponenziale).

nb: I numeri di Catalan partono da 0, quindi per $(n=0, 1, 2, \dots)$ avremo 1 ($n=0$), 1 ($n=1$), 2 ($n=2$), 5 ($n=3$), ecc.

Per questo motivo, consideriamo $P(n) = C(n-1)$.

Caratterizzazione della sottostruttura ottima (prodotto matrici)

Fase 1 – Caratterizzazione della sottostruttura ottima

Con la notazione $A_{i..j}$, dove $i \leq j$, indichiamo una parentesizzazione che è il risultato del prodotto $A_i \cdot \dots \cdot A_j$.

Indichiamo con $A_{1..n}$ una matrice che è il risultato fra il prodotto di tutte le n matrici della sequenza. Sapendo che “la soluzione ottima del problema contiene la soluzione ottima dei sottoproblemi”, possiamo suddividere tale problema in due sottoproblemi anche loro ottimi:

- quello della parentesizzazione ottima delle prime k matrici $A_{1..k}$;
- quello della parentesizzazione ottima delle rimanenti $n-k$ matrici $A_{k+1..n}$;

Dal loro prodotto infatti si ottiene la soluzione finale $A_{1..n}$.

Dunque, il numero di prodotti totali che verranno effettuati è dato dal numero di prodotti dato da $A_{1..k}$ (ottima) sommato al numero di prodotti di $A_{k+1..n}$ (ottima), a cui va aggiunto il costo per moltiplicare queste due matrici.

In pratica, stiamo dicendo che se il sottoproblema sinistro è ottimo e il sottoproblema destro è ottimo, allora il loro prodotto darà una soluzione ottima.

Formalmente:

Teorema: se $A_{1..n} = A_{1..k} \cdot A_{k+1..n}$ è una parentesizzazione ottima del prodotto $A_1 \cdot A_2 \cdot \dots \cdot A_n$, allora $A_{1..k}$ e $A_{k+1..n}$ sono parentesizzazioni ottime dei prodotti $A_1 \cdot \dots \cdot A_k$ e $A_{k+1} \cdot \dots \cdot A_n$.

Questo teorema ci dice che ogni soluzione ottima al problema della parentesizzazione contiene al suo interno le soluzioni ottime dei due sottoproblemi.

Fase 2 – Definizione ricorsiva del valore di una soluzione ottima

Per il problema della moltiplicazione di una sequenza di matrici, scegliamo come sottoproblemi i problemi di determinare il costo minimo di una parentesizzazione di $A_i \cdot A_{i+1} \cdot \dots \cdot A_j$.

Sia $m[i,j]$ il numero di minimo dei prodotti necessari a calcolare il prodotto $A_{i..j}$.

Per il problema principale il costo per calcolare $A_{1..n}$ sarà $m[1,n]$.

- Se $i = j$, il problema è banale in quanto la sequenza è $A_{i..i}$, ovvero si tratta di una sola matrice (A_i), quindi non occorre effettuare alcun prodotto scalare. Allora $m[i, i] = 0$ per tutti gli $i = 1, \dots, n$.
- Per calcolare $m[i,j]$ quando $i < j$ il costo sarà ottenuto sfruttando la struttura della parentesizzazione ottima. Supponiamo che la parentesizzazione ottima $A_{i..j}$ si suddivida in due parti, $A_{i..k}$ e $A_{k+1..j}$ dove $i \leq k < j$.

Quindi il costo è dato dalla somma del costo delle p.o. di $A_{i..k}$ e $A_{k+1..j}$, ossia $m[i,k]$ e $m[k+1,j]$, più il costo per moltiplicare $A_{i..k}$ e $A_{k+1..j}$ ossia $p_{i-1}p_kp_j$ (ogni matrice A_i ha p_{i-1} righe e p_i colonne). Quindi: $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$.

Perché $p_{i-1}p_kp_j$?

Abbiamo detto che $m[i,j]$ è dato dal prodotto fra $A_{i \dots k}$ e $A_{k+1 \dots j}$, ovvero:

$$(A_i \cdot \dots \cdot A_k) \cdot (A_{k+1} \cdot \dots \cdot A_j)$$

Ricordiamo che una matrice A_i ha $p(i-1)$ righe e $p(i)$ colonne, avremo:

$$(A_{p(i-1,i)} \cdot \dots \cdot A_{p(k-1,k)}) \cdot (A_{p(k,k+1)} \cdot \dots \cdot A_{p(j-1,j)})$$

Effettuiamo i prodotti delle matrici fra parentesi, ci rimarrà dunque:

$$A_{p(i-1,k)} \cdot A_{p(k,j)}$$

Ricordando che il numero di prodotti da effettuare fra due generiche matrici $A_{p,q}$ e $B_{q,r}$ è pqr , avremo equivalentemente:

$$p_{i-1}p_kp_j$$

Equazione di ricorrenza

Questa equazione presuppone che k sia conosciuto, in realtà k può assumere $j - i$ valori essendo $k = i, i+1, \dots, j-1$. Pertanto dobbiamo esaminarli tutti per ottenere il migliore.

Quindi, l'equazione di ricorrenza per il costo minimo della parentesizzazione di $A_{i \dots j}$ è:

$$m[i,j] = \begin{cases} 0 & \text{se } i = j \\ \min_{i \leq k < j} m[i,k] + m[k+1,j] + p_{i-1}p_kp_j & \text{se } i < j \end{cases}$$

nb: in " $i \leq k < j$ ", abbiamo che j non è compreso perché altrimenti potremmo avere $i=j$, che sarebbe in realtà il caso base.

I valori di $m[i,j]$ non ci forniscono tutte le informazioni per ricostruire la soluzione ottima. Pertanto definiamo $s[i,j]$ come il valore di k in cui è stato suddiviso il prodotto $A_{i \dots j}$ per ottenere una parentesizzazione ottima, ossia il k per cui: $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$.

Riepilogando:

1. Sia $m[i,j]$ il numero di minimo dei prodotti necessari a calcolare il prodotto $A_{i \dots j}$.
2. *Caso base:* $i = j$.
3. *Passo ricorsivo:* $i < j$ (esiste una parentesizzazione ottima).
4. $p_{i-1}p_kp_j$ è il costo per moltiplicare:
 - $A_{i \dots k}$: p_{i-1} righe, p_k colonne.
 - $A_{k+1 \dots j}$: p_k righe, p_j colonne.

Problema della moltiplicazione di matrici (Ricorsivo Divide et Impera)

```
RMC(p,i,j)          //RECURSIVE_MATRIX_CHAIN(p,i,j)
1.  if i=j
2.      return 0
3.  min = ∞
4.  for k=i to j-1
5.      q = RMC(p,i,k) + RMC(p,k+1,j) + pi-1pkpj
6.      if q < min
7.          min = q
8.  return min
```

nb: all'inizio min è ∞ perché rappresenta il valore minimo, e quindi all'inizio vogliamo inizializzarlo al valore più alto possibile.

Essendo un approccio divide et impera puro, non abbiamo bisogno di usare strutture di appoggio per salvare le soluzioni dei sottoproblemi, ed è per questo che non usiamo $m[i, j]$ ma una generica variabile min (che comunque quando verrà restituita alla fine corrisponderà al minimo per $A_i \dots A_j$).

La chiamata iniziale della RMC sarà $RMC(p, 1, n)$.

Ricorrenza RMC

All'inizio avremo $i=1$ e $j=n$. Faremo subito un confronto per verificare se siamo nel caso base (costo 1). Se non siamo nel caso base, dovremo inoltre considerare il costo del ciclo for. Il ciclo for si ripete $n-1$ volte, nel quale dobbiamo considerare il costo delle due chiamate ricorsive, una di dimensione k e l'altra di dimensione $n-k$, oltre che al costo della moltiplicazione (1).

La **ricorrenza** è:

$$T(n) \geq \begin{cases} 1 & \text{se } n = 1 \\ 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) & \text{se } n > 1 \end{cases}$$

Risoluzione ricorrenza RMC:

$$T(n) \geq 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1)$$

Per la proprietà delle sommatorie avremo:

$$T(n) \geq 1 + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k) + \sum_{k=1}^{n-1} 1$$

Le prime due sommatorie sono identiche, solo che effettuano le somme in senso opposto ($T(1) + T(2) + \dots + T(n-1)$ per la prima; $T(n-1) + T(n-2) + \dots + T(1)$ per la seconda). La terza è banale.

$$T(n) \geq 1 + 2 \sum_{k=1}^{n-1} T(k) + (n-1)$$

$$T(n) \geq 2 \sum_{k=1}^{n-1} T(k) + n$$

Dimostriamo $T(n) = \Omega(2^n)$ per sostituzione, ossia che $T(n) \geq c2^n$. Infatti guardando la sommatoria, notiamo che $T(k)$ viene richiamata $n-1$ volte, e la sommatoria viene ripetuta 2 volte.

Sostituendo avremo:

$$T(n) \geq 2c \sum_{k=1}^{n-1} 2^{k-1} + n$$

Facciamo partire questa serie da 0 così da poterla risolvere con la serie geometrica:

$$\text{Ricordando la serie geometrica: } \sum_{k=0}^N x^k = \frac{x^{N+1}-1}{x-1}$$

$$T(n) \geq 2c \frac{2^{n-2+1} - 1}{2 - 1} + n$$

$$T(n) \geq 2c(2^{n-1} - 1) + n$$

$$T(n) \geq c2^n - 2c + n$$

$$T(n) \geq c2^n + (-2c + n)$$

Vogliamo liberarci di quel fastidioso “ $-2c + n$ ”, come ricordiamo possiamo rimuoverlo se è una quantità positiva (vedi “metodo di sostituzione - vademecum”).

$$-2c + n \geq 0$$

$$2c \leq n$$

$$c \leq \frac{n}{2}$$

Quindi: $T(n) = \Omega(2^n)$, per $0 < c \leq \frac{n}{2}$

Notiamo come, avendo utilizzato un approccio Divide et impera, la complessità di tale algoritmo è esponenziale ($T(n) = \Omega(2^n)$), in quanto risolve molti più problemi del necessario.

Infatti, il numero dei problemi distinti è basso, ossia:

$$\binom{n}{2} + n = \frac{n(n-1)}{2} + n = \frac{n(n+1)}{2} = \Theta(n^2)$$

Problema della moltiplicazione di matrici (PD con Tabulation)

La procedura assume che la matrice A_i abbia p_{i-1} righe e p_i colonne.

L'input è una sequenza $p = p_0, p_1, \dots, p_n$.

La procedura usa una tabella ausiliaria $m[1 \dots n, 1 \dots n]$ per memorizzare i costi $m[i,j]$ ed $s[1 \dots n, 2 \dots n]$ che registra l'indice del costo ottimo nel calcolo di $m[i,j]$.

MATRIX_CHAIN_ORDER(p)

```
1.  n = p.length-1
2.  crea tabelle m[1...n, 1...n] e s[1...n-1, 2...n]
3.  for i=1 to n
4.      m[i,i] = 0
5.  for l=2 to n                //l è la lunghezza della catena
6.      for i=1 to n-l+1
7.          j = i+l-1
8.          m[i,j] = ∞
9.          for k=i to j-1
10.             q = m[i,k] + m[k+1,j] + pi-1pkpj
11.             if q < m[i,j]
12.                 m[i,j] = q
13.                 s[i,j] = k
14. return m and s
```

Ho 3 cicli for innestati. Nell'ultimo, se $q < m[i,j]$ significa che ho ottenuto un valore migliore della parentesizzazione per $m[i,j]$, e a questo punto vado a memorizzarmi il k per cui ho ottenuto $m[i,j]$. Quindi mi segnero in $s[i,j]$ il valore k ottimale per cui la matrice $A_{i \dots j}$ viene spezzata in maniera ottima in due parti $A_{i \dots k}$ e $A_{k+1 \dots j}$.

Notiamo inoltre come abbiamo inizializzato a 0 tutte le posizioni ove $i = j$ ($m[i,j] \leftarrow 0$).

nb: la tabella s è una cosa in più che possiamo anche omettere se siamo interessati solo al numero di prodotti.

La complessità di tale algoritmo è: $O(n^3)$.

Problema della moltiplicazione di matrici (PD con Memoization)

La prima cosa che andremo a fare sarà inizializzare tutti i valori di $m[i,j]$ all'infinito. Infatti, se fossero infinito, significa che non sono stati ancora calcolati.

Da qui, poi chiamiamo la LOOKUP-CHAIN, nella quale:

1. controlleremo che $m[i,j]$ non sia stato già calcolato, in caso ritorneremo il valore già calcolato in precedenza.
2. controlleremo se $i = j$, in quanto se fossero uguali ritorneremo 0.
3. Altrimenti, procederemo al calcolo di $m[i,j]$, attraverso delle chiamate ricorsive per $K=i$ a $j-1$, e controllando se il valore $q < m[i,j]$, che sappiamo si verificherà almeno una volta in quanto $m[i,j]$ è $+\infty$.

LOOKUP-CHAIN viene chiamata soltanto quando il valore di $m[i,j]$ non risulta essere già calcolato.

Quindi, non ha più una complessità di tipo esponenziale $\Omega(2^n)$, ma polinomiale $O(n^3)$.

MEMOIZED_MATRIX_CHAIN(p)

```
1. n = p.length-1
2. crea tabella m[1...n,1...n]
3. for i=1 to n
4.     for j=1 to n
5.         m[i,j] = ∞
6. return LC(m,p,1,n)
```

LC(m,p,i,j)

//LOOKUP_CHAIN(m,p,i,j)

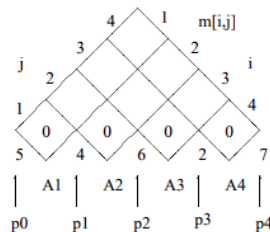
```
1. if m[i,j] < ∞
2.     return m[i,j]
3. if i=j
4.     m[i,j] = 0
5. else
6.     for k=i to j-1
7.         q = LC(m,p,i,k) + LC(m,p,k+1,j) + pi-1pkpj
8.         if q < m[i,j]
9.             m[i,j] = q
10. return m[i,j]
```

ESEMPIO (opzionale, serve per capire il funzionamento)

Sia data una catena A di 4 matrici (A_1, A_2, A_3, A_4) con il seguente array di sizes $p = (5, 4, 6, 2, 7)$, ricordando che p parte da 0. Trovare $m[1, n]$, ovvero la soluzione.

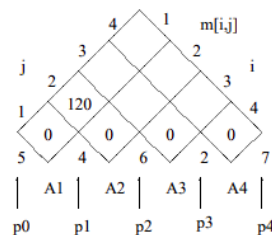
Notare che la matrice sia ruotata in senso antiorario di 45 gradi, assumendo una forma piramidale. Serve solo per indicare che la cella importante è $m[1, n]$ (la punta della piramide), ma possiamo usare anche una rappresentazione “normale”.

S0: Initialization



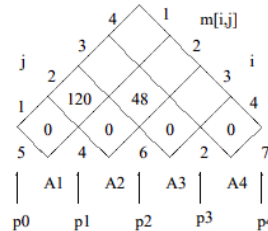
Stp 1: Computing $m[1, 2]$ By definition

$$m[1, 2] = \min_{1 \leq k < 2} (m[1, k] + m[k + 1, 2] + p_0 p_k p_2) \\ = m[1, 1] + m[2, 2] + p_0 p_1 p_2 = 120.$$



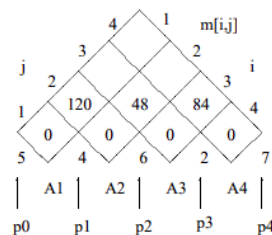
Stp 2: Computing $m[2, 3]$ By definition

$$m[2, 3] = \min_{2 \leq k < 3} (m[2, k] + m[k + 1, 3] + p_1 p_k p_3) \\ = m[2, 2] + m[3, 3] + p_1 p_2 p_3 = 48.$$



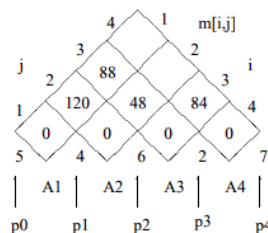
Stp3: Computing $m[3, 4]$ By definition

$$m[3, 4] = \min_{3 \leq k < 4} (m[3, k] + m[k + 1, 4] + p_2 p_k p_4) \\ = m[3, 3] + m[4, 4] + p_2 p_3 p_4 = 84.$$



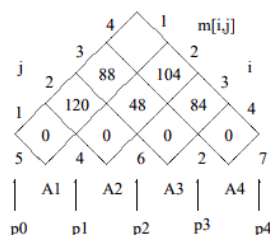
Stp4: Computing $m[1, 3]$ By definition

$$m[1, 3] = \min_{1 \leq k < 3} (m[1, k] + m[k + 1, 3] + p_0 p_k p_3) \\ = \min \left\{ \begin{array}{l} m[1, 1] + m[2, 3] + p_0 p_1 p_3 \\ m[1, 2] + m[3, 3] + p_0 p_2 p_3 \end{array} \right\} \\ = 88.$$



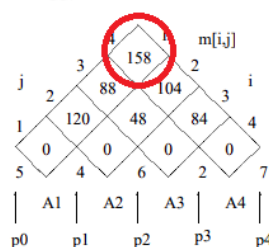
Stp5: Computing $m[2, 4]$ By definition

$$m[2, 4] = \min_{2 \leq k < 4} (m[2, k] + m[k + 1, 4] + p_1 p_k p_4) \\ = \min \left\{ \begin{array}{l} m[2, 2] + m[3, 4] + p_1 p_2 p_4 \\ m[2, 3] + m[4, 4] + p_1 p_3 p_4 \end{array} \right\} \\ = 104.$$



St6: Computing $m[1, 4]$ By definition

$$m[1, 4] = \min_{1 \leq k < 4} (m[1, k] + m[k + 1, 4] + p_0 p_k p_4) \\ = \min \left\{ \begin{array}{l} m[1, 1] + m[2, 4] + p_0 p_1 p_4 \\ m[1, 2] + m[3, 4] + p_0 p_2 p_4 \\ m[1, 3] + m[4, 4] + p_0 p_3 p_4 \end{array} \right\} \\ = 158.$$



ESEMPIO 2 (Abdul Bari)

Stesso ragionamento, ma la matrice m è invece indicata con c , la matrice s è indicata con k , e i sizes p sono invece indicati con d .

Matrix Chain Multiplication

$$C[i, j] = \min_{i \leq k < j} \{ C[i, k] + C[k+1, j] + d_{i-1} \times d_k \times d_j \}$$

$$A_1 \bullet A_2 \bullet A_3 \bullet A_4$$

$$\begin{matrix} 3 & 2 & 2 & 4 & 4 & 2 & 2 & 5 \\ d_0 & d_1 & & d_2 & & d_3 & & d_4 \end{matrix}$$

		j →			
		1	2	3	4
i ↓	1	0			
	2		0		
	3			0	
	4				0

		K			
		1	2	3	4
i	1				
	2				
	3				
	4				

$$C[1, 2] = \min_{1 \leq k < 2} \left\{ C[1, 1] + C[2, 2] + d_0 \times d_1 \times d_2 \right\}$$

$$0 + 0 + 3 \times 2 \times 4$$

$$C[2, 3] = \min_{2 \leq k < 3} \left\{ C[2, 2] + C[3, 3] + d_1 \times d_2 \times d_3 \right\}$$

$$0 + 0 + 2 \times 4 \times 2 = 16$$

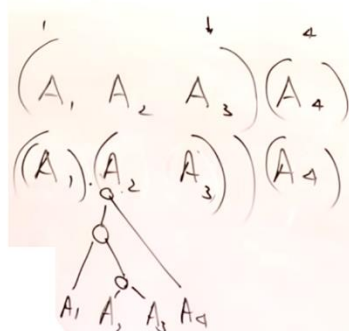
$$C[3, 4] = \min_{3 \leq k < 4} \left\{ C[3, 3] + C[4, 4] + d_2 \times d_3 \times d_4 \right\}$$

$$0 + 0 + 4 \times 2 \times 5 = 40$$

$$C[1, 3] = \min_{1 \leq k < 3} \left\{ \begin{array}{l} k=1: C[1, 1] + C[2, 3] + d_0 \times d_1 \times d_3 = 28 \\ \quad 0 + 16 + 3 \times 2 \times 2 \\ k=2: C[1, 2] + C[3, 3] + d_0 \times d_2 \times d_3 = 48 \\ \quad 24 + 0 + 3 \times 4 \times 2 \end{array} \right.$$

$$C[2, 4] = \min_{2 \leq k < 4} \left\{ \begin{array}{l} k=2: C[2, 2] + C[3, 4] + d_1 \times d_2 \times d_4 = 80 \\ \quad 0 + 40 + 2 \times 4 \times 5 \\ k=3: C[2, 3] + C[4, 4] + d_1 \times d_3 \times d_4 = 36 \\ \quad 16 + 0 + 2 \times 2 \times 5 \end{array} \right.$$

$$C[1, 4] = \min_{1 \leq k < 4} \left\{ \begin{array}{l} k=1: C[1, 1] + C[2, 4] + d_0 \times d_1 \times d_4 = 86 \\ \quad 0 + 36 + 3 \times 2 \times 5 \\ k=2: C[1, 2] + C[3, 4] + d_0 \times d_2 \times d_4 = 124 \\ \quad 24 + 40 + 3 \times 4 \times 5 \\ k=3: C[1, 3] + C[4, 4] + d_0 \times d_3 \times d_4 = 58 \\ \quad 28 + 0 + 3 \times 2 \times 5 \end{array} \right.$$



		j →			
		1	2	3	4
i ↓	1	0	24	28	58
	2		0	16	36
	3			0	40
	4				0

		K			
		1	2	3	4
i	1		1	1	3
	2			2	3
	3				3
	4				

LEZ 12 – Tecniche di programmazione Greedy (Intro e Scheduling)

Algoritmi Greedy

La tecnica Greedy è un approccio utilizzato per la risoluzione di problemi di ottimizzazione, così come la programmazione dinamica. È possibile **applicare** la tecnica Greedy se è possibile dimostrare che **esiste una scelta golosa** e quando il problema ha **sottostruttura ottima**.

Gli algoritmi greedy si basano sulla strategia di prendere quella decisione che, al momento, appare come la migliore. In altri termini, viene sempre presa quella decisione che localmente è la decisione ottima, con la speranza che tale decisione porti ad una soluzione ottima del problema globale.

I problemi che godono della scelta greedy sono problemi che, risolti attraverso algoritmi greedy, portano alla soluzione ottima.

Approccio Greedy vs Programmazione Dinamica

La programmazione dinamica e gli algoritmi greedy vengono usati per risolvere problemi di ottimizzazione per cui vale la proprietà della sottostruttura ottima:

- nella programmazione dinamica, le scelte dipendono dalla soluzione dei sottoproblemi;
- negli algoritmi greedy, le scelte ad ogni passo dipendono da un criterio esterno (**appetibilità**), dove ognuna di queste scelte determina un sottoproblema.

Il motivo per cui vorremo usare la tecnica greedy rispetto alla programmazione dinamica per alcuni problemi, è che talvolta la PD risulta essere inutilmente onerosa per decidere la scelta migliore (hanno normalmente complessità polinomiale), mentre lo stesso risultato può essere raggiunto con algoritmi più semplici ed efficienti (hanno normalmente complessità $n \log n$).

Non sempre è possibile risolvere un problema con un algoritmo greedy o con un algoritmo basato sulla programmazione dinamica (ad es, problema del commesso viaggiatore).

Non sempre problemi che soddisfano la proprietà della sottostruttura ottima possono essere risolti con entrambi i metodi. Ci sono problemi che non possono essere risolti con algoritmi greedy ma possono essere risolti con la programmazione dinamica.

Inoltre, se un dato problema può essere risolto mediante un algoritmo greedy è inutile scomodare la programmazione dinamica.

Proprietà della scelta ottima

Proprietà della scelta golosa: stabilisce che una soluzione globalmente ottima può essere ottenuta facendo scelte localmente ottime.

Quando valutiamo la scelta da fare, facciamo sempre la scelta che sembra migliore per il problema corrente, cioè consideriamo solo il problema corrente senza considerare nessun sottoproblema. Trovata la soluzione localmente ottima, passiamo all'analisi del sottoproblema, sperando che abbia la stessa struttura del problema principale.

Quindi, la scelta fatta da un algoritmo greedy può dipendere solo dalle scelte fatte in precedenza (che determinano il problema corrente) ma non dalle scelte future (soluzioni dei sottoproblemi).

In pratica, l'approccio greedy è una "speranza" e difficilmente porta ad una soluzione ottima. Spesso, però, questo approccio porta a soluzioni più semplici e meno onerose delle soluzioni basate sulla programmazione dinamica.

Elementi della strategia Greedy

I passi per sviluppare un algoritmo greedy sono:

1. Determinare la sottostruttura ottima del problema;
2. Sviluppare una soluzione ricorsiva;
3. Dimostrare che se facciamo la scelta golosa, resta un solo sottoproblema;
4. Dimostrare che la scelta golosa è sempre sicura;
5. Sviluppare un algoritmo iterativo che implementa la strategia greedy.

PROBLEMA DELLA SELEZIONE DI ATTIVITÀ (SCHEDULING)

Sono situazioni del tipo: siano date n attività in competizione tra loro per l'utilizzo di una certa risorsa (es. processi in competizione per la CPU).

Bisogna trovare l'*allocazione ottimale della risorsa*, cioè il più grande sottoinsieme di attività che possono condividere la risorsa senza creare conflitti.

Scheduling – Definizione del problema

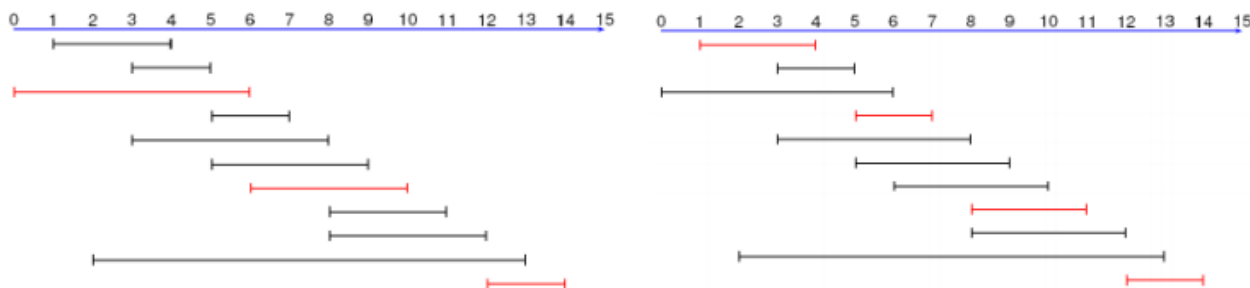
Supponiamo di avere un insieme $S = (a_1, a_2, \dots, a_n)$ di n attività che devono utilizzare la stessa risorsa, ove tale risorsa non può essere utilizzata contemporaneamente da due o più attività.

Ogni attività a_i ha un tempo di inizio s_i ed un tempo di fine f_i , con $0 \leq s_i < f_i < \infty$.

Quando viene selezionata, l'attività a_i si svolge durante l'intervallo semiaperto $[s_i, f_i)$.

Le attività a_i e a_j sono **compatibili** se gli intervalli $[s_i, f_i)$ e $[s_j, f_j)$ non si sovrappongono, ossia se $s_i \geq f_j$ oppure $s_j \geq f_i$.

Il nostro **scopo** consiste nel *selezionare il sottoinsieme che contiene il maggior numero di attività mutuamente compatibili*.



Sottostruttura ottima

Indichiamo con S_{ij} l'insieme delle attività che iniziano dopo la fine delle attività a_i e finiscono prima delle attività a_j .

Supponiamo che A_{ij} sia un insieme di attività mutuamente compatibili in S_{ij} e che includa a_k .

Poiché a_k è nella soluzione ottima, il problema viene suddiviso in **due sottoproblemi**:

1. Trovare le attività mutuamente compatibili nell'insieme S_{ik} (le attività iniziano dopo a_i e finiscono con a_k);
2. Trovare le attività mutuamente compatibili nell'insieme S_{kj} (le attività iniziando dopo a_k e finiscono con a_j).

Siano: $A_{ik} = A_{ij} \cap S_{ik}$ e $A_{kj} = A_{ij} \cap S_{kj}$

Pertanto: $A_{ij} = A_{ik} \cup a_k \cup A_{kj}$

Ossia A_{ik} contiene le attività in A_{ij} che finiscono prima dell'inizio di a_k , e A_{kj} contiene le attività che iniziano dopo la fine di a_k . Pertanto, l'insieme delle attività mutuamente incompatibili A_{ij} , dato dall'unione di A_{ik} con a_k e A_{kj} , avrà un **numero di attività** pari a:

$|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$. (ove 1 sarebbe a_k).

dimostrazione (per assurdo)

Supponendo che A_{ij} è ottima, vogliamo dimostrare che anche A_{ik} e A_{kj} sono ottime. Supponiamo per assurdo che A_{ik} non sia ottima.

Se A_{ik} non è ottima, esisterà pertanto una nuova soluzione A_{ik}^* che avrà un numero di attività maggiore di A_{ik} , ossia $|A_{ik}^*| > |A_{ik}|$.

Ma a questo punto, possiamo costruire una nuova soluzione $A_{ij}^* = A_{ik}^* \cup a_k \cup A_{kj}$ che avrà un numero di attività $|A_{ik}^*| + |A_{kj}| + 1 > |A_{ij}|$, che è assurdo in quanto abbiamo supposto che A_{ij} è ottima. Quindi A_{ik} è necessariamente ottima.

Possiamo dimostrare che anche A_{kj} sia ottima seguendo lo stesso ragionamento.

Equazione di ricorrenza (calcolo del valore della soluzione ottima)

Se indichiamo con $c[i,j]$ la dimensione di una soluzione ottima di S_{ij} otteniamo la ricorrenza: $c[i,j] = c[i,k] + c[k,j] + 1$.

Poiché dobbiamo esaminare tutte le attività in S_{ij} per scegliere a_k , abbiamo:

$$c[i,j] = \begin{cases} 0 & \text{se } S_{ij} = \emptyset \\ \max_{a_k \in S_{ij}} c[i,j] = c[i,k] + c[k,j] + 1 & \text{se } S_{ij} \neq \emptyset \end{cases}$$

Teorema della scelta golosa

Consideriamo un sottoproblema non vuoto S_k e sia a_m l'attività in S_k che ha il primo tempo di fine; allora l'attività a_m è inclusa in qualche sottoinsieme massimo di attività mutuamente compatibili di S_k .

dimostrazione

Supponiamo che A_k sia un sottoinsieme massimo di attività mutuamente compatibili in S_k , e sia a_j un'attività in A_k con il più piccolo tempo di fine.

Possono verificarsi **due situazioni**:

- $a_j = a_m$
- $a_j \neq a_m$

Nella **prima situazione**, essendo $a_j = a_m$, la dimostrazione è immediata. Infatti a_m fa parte di un sottoinsieme massimo di attività mutuamente compatibili.

Nella **seconda situazione**, definiamo un nuovo insieme A'_k , ottenuto rimuovendo a_j ed inserendo a_m :

$$A'_k = A_k - \{a_j\} \cup \{a_m\}$$

Le attività in A'_k sono disgiunte, questo perché anche in A_k le attività sono disgiunte e perché l'attività a_m , inserita al posto di a_j , ha un tempo di fine inferiore di a_j ($f_m \leq f_j$).

Poiché $|A'_k| = |A_k|$, allora A'_k è un sottoinsieme massimo di attività mutuamente compatibili di S_k che include a_m .

Problema della selezione di attività (Greedy Activity Selector)

Il *problema della selezione di attività* consiste nell'individuazione di un sottoinsieme A che contiene il massimo numero di attività mutuamente compatibili.

La **Greedy Activity Selector** è una procedura che suppone di avere le attività ordinate in base al loro tempo di fine, per poi scegliere la prima attività a_1 . A questo punto è possibile eliminare tutte le attività che intersecano con a_1 , cioè quelle che iniziano prima che a_1 finisca. Si ripetono questi passaggi iterativamente anche per le restanti attività.

Greedy_Activity_Selector($s[1..n]$, $f[1..n]$)

1. $n \leftarrow s.length$
2. $A \leftarrow \{a_1\}$
3. $k \leftarrow 1$
4. **for** $m \leftarrow 2$ **to** n
5. **if** $s[m] \geq f[k]$
6. $A \leftarrow A \cup \{a_m\}$
7. $k \leftarrow m$
8. **return** A

Ad ogni passo, l'algoritmo inserisce in A la prima attività compatibile con l'attività appena terminata, cioè quella localmente ottima. In pratica, l'algoritmo seleziona l'attività a_1 , poi la prima attività compatibile con a_1 , poi la prima attività compatibile con quella appena conclusa, e così via.

nb: il GAS produce un sottoinsieme A che contiene il massimo numero di attività mutuamente compatibili, questo significa che non è detto che conterra tutte le attività, e anzi è improbabile che accada.

Es.

i	s_i	f_i
1	1	4
2	3	5
3	0	6
4	5	7
5	3	9

Se seguiamo l'algoritmo, ci renderemo conto come l'insieme S sarà costituito solo da $\{a_1, a_4\}$, che rappresenta appunto il massimo numero di attività mutuamente compatibili. Le altre finirebbero per sovrapporsi.

LEZ 13 – Tecniche di programmazione Greedy (Problema dello zaino)

Problema dello zaino (PD vs Greedy)

Un ladro entra in un magazzino e trova n oggetti; il k -esimo oggetto vale v_k euro e pesa w_k chilogrammi, dove v_k e w_k sono numeri interi. Il ladro vorrebbe realizzare il furto di maggior valore compatibilmente con il peso W del suo zaino. Quali oggetti deve prendere?

Esistono due versioni del problema dello zaino:

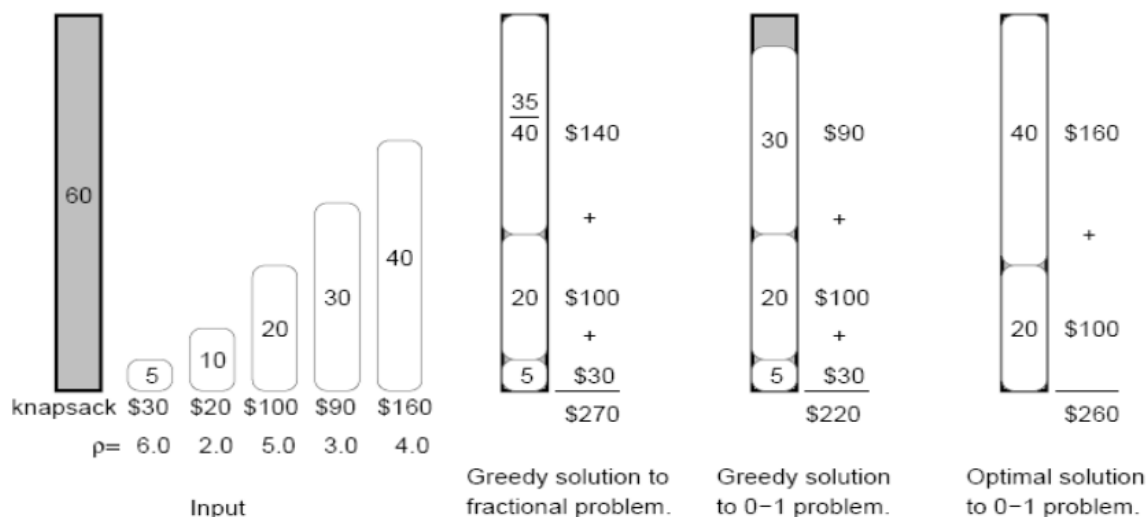
1. Il problema dello zaino 0-1

La dicitura “0-1” indica che ciascun oggetto può essere preso o lasciato; il ladro non può prendere frazioni di oggetti e non può prendere più volte lo stesso oggetto.

2. Il problema dello zaino frazionario

La dicitura “frazionario” indica che il ladro può prendere frazioni di oggetti anziché fare una scelta binaria (0-1).

I problemi sono molto simili ed entrambi verificano la sottostruttura ottima, con la differenza che il problema dello zaino 0-1 non gode della scelta greedy. Infatti, utilizzando la scelta greedy si prendono dallo zaino gli oggetti che hanno il più alto rapporto peso/valore, tuttavia tale ragionamento non funziona con lo zaino 0-1 perché non possiamo prendere frazioni di un oggetto, e non è detto che il valore totale degli oggetti presi fosse il più alto possibile. Per questo motivo, si dice che con l’approccio greedy si ottiene una scelta “quasi” ottimale.



(Utilizzando la tecnica greedy si prendono dallo zaino gli oggetti che hanno il più alto rapporto peso/valore. Prenderemmo quindi il primo, il terzo e il quarto oggetto, andando così a totalizzare un peso totale di 55kg e un valore di 220\$. In pratica, si ottengono soluzioni ,quasi' ottimali con l'approccio greedy.

Con la programmazione dinamica, invece, sarebbero 3 le possibili strade: 35 kg per un valore di 150\$, 60 kg per un valore di 210\$ e 60kg per un valore di 260\$)

ZAINO FRAZIONARIO

La procedura assume che gli oggetti siano ordinati per rapporto $\frac{v_k}{w_k}$.

- Input: $V = \{v_1, v_2, \dots, v_k\}$ e $W = \{w_1, w_2, \dots, w_k\}$
tali che $\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots \geq \frac{v_n}{w_n}$.
- Output: $X[k]$ – ossia la frazione dell'oggetto k da includere; *Value* – il valore totale dello zaino.

In pratica, vado ad inserire nello zaino solo quegli oggetti con il rapporto valore/peso maggiore. Essendo che gli oggetti sono già ordinati per questo rapporto, mi basterà cercare di inserirli partendo dal primo e passando di volta in volta al successivo finché non c'è più spazio nello zaino.

$X[k]$ può assumere **3 valori**:

- $X[k] = 1$: lo zaino riesce a sopportare il peso dell'oggetto che si vuole inserire per intero;
- $X[k] = 0$: lo zaino è completamente pieno, dunque l'oggetto non può essere inserito;
- $0 < X[k] < 1$: lo zaino NON riesce a sopportare il peso dell'oggetto che si vuole inserire per intero, ma è comunque possibile inserire una frazione dell'oggetto; dopo che si è verificata questa situazione, non sarà più possibile inserire oggetti nello zaino in quanto è completamente pieno.

fractional_knapsack(V,W,X,n,TOT)

```
1. for k=1 to n
2.     X[k] = 0.0
3. value = 0.0
4. capacity = TOT
5. k = 1
6. while k ≤ n and capacity > 0.0
7.     if W[k] ≤ capacity
8.         X[k] = 1.0
9.         capacity = capacity - W[k]
10.    else
11.        X[k] = capacity/W[k]
12.        capacity = 0.0
13.    value = value + X[k] * V[k]
14.    k = k+1
15. return value
```

nb: possiamo immaginare come l'array X sia passato per riferimento.

ZAINO 0-1 (Programmazione dinamica)

La programmazione dinamica può risolvere il problema dello zaino 0-1 quando sia i pesi che i valori degli oggetti sono interi, perché se non fossero interi lo spazio dei problemi non sarebbe più polinomiale, e quindi anche se ha la sottostruttura ottima, non può essere risolto con la PD in quanto è improbabile che possa trovare problemi ripetuti (e quindi diventa esponenziale).

(nb: in tal caso, potrà invece essere risolto con tecniche che vedremo più avanti).

Per prima cosa, dobbiamo verificare la *sottostruttura ottima*, e quindi dobbiamo identificare lo *spazio dei sottoproblemi*.

Spazio dei sottoproblemi

Il ladro deve scegliere fra n oggetti tenendo conto di avere uno zaino di capacità W . Il problema originale è quindi $S(n, W)$.

Denotiamo con $S(i, w)$ il sottoproblema. Il ladro può scegliere gli oggetti nel sottoinsieme di A , che va da $a_1 \dots a_i$, per realizzare il furto di maggior valore tenendo conto della capacità residua dello zaino w , dove $1 \leq i \leq n$ e $1 \leq w \leq W$.

Possono verificarsi due **casi banali** in cui non è possibile scegliere l'oggetto:

- $S(0, w)$: l'appartamento è vuoto (non sono presenti oggetti);
- $S(i, 0)$: lo zaino è pieno.

Sottoproblemi generati

Sia $S(i, w)$ un sottoproblema non banale (altrimenti non potremmo scegliere l'oggetto). Esaminiamo il peso w_i dell' i -simo oggetto e consideriamo i seguenti possibili casi:

- se $w_i > w$, l'oggetto a_i non può essere selezionato. Risolviamo quindi il sottoproblema $S(i-1, w)$;
- se $w_i \leq w$, l'oggetto a_i può essere selezionato. Abbiamo due possibili alternative:
 1. **prendiamo** l'oggetto a_i , dunque risolviamo $S(i-1, w-w_i)$;
 2. **ignoriamo** l'oggetto a_i , dunque risolviamo $S(i-1, w)$.

Tra le due, scegliamo quella con valore maggiore.

In ogni caso, una soluzione ottima per $S(i, w)$ deve contenere una soluzione per uno dei sottoproblemi $S(i-1, w-w_i)$ e $S(i-1, w)$.

• **Sottostruttura ottima: 1° caso (prendiamo a_i)**

Sia $A(i,w)$ una soluzione ottima $S(i,w)$ con $i, j > 0$.

Consideriamo il caso:

$A(i,w) = \{a_i\} \cup A(i-1, w-w_i)$, con $A(i-1, w-w_i)$ che è una soluzione ottima per $S(i-1, w-w_i)$. Possiamo dimostrare che sia ottima per assurdo.

Se per assurdo $A(i-1, w-w_i)$ non è ottima, allora deve esistere una soluzione alternativa $A^*(i-1, w-w_i)$ per lo stesso sottoproblema il cui valore è maggiore di quello di $A(i-1, w-w_i)$. È pertanto possibile costruire una nuova soluzione del problema principale:

$A^*(i,w) = \{a_i\} \cup A^*(i-1, w-w_i)$ il cui valore v è maggiore rispetto a quello principale. Infatti:

$$v(A^*(i,w)) = v_i + v(A^*(i-1, w-w_i))$$

$$v(A^*(i,w)) > v_i + v(A(i-1, w-w_i))$$

$$v(A^*(i,w)) = v_i + v(A(i,w))$$

che però **contraddice che $A(i,w)$ sia una soluzione**, e dunque abbiamo dimostrato $A(i-1, w-w_i)$ è ottima.

• **Sottostruttura ottima: 2° caso (ignoriamo a_i)**

Sia $A(i-1,w)$ una soluzione ottima $S(i,w)$ con $i, j > 0$.

Consideriamo il caso in cui **a_i non appartiene all'insieme $A(i,w)$** , quindi $A(i,w)$ contiene al suo interno una soluzione $A(i-1,w)$ per $S(i-1,w)$.

Se per assurdo $A(i-1,w)$ non è ottima, allora deve esistere una soluzione alternativa $A^*(i-1,w)$ per $S(i-1,w)$ il cui valore v è maggiore rispetto a quello principale. Ma:

$$v(A(i,w)) = v(A(i-1,w))$$

//stesso valore perché abbiamo ignorato l'oggetto

però $v(A(i-1,w)) < v(A^*(i-1,w))$

//lo abbiamo stabilito per assurdo

$$v(A^*(i-1,w)) = v(A^*(i,w))$$

che però **contraddice che $A(i,w)$ sia una soluzione**, e dunque abbiamo dimostrato $A(i-1, w)$ è ottima.

Numero dei sottoproblemi

Abbiamo **Wn** sottoproblemi della forma $S(i,w)$, dove $1 \leq i \leq n$ e $1 \leq w \leq W$.

Ricordando che il problema originale è $S(n,W)$, dai casi banali avremo:

- **W** sottoproblemi della forma $S(0,w)$;
- **n** sottoproblemi della forma $S(i,0)$.

In totale, avremo: $Wn + W + n = O(Wn)$ sottoproblemi.

Equazione di ricorrenza (calcolo del valore della soluzione ottima)

Sia $V(i,w)$ il valore della soluzione ottima per il sottoproblema $S(i,w)$, dove $i = 0, \dots, n$ e $w = 0, \dots, W$.

Avremo:

$$V(i,w) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ \max\{V(i-1,w), V(i-1,w-w_i) + v_i\} & \text{se } w_i \leq w \\ V(i-1,w) & \text{altrimenti} \end{cases}$$

Pseudocodice

Knapsack_01(w,v,W,n)

```
1. for w=0 to W
2.   V[0,w] = 0
3. for i=1 to n
4.   V[i,0] = 0
5. for i=1 to n
6.   for w=1 to W
7.     if w[i] ≤ w
8.       V[i,w] = max(V[i-1,w], V[i-1, w-w[i]]+v[i])
9.     else
10.      V[i,w] = V[i-1, w]
12. Return V
```

Nb: “w” è un indice, $w[i]$ rappresenta il peso dell’i-simo elemento, W rappresenta la capacità totale.

S e V non sono la stessa cosa. Usavamo S per fare dei ragionamenti sui sottoproblemi generati, mentre usiamo V per calcolare il valore della soluzione ottima (e infatti in V consideriamo anche il valore di un oggetto “ $v[i]$ ”).

LEZ 14 – Tecniche di programmazione Greedy (Codici di Huffman)

Codici di Huffman

L'algoritmo di Huffman è un *algoritmo greedy*, ed è un metodo usato per la compressione dei dati in quanto consente, a seconda delle caratteristiche del file che si vuole comprimere, un risparmio di spazio che va dal 20% al 90%

Per la compressione di stringhe, Huffman utilizza un **codice a lunghezza variabile**, dove si assegna ai caratteri più frequenti una sequenza binaria più corta, ai caratteri meno frequenti una sequenza binaria più lunga. La particolarità di queste stringhe binarie è che nessuna di esse risulta essere prefisso di un'altra stringa binaria. Huffman è, quindi, un **codice prefisso**.

I codici prefisso possono essere rappresentati mediante alberi binari, dove:

- le foglie sono i caratteri da codificare;
- gli archi che scendono a sinistra hanno etichetta 0;
- gli archi che scendono a destra hanno etichetta 1;
- i percorsi radice-foglia corrispondono alla sequenza codificata;

I caratteri più frequenti si troveranno in alto nell'albero (avranno una minore profondità, meno bit).

I caratteri meno frequenti si troveranno in basso nell'albero (avranno una maggiore profondità, molti bit).

La profondità di una foglia indica la lunghezza della stringa binaria utilizzata per codificare il carattere associato alla foglia.

Per decodificare un carattere basta partire dalla foglia associata a quel carattere e risalire fino alla radice.

ESEMPIO

Con un codice a *lunghezza fissa*, per rappresentare 5 caratteri distinti abbiamo bisogno di 3 bit. Infatti E=(000), F=(001), G=(010), H=(011), I=(100). Per 100.000 caratteri, quindi si ha bisogno di 300.000 bit.

Con un codice prefisso, invece, notiamo che la memoria richiesta è *inferiore*, ed è di 210.000 bit.

3. E: $100.000 * 0.40 * 1\text{bit} = 40.000 \text{ bit}$
4. F: $100.000 * 0.05 * 4\text{bit} = 20.000 \text{ bit}$
5. G: $100.000 * 0.25 * 2\text{bit} = 50.000 \text{ bit}$
6. H: $100.000 * 0.10 * 4\text{bit} = 40.000 \text{ bit}$
7. I: $100.000 * 0.20 * 3\text{bit} = 60.000 \text{ bit}$

= 210.000 bit

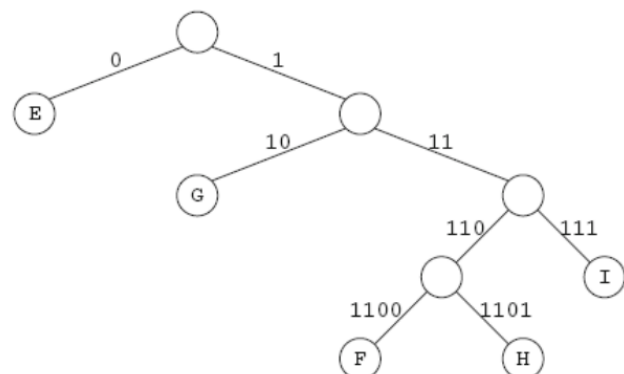


Table 3.1 Frequency of each character in the file

	E	F	G	H	I
Frequency	40 %	5 %	25 %	10 %	20 %
Fix length code	000	001	010	011	100
Huffman code	0	1100	10	1101	111

A partire dall'albero T che rappresenta un codice prefisso, è semplice calcolare il numero di bit necessari per codificare il file.

Per ogni carattere c nell'alfabeto C , indichiamo con:

- $f(c)$ la frequenza del carattere c nel file
- $d_T(c)$ la profondità della foglia etichettata con c ; inoltre esso rappresenta anche la lunghezza della parola che codifica il carattere c .

Il numero di bit necessari per codificare un file risulta:

$$B(T) = \sum_{c \in C} f(c) * d_T(c)$$

dove $B(T)$ rappresenterà il costo dell'albero T , cioè il numero di bit necessari per codificare il file.

COSTRUZIONE DI UN CODICE DI HUFFMAN

1. $\text{Huffman}(C)$
 2. $n \leftarrow |C|$
 3. $Q \leftarrow C$
 4. **for** $i \leftarrow 1$ **to** $n - 1$
 5. $z \leftarrow \text{ALLOCATE_NODE}()$
 6. $x \leftarrow \text{LEFT}[z] \leftarrow \text{EXTRACT_MIN}(Q)$
 7. $y \leftarrow \text{RIGHT}[z] \leftarrow \text{EXTRACT_MIN}(Q)$
 8. $f[z] \leftarrow f[x] + f[y]$
 9. $\text{INSERT}(Q, z)$
 10. **return** $\text{EXTRACT_MIN}(Q)$
- nb: f = frequenza

C rappresenta l'alfabeto (array di caratteri) e n la sua lunghezza. Presupponiamo di inserire tali simboli in una *coda di priorità di minimo*, utilizzando come chiave la *frequenza*. Ad ogni passo creo un nodo, estraggo il minimo e lo metto come figlio sinistro del nodo, poi estraggo di nuovo il minimo e lo metto come figlio destro. Il valore del nodo sarà pari alla somma dei due figli. A questo punto inserisco il nodo nella coda perché per rivalutare la nuova fusione dei due minimi devo considerare anche i nodi. Ripeto tutto finché non sono finiti tutti i caratteri.

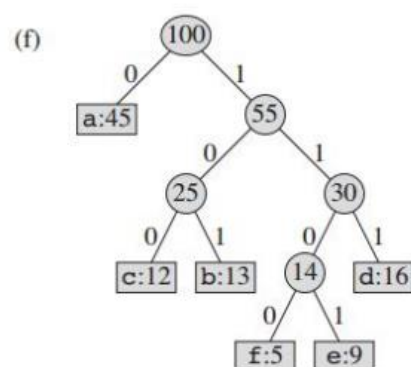
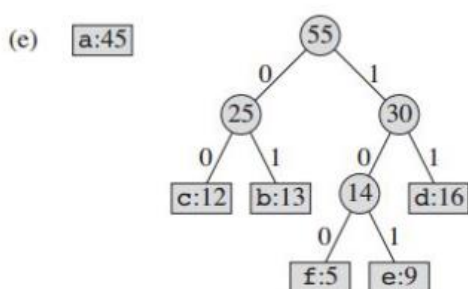
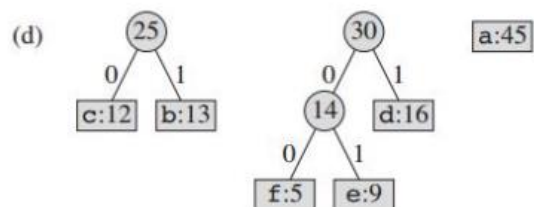
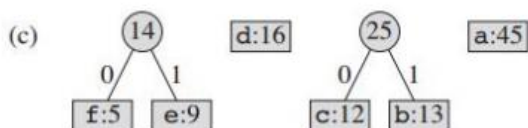
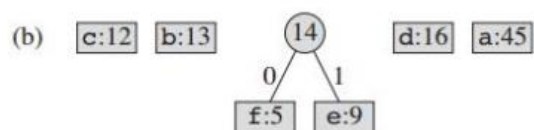
Per l'estrazione dei minimi impiego un tempo $\log n$ (se utilizzo una coda di priorità basata su heap). Il numero dei caratteri è n .

Ergo, la complessità dell'algoritmo è **$O(n \log n)$** .

nb: Sebbene il padre sia sempre maggiore dei nodi figli, *non* si tratta di un heap, in quanto non è completo e non abbiamo il vincolo per cui tutti i nodi terminali siano compresi fra l'ultimo e il penultimo livello.

ESEMPIO DI COSTRUZIONE DI UN CODICE DI HUFFMAN

(a) f:5 e:9 c:12 b:13 d:16 a:45



Codici prefisso

I **codici prefisso** sono dei sistemi di codifica nei quali nessuna stringa binaria è inizio (*prefisso*) di un'altra stringa binaria.

Tali *codici prefisso* hanno delle buone proprietà che permettono di semplificare la codifica e la decodifica:

Per quanto riguarda la codifica, essa risulta essere semplice per un qualunque codice di caratteri binario, in quanto si concatenano le stringhe di bit che rappresentano i caratteri del file.

Per quanto riguarda la decodifica, essa risulta piuttosto semplice nel caso di un codice prefisso. Dato che nessuna stringa binaria del codice è prefisso di una qualunque altra stringa binaria, la stringa con cui inizia il file codificato è identificata senza alcuna ambiguità. Per questo motivo, potremmo identificare la prima stringa del codice codificato, trasformarla nel carattere originario, rimuoverla dal codice codificato e ripetere questo procedimento sulla parte rimanente del file codificato.

Correttezza dell'algoritmo della codifica di Huffman

Per dimostrare che il codice di Huffman è un *algoritmo greedy* dobbiamo verificare la *greedy choice* (scelta golosa) e la *sottostruttura ottima*.

LEMMA (SOTTOSTRUTTURA OTTIMA)

Sia C un alfabeto che definisce una frequenza $f(c)$ per ogni carattere $c \in C$, e siano x e y i due caratteri in C con **frequenza minima**.

Sia C' l'alfabeto ottenuto rimuovendo x e y da C ed aggiungendo il carattere z , ovvero $C' = C - \{x, y\} \cup \{z\}$. per C' , il carattere z avrà $f(z) = f(x) + f(y)$.

Sia T' un albero che rappresenta un **codice prefisso ottimo per C'** .

Allora l'albero T , ottenuto da T' sostituendo z con un nodo interno che ha come figli x e y , è un albero che rappresenta un codice prefisso ottimo per l'alfabeto C .

dimostrazione

Per prima cosa, mostriamo che il costo dell'albero T (cioè $B(T)$) può essere espresso in termini del costo dell'albero T' (cioè $B(T')$):

Per ogni $c \in C'$ avremo che $d_T(c) = d_{T'}(c)$, e quindi $f(c)d_T(c) = f(c)d_{T'}(c)$.

Poiché $d_T(x) = d_T(y) = d_{T'}(z) + 1$ (la profondità di x e y su T è uguale alla profondità di $z + 1$ su T'), avremo:

$$f(x)d_T(x) + f(y)d_T(y) = (f(x) + f(y))(d_{T'}(z) + 1)$$

ricordando che $f(x) + f(y) = f(z)$, avremo:

$$f(x)d_T(x) + f(y)d_T(y) = f(z)(d_{T'}(z) + 1)$$

$$f(x)d_T(x) + f(y)d_T(y) = f(z)d_{T'}(z) + f(z)$$

ovvero:

$$f(x)d_T(x) + f(y)d_T(y) = f(z)d_{T'}(z) + f(x) + f(y)$$

Dalla quale si deduce che il costo dell'albero T (cioè $B(T)$) può essere espresso in termini del costo dell'albero T' (cioè $B(T')$), e infatti:

$$B(T) = B(T') + f(x) + f(y)$$

e viceversa:

$$B(T') = B(T) - f(x) - f(y)$$

Supponiamo **per assurdo** che T non sia un codice prefisso ottimo. Allora deve esistere un albero T'' tale che $B(T'') < B(T)$. L'albero T'' ha x e y come foglie sorelle (hanno stesso padre). Sia T''' l'albero T'' con il padre comune di x e y sostituito da una foglia z , con frequenza $f(z) = f(x) + f(y)$. Allora:

$$B(T''') = B(T'') - f(x) - f(y)$$

$$< B(T) - f(x) - f(y)$$

$$< B(T')$$

che contraddice l'ipotesi per cui T' sia un codice prefisso ottimo per C' .

LEMMA (SCELTA GOLOSA)

Sia C un alfabeto che definisce una frequenza $f(c)$ per ogni carattere $c \in C$, e siano x e y i due caratteri in C con **frequenza minima**.

Allora esiste un codice prefisso ottimo per C in cui le parole in codice per x e y hanno la stessa lunghezza e differiscono soltanto nell'ultimo bit.

dimostrazione

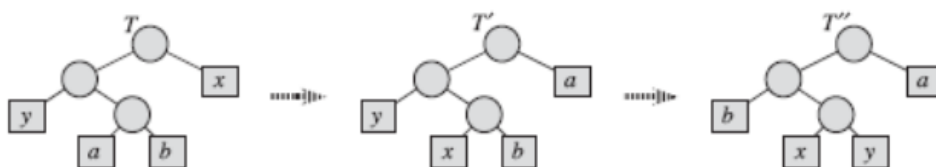
L'idea è di prendere un albero T che rappresenta un arbitrario codice prefisso ottimo e di modificarlo in modo tale da creare un altro albero, che rappresenta un altro codice prefisso ottimo, in modo tale che i caratteri x e y appaiano come foglie sorelle con profondità massima nel nuovo albero.

Se questa trasformazione sarà possibile, allora i codici avranno la stessa lunghezza e differiranno solo nell'ultimo bit.

Siano a e b due caratteri che sono foglie sorelle con profondità massima in T , e supponiamo che $f(a) \leq f(b)$. Supponiamo inoltre che in T ci siano anche i caratteri x e y con $f(x) \leq f(y)$.

Poiché $f(x)$ e $f(y)$ sono minime, abbiamo che: $f(x) \leq f(a)$ e $f(y) \leq f(b)$.

Otteniamo un albero T' scambiando le posizioni di a ed x , ed ancora otteniamo un albero T'' scambiando le posizioni di b e y .



La differenza di costo tra T e T' è:

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c)d_T(c) - \sum_{c \in C} f(c)d_{T'}(c) \\ &= f(x)d_T(x) + f(a)d_T(a) - f(x)d_{T'}(x) - f(a)d_{T'}(a) \\ &= (f(a) - f(x))(d_T(a) - d_T(x)) \geq 0 \end{aligned}$$

Siamo sicuri che il **risultato** sia **non negativo** perché entrambi i fattori sono non negativi, infatti:

- $f(a) - f(x)$ è non negativo perché x è una foglia con frequenza minima;
- $d_T(a) - d_T(x)$ è non negativo perché a è una foglia con profondità massima in T .

Dunque, $B(T') \leq B(T)$.

In maniera analoga, possiamo dimostrare che scambiando y e b , l'albero T'' avrà un costo inferiore di T' , cioè $B(T'') \leq B(T')$.

La **scelta Greedy** sta nel fatto che se scambiamo la foglia con profondità maggiore con quella con frequenza minore, l'albero risultante ha un costo totale minore o uguale a quello dell'albero di partenza.

LEZ 15 – GRAFI e ALGORITMI DI VISITA DI UN GRAFO

GRAFI

Un **grafo** G è una coppia di insiemi (V, E) dove:

- V è l'insieme dei vertici (o nodi);
- E è l'insieme degli archi.

Un **arco** (v, w) è una coppia di vertici appartenenti entrambi a V .

Se $v, w \in V$ sono collegati da un arco, si dice che v è adiacente a w .

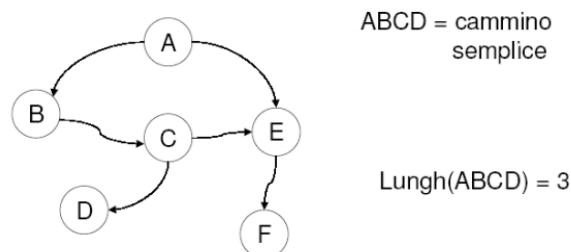
Un grafo si dice **orientato** se presenta archi a cui è assegnato un verso di percorrenza, ove un arco è una *coppia ordinata* (v, w) di nodi.

Un grafo si dice **non orientato** se presenta archi che possono essere percorsi in entrambi i sensi, ove un arco è una *coppia non ordinata* (v, w) di nodi.

Con **cammino di un grafo**, si intende la sequenza di nodi $v_0, v_1, \dots, v_{n-1}$, tali che $(v_0, v_1), (v_1, v_2), \dots, (v_{n-2}, v_{n-1})$ sono archi;

Con **lunghezza del cammino**, si intende il numero di archi del cammino (oppure, il numero di nodi *del cammino* $- 1$);

Con **cammino semplice**, cammino in cui tutti i nodi, eccetto eventualmente il primo e l'ultimo, sono distinti;



Rappresentazione di un grafo

Vi sono due modi per rappresentare un grafo: come *liste di adiacenza* o come *matrice di adiacenza*.

Visita di un grafo

Per visita di un grafo si intende l'attraversamento di tutti i nodi seguendo un particolare criterio. Distinguiamo due tipi di visita:

- in ampiezza: BFS (breadth first search).
- in profondità: DFS (depth first search);

Liste di adiacenza

I nodi di un grafo possono essere elencati mediante liste di adiacenza, in cui ogni nodo è descritto da una lista di elementi contenenti il nome dei nodi ad esso adiacenti.

In pratica c'è un vettore di V liste, una per ogni vertice. Ciascuna lista contiene (puntatori a) tutti i vertici adiacenti a quello associato alla lista.

Matrici di adiacenza

I nodi di un grafo possono essere elencati mediante matrice di adiacenza. Questa matrice conterrà un numero di righe e di colonne pari al numero di vertici del grafo. Gli '1' presenti all'interno della matrice indicano che c'è un collegamento tra i due nodi coinvolti, mentre gli '0' presenti all'interno della matrice indicano che non c'è nessun collegamento tra i due nodi coinvolti.

Se il grafo è non orientato allora la matrice sarà **simmetrica** (il triangolo superiore e quello inferiore coincidono) ed, inoltre, presenterà sulla diagonale principale tutti 0 (il grafo non ha cappi, cioè non ha nodi collegati con se stessi).

Se il grafo è orientato allora la matrice sarà **asimmetrica** e potrebbe non presentare tutti 0 sulla diagonale principale.

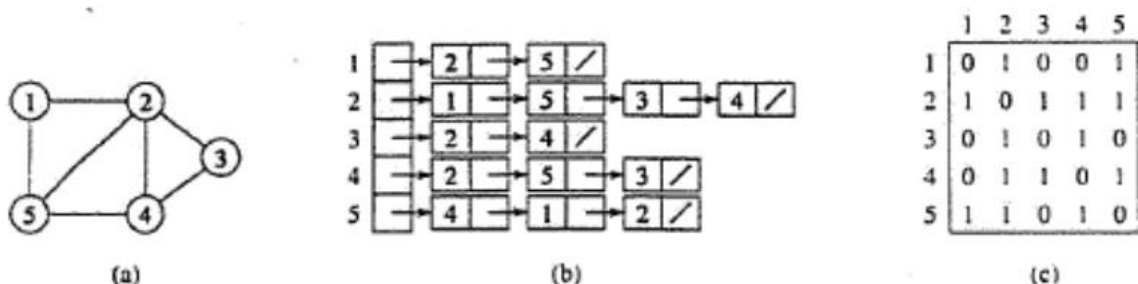


Figura 23.1 Due rappresentazioni di un grafo non orientato. (a) Un grafo orientato G con cinque vertici e sette archi. (b) Una rappresentazione con liste di adiacenza di G . (c) La rappresentazione con matrice di adiacenza di G .

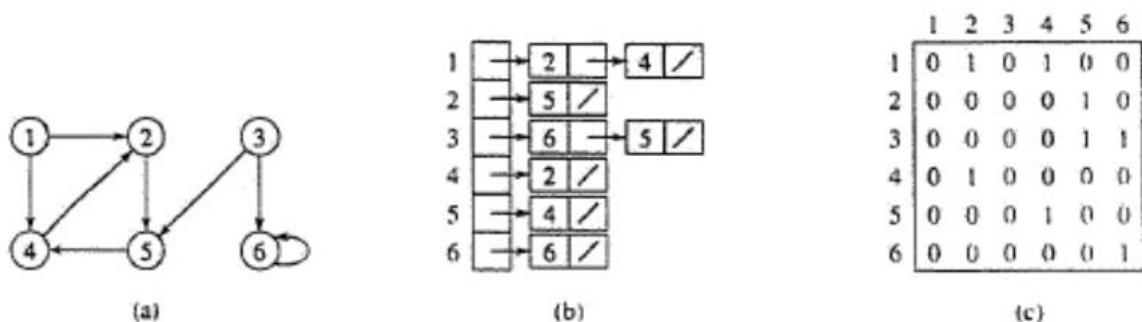


Figura 23.2 Due rappresentazioni di un grafo orientato. (a) Un grafo orientato G con sei vertici ed otto archi. (b) Una rappresentazione con liste di adiacenza di G . (c) La rappresentazione con matrice di adiacenza di G .

Visita in ampiezza – Algoritmo BFS

L'**algoritmo BFS** (Breadth-First Search) è uno degli algoritmi di visita di un grafo (sia orientato che non orientato). Esso visita il grafo in ampiezza.

La visita in ampiezza è chiamata così perché essa espande la frontiera tra i vertici scoperti e quelli ancora da scoprire in modo uniforme lungo l'ampiezza della frontiera stessa.

L'idea è quella di partire da un vertice/nodo sorgente s , per poi esplorare tutti i vertici del grafo, visitando prima i nodi adiacenti di s in modo da esplorare tutti i vertici che hanno distanza k prima di iniziare a visitare quelli che hanno distanza $k+1$.

E' necessario distinguere tra i vertici non visitati, quelli visitati e quelli processati. Per fare ciò si associa ad ogni vertice un **colore** per indicare lo stato dello stesso:

1. **bianco** vertice non ancora visitato
2. **grigio** vertice visitato ed inserito nella coda
3. **nero** vertice processato per cui è terminata la visita

Un vertice viene scoperto quando viene incontrato per la prima volta; una *volta scoperto, cessa di essere bianco*.

Un vertice è nero se *tutti i vertici adiacenti sono stati scoperti*.

Un vertice è grigio se tra i suoi adiacenti ci sono vertici bianchi, ovvero vertici che non sono stati ancora scoperti. Tali vertici grigi rappresentano la frontiera tra i vertici scoperti e quelli da scoprire.

La visita in ampiezza costruisce un albero *BF* che inizialmente è costituito dalla sorgente s che è la radice.

Se un nodo v viene scoperto, durante l'ispezione della lista di adiacenza del nodo u (già scoperto), vengono aggiunti a **BF** il nodo v e l'arco (u, v) . Il vertice u è il padre di v nell'albero *BF*.

Un vertice può avere un solo padre. Se u è lungo il cammino che va dalla sorgente s al nodo v , allora u è un antenato di v e v è un discendente di u .

Si suppone che il grafo sia rappresentato mediante una *lista di adiacenza*.

Esso utilizza una struttura dati CODA, che attua una politica FIFO.

Inoltre, per ogni nodo non visitato:

1. si inserisce il nodo in un coda;
2. si estrae dalla coda un nodo e si visita;
3. si inseriscono in coda tutti gli elementi adiacenti al nodo corrente.

Riassumendo:

Si assume che un grafo G sia rappresentato usando liste di adiacenza.

Per ogni vertice, si vanno a memorizzare le seguenti informazioni:

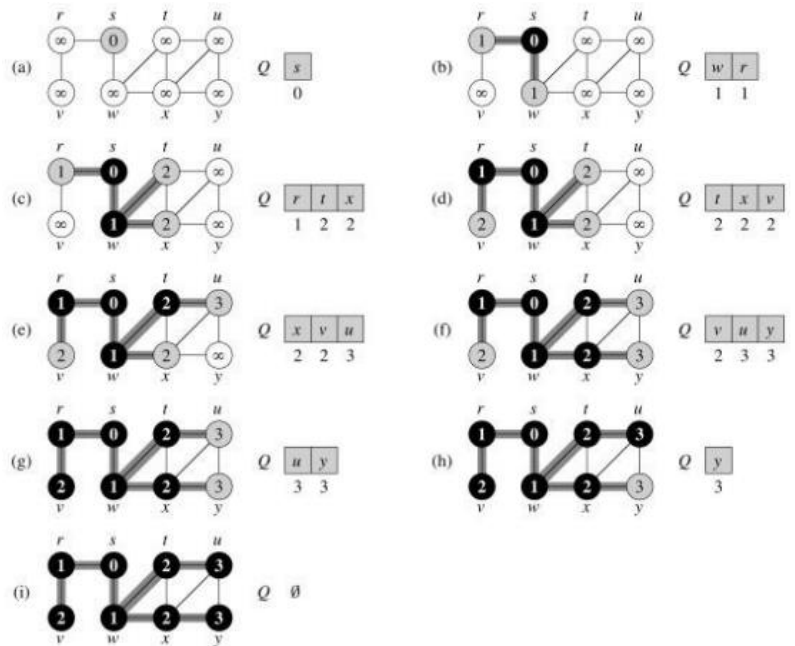
- *colore* $u.color$
- *distanza* $u.d$ //dal nodo sorgente
- *padre* $u.\pi$

Inoltre, la coda Q contiene l'insieme dei vertici di colore grigio.

BFS(G, s)

```

1. for each  $u$  in  $(G.V - \{s\})$ 
2.    $u.color = WHITE$ 
3.    $u.\pi = NIL$ 
4.    $u.d = \infty$ 
5.  $s.color = GRAY$ 
6.  $s.\pi = NIL$ 
7.  $s.d = 0$ 
8.  $Q = \emptyset$ 
9. ENQUEUE( $Q, s$ )
10. while( $Q \neq \emptyset$ )
11.   DEQUEUE( $Q, u$ )
12.   for each  $v$  in  $u.adj$ 
13.     if  $v.color = WHITE$ 
14.        $v.color = GRAY$ 
15.        $v.\pi = u$ 
16.        $v.d = u.d + 1$ 
17.       ENQUEUE( $Q, v$ );
18.    $u.color = BLACK$ ;
```



- Con l'eccezione del vertice sorgente s , le righe 1-4 colorano tutti i vertici di bianco, assegnano alla variabile $d[u] \leftarrow \infty$ e $\pi[u] \leftarrow \text{NULL}$ per ogni vertice u .
- Le righe 6-9 inizializzano $d[s] \leftarrow 0$, assegnano il predecessore della sorgente a NULL ed inizializzano la coda Q in modo che contenga solo la sorgente s .
- Il ciclo while (righe 10-18) si ripete finchè restano vertici grigi.
- Il ciclo for (linee 12-17) esamina ciascun vertice della lista di adiacenza di u . Se è bianco lo colora di grigio, la sua distanza è impostata a $d[u] + 1$, il suo predecessore diventa u , ed infine viene accodato alla coda Q .
- Dopo che **tutti i nodi della sua lista di adiacenza sono stati esaminati**, u diventa **NERO**.

Fondamentalmente, il ragionamento dietro il BFS è questo: dato un certo grafo e tenendo in considerazione una CODA, la prima cosa da fare è partire da uno qualsiasi dei vertici del grafo (non è importante quale) e lo aggiungiamo alla coda. Da qui eseguiremo dei passi ripetitivi: *estriamo* tale elemento dalla coda e iniziamo ad esplorarlo, ovvero visitiamo i nodi adiacenti e li aggiungeremo nella coda. L'ordine di visita non è importante. Una volta che il nodo è stato completamente esplorato, passeremo al prossimo elemento della coda, quindi lo estraiamo e ripetiamo il processo finché non abbiamo visitato tutti i vertici del grafo.

Complessità computazionale della BFS

Le operazioni di inserimento e di cancellazione di un elemento dalla coda è $O(1)$, pertanto la coda in totale richiede $O(V)$.

La lista di adiacenza di ogni vertice è esaminata al più una volta, quando questo viene eliminato dalla coda. Il tempo totale per esaminare tutte le liste di adiacenza è pari alla loro somma, ossia $\Theta(E)$.

Il tempo di esecuzione totale è $O(V + E)$.

PERCORSI MINIMI

Si definisce **distanza di cammino minimo** $\delta(s, v)$ come il **minor numero di archi** in un cammino qualsiasi che va dal vertice s al vertice v .

Se non esiste alcun cammino che va da s a v , allora $\delta(s, v) = \infty$.

Un **cammino di lunghezza** $\delta(s, v)$ è detto **cammino minimo** da s a v .

nb: $d[v]$ è la distanza (qualsiasi) fra v e s , mentre $\delta(s, v)$ è la distanza **minima** fra v e s .

LEMMA 1 (disuguaglianza triangolare)

Dato un grafo $G = (V, E)$ e un vertice qualsiasi $s \in V$.

Allora per ogni arco $(u, v) \in E$ vale:

$$\delta(s, v) \leq \delta(s, u) + 1$$

dim.

- Se u è un vertice raggiungibile da s , anche v lo è. Pertanto il cammino da s a v non può essere maggiore del cammino da s ad u più 1, ossia l'arco (u, v) .
- Viceversa, se u NON è raggiungibile, allora $\delta(s, u) = \infty$ e quindi la disuguaglianza è valida nel caso che v sia raggiungibile da s , sia che non lo sia.

LEMMA 2 (del limite superiore)

Dato un grafo $G = (V, E)$ e un vertice qualsiasi $s \in V$,
supponiamo di eseguire BFS(G, s).

Allora al termine della procedura, per ogni vertice $v \in V$, il valore $d[v]$ soddisfa la relazione:

$$\delta(s, v) \leq d[v]$$

dim.

Si dimostra per **induzione**:

- **caso base:** si ha quando la sorgente s è stata inserita nella coda.
L'ipotesi è verificata perché si ha che:
 - $d[s] = \delta(s, s) = 0$
 - $d[v] = \infty \geq \delta(s, v)$ per ogni $v \in V - \{s\}$.
- **passo induttivo:** supponiamo che a partire dal vertice u viene scoperto un vertice bianco v . L'ipotesi induttiva implica che $\delta(s, u) \leq d[u]$.
Per l'ipotesi induttiva e per il Lemma 1 avremo che:

$$d[v] = d[u] + 1 \geq \delta(s, u) + 1 \geq \delta(s, v).$$

Il vertice v sarà inserito nella coda e non sarà più reinserito, visto che viene colorato di grigio. Pertanto il valore di $d[v]$ non cambierà più e l'ipotesi resterà valida.

LEMMA 3

Dato un grafo $G = (V, E)$ e un vertice qualsiasi $s \in V$,
supponiamo di eseguire $\text{BFS}(G, s)$,
e supponiamo che la coda Q contenga i vertici $[v_1, v_2, \dots, v_r]$, dove v_1 è l'inizio della coda e v_r è la fine. **Allora** valgono:

$$d[v_r] \leq d[v_1] + 1$$

$$d[v_i] \leq d[v_{i+1}] \quad \text{per } i = 1, \dots, r-1$$

dim.

Si dimostra per **induzione**:

- **caso base**: si ha quando la sorgente s è stata inserita nella coda.
Visto che la coda contiene solo s , il lemma è valido.
Infatti: $d[s] \leq d[s] + 1$ con $v_r = v_1$
- **passo induttivo**: si deve dimostrare che il lemma è valido dopo l'inserimento e la cancellazione di un vertice della coda.
 - se il vertice v_1 viene **eliminato** dalla coda, v_2 è la nuova testa della coda.
Per l'ipotesi induttiva abbiamo che: $d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1$.
Il lemma è dimostrato se v_2 è l'inizio della coda.
 - Se un vertice v viene **inserito** nella coda, questo diventa v_{r+1} .
Viene eliminato u dalla coda, la cui lista di adiacenza è attualmente presa in esame.
 - Se u era l'unico elemento della coda, la sua rimozione svuota la coda, dopodiché ogni vertice v avrà lo stesso valore $d[v] = d[u] + 1$ e le disuguaglianze sono verificate.
 - Altrimenti per l'ipotesi induttiva il nuovo inizio di coda ha $d[v_1] \geq d[u]$.
Quindi: $d[v_{r+1}] = d[v] = d[u] + 1 \leq d[v_1] + 1$.
Per l'ipotesi induttiva si ha: $d[v_r] \leq d[u] + 1 = d[v] = d[v_{r+1}]$
Il lemma è dimostrato quando v è inserito nella coda.

COROLLARIO: Supponiamo che i vertici v_i e v_j siano inseriti nella coda durante l'esecuzione di BFS e che v_i sia inserito prima di v_j . Allora $d[v_i] \leq d[v_j]$.

Teorema della Correttezza della Visita in Ampiezza

Sia $G = (V, E)$ un grafo e supponiamo che la procedura *BFS* venga eseguita sul grafo G da un dato vertice sorgente $s \in V$. Allora, durante la sua esecuzione, *BFS* scopre tutti i vertici $v \in V$ che sono raggiungibili dalla sorgente s , e alla fine dell' esecuzione, $d[v] = \delta(s, v)$ per ogni $v \in V$. Inoltre, per qualsiasi vertice $v \neq s$ che è raggiungibile da s , uno dei cammini minimi da s a v è un cammino minimo da s a $\pi[v]$ seguito dall' arco $(\pi[v], v)$.

dimostrazione

Supponiamo **per assurdo** che esista un qualche nodo $v \in V$ per cui $d[v] \neq \delta(s, v)$.

Per il Lemma 2 sappiamo che $\delta(s, v) \leq d[v]$, ma avendo supposto che $d[v] \neq \delta(s, v)$ allora $\delta(s, v) < d[v]$. Il vertice deve essere raggiungibile altrimenti si avrebbe l'assurdo $\delta(s, v) = \infty \geq d[v]$.

Sia u il padre di v in un cammino minimo da s a v , con $\delta(s, v) = \delta(s, u) + 1$.

Mettendo insieme queste proprietà, abbiamo che: $d[v] > \delta(s, v) = \delta(s, u) + 1 = d[u] + 1$.

Consideriamo il momento in cui il *BFS* elimina il vertice v dalla coda:

1. Se v è bianco allora *BFS* imposta $d[v] = d[u] + 1$, che contraddice l' affermazione iniziale.
2. Se v è nero, per il corollario, il *BFS* imposta $d[v] \leq d[u]$, che pure contraddice l'affermazione iniziale.
3. Se v è grigio, allora è stato colorato dopo la cancellazione dalla coda di qualche vertice w (che era stato cancellato prima di u) e per il quale $d[v] = d[w] + 1$. Ma per il corollario $d[w] \leq d[u]$ e quindi $d[v] \leq d[u] + 1$ che pure contraddice l'affermazione iniziale.

Pertanto, concludiamo che $d[v] = \delta(s, v)$ per ogni $v \in V$.

Tutti i vertici che sono raggiungibili da s devono essere scoperti, altrimenti avrebbero il d infinito.

nb: Infine osserviamo che se vale $\pi[v] = u$ allora $d[v] = d[u] + 1$. Quindi si può ottenere un cammino minimo da s a v prendendo un cammino minimo da s a $\pi[v]$ e poi attraversando l' arco $(\pi[v], v)$.

Visita in profondità – Algoritmo DFS

La strategia seguita dalla visita in profondità (depth-first-search) consiste nell' esplorare il grafo (che sia orientato o non orientato) andando ad ogni istante il più possibile in profondità.

Nella DFS gli archi vengono scoperti a partire dall'ultimo vertice scoperto che abbia ancora vertici non scoperti uscenti da esso. Quando tutti gli archi uscenti da v sono stati scoperti, la visita torna indietro per esplorare gli archi uscenti dal vertice dal quale v era stato scoperto.

Il processo continua fino a scoprire tutti i vertici che sono raggiungibili dal vertice sorgente originario. Se rimane qualche vertice non ancora scoperto, viene selezionato uno di essi e diventa la nuova sorgente e la ricerca riparte da esso.

L'intero processo viene ripetuto sino a quando tutti i vertici del grafo non sono stati scoperti.

I vertici possono essere colorati di bianco (non ancora scoperto), grigio (scoperto) e nero (processato). All'inizio tutti i vertici sono bianchi, e successivamente possono diventare grigi e poi neri.

Questo algoritmo può restituire una foresta di diversi **alberi DF**, in quanto se il grafo è sconnesso, si avrà per ogni sottografo un albero DF, ognuno con la propria radice (sorgente). Questi alberi saranno disgiunti.

Come nella ricerca in ampiezza, quando un vertice v viene scoperto durante la scansione di una lista di adiacenza di un vertice u già scoperto, la visita in profondità memorizza questo evento assegnando u al campo padre $\pi[v]$ di v .

L'algoritmo genera un **sottografo dei predecessori**, indicato con $G_\pi = (V, E_\pi)$, dove:

$$E_\pi = \{(\pi[v], v) : v \in V \text{ and } \pi[v] \neq NIL\}$$

Gli archi sono detti **archi d'albero**.

Oltre al colore, ad ogni vertice v si associano due **informazioni temporali**:

- *tempo di inizio visita* $d[v]$, cioè quando è reso grigio per la prima volta;
- *tempo di fine visita* $f[v]$, cioè quando è reso nero.

Ciò significa, che un vertice u è:

- *bianco* prima di $d[v]$;
- *grigio* fra $d[v]$ e $f[v]$;
- *nero* dopo $f[v]$.

Un vertice può essere scoperto e visitato completamente una sola volta.

Dunque, si avrà sempre: $1 \leq d[v] < f[v] \leq 2|V|$.

DFS(G)

```
1. for each u in G.V
2.     u.color = WHITE
3.     u.π = NIL
4. time = 0
5. for each u in G.V
6.     if u.color = WHITE
7.         DFS_VISIT(u);
```

DFS_VISIT(u)

```
1. u.color = GRAY
2. u.d = time = time + 1
3. for each v in u.adj
4.     if v.color = WHITE
5.         v.π = u
6.         DFS_VISIT(v);
7. u.color = BLACK
8. u.f = time = time + 1
```

- Nelle righe 1-3, si inizializza ogni vertice del grafo settando il colore a bianco e il padre a NULL.
- Nella riga 4, si inizializza la variabile globale *tempo* a 0.
- In questo ciclo eseguiamo la procedura DFS-Visit su ogni nodo non ancora scoperto del grafo, creando un albero DF ogni volta che viene invocata la procedura.

nb: le righe 2 e 8 della DFSvisit sono in realtà 2 istruzioni in una, in quanto prima incrementano *time* e poi lo assegnano a *u.d* (o *u.f*).

spiegazione DFS

In ogni chiamata della procedura, il vertice *u* è inizialmente bianco. La prima operazione che si compie è quella aggiornare il colore (settrandolo a grigio) e il tempo di inizio visita (settrandolo pari a *tempo*+1).

Successivamente, vengono esaminati tutti gli archi uscenti dal vertice *u*, cioè tutti i vertici adiacenti, e per ciascuno di essi si controlla il colore. Se il vertice adiacente *v* che si sta considerando è bianco (vuol dire che non è stato ancora esplorato), allora viene aggiornato il padre di *v* (impostando il vertice *u* come padre) e viene invocata ricorsivamente la procedura DFS-Visit passandogli il vertice *v*.

Dopo aver visitato tutti i nodi adiacenti, *u* viene colorato di nero e viene aggiornato il tempo di fine visita (settrandolo pari a *tempo*+1).

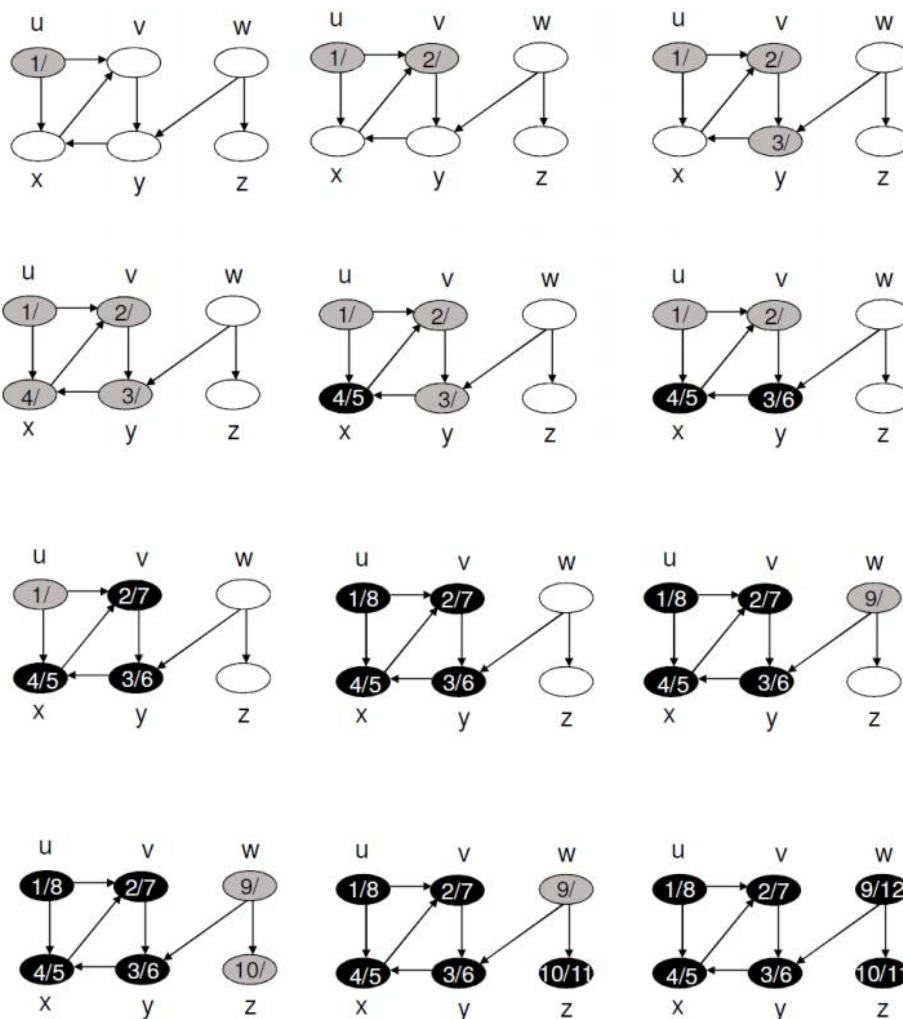
complessità computazionale dalla DFS

I cicli della DFS, a eccezione della DFS_visit, impiegano entrambi un tempo $\Theta(V)$, cioè nell'ordine del numero di vertici.

DFS_visit viene chiamata solo una volta per ogni vertice. Il ciclo *nella* DFS_visit costa $\Theta(\text{Adj}[v])$, ma $\text{Adj}[v]$ non è altro che la lista di adiacenza, e la somma delle liste di adiacenza è uguale a E , quindi il costo è $\Theta(E)$.

La complessità generale dell'algoritmo è $\Theta(V+E)$.

ESEMPIO



Teorema della Parentesi

In ogni DFS di un grafo $G = (V, E)$, per ogni coppia di vertici u e v , può essere soddisfatta solo una delle **tre condizioni**:

- $[d[v], f[v]]$ è **contenuto** in $[d[u], f[u]]$; inoltre v è un discendente di u in un *albero DF*.
($d[u] < d[v] < f[v] < f[u]$)
- $[d[u], f[u]]$ è **contenuto** in $[d[v], f[v]]$; inoltre u è un discendente di v in un *albero DF*.
($d[v] < d[u] < f[u] < f[v]$)
- Gli intervalli $[d[u], f[u]]$ e $[d[v], f[v]]$ sono **disgiunti**; inoltre u e v non sono discendenti l'uno dell'altro nella *Foresta DFS*.

(nb: 1° condizione sarebbe tipo con u e v nel grafo di prima, la 2° sarebbe il contrario, mentre la 3° sarebbe tipo fra u e w . Nessuna di queste condizione può verificarsi insieme data una coppia (u, v)).

dimostrazione

(NB: Tali concetti sono più semplici rispetto al caso della BFS, in quanto non sono legati al concetto dei cammini minimi e della sottostruttura ottima).

- Iniziamo con il caso in cui $d[u] < d[v]$. Ci sono due sottocasi da considerare a seconda se $d[v] < f[u]$.
 1. Nel primo sottocaso $d[v] < f[u]$: si verifica quando v è stato scoperto mentre u era grigio. Quindi v è un discendente di u , e la visita di v deve essere completata prima che sia finita la visita di u , e pertanto $[d[v], f[v]]$ è contenuto in $[d[u], f[u]]$.
 2. Nel secondo sottocaso $d[v] > f[u]$, allora i due intervalli sono disgiunti.
- Il caso $d[v] < d[u]$ è analogo al caso $d[u] < d[v]$.

COROLLARIO (annidamento degli intervalli dei discendenti)

Il vertice v è un discendente proprio del vertice u nella *foresta DF* per un grafo $G = (V, E)$, se e soltanto se $d[u] < d[v] < f[v] < f[u]$.

La dimostrazione è conseguenza del teorema delle parentesi.

Teorema del Cammino bianco

In una *foresta DFS* di un grafo $G = (V, E)$ il vertice v è un discendente di u se e solo se al tempo $d[u]$ in cui si scopre u , il vertice v è raggiungibile da u lungo un cammino contenente esclusivamente vertici bianchi.

dimostrazione

- Supponiamo che v sia un discendente di u . Sia w un qualsiasi vertice lungo il cammino che è tra u e v nell' albero *DFS*. Quindi w è un discendente di u e per il corollario precedente $d[u] < d[w]$ quindi w è bianco al tempo $d[u]$.
- Supponiamo che il vertice v sia raggiungibile da u lungo un cammino di vertici bianchi al tempo $d[u]$. Supponiamo che lungo il cammino tutti i vertici diventino discendenti di u .
- Sia w il predecessore di v , e ciò implica, per il corollario, che $f[w] \leq f[v]$.

Quindi v deve essere scoperto prima che sia ultimata la visita di w e perciò deve essere $d[u] < d[v] < f[w] < f[u]$.

Il teorema della parentesi implica che $d[v], f[v]$ sia contenuto in $[d[u], f[u]]$ e quindi per il corollario precedente v è discendente di u .

LEZ 16 – ARCHI e CONNESSIONE DI UN GRAFO

Classificazione degli Archi

La DFS può essere utilizzata per classificare gli archi del grafo di input $G = (V, E)$. Questa classificazione può essere utilizzata per raccogliere informazioni importanti su un grafo. Ad esempio un grafo orientato è **aciclico** se e solo se una visita in profondità non genera archi all'indietro.

Possiamo definire quattro tipi di archi in base alla foresta DF prodotta da una visita in profondità dal grafo G : *archi d'albero*, *archi all'indietro*, *archi in avanti* e *archi trasversali*.

1. **Archi d'albero**: sono gli archi nella foresta DF. L'arco (u, v) è un arco d'albero se v viene scoperto per la prima volta durante l'esplorazione di (u, v) . (insomma, quelli che collegano un vertice grigio ad un vertice bianco).
2. **Archi all'indietro**: sono quegli archi (u, v) che collegano un vertice u ad un antenato v in un albero DF.
I *cappi*, che possono presentarsi nei grafi orientati, sono considerati archi all'indietro.
3. **Archi in avanti**: sono quegli archi (u, v) (che non sono archi d'albero) che collegano un vertice u ad un discendente v in un albero DF.
4. **Archi trasversali**: sono tutti gli altri archi: possono connettere vertici nello stesso albero DF, purché un vertice non sia antenato dell'altro, oppure possono connettere vertici in alberi DF distinti.

Colore degli Archi

L' algoritmo DFS possiede quindi le informazioni necessarie per classificare gli archi che incontra.

L' idea chiave è che ogni arco (u, v) può essere classificato in base al *colore del vertice* v che viene raggiunto quando l' arco viene ispezionato la prima volta:

- BIANCO: indica un **arco d'albero**.
- GRIGIO: indica un **arco all'indietro**.
- NERO: indica un **arco in avanti** o **trasversale**.

dimostrazione

1. Il primo caso è immediato dalla specifica dell'algoritmo;
2. Per il secondo caso, i vertici grigi formano sempre una catena lineare di discendenti che corrisponde alle chiamate attive della DFS_VISIT. L'ispezione dell'arco parte sempre dal vertice grigio più profondo, quindi: *un arco, partente da un vertice grigio, che raggiunge un altro vertice grigio, raggiunge un antenato*;
3. Per il terzo caso, si può dimostrare che un certo arco (u, v) è un arco in avanti se $d[u] < d[v]$ ed un arco trasversale se $d[u] > d[v]$.

TEOREMA (sugli archi)

In una visita in profondità in un grafo non orientato G , gli archi di G possono essere archi d'albero o archi all'indietro.

dimostrazione

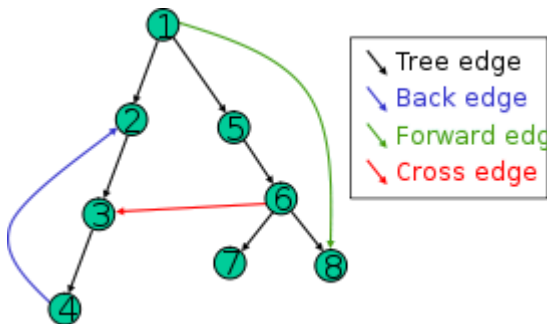
Sia (u, v) un arco arbitrario di G e supponiamo che $d[u] < d[v]$.

Allora, il vertice v deve essere scoperto e completato prima che sia completato u (mentre u è grigio), perchè v si trova nella lista di adiacenza di u .

1. Se l'arco (u, v) viene ispezionato prima nella direzione da u a v , allora v è un vertice non scoperto (bianco) e l'arco è un arco d'albero.
2. Se l'arco (u, v) viene ispezionato prima nella direzione da v a u , allora u è un vertice grigio e l'arco è un arco all'indietro.

nb: in questo caso, consideriamo l'arco (u,v) come un unico arco e non, come in una versione implementativa, due archi (u,v) e (v,u) . Infatti in tal caso potremmo avere degli archi in avanti.

Esempio di esplorazione di un grafo (1)

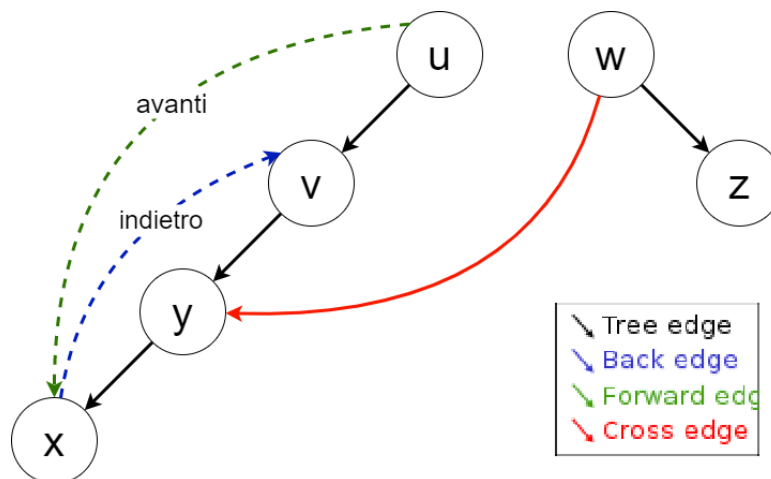


Esempio di esplorazione di un grafo, nel dettaglio (2)

Dato il seguente grafo...



Grafo costruito dall'esplorazione



Andando passo per passo...

- (u,v) : u è un vertice grigio che scopre un vertice bianco v (ovvero, v viene scoperto per la prima volta dall'esplorazione di u). Arco d'albero.
- (v,y) : v è un vertice grigio che scopre un vertice bianco y . Arco d'albero.
- (y,x) : y è un vertice grigio che scopre un vertice bianco x . Arco d'albero.
- (x,v) : x è un nodo grigio che scopre un nodo grigio v , dunque x raggiunge un suo antenato (v , per l'appunto). Arco all'indietro.

x non ha più nodi esplorabili, dunque diventa nero e risaliamo a y . Nemmeno y ha più nodi esplorabili, così come v , dunque diventano neri e risaliamo a u .

- (u,x) : u è un nodo grigio collegato ad un nodo nero x . Può trattarsi sia di un arco in avanti che di un arco trasversale. Nello specifico:
 - o se $d[u] < d[x]$ arco in avanti;
 - o se $d[u] > d[x]$ arco trasversale:

Se vediamo le distanze (di inizio), ci rendiamo conto che $d[u] = 1$, mentre $d[x] = 4$. $d[u] < d[x]$, perciò si tratta di un arco in avanti.

u non ha più nodi esplorabili, dunque diventa nero. u non ha più vicini, ma i nodi non sono finiti, dunque selezioniamo un altro nodo, ad esempio w .

- (w,y) : w è un nodo grigio collegato ad un nodo nero y . Può trattarsi sia di un arco in avanti che di un arco trasversale.
Se vediamo le distanze (di inizio), ci rendiamo conto che $d[w] = 9$, mentre $d[y] = 3$. $d[w] > d[y]$, perciò si tratta di un arco trasversale.
- (w,z) : w è un nodo grigio che scopre un nodo bianco z . Arco d'albero.

CONNESSIONE DI UN GRAFO

In un grafo orientato $G=(V,E)$, due vertici u e v sono **connessi** se in G esiste un **cammino tra u e v** . Altrimenti, sono **non connessi**.

Un grafo è **connesso** se qualunque coppia di vertici u e v distinti del grafo è connessa.

Una **componente connessa** di un grafo G è un subgrafo connesso massimo di G .

Ciascun vertice e ciascun arco appartiene esattamente ad una ed una sola componente connessa.

Distinguiamo inoltre *Grafi debolmente connessi* e *Grafi fortemente connessi*:

1. Un grafo orientato è **debolmente connesso** se rimpiazzando tutti gli archi orientati con archi non orientati si ottiene un *grafo connesso*.
2. Un grafo è **fortemente connesso** se contiene un cammino orientato da u a v per ogni coppia di vertici u,v . Le componenti forti di un grafo sono i subgrafi fortemente connessi massimi.

Calcolo delle componenti fortemente connesse

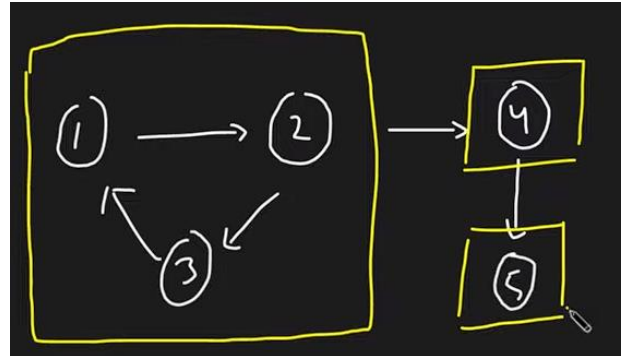
Informalmente, diciamo che un'insieme di nodi è fortemente connessa se ognuno di questi nodi è raggiungibile da qualsiasi altro degli altri nodi (direttamente o indirettamente).

Ad esempio:

Il nodo 1 può raggiungere sia 2 che 3, 2 può raggiungere sia 3 che 1, 3 può raggiungere sia 1 che 2.

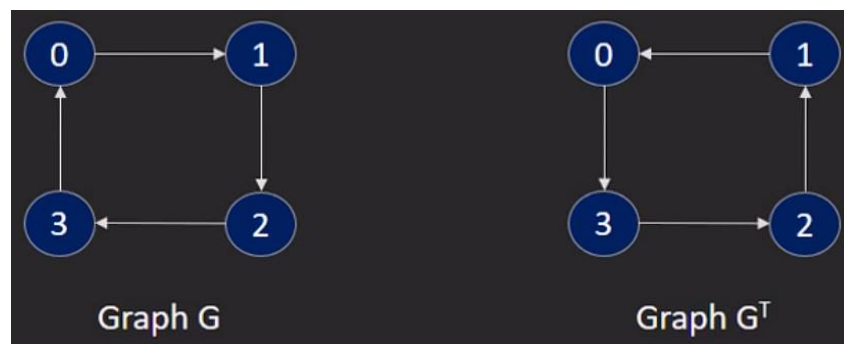
2 può raggiungere 4, tuttavia 4 non può raggiungere 2. L'insieme 1,2,3 è una componente fortemente connessa.

4 può raggiungere 5, ma 5 non può raggiungere 4, di conseguenza, 4 e 5 sono due componenti fortemente connesse costituite da 1 solo elemento.



L'algoritmo per trovare le componenti fortemente connesse di un grafo $G = (V, E)$ utilizza il grafo *trasposto* di G , ossia $G^T = (V, E^T)$, dove $E^T = \{(u, v) : (v, u) \in E\}$.

(ossia un grafo dove l'insieme dei vertici è lo stesso, ma l'insieme degli archi è *trasposto*, ovvero quello che era il predecessore diventa il successore, e viceversa. Infatti " E^T è trasposto da tutte le coppie (u, v) tali che l'arco (v, u) appartiene ad E ", per tutti gli archi).

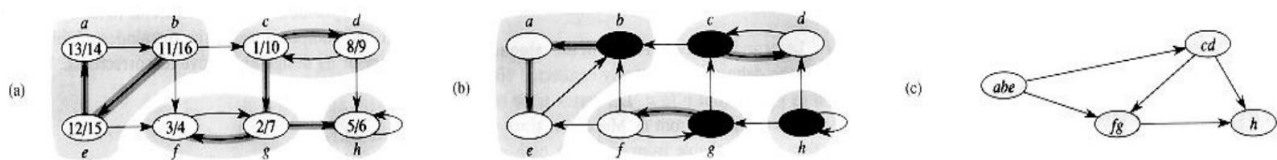


Le componenti connesse possono essere calcolate attraverso il seguente **algoritmo**, in tempo $\Theta(V + E)$

STRONGLY_CONNECTED_COMPONENTS(G)

1. Chiama DFS(G) per calcolare i tempi di completamento $f[u]$ per ciascun vertice u .
2. Calcola il grafo trasposto G^T .
3. Chiama DFS(G^T), ma nel ciclo principale di DFS, considera i vertici in tempo decrescente rispetto ai tempi calcolati nella riga 1.
(Sostanzialmente, il primo vertice che viene analizzato è quello con il tempo di fine più elevato e si prosegue in ordine decrescente).
4. Genera l'output dei vertici di ciascun albero della foresta DF che è stata prodotta nella riga 3 come una singola componente fortemente connessa.

Alla fine di tale processo, ci ritroveremo un insieme di sottografi (e/o nodi, se ce ne è uno solo).



Connettività di un grafo

In particolare, possiamo parlare di connettività di vertici e di archi:

1. **taglio di vertici (vertex cut)**: è l'insieme dei vertici la cui rimozione rende G disconnesso.
2. **connettività di vertici**: indicata con $k(G)$, è la cardinalità del più piccolo taglio di vertici
3. **connettività di vertici locale**: indicata con $k(u,v)$, è la cardinalità del più piccolo taglio di vertici che separa u e v .

Tali nozioni esistono anche in relazioni agli archi. Infatti:

1. **taglio di archi (edge cut)**: è l'insieme degli archi la cui rimozione rende il grafo disconnesso.
2. **connettività di archi**: indicata con $k'(G)$, è la cardinalità del più piccolo taglio di archi;
3. **connettività di archi locale**: indicata con $k'(u,v)$, è la cardinalità del più piccolo taglio di archi che disconnette u da v .

Un grafo è k -connesso se la connettività dei suoi vertici è k o più grande.

Un grafo è k -arco-connesso se la connettività dei suoi archi è k o più grande.

Insiemi Disgiunti

Una struttura dati per insiemi disgiunti mantiene una collezione $S = \{S_1, S_2, \dots, S_r\}$ di insiemi a due a due disgiunti. Ogni insieme S_i è identificato da un rappresentante che è dato da un elemento dell'insieme e viene denotato con $rap[S_i]$.

In alcune applicazioni, non è importante quale elemento sia il rappresentante, in altre invece sì, per cui il rappresentante può essere o un elemento qualsiasi dell'insieme o UN particolare elemento dell'insieme (ad esempio, l'elemento più piccolo).

Ogni elemento di un insieme dinamico è rappresentato da un oggetto. Dato un oggetto x , le operazioni definibili su una struttura dati per insiemi disgiunti sono le seguenti:

1. **MAKE-SET**(x)

Permette di creare un insieme S costituito dal solo elemento x , che sarà anche il rappresentante del nuovo insieme. Poiché gli insiemi nella collezione S devono essere disgiunti, allora x non deve appartenere a nessun altro insieme della collezione.

2. **UNION**(x, y)

Unisce due insiemi S_x e S_y della collezione S , che contengono gli elementi x ed y , formando un nuovo insieme $S = S_x \cup S_y$ e sceglie un nuovo rappresentante, solitamente tra i rappresentanti dei due insiemi.

3. **FIND-SET**(x)

Ritorna il rappresentante dell'insieme S_x , cioè $rap[S_x]$.

Calcolo delle componenti Connesse mediante insiemi disgiunti

La procedura **CONNECTED_COMPONENTS** usa le operazioni degli insiemi disgiunti per calcolare le componenti connesse di un grafo.

Una volta che **CONNECTED_COMPONENTS** ha preelaborato il grafo, la procedura **SAME_COMPONENT** è in grado di determinare se due vertici sono nella stessa componente connessa.

CONNECTED_COMPONENTS(G)

1. **for** ogni vertice $v \in G.V$
2. $MAKE_SET(v)$
3. **for** ogni arco $(u, v) \in G.E$
4. **if** $FIND_SET(u) \neq FIND_SET(v)$
5. $UNION(u, v)$

SAME_COMPONENT(u, v)

1. **if** $FIND_SET(u) = FIND_SET(v)$
2. **return** TRUE
3. **else return** FALSE

Rappresentazione di insiemi disgiunti mediante alberi

Una rappresentazione efficace degli insiemi disgiunti prevede l'uso di alberi. In pratica, ogni insieme è rappresentato mediante un albero.

La **radice** di ogni albero è il **rappresentante** di quell'insieme.

Il **padre del rappresentante** è il rappresentante stesso (si tratta di un **cappio**).

Ogni elemento è un nodo dell'albero ed ha solo un puntatore al nodo padre.

Una popolare implementazione dell'**unione** fra due *set* può avvenire attraverso la combinazione di due **euristiche**: l'*euristica dell'unione per rango* e l'*euristica della compressione dei cammini*:

- nell'euristica dell'unione per rango, ogni nodo ha un ulteriore campo: il rango. Il rango rappresenta il numero di archi che devono essere attraversati per raggiungere il nodo corrente partendo dalla foglia più distante (quindi, il cammino più lungo fra il nodo e una sua foglia);
Quando si uniscono due set S_x e S_y , si considera il rango dei due rappresentanti (ovvero il rango delle radici di questi due set). La radice con il rango più piccolo verrà fatta puntare alla radice con il rango più grande.
 - o se si uniscono due insiemi con lo stesso *rango*, l'insieme risultante ha rango incrementato di 1;
 - o se si uniscono due insiemi con *rango diverso*, l'insieme risultante ha rango pari al maggiore dei due.

nb: a noi interessa solo il rango dei rappresentanti, quello degli altri nodi diventa irrilevante. Se i due insiemi hanno lo stesso rango, è irrilevante chi diventa il rappresentante.

- nell'euristica della compressione dei cammini, vogliamo che ogni nodo abbia un solo padre. Dunque tutti i nodi di un set punteranno solo al loro rappresentante, ovvero alla radice.
Così facendo possiamo ridurre al massimo la profondità dell'albero, ed è possibile accedere ad ogni nodo di un set scendendo al massimo di un livello

In pratica, una volta che vengono effettuate le operazioni di unione attraverso l'unione per rango, utilizziamo la compressione dei cammini per fare in modo che ogni nodo punti al proprio rappresentante.

Un nodo, generalmente, è così composto:

```
Node{
    int rank;
    int data;
    Node parent;
}
```

Esempio Unione (attraverso il rango e la compressione dei cammi)

Abbiamo 7 set creati attraverso la funzione $\text{MAKE_SET}(x)$. Ovviamente all'inizio saranno costituiti da un unico elemento e il loro rango sarà 0.

Vogliamo effettuare le seguenti unioni:

UNION(1,2)

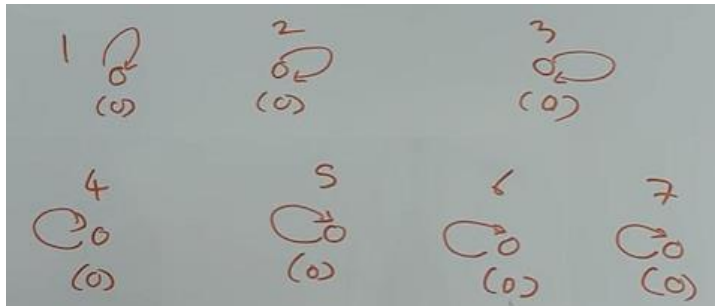
UNION(2,3)

UNION(4,5)

UNION(6,7)

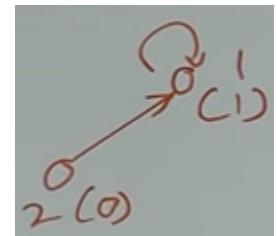
UNION(5,6)

UNION(3,7)



UNION(1,2)

Utilizziamo la $\text{FIND_SET}(x)$ per trovare il rappresentante di ambo gli insiemi. Essendo costituiti da un solo elemento questi saranno banalmente "1" per il primo insieme, e "2" per il secondo insieme. Dobbiamo comparare il loro rango. Essendo per entrambi 0, dunque è uguale, possiamo scegliere indipendentemente uno dei due.



Supponiamo che sia "1" il nuovo rappresentante. "2" punterà adesso ad "1", e il rango di "1" diventerà 1.

Non è necessaria la compressione dei cammini.

UNION(2,3)

Con $\text{FIND_SET}(x)$ scopriamo che il rappresentante dell'insieme di cui fa parte il nodo "2" è "1", mentre l'altro insieme essendo costituito da un solo elemento sarà banalmente "3".

Il rango del rappresentante del primo insieme è 1, mentre quello del secondo insieme è 0. Dunque il rappresentante diventerà "1", e "3" sarà suo figlio.



Non è necessaria la compressione dei cammini.

UNION(4,5) e UNION(6,7) sono la stessa cosa di UNION(1,2)



UNION(5,6)

Applichiamo FIND_SET(5) e FIND_SET(6).

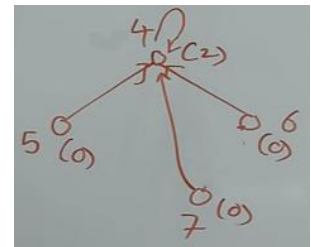
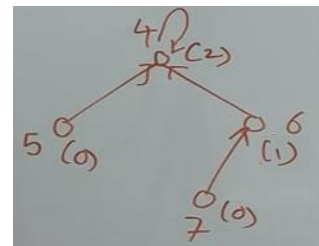
Scopriamo che "5" appartiene ad un insieme il cui rappresentante è "4", mentre "6" appartiene ad un insieme il cui rappresentante è proprio "6".

Il rango di ambo i rappresentanti di questi due insiemi è 1, quindi possiamo scegliere indipendentemente l'uno o l'altro.

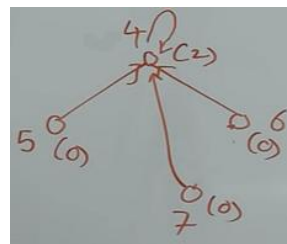
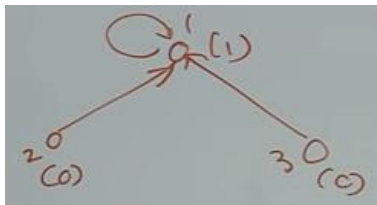
Supponiamo che sia "4" il nuovo rappresentante. "6" punterà adesso a "4", e il rango di "4" diventerà 2 (ricordo che il rango dei figli diventa irrilevante).

In questo caso ha senso applicare la compressione dei cammini, così che ogni nodo sia collegato direttamente al suo rappresentante.

Avremo dunque:



Abbiamo la seguente situazione:



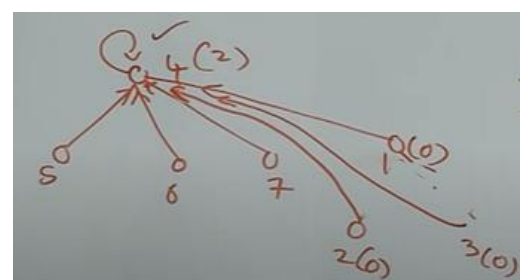
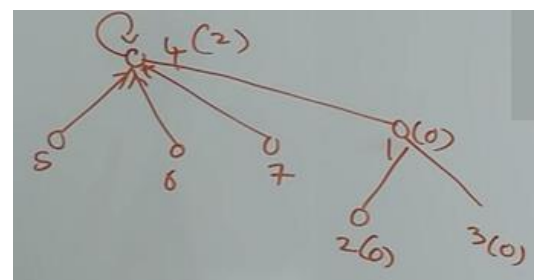
UNION(3,7)

Applichiamo FIND_SET(3) e FIND_SET(7).

Scopriamo che "3" appartiene ad un insieme il cui rappresentante è "1", mentre "7" appartiene ad un insieme il cui rappresentante è "4".

Il rango del rappresentante del primo insieme è 1, mentre quello del secondo insieme è 2. Dunque il rappresentante diventerà "4", e "1" sarà suo figlio.

Applichiamo la compressione dei cammini:



Abbiamo completato le unioni.

TEOREMA SULLE OPERAZIONI SU INSIEMI DISGIUNTI

Una sequenza di m operazioni MAKE_SET, UNION, FINDSET, n delle quali sono operazioni MAKE_SET, può essere eseguita, nel caso peggiore, in una foresta di insiemi disgiunti con l'unione per rango e la compressione del cammino nel tempo $O(m \alpha(n))$, ove $\alpha(n)$ rappresenta l'inverso della Funzione di Ackerman $A_k(n)$.

Funzione di Ackerman-like

Per $k \geq 0$ e $j \geq 1$ interi qualsiasi, definiamo la funzione $A_k(j)$ come:

$$A_k(j) = \begin{cases} j + 1 & \text{se } k = 0 \\ A_{k-1}^{(j+1)}(j) & \text{se } k \geq 1 \end{cases}$$

Valgono:

- $A_{k-1}^{(0)}(j) = j$
- $A_{k-1}^{(i)}(j) = A_{k-1}(A_{k-1}^{(i-1)}(j))$

• Proprietà della funzione

Per ogni intero $j \geq 1$, $A_1(j) = 2j + 1$

Per ogni intero $j \geq 1$, $A_2(j) = 2^{j+1}(j + 1) - 1$

Quindi:

- $A_0(1) = 1 + 1 = 2$
- $A_1(1) = 2 * 1 + 1 = 3$
- $A_2(1) = 2^2(2) - 1 = 7$
- $A_3(1) = A_2^2(1) = A_2(A_2(1)) = A_2(7) = 2^8(8) - 1 = 2047$
- $A_4(1) = A_3^2(1) = A_3(A_3(1)) = A_3(2047) = A_2^{2048}(2047) \gg A_2(2047)$
- $A_4(1) \gg 2^{2048}(2048) - 1$
- $A_4(1) \gg 2^{2059} - 1 \gg 2^{2058} \gg 2^{2000} 2^{58}$
- $A_4(1) \gg (2^{10})^{200} (2^{10})^5 256 \gg 10^{600} 10^{15} 256$
- $A_4(1) \gg 256 * 10^{615}$

Inverso della funzione di Ackerman

L'inverso della funzione $A_k(n)$, per qualsiasi intero $n \geq 0$, è definita come:

$$\alpha(n) = \min(k: A_k(1) \geq n)$$

Ovvero $\alpha(n)$ è il livello k più basso per il quale $A_k(1)$ è almeno pari a n . Abbiamo:

$$\alpha(n) = \begin{cases} 0 & \text{per } 0 \leq n \leq 2 \\ 1 & \text{per } n = 3 \\ 2 & \text{per } 4 \leq n \leq 7 \\ 3 & \text{per } 8 \leq n \leq 2047 \\ 4 & \text{per } 2048 \leq n \leq A_4(1) \end{cases}$$

Tale funzione è una funzione che cresce molto lentamente, se non per quei valori eccezionalmente grandi di n per cui $\alpha(n) > 4$.

Per questo motivo, l'inversa di Ackerman è assimilabile ad una costante, e infatti nella pratica è poco più di un $O(1)$.

Quindi, possiamo ritenere che $\alpha(n) \leq 4$ in tutte le applicazioni pratiche.

LEZ 17 – ALBERI DI COPERTURA MINIMO, KRUSKAL e PRIM

Albero di copertura minimo

Un **albero ricoprente** T (spanning tree) di G , ove G è un grafo connesso pesato e non orientato, è un subgrafo che è un albero, e connette tutti i vertici.

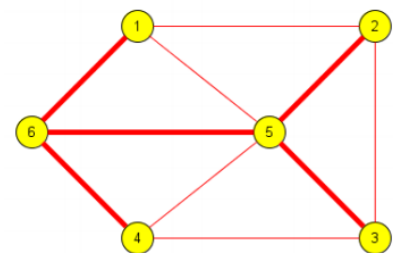
Un grafo può avere differenti **spanning trees**.

Se assegniamo un peso (weight) a ciascun arco, possiamo ottenere il peso totale dell'*albero ricoprente* sommando i pesi degli archi di tale albero.

L'albero ricoprente che ha il peso minimo prende il nome di **albero ricoprente minimo** (minimum spanning tree, MST).

Ogni grafo non orientato (non necessariamente connesso) ha un **MSF (Minimum Spanning Forest)**, che è l'unione degli MST delle sue componenti connesse.

Esempi di applicazione del MST sono la progettazione dei circuiti elettronici o l'installazione della fibra in una città, ove lo scopo è quello di trovare il percorso con il costo (peso) complessivo più basso.



CALCOLO MST

Un MST è un sottografo di $G(V,E)$ composto da $|V|$ nodi e da $(|V| - 1)$ archi.

Il primo algoritmo ideato per trovare un MST è da attribuire a Boruvka, sul quale si basa l'algoritmo di Chezelles che il più veloce algoritmo per trovare un MST (con una complessità $O(E \alpha(E, V))$, dunque approssimativamente $O(E)$).

Tra gli algoritmi più usati al giorno d'oggi ci sono quelli Kruskal e di Prim. Da notar bene che in un MST non si verificano mai dei "cicli" fra i vertici.

(ad es. sopra compare un "ciclo" fra 2,5, e 3. Quindi gli archi (2,5),(2,3),(3,5) formano un ciclo, e quindi sapremo per certo che questo trio di archi non potrà mai comparire tutto insieme in un eventuale MST).

MST – PROPRIETÀ *(quelle importanti sono le prime due)

1. **Possible multiplicity:** Possono esistere diversi MST. In particolare, se tutti i pesi degli archi sono gli stessi, ciascun spanning tree è un MST.
2. **Uniqueness:** Se ciascun arco ha un peso differente, allora esiste uno ed un solo MST (vero in molte situazioni realistiche).
3. **Minimum-cost subgraph:** Se tutti i pesi sono non negativi, allora un MST è il minimum-cost subgraph che connette tutti i vertici.
4. **Cycle property:** Per ogni ciclo C nel grafo, se il peso di un arco e di C è più grande degli altri archi di C , allora l'arco non può appartenere ad un MST.
5. **Cut property:** Per ogni cut C nel grafo, se il peso di un arco e di C è più piccolo dei pesi degli altri archi di C , allora l'arco appartiene a tutti gli MST del grafo.

DEFINIZIONE FORMALE

Sia $G = (V, E)$ un grafo non orientato e connesso. Ad ogni arco (u, v) di E è assegnato un peso $w(u, v)$ che specifica il costo per connettere i due vertici u e v . Si vuole trovare un albero $T \subseteq E$ che connette tutti i vertici in modo da minimizzare il peso totale.

Il **peso totale** sarà dato da:

$$w(t) = \sum_{(u,v) \in T} w(u, v)$$

Algoritmo generico

Si tratta di un algoritmo generico, di tipo greedy, che fa crescere progressivamente un albero di copertura aggiungendo un arco alla volta.

L'algoritmo gestisce un insieme A che è sempre un sottoinsieme di un qualche albero di copertura minimo. L'idea è quella di accrescere il sottoinsieme A di archi aggiungendo un arco alla volta. In pratica, ad ogni passo, si determina un arco (u, v) che può essere aggiunto ad A mantenendo la proprietà per A di essere un sottoinsieme di un qualche albero di copertura. Un arco del genere è detto **arco sicuro**.

Generic-MST(G, w)

1. $A = \emptyset$
2. **while** A non forma un albero di copertura
3. trova un arco sicuro (u, v)
4. $A = A \cup \{(u, v)\}$
5. **return** A

Invariante di ciclo (A è un sottoinsieme di un qualche MST).

inizializzazione: Dopo la riga 1, l'insieme A soddisfa banalmente l'invariante di ciclo (in quanto l'insieme A è vuoto).

Conservazione: Il ciclo nelle righe 2-4 conserva l'invariante perché aggiunge solo archi sicuri.

Conclusione: Tutti gli archi aggiunti ad A si trovano in un albero di connessione minimo, quindi l'insieme A restituito nella riga 5 deve essere un albero di connessione minimo.

nb: la parte più complicata dell'algoritmo sta proprio nel trovare un arco sicuro

ARCO SICURO

Per definire un arco sicuro, abbiamo bisogno di alcune definizioni:

1. un **taglio** $(S, V-S)$ di un grafo non orientato G è una partizione di V .
2. un arco **attraversa** il taglio se uno dei suoi estremi è in S e l'altro è in $V-S$
3. un taglio **rispetta** un insieme di archi A se nessun arco di A attraversa il taglio.
4. un arco **leggero** è un arco con peso minimo (tra gli archi che attraversano quel taglio).

Detto ciò:

Sia $G = (V, E)$ un grafo non orientato e connesso con una funzione peso w a valori reali definita su E . Sia A un sottoinsieme di E contenuto in un qualche albero di copertura minimo per G , sia $(S, V-S)$ un qualunque *taglio* che *rispetta* A , e sia (u, v) un arco *leggero* che *attraversa* $(S, V-S)$.

Allora l'arco (u, v) è sicuro per A .

dimostrazione

Sia T un albero di connessione minimo che contiene A , e supponiamo che T non contenga l'arco leggero (u, v) , perché se lo contenesse il teorema sarebbe dimostrato.

Sia T' un altro albero di connessione minimo, che include $A \cup (u, v)$.

L'arco (u, v) forma un ciclo con gli archi del cammino semplice p che va da u a v in T . Poiché u e v si trovano sui lati opposti del taglio $(S, V-S)$ (e ciò è vero perché ipotizziamo che l'arco (u, v) è leggero e attraversa il taglio) c'è almeno un altro arco in T che appartiene al cammino semplice p che attraversa il taglio (altrimenti non formerebbe un ciclo).

Supponiamo che un certo arco (x, y) sia uno di questi archi. L'arco (x, y) non appartiene ad A , perché il taglio rispetta A .

Poiché (x, y) si trova nel cammino semplice unico da u a v , le due componenti si ricongiungono per fornire un nuovo albero di copertura $T' = T - \{(x, y)\} \cup (u, v)$.

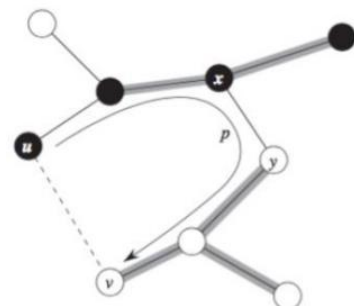
Poiché (u, v) è un arco leggero che attraversa $(S, V-S)$, e poiché anche l'arco (x, y) attraversa lo stesso taglio, allora si ha che:

$$w(u, v) \leq w(x, y)$$

e quindi:

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$$

Ma poiché T' è un albero di connessione minimo, allora $w(T) \leq w(T')$; di conseguenza anche T' deve essere un albero di connessione minimo.

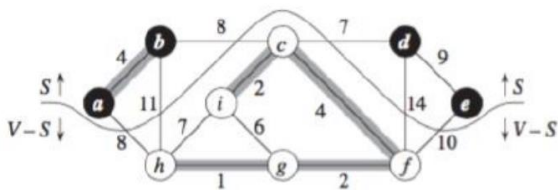


In pratica:

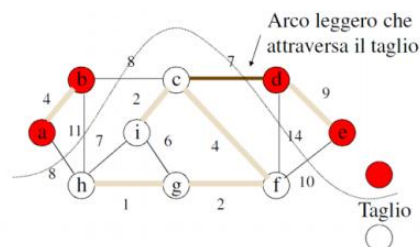
1. il taglio deve rispettare il sottoinsieme di archi A;
2. considerato un arco (u,v) , per essere un **arco sicuro**, questo deve essere l'arco leggero (ovvero, l'arco con peso minore fra quelli che attraversano il taglio).

nb: un taglio, per essere valido, deve creare una partizione di nodi. Il taglio possiamo farlo come vogliamo, basta che rispetti A.

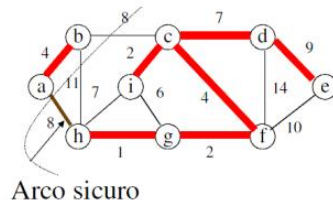
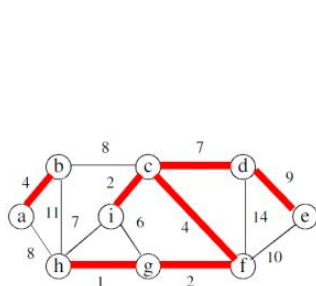
ESEMPLI:



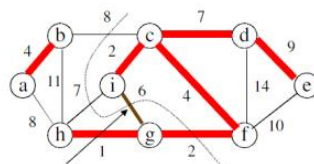
nb: gli archi in grigio/rosso rappresentano il sottoinsieme insieme di archi A.



Insieme A: archi in grigio
il taglio rispetta A

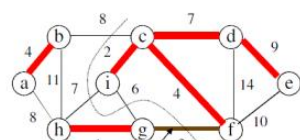


Arco sicuro

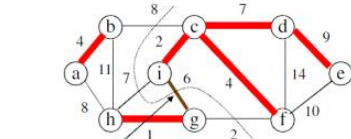
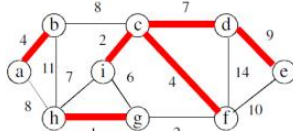


Arco non sicuro

Arco (i,g) non sicuro perché il taglio non rispetta A.



Arco sicuro



Arco non sicuro

Arco (i,g) non sicuro perché non leggero (che invece è l'arco (g,f)).

Algoritmo di Kruskal

Nell'algoritmo di Kruskal, l'insieme A è una foresta (grafo non orientato, aciclico, dato dall'unione di più alberi).

L'idea è quella di ingrandire, ad ogni passo, i sottoinsiemi disgiunti di A , aggiungendo un arco (u,v) di peso minimo che connette due sottoinsiemi disgiunti. In questo modo, ad ogni passo, si ingrandiscono i sottoinsiemi connettendoli tra di loro attraverso l'arco che si è scelto.

L'algoritmo risulta essere greedy perché ad ogni passo si aggiunge alla foresta un arco con il minor peso.

Nell'**implementazione** dell'algoritmo si usa una *struttura dati per insiemi disgiunti*, dove ogni insieme contiene i vertici di un albero della foresta corrente.

MST_KRUSKAL(G, w, r)

```
1.   $A = \emptyset$ 
2.  for each  $u$  in  $G.V$ 
3.       $\text{MAKE\_SET}(u)$ 
4.   $\text{SORT}(E)$  //order edges by non-desc weight
5.  for each  $\{(u,v)\}$  in  $G.E$ 
6.      if  $\text{FIND\_SET}(u) \neq \text{FIND\_SET}(v)$ 
7.           $A = A \cup \{(u,v)\}$ 
8.           $\text{UNION}(u,v)$ 
9.  return  $A$ 
```

- Linee 1-4: A viene inizializzato con l'insieme vuoto; MAKE-SET crea $|V|$ alberi, uno per ogni vertice; gli archi di E vengono ordinati in ordine di peso non decrescente.

- Linee 5-8: $\text{FIND_SET}(u)$ restituisce un rappresentante dell'insieme che contiene u ; perciò si determina se due vertici u e v appartengono allo stesso albero. Se i due vertici appartengono ad alberi diversi, l'arco (u, v) viene aggiunto ad A ed i due alberi vengono fusi, mediante la UNION , in un unico albero.

- Linea 9: Viene restituito A .

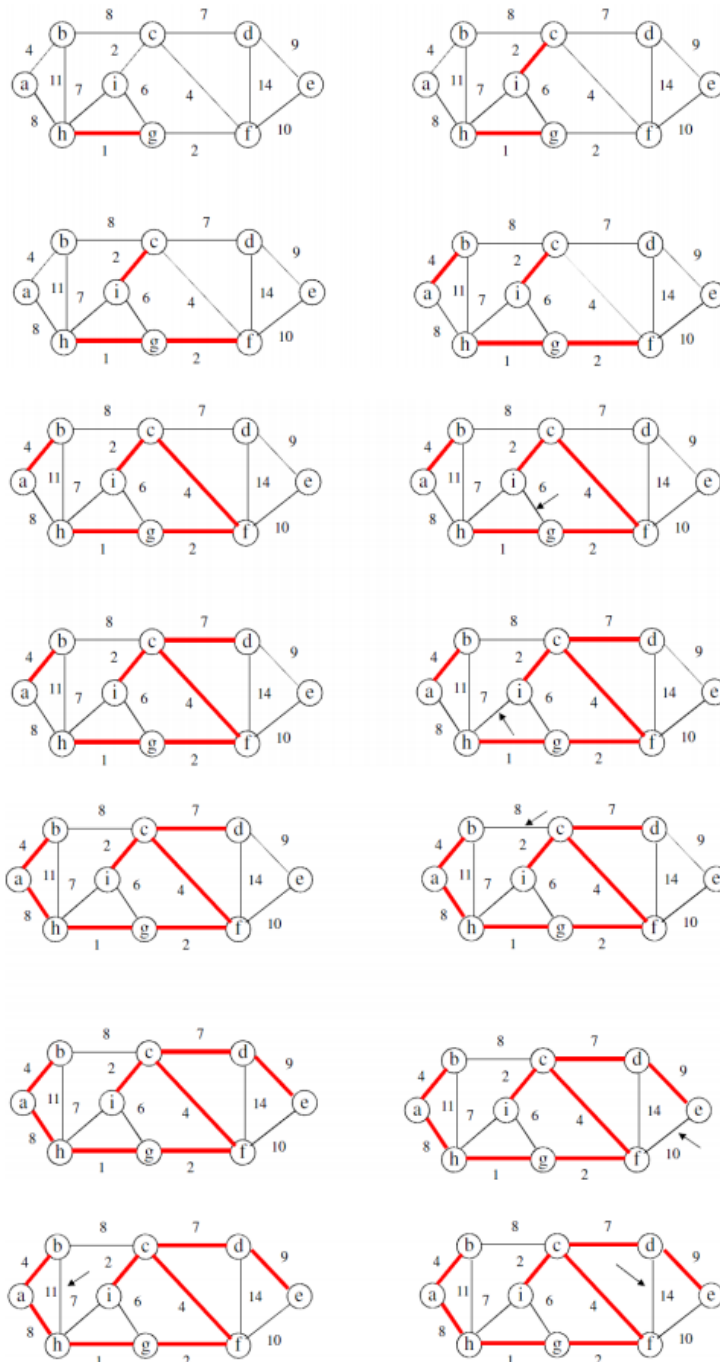
Per quanto riguarda la complessità:

1. L' inizializzazione richiede tempo $O(V)$;
2. Il tempo necessario per ordinare gli archi è $O(E \log E) = O(E \log V^2) = O(E \log V)$;
3. Le operazioni sulla foresta di insiemi disgiunti è $O(E \alpha(E, V))$, circa $O(E)$.

Quindi, il **tempo di esecuzione** totale è pari a: $O(V + E \log V + E) = O(E \log V)$.

(nb: nel caso peggiore, possiamo considerare $E = V^2$ perché ogni vertice sarà adiacente a tutti i vertici del grafo).

ESEMPIO:



Algoritmo di Prim

Nell'algoritmo di Prim, l'insieme A è un singolo albero (grafo non orientato, aciclico, connesso). Tale albero parte da un vertice arbitrario r (la *radice*) e cresce fino a quando non ricopre tutti i vertici.

Ad ogni passo, viene aggiunto un arco leggero che collega un vertice in VA con un vertice in $V - VA$, dove VA è l'insieme di nodi raggiunti da degli archi di A .

(Da notar bene, che $(VA, V - VA)$ è un taglio che rispetta A , per definizione, ed inoltre che gli archi leggeri che attraversano il taglio sono sicuri).

Implementazione

MST_PRIM(G, w, r)

```
1. for each  $u$  in  $G.V$ 
2.      $u.key = \infty$ 
3.      $u.p = NIL$ 
4.  $r.key = 0$ 
5.  $Q = G.V$ 
6. while  $Q \neq \emptyset$ 
7.      $u = \text{EXTRACT\_MIN}(Q)$ 
8.     for each  $v$  in  $u.adj$ 
9.         if  $v \in Q$  and  $w(u,v) < v.key$ 
10.             $v.p = u$ 
11.             $v.key = w(u,v)$  //decrease key
```

- **Linee 1-4:** a tutti i vertici del grafo viene posto a infinito il valore “chiave”, e il padre viene posto a NIL. Solo il valore chiave della radice viene posto a 0 in modo da estrarre la radice come elemento minimo nella fase iniziale.

- **Linea 5:** Viene inizializzata una coda di priorità di minimo che contiene i vertici di G .

- **Linee 6-7:** Finchè ci sono vertici nella coda Q , si estrae (elimina) ad ogni passo un vertice dalla coda, quello con chiave minore. Durante l'algoritmo, l'insieme $V-Q$ contiene i vertici dell'albero che si sta costruendo.

- **Linee 8-11:** Per ogni vertice v adiacente al vertice estratto u , si controlla se v si trova nella coda Q e se il peso dell'arco (u,v) è minore del valore della chiave di v .

In caso affermativo, si aggiornano i campi key (chiave) e p (parent) di v :

- il vertice estratto u diventa il genitore di v
- il peso dell'arco (u,v) diventa la chiave di v

In pratica, in queste linee si aggiornano i campi key (chiave) e p (parent) di ogni vertice v adiacente al vertice estratto u .

Al ciclo successivo, si esaminerà la coda Q e si troverà che uno dei v esaminati precedentemente è diminuito tanto da essere il vertice con chiave più piccola, allora si aggiungerà implicitamente v all'albero.

Per quanto riguarda la **complessità**, esso dipende da Q.

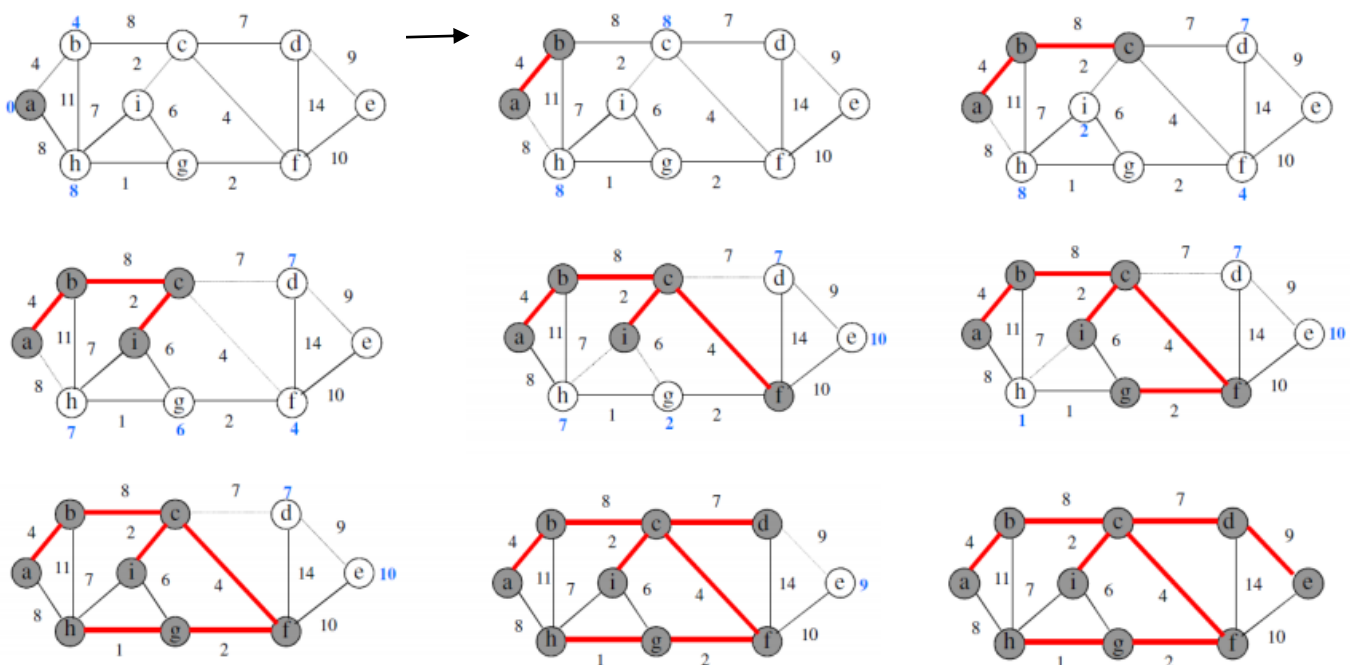
e utilizziamo un heap binario:

- l'inizializzazione, eseguita mediante *Build-Heap*, impiega $O(V)$;
- il ciclo While viene eseguito V volte ed ogni operazione *Extract-Min* impiega $O(\log V)$;
- il ciclo For interno viene eseguito $O(E)$ volte;
- il controllo di appartenenza costa $O(1)$ se usiamo l'indirizzamento diretto;
- l'assegnazione nella riga 11 costa $O(\log V)$ essendo un'operazione *Decrease-Key* su un heap;

Il **tempo di esecuzione** totale sarà quindi: $O(V + V \log V + E \log V) = O(E \log V)$

Ed è uguale a quello di Kruskal.

ESEMPIO:



Differenze fra l'algoritmo di Prim e di Kruskal

PRIM

L'albero che stiamo costruendo rimane sempre connesso.

Attraversa un vertice più di una volta per ottenere la distanza minima.

Costruisce una soluzione partendo da un vertice arbitrario r , ed aggiunge di volta in volta all'albero il successivo vertice col costo minore.

Risulta essere più veloce per grafi densi¹.

KRUSKAL

L'albero che stiamo costruendo rimane solitamente sconnesso (foresta).

Attraversa un vertice solo una volta.

Costruisce una soluzione partendo dall'arco col costo minore, ed aggiunge di volta in volta all'albero/foresta il successivo arco col costo minore.

Risulta essere più veloce per grafi sparsi².

Entrambi sono algoritmi Greedy.

¹ Informalmente, un grafo denso è un grafo con molti archi.
Più precisamente, si tratta di un grafo dove il numero di vertici è $\Theta(|V|^2)$.

² Informalmente, un grafo sparso è un grafo con pochi archi.
Più precisamente, si tratta di un grafo dove il numero di vertici è $O(|V|)$.

LEZ 18/19 – TABELLE HASH

Tabelle Hash - introduzione

Una **tabella hash** è una struttura dati utilizzata per mettere in corrispondenza una data *chiave* con un dato *valore*. Viene usata per l'implementazione di strutture dati astratte associative come Map o Set.

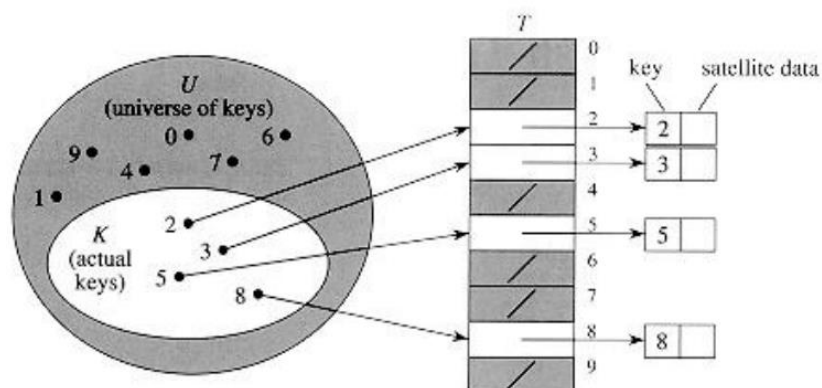
Una tabella hash è una struttura dati efficace per implementare quei dizionari che implementano solo le operazioni di INSERT, DELETE e SEARCH (che comunque sono sufficienti per la maggior parte delle applicazioni).

Nel caso peggiore, l'hashing richiede un tempo lineare per svolgere un'operazione, ma il tempo atteso è $O(1)$.

TABELLE A INDIRIZZAMENTO DIRETTO

L'**indirizzamento diretto** è una tecnica semplice che funziona quando l'universo delle chiavi è ragionevolmente piccolo.

Una tabella ad indirizzamento diretto corrisponde al concetto di array, infatti ad ogni chiave possibile corrisponde una sola posizione, o slot, nella tabella.



Rappresentiamo con U l'universo di tutte le chiavi, anche quelle non contenute nell'array, mentre con K rappresentiamo le *chiavi effettive*.

Le dimensioni di U andranno da $0, \dots, m-1$.

La *tabella di indirizzamento diretto*, che altro non è che un array, sarà indicata con $T[0, \dots, m-1]$, in cui ogni cella corrisponde ad una chiave dell'Universo U .

Se l'insieme non contiene elementi con la chiave k , allora $T[k] = \text{NULL}$.

Le operazioni sulla Tabella, le quali hanno complessità $O(1)$, sono:

- $\text{Direct_Address_Search}(T, k)$
return $T(k)$
- $\text{Direct_Address_Insert}(T, x)$
 $T[\text{key}[x]] \leftarrow x$
- $\text{Direct_Address_Delete}(T, x)$
 $T[\text{key}[x]] \leftarrow \text{NULL}$

TABELLE HASH

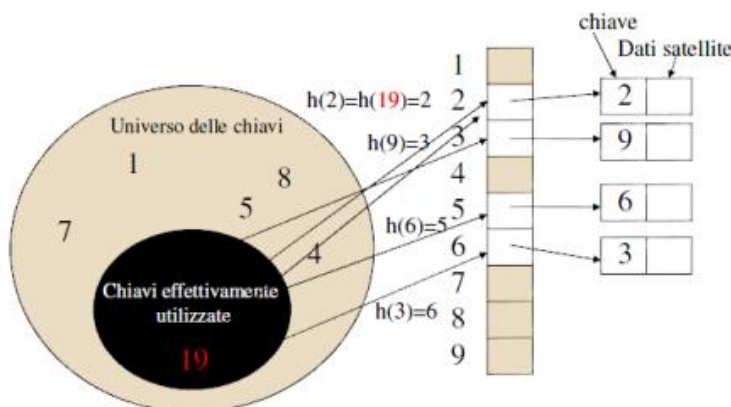
L'indirizzamento diretto ha, come già accennato, una limitazione piuttosto ovvia: se l'universo delle chiavi è molto grande non è possibile (o conveniente) usare una tabella a indirizzamento diretto in quanto le sue dimensioni sarebbero spropositate.

Invece, le **tabelle hash** sono strutture dati che permettono di avere un giusto compromesso tra i requisiti di memoria e l'efficienza nelle operazioni.

Con il metodo *hash*, un elemento con chiave k è memorizzato nello slot $h(k)$ della tabella. In pratica si utilizza una funzione hash $h()$ per calcolare lo slot della chiave. In particolare, la funzione hash definisce una corrispondenza tra l'universo U delle chiavi e le posizioni di una tabella hash $T[0,1, \dots, m-1]$.

La funzione hash, quindi, riduce l'intervallo degli indici che devono essere gestiti. Questo intervallo passa da U valori ad m valori. Di conseguenza diminuisce anche lo spazio di memoria richiesto.

C'è, però, un problema: due chiavi distinte possono produrre lo stesso valore hash, cioè due chiavi possono essere associate allo stesso slot. In questo caso si ha una **collisione**.



FATTORE DI CARICO

Data una tabella hash T con m posizioni che memorizza n elementi, si definisce il fattore di carico $\alpha = n/m$ ossia il numero medio di elementi per ogni lista memorizzata.

Si assume che qualunque elemento corrisponda in modo equamente probabile a una delle m posizioni, indipendentemente dalla posizione degli altri elementi (*uniformità semplice della funzione hash*).

Funzioni Hash

Una buona funzione hash deve soddisfare due caratteristiche:

1. La prima caratteristica che una buona funzione hash soddisfa è il **criterio di uniformità semplice**: il valore hash di una chiave k è uno dei valori $0 \dots m-1$ in modo equiprobabile.
In pratica ogni chiave ha la stessa probabilità di essere associata a uno qualsiasi degli m slot.
2. La seconda caratteristica è quella in cui una funzione hash dovrebbe utilizzare tutte le cifre della chiave per produrre un valore hash.

Sono diversi i metodi per trovare una buona funzione hash, fra cui l'*hashing per divisione*, l'*hashing per moltiplicazione* e l'*hashing universale*.

HASHING PER DIVISIONE

La funzione hash è del tipo $h(k) = k \bmod m$ cioè il valore hash è il resto della divisione di k per m .

In pratica, una chiave k viene associata a uno dei m slot prendendo il resto della divisione tra k ed m .

Il metodo risulta essere veloce, ma occorre far attenzione ai valori di m . Infatti, quando si utilizza il metodo della divisione, di solito, si cerca di evitare alcuni valori di m . Buoni valori di m sono i numeri primi non troppo vicini a potenze del 2.

Es. Supponiamo di voler memorizzare 2000 chiavi. È possibile scegliere $m=701$ (ammettendo mediamente 3 collisioni per ogni cella). Allora, $h(k) = k \bmod 701$.

HASHING PER MOLTIPLICAZIONE

Il metodo della moltiplicazione opera in due passi:

1. si moltiplica la chiave per una costante A , tale che $0 < A < 1$ e si estrae la parte frazionaria del risultato;
2. si moltiplica questo valore per m e si prende la parte intera inferiore.

La funzione di hash è $h(k) = \lfloor m (kA \bmod 1) \rfloor$ dove $kA \bmod 1$ è la parte frazionaria di $kA - \lfloor kA \rfloor$.

Un vantaggio del metodo della moltiplicazione è che il valore di m non è critico. Pertanto si sceglie una potenza di 2, ossia $m = 2^p$.

Ad esempio, se $m = 90$, $k = 19$ e $A = 1/4$

- $kA = 4,75$, $\lfloor kA \rfloor = 4$ e $kA - \lfloor kA \rfloor = 0,75$
- $m \cdot (kA - \lfloor kA \rfloor) = 90 \cdot 0,75 = 67,5$ e $\lfloor 67,5 \rfloor = 67$

Sebbene il metodo funziona con qualsiasi valore di A , si propone come valore ottimale il **valore della sezione aurea**:

$$A = \frac{\sqrt{5} - 1}{2} = 0.618033988749 \dots$$

HASHING UNIVERSALE

L'hashing universale consiste nel selezionare nel modo casuale la funzione hash da una classe di funzioni accuratamente progettate, all'inizio dell'esecuzione dell'algoritmo.

La randomizzazione garantisce che nessun input possa provocare sistematicamente il comportamento nel caso peggiore dell'algoritmo.

“Sia $\mathcal{H}$ una collezione finita di simboli hash che associano un dato universo U di chiavi nell'intervallo $\{0, \dots, m-1\}$.

Tale collezione è detta universale, se per ogni coppia di chiavi distinte $k, l \in U$, il numero di funzioni hash $h \in \mathcal{H}$ per le quali $h(k) = h(l)$ è al massimo $\frac{|\mathcal{H}|}{m}$ ”.

Ovvero, con una funzione hash scelta a caso da $\mathcal{H}$, la probabilità di una collisione fra due chiavi distinte k e l non è maggiore della probabilità $1/m$ di una collisione nel caso in cui $h(k)$ e $h(l)$ fossero scelte in modo casuale e indipendente dall'insieme $\{0, \dots, m-1\}$.

Progettare una classe universale di funzioni hash

Si inizia scegliendo un numero primo p sufficientemente grande tale che ogni possibile chiave sia compresa tra 0 e $p-1$.

Siano $Z_p = \{0, \dots, p-1\}$ e $Z_p^* = \{1, \dots, p-1\}$.

Poiché supponiamo che la dimensione dell'universo delle chiavi sia maggiore del numero di celle della tabella hash, abbiamo $p > m$.

Definiamo la funzione hash h_{ab} per ogni $a \in Z_p^*$ e per ogni $b \in Z_p$, avremo che: $h_{ab}(k) = ((ak + b) \bmod p) \bmod m$.

ad esempio: se $p=17$, $m=6$, $a=3$, $b=4$ e $k=8$, avremo che:

$$h_{34}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5.$$

La *famiglia* di tutte queste funzioni di hash è:

$$\mathcal{H}_{p,m} = \{h_{ab} : a \in Z_p^* \text{ e } b \in Z_p\}$$

OSS.

Ogni funzione hash h_{ab} manda da Z_p a Z_m .

Questa classe di funzioni hash ha la proprietà che la dimensione m dell'output è arbitraria (non è necessariamente un numero primo).

Poiché le scelte possibili per a sono $p-1$ e le scelte possibili per b sono p , la collezione $\mathcal{H}_{p,m}$ contiene $p(p-1)$ funzioni hash.

(nb: questo e il corollario dopo, *Camastra non lo chiede da anni all'esame*)

Teorema

Supponiamo che una funzione hash h sia scelta da una collezione universale di funzioni hash e sia utilizzata per inserire n chiavi in una tabella T di dimensione m , applicando il concatenamento per risolvere le collisioni:

- **se la chiave k non è nella tabella**, allora la lunghezza attesa $E[n_{h(k)}]$ della lista è al massimo α ;
- **se la chiave k è nella tabella**, allora la lunghezza attesa $E[n_{h(k)}]$ della lista (che contiene la chiave k) è al massimo $1+\alpha$.

dimostrazione

Per ogni coppia k e l di chiavi distinte, definiamo la variabile casuale indicatrice:

$$X_{kl} = I\{h(k) = h(l)\}.$$

Poiché, per definizione, la collisione tra una singola coppia di chiavi ha probabilità al più $\frac{1}{m}$, ossia $P\{h(k) = h(l)\} \leq \frac{1}{m}$.

Per ogni chiave k , si definisce la variabile casuale Y_k , che è uguale al numero di chiavi diverse da k che sono associate alla stessa cella di k ,

$$\text{ossia } Y_k = \sum_{\substack{l \in T \\ l \neq k}} X_{kl} \leq \sum_{\substack{l \in T \\ l \neq k}} \frac{1}{m}.$$

Ora si hanno due casi:

- **se $k \notin T$** , allora $n_{h(k)} = Y_k$ e $|\{l: l \in T\} \cap l \neq k| = n$.
Quindi $E[n_{h(k)}] = E[Y_k] \leq \alpha$
- **se $k \in T$** , allora la chiave k non appare nella lista $T[h(k)]$ ed il numero indicato da Y_k non include la chiave k , abbiamo $n_{h(k)} = Y_k + 1$ e $|\{l: l \in T\} \cap l \neq k| = n - 1$.
Quindi $E[n_{h(k)}] = 1 + E[Y_k] \leq 1 + \alpha$.

Corollario (hashing universale)

Utilizzando l'hashing universale e la risoluzione delle collisioni mediante concatenamento in un tabella con m celle inizialmente vuote, occorre un tempo atteso $\theta(n)$ per eseguire qualsiasi sequenza di n operazioni INSERT, SEARCH o DELETE.

Dimostrazione

Poiché il numero di inserimenti è $O(m)$, si ha $n = O(m)$ e $\alpha = O(1)$.

Le operazioni INSERT e DELETE richiedono un tempo costante e, per il teorema precedente, il tempo atteso per un'operazione SEARCH è $O(1)$.

Per la linearità del valore atteso, quindi, il tempo atteso dell'intera sequenza di operazioni è $O(n)$. Poiché ogni operazione richiede il tempo $\Omega(1)$, è dimostrato il limite $\theta(m)$.

RISOLUZIONE DELLE COLLISIONI PER CONCATENAZIONE

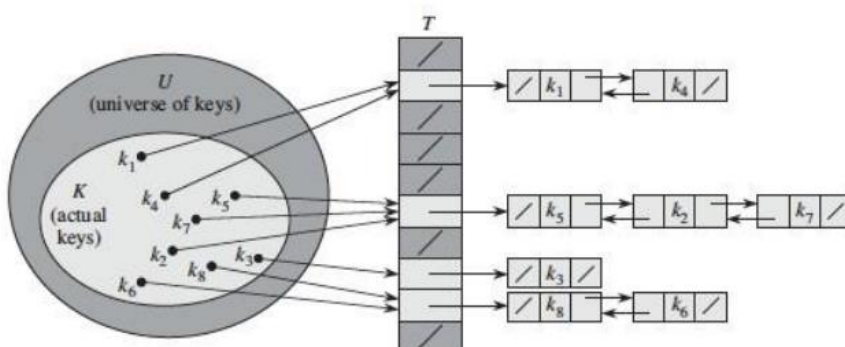
Vediamo l'hash table come un **insieme di liste**.

Ogni **cella** contiene un **puntatore** alla **testa della lista** di tutti gli elementi che vengono mappati nella stessa cella, ovvero tutti quegli elementi che corrispondono alla stessa posizione $h(k)$.

L'**inserimento** avviene in testa alla lista e si suppone che l'elemento da inserire non sia già presente nella hash table.

La **cancellazione** può essere implementata in maniera efficiente se le liste sono **doppiamente linkate**.

- CHAINED-HASH-INSERT(T, x)
 - **inserisce x in testa alla lista** $T[h(key[x])]$ (si suppone che l'elemento non sia già presente)
- CHAINED-HASH-SEARCH(T, k)
 - ricerca un elemento con chiave k nella lista $T[h(k)]$
- CHAINED-HASH-DELETE(T, k)
 - cancella x dalla lista $T[h(key[x])]$. (più efficiente se le liste sono doppiamente linkate)



Per quanto riguarda la **complessità computazionale**:

3. INSERT impiega un tempo $O(1)$;
4. SEARCH impiega un tempo $O(l)$, ove l è la lunghezza della lista;
5. DELETE impiega un tempo $O(l)$.

(nb: la lista in questo caso è unidirezionale. Se fosse bidirez., sarebbero tutte $O(1)$)

TEOREMA (ricerca senza successo)

In una tabella hash, le cui collisioni sono risolte con il concatenamento, una ricerca senza successo richiede un tempo atteso $\Theta(1 + \alpha)$, nell'ipotesi di hashing uniforme.

dimostrazione

Nell'ipotesi di hashing uniforme, qualsiasi chiave k non ancora memorizzata nella tabella ha la stessa probabilità di essere associata ad una qualsiasi delle m celle.

Il tempo atteso per ricercare senza successo una chiave k è il tempo atteso per svolgere le ricerche fino alla fine della lista $T[h(k)]$, che ha lunghezza attesa pari ad α .

Quindi il numero atteso di elementi esaminato in una ricerca senza successo è α e il tempo totale richiesto, incluso quello per calcolare $h(k)$, è $\Theta(1 + \alpha)$.

TEOREMA (ricerca con successo)

In una tabella hash, le cui collisioni sono risolte con il concatenamento, una ricerca con successo richiede un tempo atteso $\Theta(1 + \alpha)$, nell'ipotesi di hashing uniforme.

dimostrazione

Supponiamo che l'elemento da ricercare abbia la stessa probabilità di essere uno qualsiasi degli n elementi memorizzati nella tabella.

Il numero di elementi esaminati durante una *ricerca con successo* di un elemento x è uno in più del numero di elementi che si trovano prima di x nella lista di x .

Gli elementi che precedono x nella lista sono stati inseriti tutti dopo di x , perché i nuovi elementi vengono posti all'inizio della lista.

Per trovare il numero atteso di elementi esaminati, sugli n elementi x della tabella, prendiamo la media di 1 più il numero atteso di elementi aggiunti alla lista di x , dopo che x è stato aggiunto alla lista.

Indichiamo con x_i l' i -esimo elemento inserito nella tabella (per $i = 1, \dots, n$) e sia $k_i = \text{key}[x_i]$. Per le chiavi k_i e k_j , definiamo la variabile casuale indicatrice $X_{ij} = I\{h(k_i) = h(k_j)\}$.

Nell'ipotesi di *hashing uniforme semplice*, abbiamo che $P\{h(k_i) = h(k_j)\} = \frac{1}{m}$, quindi

$$E[X_{ij}] = \frac{1}{m}.$$

Svolgimento: ↓

Svolgimento:

$$\begin{aligned}
& E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] = \\
& \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}] \right) = \\
& \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) = \\
& 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) = \\
& 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right) = \\
& 1 + \frac{1}{nm} \left(n^2 - n \frac{n+1}{2} \right) = \\
& 1 + \frac{1}{m} \left(n - \frac{n+1}{2} \right) = \\
& 1 + \frac{n-1}{2m} = \\
& 1 + \frac{a}{2} - \frac{a}{2n} = \theta(1 + \alpha)
\end{aligned}$$

Il significato di questa analisi è che se il numero di slot della tabella hash è proporzionale al numero di elementi della tabella, la ricerca richiede in media un tempo costante.

Poiché l'inserimento richiede il tempo $O(1)$ nel caso peggiore e la cancellazione richiede il tempo $O(1)$ nel caso peggiore, quando le liste sono doppiamente concatenate (bidirezionali), allora tutte le operazioni di dizionario possono essere svolte, in media, nel tempo $O(1)$.

RISOLUZIONE DELLE COLLISIONI CON INDIRIZZAMENTO APERTO

Nell'indirizzamento aperto tutti gli elementi sono memorizzati nella stessa tabella hash, ovvero ogni voce della tabella contiene un elemento dell'insieme dinamico o la costante NIL.

Quando cerchiamo un elemento, esaminiamo sistematicamente gli slot della tabella finchè non troviamo l'elemento desiderato o finchè ci accorgiamo che l'elemento non si trova nella tabella.

Diversamente dal concatenamento, non ci sono liste né elementi memorizzati all'esterno della tabella.

Quindi, nell'indirizzamento aperto, la tabella hash può "riempirsi" al punto tale che non possono essere effettuati altri inserimenti; il fattore di carico è, perciò, sempre ≤ 1 .

Le operazioni di inserimento, ricerca e cancellazione sono più complicate rispetto al caso precedente:

Inserimento

Per effettuare un inserimento con l'indirizzamento aperto, data una chiave si genera un valore di hash:

- se lo slot associato a questo valore è vuoto, allora inserisco la chiave;
- se lo slot associato a questo valore è occupato, allora si scorre/esamina la tabella (scansione) a partire dallo slot occupato fino a trovare uno slot vuoto dove inserire la chiave;

Per determinare quali slot esaminare si estende la funzione hash in modo da farle generare una sequenza di valori (sequenza di scansione) invece che un singolo valore. La funzione hash prenderà in input la chiave e un indice di posizione per produrre in output una nuova posizione.

hash_insert(T,k)

```
1.  i = 0
2.  repeat
3.      j = h(k,i)
4.      if T[j] = NIL
5.          T[j] = k
6.          return j
7.      i = i+1
8.  until i = m
9.  error "overflow tabella hash"
```

Ricerca

L'algoritmo di ricerca con indirizzamento aperto, esamina/scorre la stessa sequenza di slot che ha esaminato l'algoritmo di inserimento quando ha inserito la chiave k . Quindi, la ricerca può terminare (senza successo) quando trova uno slot vuoto, perché la chiave k sarebbe stata inserita lì e non dopo nella sua sequenza di scansione (questo ragionamento presuppone che le chiavi non vengano cancellate dalla tabella hash).

hash_search(T, k)

```
1.   $i = 0$ 
2.  repeat
3.       $j = h(k, i)$ 
4.      if  $T[j] = k$ 
5.          return  $j$ 
6.       $i = i + 1$ 
7.  until  $i = m$  or  $T[j] = \text{NIL}$ 
8.  return NIL
```

Cancellazione

La cancellazione risulta essere più difficile. Se eliminiamo una chiave dallo slot i della tabella, non possiamo semplicemente marcare quello slot come vuoto (inserendo NIL), perché così facendo spezzerebbero la sequenza di scansione di una chiave k inserita precedentemente, che ha saltato quello slot perché occupato.

Una soluzione sarebbe quella di marcare lo slot come DELETED e modificare l'algoritmo dell'inserimento affinché possa inserire una nuova chiave in questo slot.

Se si prevede che le chiavi debbano essere cancellate, si preferisce gestire le collisioni attraverso il concatenamento.

SEQUENZE DI ISPEZIONE

Nei casi considerati, abbiamo ipotizzato un **hashing uniforme**.

Poiché è difficile implementare funzioni h che rispettino la proprietà di uniformità, si usano delle approssimazioni accettabili.

Esistono 3 tecniche che ci consentono di effettuare l'ispezione di una tabella con indirizzamento aperto, e sono:

- ispezione lineare;
- ispezione quadratica;
- hashing doppio.

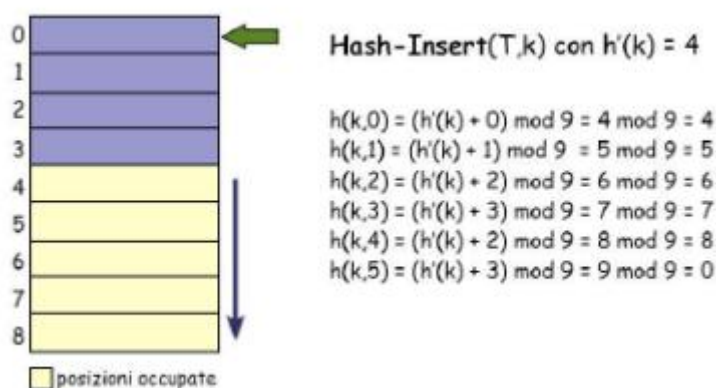
Tali ispezioni garantiscono che la sequenza sia una permutazione di $[0, 1, \dots, m-1]$, ovvero garantiscono che tutte le celle vengano esaminate.

ISPEZIONE LINEARE

Sia $h' : U \rightarrow \{0, 1, \dots, m-1\}$ una funzione hash ordinaria, la *scansione lineare* usa la funzione hash: $h(k, i) = (h'(k) + i) \bmod m$ per $i = 1, \dots, m$.

In poche parole, io genero la funzione hash tramite h' , se è occupata avanzo di una posizione (incrementando la i).

Risulta facile da implementare, ma presenta un fenomeno conosciuto come **agglomerazione primaria**: si formano lunghe file di slot occupati, che aumentano il tempo medio di ricerca.

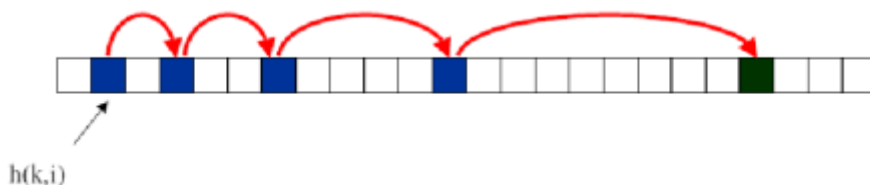


SCANSIONE QUADRATICA

Sia $h' : U \rightarrow \{0, 1, \dots, m-1\}$ una funzione hash ordinaria, la *scansione quadratica* usa la funzione hash: $h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$.

In poche parole, la posizione iniziale esaminata è $T[h'(k)]$, poi le posizioni successivamente esaminate sono distanziate da quantità che dipendono in modo quadratico dal numero di scansione i .

Il metodo della scansione quadratica funziona meglio della scansione lineare perché evita l'agglomerazione primaria, ma non risolve del tutto il problema dell'agglomerazione. Si ha infatti una **agglomerazione secondaria**: se si ha una collisione sul primo elemento, le due sequenze di scansione risultano comunque identiche, quindi se si hanno due chiavi con la stessa posizione iniziale, entrambe avranno la stessa sequenza di scansione e le collisioni si ripeteranno ogni volta lungo tutta la sequenza.



HASHING DOPPIO

Siano $h_1 : U \rightarrow \{0, 1, \dots, m-1\}$ e $h_2 : U \rightarrow \{0, 1, \dots, m-1\}$ due funzioni hash, l'*hashing doppio* usa la funzione hash: $h(k, i) = (h_1(k) + i h_2(k)) \bmod m$.

Questo è il metodo che più si avvicina alla proprietà di uniformità. Inoltre risolve i problemi di agglomerazione che si verificano nella scansione lineare e in quella quadratica.

L'idea è quella di partire da un valore hash e fare in modo che i valori delle posizioni successive dipendano proprio da un'altra funzione. Grazie a questa caratteristica **non vi è agglomerazione** proprio perché la distanza cambia ogni volta.

Tuttavia, dobbiamo assicurarci che h_2 non sia multiplo di m (numero di slot della tabella). Infatti, se m e $h_2(k)$ avessero un massimo comun divisore, allora la sequenza cercata per k non esaminerebbe tutti gli elementi.

Un modo pratico è di scegliere m potenza di 2 e scegliere h_2 in modo che produca un numero dispari.

Per esempio: $h_1(k) = k \bmod m$ e $h_2(k) = 1 + (k \bmod m)$

Teorema (sul numero atteso di ispezioni senza successo)

Nell'ipotesi di hashing uniforme, data una tabella hash ad indirizzamento aperto con fattore di carico $\alpha < 1$ allora il numero atteso di ispezioni senza successo è al massimo $\frac{1}{1-\alpha}$.

dim

In una ricerca senza successo, ogni ispezione (tranne l'ultima) accede a una cella occupata che non contiene la chiave desiderata e l'ultima cella è vuota.

Definiamo la variabile casuale X come il numero di ispezioni fatte in una ricerca senza successo; definiamo inoltre A_i come l'evento in cui l' i -sima ispezione viene eseguita e trova una cella occupata. Allora l'evento $\{X \geq i\}$ è l'intersezione degli eventi $A_1 \cap \dots \cap A_{i-1}$.

Limitando $P\{X \geq i\}$ e limitando $P\{A_1 \cap \dots \cap A_{i-1}\}$, si ha

$$P\{A_1 \cap \dots \cap A_{i-1}\} = P\{A_1\} \cdot P\{A_2|A_1\} \cdot P\{A_3|A_1 \cap A_2\} \cdot \dots \cdot P\{A_{i-1}|A_1 \cap \dots \cap A_{i-2}\}.$$

Poiché ci sono n elementi ed m celle, $P\{A_1\} = \frac{n}{m}$.

Per $j > 1$, la probabilità che la j -sima ispezione trovi una cella occupata, dopo che le prime $j-1$ ispezioni hanno trovato celle occupate, è (dall'ipotesi di hashing uniforme) $\frac{n-j+1}{m-j+1}$.

Osservando che $n < m$, implica che $\frac{n-j}{m-j} \geq \frac{n}{m}$ per ogni j tale che $0 \leq j \leq m$.

Allora, per ogni i tale che $1 \leq i \leq m$ si ha:

$$P\{X \geq i\} = \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \dots \cdot \frac{n-i+2}{m-i+2} \leq \left(\frac{n}{m}\right)^{i-1} \leq \alpha^{i-1}$$

Il numero atteso di ispezioni è:

$$E[X] = \sum_{i=1}^{\infty} Pr\{X \geq i\} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}$$

Quindi è: $E[X] \leq \frac{1}{1-\alpha}$

Corollario

L'inserimento di un elemento in una tabella Hash a indirizzamento aperto con un fattore di carico α richiede in media non più di $\frac{1}{1-\alpha}$ ispezioni nell'ipotesi di hash uniforme.

Teorema (sul numero atteso di ispezioni con successo)

Data una tabella hash ad indirizzamento aperto con fattore di carico α , il numero atteso di ispezioni in una ricerca con successo è $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$.

dim

Per il corollario precedente, se k era la $(i + 1)$ -sima chiave inserita nella tabella hash, il numero atteso di ispezioni fatte in una ricerca di k è al massimo $\frac{1}{1-\frac{i}{m}} = \frac{m}{m-i}$.

Calcolando la media su tutte le n chiavi della tabella hash, si ha:

$$\frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} =$$

$$\frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} =$$

$$\frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \leq$$

$$\frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx \leq \frac{1}{\alpha} \ln \frac{m}{m-n} \leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$$

CONSIDERAZIONI FINALI

Tra tutti i metodi studiati conviene usare:

1. Il concatenamento quando non è noto il fattore di carico α ;
2. L'indirizzamento aperto quando si conosce la dimensione dell'insieme delle chiavi.

Inoltre, il fattore di carico α deve considerare che nel caso di indirizzamento aperto si memorizza direttamente la chiave mentre nel caso di concatenazione si ha la memorizzazione aggiuntiva dei puntatori per gestire le liste.

LEZ 20 – ALGORITMI DI RICERCA ESAUSTIVA (FORZA BRUTA)

Gli algoritmi di ricerca esaustiva sono algoritmi di risoluzione di un dato problema che consistono nel verificare tutte le soluzioni teoricamente possibili fino a che si trova quella effettivamente corretta.

Possiamo distinguere 2 tecniche di programmazione di questo tipo: il **backtracking** e il **branch and bound**.

Notare che queste due tecniche risultano essere delle versioni migliori dell'approccio puramente brute force, in quanto adottano dei meccanismi che in determinate situazioni li permettono di ignorare una soluzione se già si "accorgono" che questa non può essere la soluzione corretta (il branch and bound ci riesce meglio rispetto al backtracking, ma un algoritmo BB risulta più complesso da sviluppare).

BACKTRACKING

Il *backtracking* (ritorno all'indietro) è una tecnica che viene utilizzata per trovare soluzioni a problemi in cui devono essere soddisfatti dei **vincoli**, e consiste nel provare varie sequenze di decisione fino a trovarne una che porta alla soluzione.

Viene spesso applicata per *problemi decisionali, di ricerca e di ottimizzazione*.

Il backtracking si basa sul concetto di **soluzione ammissibile**, ossia una soluzione deve rispettare alcuni criteri (es. la massima) o vincoli (es. posso inserire un numero in una data casella solo se sulla stessa riga e colonna non c'è lo stesso numero).

Il backtracking ha il vantaggio di trovare sempre la soluzione (in quanto esplora tutte quelle ammissibili), tuttavia proprio per questo ha una complessità perlomeno **esponenziale** (nel caso del commesso viaggiatore è fattoriale), e viene utilizzato solo quando non siamo riusciti a trovare un algoritmo che abbia complessità almeno polinomiale (è un'ultima spiaggia).

L'approccio "brute force" consiste nel fatto che si esamina tutto lo spazio delle soluzioni ammissibili: provata una possibile soluzione del problema, se non va bene, si torna indietro e se ne prova un'altra.

I **problemi tipici** affrontati dal backtracking sono:

1. **Enumerazione**: si elencano tutte le soluzioni algebricamente possibili, ossia si esplora tutto lo spazio di ricerca;
2. **Costruzione di una soluzione**: usando l'algoritmo di Enumerazione, ci si ferma alla prima soluzione possibile.

ORGANIZZAZIONE GENERALE

Una soluzione viene rappresentata come un vettore $V[1, \dots, N]$.

Il contenuto degli elementi $V[i]$ è preso da un insieme di scelte S dipendente dal problema.

Ad ogni passo, partiamo da una soluzione parziale $V[1, \dots, k]$, in cui sono state prese $k \geq 1$ scelte.

Se $V[1, \dots, k]$ è una soluzione ammissibile, la “processiamo”, ovvero la visualizziamo e la contiamo. A questo punto possiamo fermarci, se ci bastava un’unica soluzione, o continuare elencando tutte le soluzioni.

Se $V[1, \dots, k]$ non è una soluzione completa:

1. Se è possibile, estendiamo $V[1, \dots, k]$ con una delle possibili scelte in una soluzione $V[1, \dots, k+1]$.
2. Altrimenti, “cancelliamo” l’elemento $V[k]$ (backtrack) e ripartiamo dalla soluzione $V[1, \dots, k-1]$.
(possiamo pensare a Pollicino. Arrivato ad un vicolo cieco, riavvolgeva il gomitolo e percorreva un’altra strada)

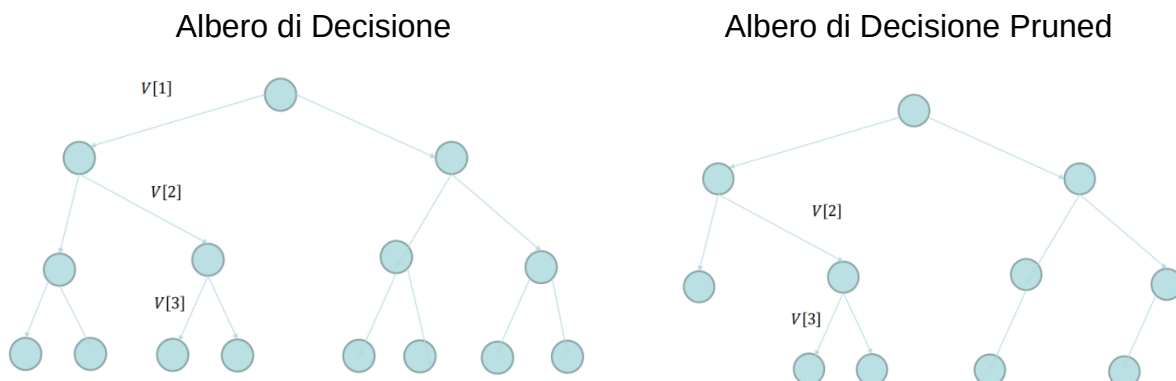
ALBERO DELLE DECISIONI

Possiamo rappresentare uno *spazio di ricerca* attraverso un **albero di decisione**, la cui **radice** è la *soluzione parziale vuota* (vuota in quanto non abbiamo fatto ancora alcuna decisione).

I **nodi interni** dell’albero di decisione rappresentano le *soluzioni parziali*, mentre le **foglie** rappresentano le *soluzioni ammissibili*.

Gli archi dell’albero che sicuramente non portano a soluzioni ammissibili possono essere eliminati (**pruning**).

La valutazione viene fatta nelle soluzioni parziali radici del sottoalbero dove fare il pruning.



Notiamo come nell’Albero Pruned, la foglia più a sinistra del livello 2 non ha più figli al livello 3 in quanto i vincoli sono stati violati, e il massimo che possiamo ottenere è una soluzione parziale.

Per trovare una soluzione completa, siamo costretti a fare backtrack, quindi risalire al nodo padre, e poi scendere al figlio destro.

APPROCCI BACKTRACKING

Nel backtracking possiamo utilizzare due approcci:

1. **Ricorsivo:** lavora tramite una visita in profondità nell'albero delle scelte;
2. **Iterativo:** Utilizza un approccio greedy, eventualmente tornando sui propri passi.

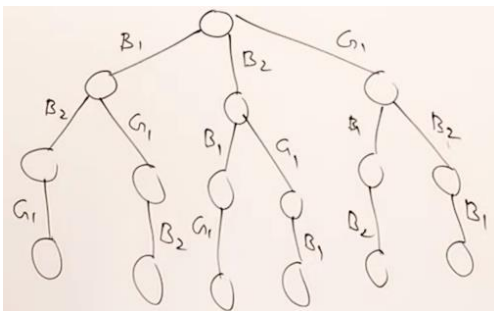
nb: il metodo ricorsivo non è sempre utilizzabile, in quanto c'è sempre il rischio che possiamo "sfondare" lo stack della ricorsione.

nb: con "approccio greedy" non intendiamo la tecnica greedy (non verifichiamo la sottostruttura ottima/scelta golosa), ma semplicemente andremo ad individuare, ogni volta, la soluzione più immediata.

ES.

Immaginiamo di avere due ragazzi (B_1, B_2) e una ragazza (G_1), e che dobbiamo decidere dove questi 3 devono sedersi dati 3 posti. Lo spazio delle soluzioni è rappresentato da tutte le possibili combinazioni di posti, ovvero (essendo 3 persone) $3! = 6$.

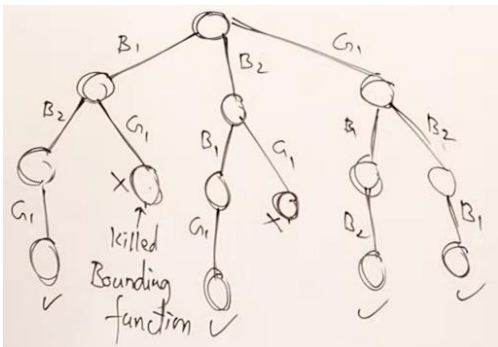
Costruiamo l'albero di decisione:



Permutazioni ammissibili:

- B_1, B_2, G_1 ;
- B_1, G_1, B_2 ;
- B_2, B_1, G_1 ;
- B_2, G_1, B_1 ;
- G_1, B_1, B_2 ;
- G_1, B_2, B_1 .

Immaginiamo però di inserire un *vincolo*, ovvero che la ragazza non può sedersi al centro. Secondo il backtracking, gli archi dell'albero che sicuramente non portano a soluzioni ammissibili possono essere eliminati, ed infatti avremo la seguente situazione.



Permutazioni ammissibili:

- B_1, B_2, G_1 ;
- B_2, B_1, G_1 ;
- G_1, B_1, B_2 ;
- G_1, B_2, B_1 .

In effetti, quando da B_1 andiamo a G_1 (sulla sinistra) o quando da B_2 andiamo a G_1 (al centro), si trattano entrambe di situazioni che porterebbero la ragazza ad essere nel posto centrale, dunque non verranno considerate come soluzioni ammissibili.

Problema delle 8 regine

Il problema consiste nel posizionare 8 regine su una scacchiera in modo tale che nessuna regina possa essere attaccata dalle altre.

Una regina può spostarsi liberamente in orizzontale, verticale e diagonale.

Il **problema generale** è quello di posizionare **N regine** su una scacchiera **NxN**.

Due regine sono sotto attacco se sono sulla stessa:

- riga;
- colonna;
- diagonale.

In un **approccio puramente bruteforce**, consideriamo ogni possibile combinazione di regine (tipo anche tutte e 4 sulla stessa riga), e dunque ogni regina può essere inserita in ognuna della NxN celle della scacchiera, ovvero consideriamo N^2 celle.

Tuttavia anche in un approccio bruteforce non ha senso considerare casi che già sappiamo portare a situazioni impossibili (se ho 2 regine sulla stessa riga, già so che non potrà essere una soluzione, dunque è inutile verificarne il caso anche mettendone una 3^a o una 4^a).

In un **approccio backtracking**:

consideriamo solo le soluzioni ammissibili, dunque non ha senso posizionare due regine sulla stessa riga o sulla stessa colonna, e quindi le soluzioni che derivano da questi inserimenti iniziali le scartiamo a prescindere (perché non sono ammissibili).

Di sicuro sarà possibile inserire una regina per ogni riga, dunque dovremo solo cercare di capire in quali colonne potremo posizionarle.

Per ogni regina, individuiamo la prima colonna senza regina partendo da sinistra e proviamo a verificarla. Trovata una riga ed una colonna libere, dovremo solo controllare che la nostra regina non venga attaccata diagonalmente:

- se la regina non viene attaccata, allora proviamo ad inserire una nuova regina alla riga successiva;
- se la regina viene attaccata ma ci sono ancora colonne da controllare, verifichiamo la colonna successiva;
- se la regina viene attaccata ma NON ci sono altre colonne da controllare, significa che questo percorso porta ad una soluzione che non è ammissibile e quindi “risaliamo” alla regina della riga precedente (spostandola alla colonna successiva).

Scacchiera	Soluzioni	Scacchiera	Soluzioni
1 x 1	1	11 x 11	2 680
2 x 2	0	12 x 12	14 200
3 x 3	0	13 x 13	73 712
4 x 4	2	14 x 14	365 596
5 x 5	10	15 x 15	2 279 184
6 x 6	4	16 x 16	14 772 512
7 x 7	40	17 x 17	95 815 104
8 x 8	92	18 x 18	666 090 624
9 x 9	352	19 x 19	4 968 057 848
10 x 10	724	20 x 20	39 029 188 884

esempio con $N=4$

- Posizioniamo la prima regina Q_1 nella prima riga, nella prima colonna.
 - o Da qui proviamo a posizionare la seconda regina Q_2 nella seconda riga, provando di colonna in colonna da sinistra verso destra. Non possiamo inserirla nella prima colonna (stessa colonna di Q_1) e nella seconda colonna verrebbe attaccata da Q_1 , dunque la inseriamo nella terza colonna;
 - Inseriamo Q_3 . Ci rendiamo conto che verrebbe attaccata in qualsiasi colonna noi la inserissimo, dunque dobbiamo risalire a Q_2 ;
- Proviamo ad inserire Q_2 nella quarta colonna. Non viene attaccata, dunque riproviamo ad inserire Q_3 :

Inseriamo Q_3 . Possiamo inserirla nella seconda colonna.

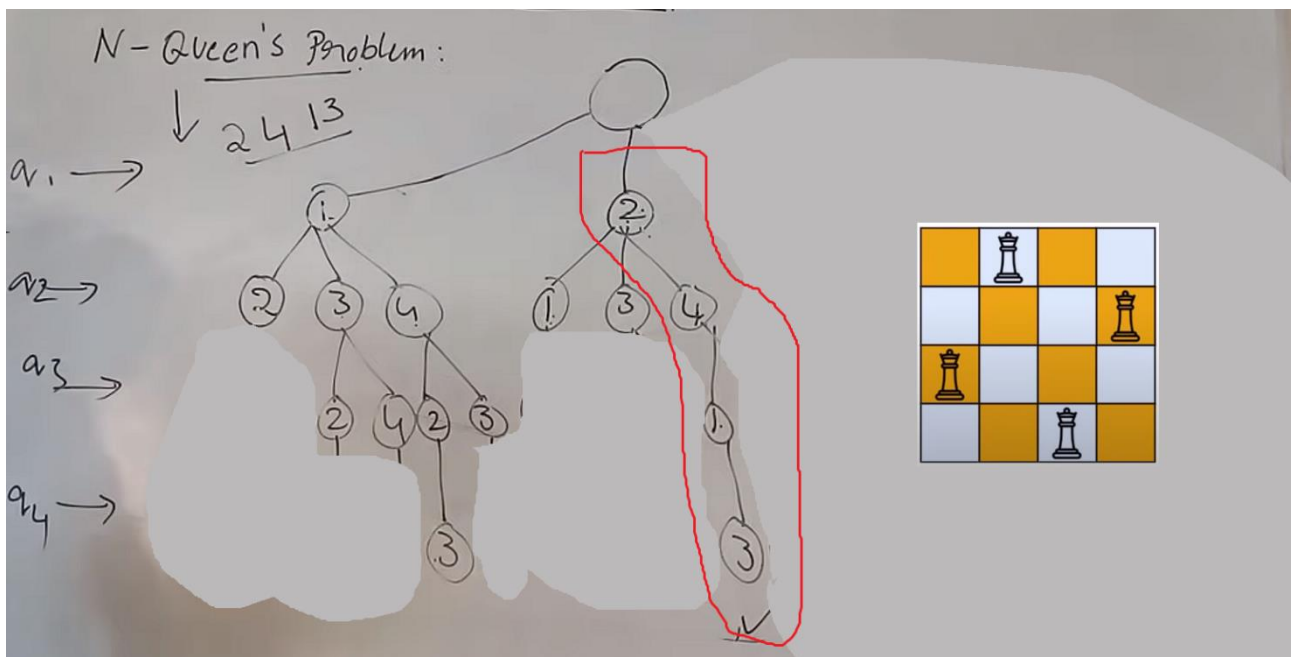
- Inseriamo Q_4 . Ci rendiamo conto che non può essere inserita da nessuna parte senza essere attaccata, dunque risaliamo a Q_3 .

Non possiamo inserire Q_3 ne nella terza che nella quarta colonna, dunque risaliamo di nuovo a Q_2 .

Q_2 non può spostarsi ulteriormente a sinistra, dunque dovremo risalire a Q_1 .

- Posizioniamo Q_1 nella seconda colonna:
 - o Per non essere attaccata, Q_2 può essere posizionata solo nella quarta colonna:
 - Inseriamo Q_3 . Può essere inserito direttamente nella prima colonna.
 - Inseriamo Q_4 : ci rendiamo conto che se lo inseriamo nella terza colonna non verrà attaccata da nessuna, dunque abbiamo raggiunto una possibile soluzione.

La soluzione trovata è dunque la seguente:



nb: è solo una soluzione, se continuassimo a cercare ne troveremmo altre.

Il problema dello zaino 0-1

Ricordiamo il problema dello zaino 0-1 (visto per la programmazione dinamica):

Un ladro entra in un magazzino e trova n oggetti; il k -esimo oggetto vale v_k euro e pesa w_k chilogrammi, dove v_k e w_k sono numeri interi. Il ladro vorrebbe realizzare il furto di maggior valore compatibilmente con il peso W del suo zaino. Quali oggetti deve prendere?

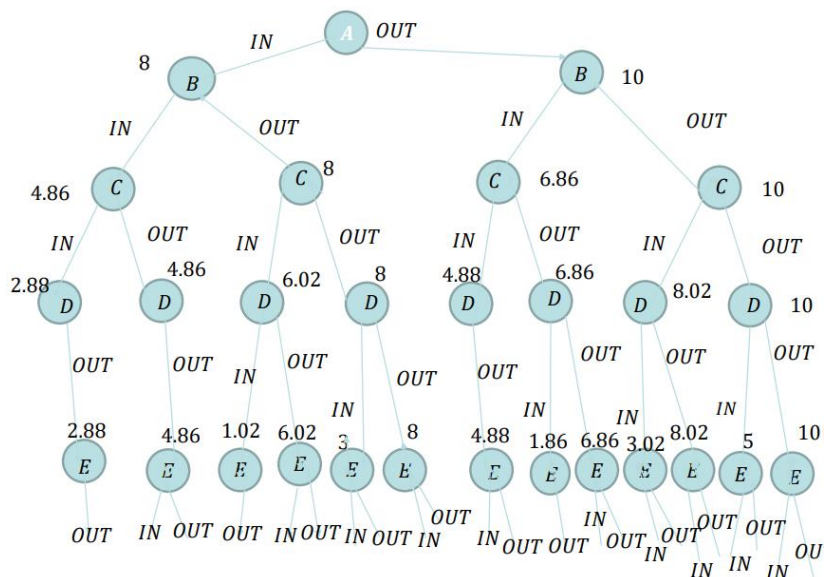
...la dicitura "0-1" indica che ciascun oggetto può essere preso o lasciato; il ladro non può prendere frazioni di oggetti e non può prendere più volte lo stesso oggetto.

Supponiamo un caso in cui il valore degli oggetti sia intero, mentre il loro peso è frazionario. In questo caso non possiamo applicare la programmazione dinamica, quindi siamo costretti ad utilizzare il backtracking.

Quello che andremo a fare è: consideriamo il primo oggetto A, e consideriamo se prenderlo o lasciarlo. Se lo inseriamo andiamo nel figlio sinistro, altrimenti nel figlio destro. Al secondo livello, consideriamo l'oggetto B e facciamo la stessa cosa, e faremo lo stesso per tutti gli oggetti rimanenti seguendo la logica del DFS, facendo attenzione a non superare la capienza dello zaino.

Oggetto	Peso	Valore
A	2.00 Kg	40 \$
B	3.14 Kg	50 \$
C	1.98 Kg	100 \$
D	5.00 Kg	95 \$
E	3.00 Kg	30 \$

capacità: 10kg



Partiamo da 10, e ogni volta che prendiamo un oggetto diminuisce la capienza dello zaino. Notare che se non prendiamo un oggetto perché supererà la capienza rimanente dello zaino, allora è inutile continuare ad esplorare quel percorso (perché sarebbe una soluzione non ammissibile).

Es. Nel percorso A,B,C... non possiamo prendere D perché il suo peso (5kg) è maggiore della capienza rimanente dello zaino (2.88kg), e dunque è inutile andare a considerare il percorso/i percorsi nel caso l'avessimo preso (perché è una soluzione non ammissibile).

Branch and Bound

Il *Branch and Bound* è una tecnica di programmazione per la risoluzione di problemi di ottimizzazione. Consiste nell'**esplorare in modo intelligente** tutte le possibili soluzioni del problema di ottimizzazione, ossia l'albero delle soluzioni.

È intelligente perché:

- riconosce quando arrestarsi, cioè quando un ottimo è raggiunto;
- riconosce che grandi parti dello spazio delle soluzioni non può contenere un ottimo;
- cerca prima nella parti più promettenti dello spazio di ricerca, per poi spostarsi in quelle meno promettenti.

Può essere visto come un'evoluzione del backtracking, in quanto implementa dentro di sé un meccanismo di previsione (lookahead) che gli permette di capire se il ramo sottostante può portare ad una soluzione migliore della corrente.

Se questo è possibile, percorre quel ramo, altrimenti rinuncia a percorrerlo, ossia fa il pruning (lo pota).

Anche il Branch and Bound prova tutte le soluzioni possibili, ma è in grado di scartare alcune soluzioni dimostrando a priori che non sono ottime.

Il Branch and Bound, per implementare il meccanismo di previsione, utilizza l'**upper bound** (limite superiore), ovvero determina e stabilisce un limite superiore al valore che la soluzione, nel problema di ottimizzazione, può assumere.

Grazie all'*upper bound* siamo in grado di fare un pruning intelligente, e quindi ci permette di evitare quei cammini che non ci permetterebbero di arrivare ad una soluzione migliore di quella che abbiamo già ottenuto.

Per poter aggiornare tale limite si utilizza la *tecnica del rilassamento* (nb: cammini minimi -> alg. di Dijkstra).

L'**albero delle soluzioni** è costruito in modo simile al backtracking, quindi le foglie più profonde dell'albero sono le soluzioni, mentre i nodi interni risultano essere soluzioni parziali.

nb: il nodo che si sta esaminando prende il nome di "nodo attivo"

nb: il B&B non è molto importante.

Funzionamento del Branch and Bound

Il Branch and Bound organizza l'attività di ricerca in modo tale che i nodi dell'albero siano espansi uno alla volta (DFS), e ad ogni nodo interno è associato l'upper bound della soluzione.

Vengono costruiti i figli solo se il suo upper bound risulta essere superiore al valore attuale della soluzione, altrimenti il nodo non viene espanso e il sottoalbero viene tagliato.

Nel Branch and Bound possiamo distinguere due **parti**:

1. **Branching**: che consiste nell'esplorazione del sottoalbero radicato in un nodo attivo.
2. **Bounding**: che consiste nello stimare, o aggiornare, il valore massimo che la soluzione dell'albero sottostante può ottenere.

L'upper bound della soluzione dovrà essere **stimato attraverso un algoritmo**

nb: in realtà esistono varie strategie con cui si può esplorare l'albero delle soluzioni (es. Breadth-First-Search o Best-First-Search), ma la più utilizzata è proprio il DFS.

PROBLEMA DELLO ZAINO 0-1 (con Branch and Bound)

Lo zaino ha una capacità di 10kg, e consideriamo di poter prendere i seguenti oggetti:

Oggetto	Peso	Valore
A	2.00 Kg	40 \$
B	3.14 Kg	50 \$
C	1.98 Kg	100 \$
D	5.00 Kg	95 \$
E	3.00 Kg	30 \$

La prima cosa da fare è calcolare l'**upper bound**. Un upper bound alla soluzione del problema è il valore che otterremmo con lo zaino frazionario applicando una tecnica greedy.

Se ordiniamo gli oggetti per il loro rapporto valore/peso, avremo in ordine: C (50.5), A (20), D (19), B (15.9), E (10).

Ovviamente, inseriamo nello zaino prima gli oggetti con il rapporto maggiore. Potremo inserire completamente C, A e D, mentre potremo inserire solo in parte B.

$$100 \$ + 40 \$ + 95 \$ + \frac{(10-8.98)}{3.14} 50\$ = \mathbf{251.24 \$}$$

(arrotondiamo a 251\$)

Pertanto l'Upper bound è di 251\$.

251\$ è il limite superiore dell'intero problema, ovvero nessuna soluzione al problema potrà avere un valore superiore a questo upper bound.

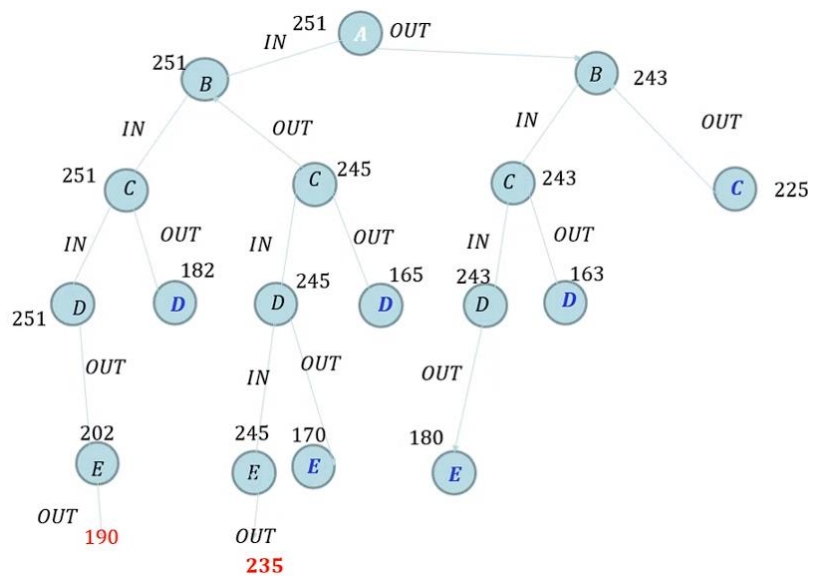
Il nostro obiettivo è esplorare l'albero (effettuiamo il *branching*), e ogni volta che troviamo un upper bound maggiore, ne aggiorniamo il valore (effettuiamo il *bouding*) .

Se ci accorgiamo che seguire un certo ramo non potrà mai portarci ad una soluzione migliore di quella corrente, potiamo quel ramo.

Esempio del prof (non sono sicuro se sia corretto):

Oggetto	Peso	Valore
A	2.00 Kg	40 \$ (20 \$/kg)
B	3.14 Kg	50 \$ (15.92 \$/kg)
C	1.98 Kg	100 \$ (50.50 \$/kg)
D	5.00 Kg	95 \$ (19 \$/kg)
E	3.00 Kg	30 \$ (10 \$/kg)

capacità: 10 kg



LEZ 21 – AUTOMI A STATI FINITI

INTRODUZIONE

Un *automa a stati finiti* è un modello matematico che permette di descrivere, con precisione e in maniera formale, il comportamento di molti sistemi.

Un automa a stati finiti ha un insieme di stati e un “controllo”, che si muove da uno stato ad un altro in risposta agli input esterni.

Possiamo pensare al software per progettare circuiti digitali, o l’analizzatore lessicale di un compilatore, o ancora la ricerca di parole o frasi in un testo.

Il più **semplice automa a stati finiti** a cui possiamo pensare è l’**interruttore**, il quale è un dispositivo che ha due stati (on e off), e che parte da un certo stato iniziale (acceso o spento, per l’appunto). Premendo il pulsante (*push*) cambiamo lo stato.

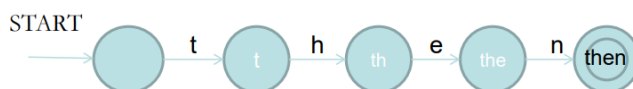


Negli **automi a stati finiti**, gli **stati** vengono identificati con i *cerchi*, mentre gli **archi tra gli stati** vengono etichettati dall’input, che rappresentano le influenze esterne sul sistema.

Lo stato in cui si trova inizialmente il sistema prende il nome di **stato iniziale**.

Gli stati che indicano una sequenza di input valida prendono il nome di **stati finali** o **stati accettanti**.

Un altro esempio di automa potrebbe essere riconoscere la parola “then”, dove gli input corrispondono alle lettere e lo stato iniziale corrisponde alla stringa vuota. Si passa allo stato “then” quando l’input ha formato la parola then, che è anche l’unico stato accettante.



TEORIA DEGLI AUTOMI

I concetti basilari della teoria degli automi sono: *alfabeto*, *stringa* e *linguaggio*.

• ALFABETO

Un *alfabeto* è un insieme non vuoto di simboli, e viene indicato solitamente con il simbolo Σ .

Esempi di alfabeto sono l’alfabeto binario $\Sigma = \{0,1\}$, o anche l’alfabeto italiano o l’insieme di tutti i caratteri ASCII.

• STRINGA

Una *stringa* è una sequenza finita di simboli scelti da un alfabeto.

Essa è rappresentata con λ .

Una *stringa vuota* è una stringa composta da nessun simbolo, e può essere scelta da qualunque alfabeto.

La **lunghezza** di una stringa è data dal numero di posizioni per i simboli della stringa, e viene indicata con $|\lambda|$.

Potenze di un alfabeto

Definiamo Σ^k come l'insieme di stringhe di lunghezza k , con i simboli tratti da Σ .

es. Se $\Sigma = \{0,1\}$, allora

- $\Sigma^1 = \{0,1\}$
- $\Sigma^2 = \{00,10,01,11\}$
- $\Sigma^3 = \{000,100,001,101,010,110,011,111\}$
-
-
- $\Sigma^0 = \{\varepsilon\}$

nb: $\Sigma^0 = \{\varepsilon\}$ è definito per qualsiasi alfabeto Σ .

Σ e Σ^1 non sono la stessa cosa. Il primo è un alfabeto, il secondo è un insieme di stringhe.

L'insieme di tutte le stringhe su un alfabeto Σ viene indicato con Σ^* .

Ossia $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$

Ad esempio, se $\Sigma = \{0,1\}$ allora : $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$

Con Σ^+ si indica l'insieme delle stringhe non vuote sull'alfabeto Σ .

Pertanto: $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots$

$$\Sigma^* = \Sigma^0 \cup \Sigma^+$$

Concatenazione di Stringhe

Siano x e y due stringhe, allora xy denota la loro concatenazione, ossia la stringa che si ottiene facendo seguire ad una copia di x una copia di y .

Da notar bene che xy è diverso da yx .

" ε " è l'identità dell'operazione di concatenazione. È come se fosse lo 0 nell'addizione.

Infatti, per qualunque stringa x vale la relazione: $x = \varepsilon x = x \varepsilon$

• LINGUAGGIO

Un *linguaggio* è un insieme di stringhe scelte da Σ^* , dove Σ è un alfabeto.

Se Σ è un alfabeto e $L \subseteq \Sigma^*$, allora L è un linguaggio su Σ .

DA NOTARE:

Un linguaggio su Σ non deve necessariamente includere stringhe con tutti i simboli di Σ .

Una volta che si è stabilito che L è un linguaggio su Σ , L è un linguaggio su tutti gli alfabeti che includono Σ .

(Pensiamo all'alfabeto italiano composto da 21 lettere, che a sua volta è compreso in quello internazionale da 26. Se un linguaggio L è definito sull'alfabeto internazionale, allora è anche definito sull'alfabeto italiano).

Il linguaggio $L = \{ \varepsilon \}$, formato dalla sola stringa vuota, è un linguaggio rispetto ad ogni alfabeto. Tale linguaggio non coincide con il linguaggio vuoto $\emptyset$, in quanto ha una stringa mentre il vuoto non ne ha.

Operazioni su Linguaggi

- **Intersezione ed unione tra linguaggi:** derivano dalle definizioni insiemistiche.
- **Complemento di L :** è definito come $\bar{L} = \Sigma^* - L$. (è la parte di Σ^* non contenuta in L).
- **Concatenazione di due linguaggi L_1 e L_2 :** è l'insieme delle stringhe che si possono formare prendendo ogni stringa in L_1 e concatenandola con tutte le stringhe in L_2 .

$$L = \{ x_1 x_2 : x_1 \in L_1, x_2 \in L_2 \}.$$

- **Chiusura (o stella di Kleene) di un linguaggio L :** rappresenta l'insieme delle stringhe che possono essere formate prendendo un numero qualsiasi di stringhe da L , eventualmente con ripetizioni¹, e concatenandole tutte. È rappresentata con L^* .

$$L^* = \{ \varepsilon \} \cup L^1 \cup L^2 \cup \dots$$

Dove L^k è il linguaggio ottenuto concatenando L con se stesso k volte.

(Notiamo come sia il corrispettivo di Σ^*)

¹la stessa stringa può essere selezionata più volte.

Esempi

- Unione di due linguaggi:

Se $L_1 = \{001, 10, 111\}$

$L_2 = \{\epsilon, 001\}$

allora $L_1 \cup L_2 = \{\epsilon, 10, 001, 111\}$

- Concatenazione di due linguaggi:

Se $L_1 = \{001, 10, 111\}$

$L_2 = \{\epsilon, 001\}$

allora $L_1 L_2 = \{001, 10, 111, 001001, 10001, 111001\}$

Le prime tre stringhe in $L_1 L_2$ sono le stringhe in L_1 concatenate con ϵ . Dato che ϵ è l'identità per la concatenazione, le stringhe risultanti sono le stesse di quelle in L_1 . Le ultime tre stringhe in $L_1 L_2$ sono date concatenando ogni stringa in L_1 con la seconda (e ultima) stringa di L_2 "001".

- Chiusura:

Se $L = \{0, 11\}$

allora L^* consiste di quelle stringhe di 0 e 1 tali che gli 1 compaiono a coppie.

Ad esempio: 011, 11110 e ϵ fanno parte della chiusura, mentre 01011 o 101 non ne fanno parte.

Generalmente la chiusura è *infinita*, ad eccezione di due casi (ad esempio, per $L = \{0\}$).

DEFINIZIONE FORMALE DI UN PROBLEMA RIGUARDANTE GLI AUTOMI

Possiamo definire formalmente un problema riguardante gli automi come:

Data una stringa w in Σ^* , decidere se w appartiene a L , ove Σ è un alfabeto e L è un linguaggio su Σ .

Automa a Stati finiti deterministico

Un **Automa a stati finiti deterministico (DFA)** è un automa che, dopo aver letto una qualunque sequenza di input, si trova in un unico stato.

Consiste dei seguenti componenti:

- Un **insieme finito Q di stati**;
- Un **insieme finito Σ di simboli input**;
- Una **funzione di stato $\delta : Q \times \Sigma \rightarrow Q$** (delta, tale che accetta un qualsiasi stato di Q in combinazione con qualsiasi simbolo di input di Σ , e che mi restituisce uno stato sempre appartenente a Q).
Ovvero, prende come input uno stato ed un simbolo di input e restituisce uno stato;
- Uno **stato iniziale $q_0 \subseteq Q$** ;
- Un **insieme di stati finali, accettanti, $F \subseteq Q$** .

ESEMPIO: definizione dell'*interruttore* in termini di “automa a Stati finiti deterministici”:

- $Q = \{ \text{on, off} \}$
- $\Sigma = \{ \text{push} \}$
- $\delta = \{ \text{on, off} \} \times \{ \text{push} \} \rightarrow \{ \text{on, off} \}$ *l'ordine di Q finale non è importante*
- $q_0 = \{ \text{on, off} \}$
- $F = \{ \text{on, off} \}$

LINGUAGGIO DI UN DFA: definizione informale

Informalmente, il linguaggio del DFA è l'insieme di tutte le stringhe che il DFA accetta.

Supponiamo che $a_1, a_2 \dots a_n$ sia una sequenza di simboli di input. Si parte dal DFA nel suo stato iniziale q_0 . Utilizzando la *funzione di transizione* $\delta(q_0, a_1)$, troviamo lo stato (ad esempio, q_1). Il successivo simbolo di input a_2 viene trattato valutando $\delta(q_1, a_2)$, producendo q_2 .

Seguendo questo ragionamento, si trovano i rimanenti stati $q_3, q_3, \dots q_n$ tali che per ogni i $\delta(q_{i-1}, a_i)$.

Allora, se q_n è un elemento di F , allora l'input è $a_1, a_2 \dots a_n$.

FUNZIONE DI TRANSIZIONE ESTESA

In una funzione di transizione estesa, il secondo argomento non è più un simbolo ma una *sequenza* di simboli, ovvero una stringa.

Tale funzione $\hat{\delta}(q_0, w)$ è una funzione che prende uno stato iniziale q_0 e una stringa w e restituisce uno stato p , ossia lo stato che l'automa raggiunge quando parte dallo stato q_0 ed elabora una sequenza di input w .

È equivalente ad avere un insieme di funzioni di transizione innestate l'una dentro l'altra.

ESEMPIO: L'automa che *riconosce la parola "then"* viene descritto attraverso una funzione di transizione estesa perché, invece di dargli un simbolo alla volta, gli diamo direttamente la stringa "then".

Le seguenti funzioni sono equivalenti, ma notiamo come quella di transizione estesa risulta essere molto più leggibile.

$$\hat{\delta}(q_0, \text{then}) = \delta(\delta(\delta(\delta(q_0, t), h), e), n) = p$$

LINGUAGGIO REGOLARE

Un linguaggio L è un **linguaggio regolare** se e solo se esiste un DFA, A , tale che $L(A) = L$. In pratica, è regolare se viene *riconosciuto* da un qualche automa a stati finiti.

nb: $\hat{\delta}$ (delta a cappuccio)

LINGUAGGIO DI UN DFA: definizione formale

Dato un DFA $A = (Q, \Sigma, \delta, q_0, F)$, il linguaggio del DFA, indicato con $L(A)$, è indicato come:

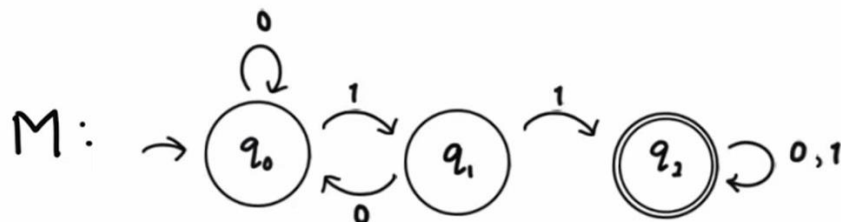
$$L(A) = \{w \mid \hat{\delta}(q_0, w) \text{ è in } F\}$$

(Dato un DFA “A”, definito dalla quintupla (di cui sopra), il linguaggio del DFA “A” sarà formato da tutte le stringhe w tali che la *funzione di transizione estesa*, a cui passiamo uno stato q_0 e la stringa w , sia in F (ovvero l’insieme degli stati accettanti).

Quindi, il linguaggio di A è l’insieme delle stringhe che portano dallo stato iniziale q_0 a uno degli stati accettanti.

ESEMPIO DFA: linguaggio regolare

Abbiamo detto che un linguaggio è regolare se viene *riconosciuto* da un qualche automa a stati finiti. Immaginiamo di avere il seguente DFA M.



Stringhe che vengono accettate da questo automa sono ad esempio:

$$L_M = \{11, 101, 011, 110, 1101, 01011, \dots\}$$

In linea generale basta dare un’idea di quali siano le stringhe accettate se come in questo caso ne possiamo avere infinite (se ignorassimo cappi e archi all’indietro, l’unica stringa accettata sarebbe 11).

Tuttavia notiamo che, qualsiasi sia la stringa che prendiamo da L_M , tutte contengono “11”, perciò possiamo ridefinire L_M come:

$$L_M = \{\text{stringhe contenenti } 11\}$$

Automa a Stati finiti non deterministico

Un **Automa a stati finiti non deterministico (NFA)** è un automa che, dopo aver letto una qualunque sequenza di input, può trovarsi in più stati contemporaneamente (al contrario del DFA).

consiste dei seguenti componenti:

- Un insieme finito Q di stati;
- Un insieme finito Σ di simboli input;
- Una **funzione di stato** $\delta : Q \times \Sigma \rightarrow P$ (delta, tale che accetta un qualsiasi stato di Q in combinazione con qualsiasi simbolo di input di Σ , e che mi restituisce un sottoinsieme di stati sempre appartenente a Q).
Ovvero, prende come input uno stato ed un simbolo di input e restituisce un sottoinsieme di stati;
- Uno **stato iniziale** $q_0 \in Q$;
- Un insieme di stati finali, **accettanti**, $F \subseteq Q$.

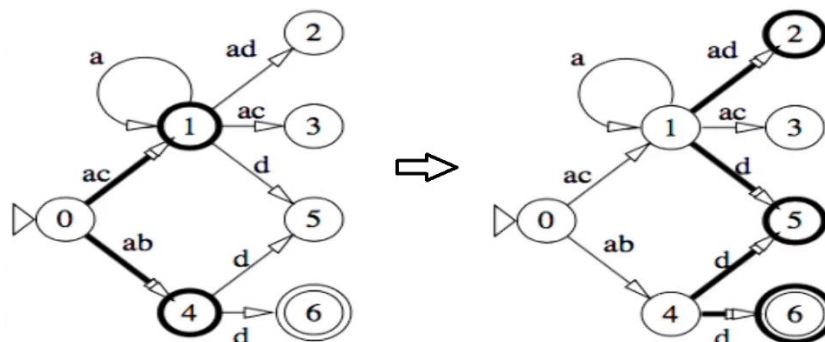
L'unica differenza con il DFA è il punto 3. Infatti sta volta non mi restituisce un solo stato di Q , ma un sottoinsieme.

Si tratta di un'**astrazione non realistica**, in quanto se do un input ad una macchina questa potrà trovarsi solo in un nuovo stato (se sto guidando e mi trovo di fronte ad un bivio, vado a destra o a sinistra, non posso andare contemporaneamente in ambo le direzioni), però sono utili concettualmente perché possono permettere di semplificare la scrittura degli automi.

Ad esempio:

Partiamo dalla stato 0. Se do come input c o b passerò rispettivamente allo stato 1 o 4, ma se do come input a passerò *sia nello stato 1 che nello stato 4*.

La stringhe continuano finché non viene trovato uno stato finale. La stringa che porta a questo stato è la **stringa accettata**.



Supponendo che gli input fossero a e d , avremo le stringhe (0,1,2), (0,1,5), (0,4,5), (0,4,6), ma di queste solo una stringa porta allo stato 6 "(0,4,6)", dunque quella sarà la stringa accettata.

LINGUAGGIO DI UN NFA: definizione formale

Detto che un NFA accetta una stringa w se è possibile fare una sequenza di scelte dello stato successivo che porta dallo stato iniziale ad uno stato accettante

Dato un **NFA** $A = (Q, \Sigma, \delta, q_0, F)$, il **linguaggio del NFA**, indicato con $L(A)$, è indicato come:

$$L(A) = \{w \mid \widehat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

(Dato un NFA “A”, definito dalla quintupla (di cui sopra), il linguaggio del NFA “A” sarà formato da tutte le stringhe w tali che la *funzione di transizione estesa*, a cui passiamo uno stato q_0 e la stringa w , intersecato con F , dia un insieme vuoto).

LEZ 22 – PROBLEMI INDECIBILI

PROBLEMI CHE I CALCOLATORI NON POSSONO RISOLVERE

Ci sono alcuni problemi che i calcolatori non sono in grado di risolvere (indecidibili).
Ad esempio, un *programma in grado di esaminare qualunque programma P , e dire se P , eseguito con I come input, stampa Ciao Mondo (o altra stringa), non esiste!*

In particolar modo, questa nostra incapacità di stabilire cosa fa un programma è data dal problema che non sappiamo *quando qualcosa avverrà, se mai avverrà*.

Ad esempio, dimostriamo che un calcolatore non è in grado di risolvere un problema specifico, che consiste nello stabilire se un programma è in grado di stampare “Ciao Mondo”.

In particolar modo, sviluppiamo un algoritmo che calcola un valore esponenziale, e attraverso un altro algoritmo andremo a stampare “Ciao Mondo” SOLO SE viene rispettata una certa condizione, ovvero che $x^n + y^n = z^n$.

```
int main()
{
    cin >> int n;
    int total = 3;
    while (true) {
        for (int x = 1; x <= total - 2; x++)
            for (int y = 1; y <= total - x - 1; y++) {
                z = total - x - y;
                if (exp(x, n) + exp(y, n) == exp(z, n))
                    cout << "Ciao mondo" << endl;
            }
        total++;
    }
}
```

```
int exp(int i, n)
{ // calcola i elevato a n
    int ans = 1;
    for (int j = 1; j <= n; j++)
        ans = ans * i;
    return(ans)
}
```

Se n è uguale a 1, sappiamo che il programma stamperà “Ciao Mondo”, in quanto troveremo sempre dei valori di interi per cui l’equazione $x^n + y^n = z^n$ è soddisfatta.

Se n è uguale a 2, pure sappiamo che il programma stamperà “Ciao Mondo”, in quanto anche in questo caso troveremo dei valori di interi per cui l’equazione è soddisfatta.
Ad esempio, pensiamo alla terna pitagorica $x=3, y=4, z=5$, dove $9+16=25$.

Se $n > 2$ il programma non sarà mai in grado di stampare “Ciao Mondo”, e questo perché per qualunque intero $n > 2$, il programma non troverà mai una tripla di interi positivi che soddisfi l’equazione $x^n + y^n = z^n$.

Tale considerazione è garantita dall’Ultimo Teorema di Fermat.

PROBLEMA DECIDIBILE

Se un problema ha un algoritmo come H , che dice sempre in maniera corretta se un'istanza del problema riceve la risposta "sì" oppure "no", allora il problema è **decidibile**. Altrimenti è detto **indecidibile**.

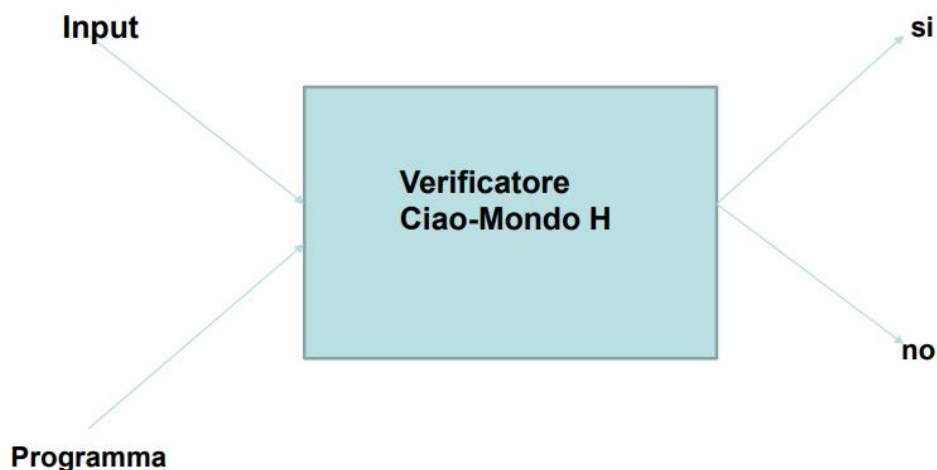
Quindi, un **problema indecidibile** è un problema che non può essere risolto attraverso un calcolatore, e non va confuso con i problemi intrattabili.

Ad esempio, il commesso viaggiatore è un problema *decidibile*, perché può essere risolto, solo che richiede tantissimo tempo per trovare una soluzione (intrattabile).

IPOTETICO VERIFICATORE CIAO_MONDO

Supponiamo che esista un programma, che chiameremo H , che prende come input un programma P e un input I , e ci dica se P , come input, stampa "Ciao Mondo".

L'unico output di H è la stampa dei due caratteri "sì" o "no".

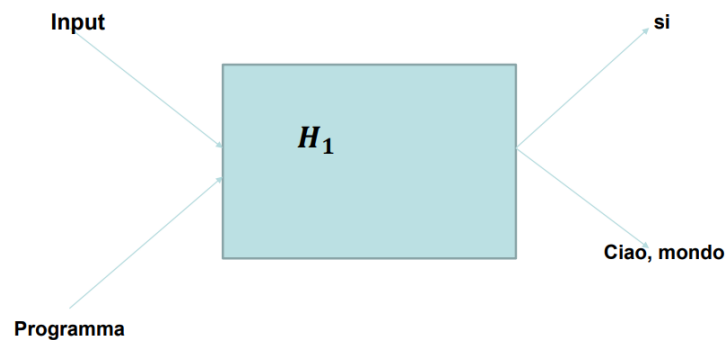


L'impossibilità di compiere la verifica di ciao-mondo si dimostra **per assurdo**.

dimostrazione:

Assumiamo ora che il programma H esista.

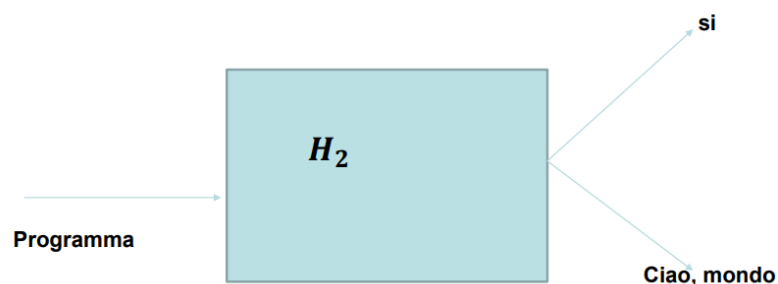
Modifichiamo il programma H in un nuovo programma H_1 che si comporta come H tranne che invece di stampare “no” stampa “Ciao Mondo”.



Dato che siamo interessati a programmi che prendono altri programmi come input, e quindi che siano in grado di verificarli, limitiamo H_1 in due situazioni:

1. L'input è ridotto a P omettendo I .
Quindi togliamo l'istanza I del programma P , così che l'input sia solo il programma.
2. Il nuovo programma deve determinare che cosa farebbe P se l'input fosse il suo stesso codice, cioè che cosa farebbe H_1 avendo in input P sia come programma che come I .

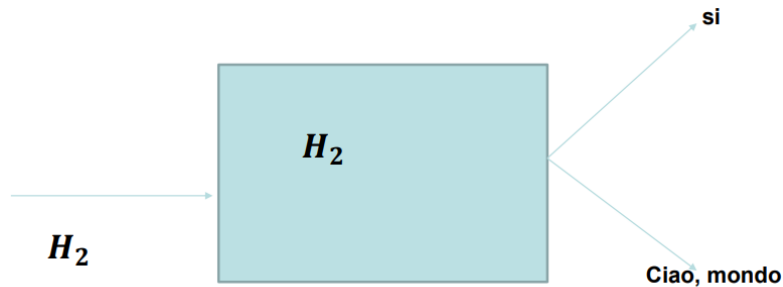
Con le conseguenti limitazioni, trasformiamo H_1 in H_2 .



A questo punto basta dimostrare che H_2 *non esiste*.

Infatti, se H_2 non esiste, allora non esiste nemmeno H_1 , e di conseguenza nemmeno H .

Presupponiamo che il programma P sia H_2 stesso. Quindi H_2 è un programma che riceve come input se stesso, e deve essere in grado di stampare “sì” se stampa “Ciao Mondo”, oppure stampa “Ciao Mondo” se non stampa “Ciao Mondo” come primo output.



Ciò significa che qualunque sia l'output ipotizzato da H_2 , possiamo dedurre che non esiste l'altro. Infatti, si tratta di una situazione paradossale, e quindi possiamo dedurre che H_2 non esiste e quindi non esiste nemmeno H .

Pertanto, abbiamo dimostrato che **nessun programma** H può dire (o meno) se un dato programma P , con input I , stampa “Ciao Mondo” come primo output.

PROBLEMA ASTRATTO

Un **problema astratto** Q si definisce come una relazione binaria sull'insieme I delle istanze del problema e l'insieme S delle soluzioni del problema.

Nello specifico, si definisce un problema astratto Q quando si ha una relazione binaria tra tutti i possibili input del problema e le soluzioni del problema.

AD ESEMPIO: Pensiamo al problema del MST, che richiede di trovare il cammino più corto tra due vertici dati, u e v , di un grafo pesato e orientato $G = (V, E)$.

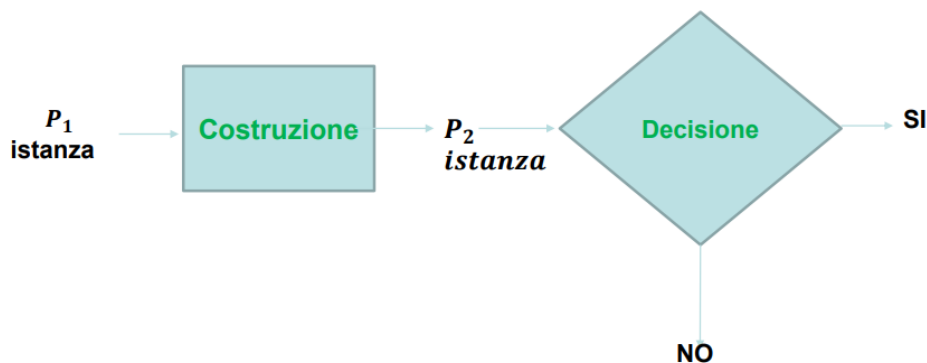
- Un'istanza del problema MST è il **grafo** G ;
- Una **soluzione** del problema è una **sequenza di archi del grafo** G .

DIMOSTRARE CHE UN PROBLEMA È INDECIDIBILE

Supponiamo di sapere che il problema P_1 è indecidibile, e sia P_2 un nuovo problema di cui vogliamo dimostrare che sia indecidibile.

Il modo più semplice che abbiamo per dimostrare che un certo problema è indecidibile, è quello di ideare una costruzione che converta le istanze di P_1 in istanze di P_2 , e che diano la stessa risposta.

Ovvero, quello che andremo a fare sarà una **riduzione** del problema P_2 al problema P_1 .



In altre parole qualunque stringa nel linguaggio P_1 viene convertita in una stringa nel linguaggio P_2 , e qualunque stringa sull'alfabeto di P_1 viene convertita in una stringa che non è nel linguaggio P_2 .

Possiamo risolvere il problema P_1 dicendo che:

1. Data un'istanza di P_1 , cioè data una stringa w che può essere o no nel linguaggio P_1 , applichiamo l' algoritmo di costruzione che produce una stringa x .
2. Verifichiamo se x si trova in P_2 e diamo la stessa risposta su w e P_1 .

Pertanto, se w è in P_1 , allora x è in P_2 e l'algoritmo dice **si**.

Se w non è in P_1 , allora x non è in P_2 e l'algoritmo dice **no**.

In ogni caso, l'algoritmo dice la verità su w . Dato che abbiamo supposto che P_1 fosse indecidibile, **abbiamo dimostrato per assurdo che P_2 è indecidibile**.

LEZ 23 – LA MACCHINA DI TURING

STORIA

Nel 1931 Gödel, nel suo teorema di incompletezza, dimostrò che una formula nel calcolo dei predicati applicati agli interi non poteva essere dimostrata né vera né falsa, e con ciò dimostrò l'esistenza dei problemi indecidibili all'interno della stessa matematica.

Nel 1936 Alan Turing propose la **Macchina di Turing** come **modello di computazione universale**.

La **tesi di Church**, che è indimostrabile, assume che qualunque modo generale di computare permette di computare solo quella che la Macchina di Turing è in grado di calcolare.

Il motivo per cui la macchina di Turing è così interessante dipende proprio dalla tesi di Church: in pratica, un programma può risolvere un problema solo se la macchina di Turing stessa è in grado di calcolarlo.

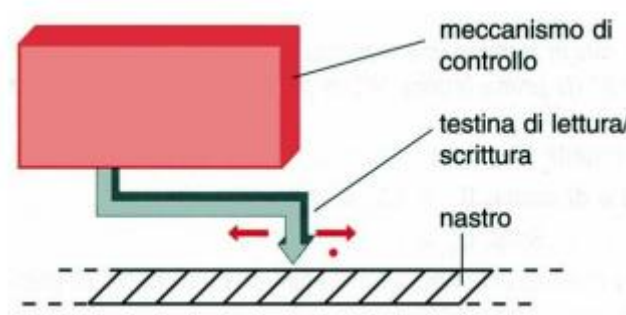
LA MACCHINA DI TURING

La **macchina di Turing** si tratta in sostanza di un automa a stati finiti con un nastro di lunghezza infinita su cui può leggere o scrivere dati. Per l'appunto, non è una macchina fisica ma un modello di una macchina ideale.

La macchina è idealmente formata da tre parti:

- un **nastro infinito**, diviso in celle, dove ogni cella può contenere un simbolo scelto da un insieme finito;
- un **meccanismo di controllo**, che può trovarsi in uno stato scelto da un insieme finito;
- una **testina di lettura/scrittura**, che permette di interagire con le celle, scrivendo un simbolo o leggendone il contenuto. Si muove sul nastro una cella per volta.

Si tratta soltanto di un **modello teorico**, poiché prevede un tempo e uno spazio (cioè il nastro) infiniti.



nb: la macchina di Turing non fa calcoli e non memorizza nemmeno i vecchi valori delle celle una volta che vengono sovrascritti. Può solo spostarsi, leggere e scrivere.

L'**input** è rappresentato da una stringa di lunghezza finita, formata da simboli scelti dall'*alfabeto di input*, che viene inizialmente posto sul nastro.

Tutte le altre celle, che si estendono senza limiti a sinistra e a destra, contengono all'inizio solo un simbolo speciale, detto **blank**. Il blank è un simbolo di nastro, ma non di input. Oltre ai simboli di input e al blank, possono esserci altri simboli.

Una **testina** mobile è collocata su una delle celle (si dice che la MT guarda quella cella). All'inizio si trova sulla cella più a sinistra di quelle che contengono l'input.

Una **mossa** della MT è una funzione $\delta(q, X)$ dello stato del controllo q e del simbolo di nastro X guardato dalla testina. In una mossa la MT compie **tre azioni**:

1. **cambia stato**. Lo stato successivo può coincidere con lo stato corrente;
2. **scrive un simbolo di nastro** nella cella che guarda, che sostituisce quello precedente. Il nuovo simbolo può essere lo stesso che si trova già nella cella;
3. **muove la testina** verso sinistra o verso destra (la testina non può rimanere ferma).

DESCRIZIONE FORMALE DELLA MT

Una macchina di Turing può essere descritta attraverso una settupla $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$, ove:

- Q è l' **insieme** finito **degli stati** del controllo;
- Σ è l' **insieme** finito **dei simboli** di input;
- Γ è l' **insieme** completo dei **simboli del nastro**; si ha sempre $\Sigma \subset \Gamma$;
- δ è la **funzione di transizione**;
- q_0 è lo **stato iniziale** del controllo; $q \in Q$;
- B è il **simbolo** detto **blank**;
- F è l' **insieme degli stati finali o accettanti**; $F \subset Q$;

FUNZIONE DI TRANSIZIONE

Gli argomenti della funzione di Transizione $\delta(q, X)$ sono uno stato q e un simbolo di nastro X . Il valore di $\delta(q, X)$, se definito, è una tripla (p, Y, D) , dove:

- $p \in Q$, è lo stato successivo;
- $Y \in \Gamma$, è il simbolo scritto nella cella guardata, al posto di qualunque simbolo vi fosse;
- D è una direzione (L o R), ossia sinistra o destra, e segnala la direzione in cui si muove la testina.

nb: notiamo come corrispondano alle 3 azioni di una mossa.

LE MACCHINE DI TURING E L'ARRESTO

La macchina di Turing continua ad accettare simboli finché non ha più mosse a disposizione, ovvero la macchina di Turing stessa si **arresta**, e si ha la cosiddetta **accettazione per arresto**.

Formalmente, diremo che: una MT si arresta se entra in uno stato q guardando un simbolo X e non ci sono mosse in questa situazione; cioè $\delta(q, X)$ è indefinita.

Se il problema **non è decidibile** con la macchina di Turing, allora tale macchina continuerà sempre a muoversi, o in avanti o all'indietro, perché troverà sempre mosse possibili che possono essere eseguite, e quindi continuerà a muoversi all'infinito.

nb:

La scelta dell'alfabeto non è limitante, in quanto la macchina di Turing ha le stesse potenzialità e le stesse funzioni indifferentemente dal sistema scelto (es. binario, decimale, o ancora simboli scelti da noi arbitrariamente).

Ovviamente, scegliere un sistema binario rispetto ad uno decimale comporterà informazioni "più lunghe".

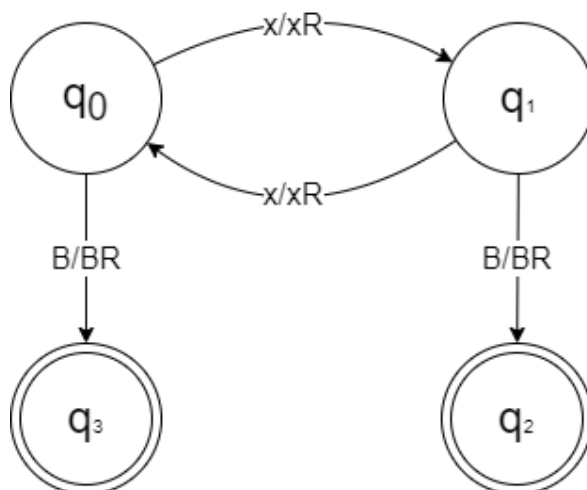
ESEMPIO DI FUNZIONAMENTO

Verificare se un numero è pari o dispari.

Il valore del numero sarà rappresentato da una serie di x (es. 5 = xxxxx).

Il blank sarà rappresentato dal simbolo B.

Il diagramma degli stati avrà questa forma:



Il ragionamento consiste nel “contare” le x:

- q_0 : è lo stato iniziale, e inoltre indica che $|x|$ è *attualmente* pari.
All'inizio abbiamo $|x| = 0$ perché non abbiamo ancora contato nulla. Se legge x va nello stato q_1 , mentre se legge B va nello stato q_3 .
- q_1 : in tale stato $|x|$ è *attualmente* dispari . Infatti noteremo che può essere raggiunto solo quando $|x|$ è dispari (es. 1, 3, 5...). Se il simbolo letto è x, ritorna nello stato q_0 (perché aggiungiamo “1” ad un numero attualmente dispari) mentre se il simbolo letto è B, va nello stato q_2 .
- q_2 : è uno stato di accettazione, e indica che il numero è dispari.
- q_3 : è uno stato di accettazione, e indica che il numero è pari.

La settupla $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$ sarà dunque:

- $Q = \{q_0, q_1, q_2, q_3\}$;
- $\Sigma = \{x\}$;
- $\Gamma = \{B, x\}$
- $\delta = \{q_0, q_1\} \times \{x, B\} \rightarrow \{q_0, q_1, q_2, q_3\}$
- $q_0 = q_0$;
- $B = B$;
- $F = \{q_2, q_3\}$;

La macchina si arresta quando incontra il simbolo blank.

Funzione di transizione:

stati/ingressi	x	B
q_0	(q_1, x, R)	(q_3, B, R)
q_1	(q_0, x, R)	(q_2, B, R)

nb: nper questo esempio, non è importante quello che scriviamo nella cella dopo aver letto (lascio quello che trovo), e Inoltre ci muoviamo sempre verso destra.

Es. 5

5 = xxxxx.

All'inizio la testina legge x, dunque va in q_1 , e si sposta a destra. La seconda volta legge di nuovo x, dunque va in q_0 . La terza volta va in q_1 , la quarta va in q_0 , la quinta va in q_1 . La sesta volta legge B, dunque essendo che si trovava in q_1 , andrà in q_2 e si arresta.

Il numero 5 è dispari.

	B	x	x	x	x	x	B	
--	---	---	---	---	---	---	---	--

DESCRIZIONE ISTANTANEA DELLA MACCHINA DI TURING

Per descrivere formalmente cosa fa una macchina di Turing dobbiamo sviluppare una notazione per le Descrizioni Istantanee (ID).

Per rappresentare una ID possiamo usare la stringa:

$$X_1 \quad X_2 \quad \dots \quad X_{i-1} \quad qX_i \quad X_{i+1} \quad \dots \quad X_{n-1} \quad X_n$$

Dove:

- q è lo stato della MT;
- la testina guarda l' i -esimo simbolo da sinistra;
- $X_1 \dots X_n$ è la porzione del nastro tra il simbolo diverso dal blank più a sinistra e quello più a destra. X_i rappresenta l' i -esima cella del nastro. Eccezionalmente, se la testina si trova a sinistra di X_1 o a destra di X_n , allora un prefisso o suffisso di $X_1 \dots X_n$ sarà costituito da blank, e i sarà rispettivamente 1 o n .

Supponiamo che $\delta(q, X_i) = (p, Y, L)$, cioè la prossima mostra è verso sinistra. Allora:

$$X_1 \dots X_{i-1} \quad qX_i \quad X_{i+1} \dots X_n \quad \xrightarrow{TM} \quad X_1 \dots X_{i-2} \quad pX_{i-1} \quad Y \quad X_{i+1} \dots X_n$$

La notazione riflette il passaggio allo stato p e il fatto che ora la testina si trova sulla cella $i-1$. Ci sono due **eccezioni** importanti:

1. Se $i=1$, allora la TM si muove verso il blank a sinistra di X_1 . In tal caso:

$$qX_1 \quad X_2 \dots X_n \quad \xrightarrow{TM} \quad pB \quad Y \quad X_2 \dots X_n$$

2. Se $i=n$ e $Y=B$, allora il simbolo B scritto su X_n si unisce alla sequenza di blank in coda e non compare nell'ID seguente. Perciò:

$$X_1 \quad X_2 \dots X_{n-1} \quad qX_n \quad \xrightarrow{TM} \quad X_1 \quad X_2 \dots X_{n-2} \quad pX_{n-1} \quad Y$$

nb: $\xrightarrow{TM}$ sarebbe "Transizione TM (Touring Machine)".

Vuole indicare come la porzione del nastro sia cambiata dopo aver effettuato una *mossa* della macchina di Touring. Infatti è da leggersi come: "data una porzione di nastro xxx, dopo che la macchina di Touring ha effettuato una *certa mossa*", otteniamo la porzione di nastro yyy".

Supponiamo che $\delta(q, X_i) = (p, Y, R)$, cioè la prossima mostra è verso destra. Allora:

$$X_1 \dots X_{i-1} qX_i X_{i+1} \dots X_n \xrightarrow{TM} X_1 \dots X_{i-1} Y pX_{i+1} \dots X_n$$

La notazione riflette il passaggio allo stato p e il fatto che ora la testina si trova sulla cella $i+1$. Ci sono due **eccezioni** importanti:

1. Se $i=n$, allora la cella $i+1$ contiene un blank e non faceva parte della ID precedente. Abbiamo perciò:

$$X_1 X_2 \dots X_{n-1} qX_n \xrightarrow{TM} X_1 X_2 \dots X_{n-1} Y pB$$

2. Se $i=1$ e $Y=B$, allora il simbolo B scritto su X_1 si unisce alla sequenza di blank in testa e non compare nell'ID seguente. Perciò:

$$qX_1 X_2 \dots X_n \xrightarrow{TM} Y pX_2 \dots X_{n-1} X_n$$

In breve,

possiamo descrivere la configurazione corrente di una MT mediante una stringa di lunghezza finita che include tutte le celle del nastro, dal simbolo diverso dal blank più a sinistra a quello più a destra. Lo stato e la posizione della testina si indicano ponendo lo stato all'interno della sequenza dei simboli di nastro, alla sinistra della cella guardata.

LEZ 24 – CLASSI P e NP (Polynomial e Nondeterministic Polynomial)

PROBLEMI ASTRATTI

Un **problema astratto** Q si definisce come una relazione binaria sull'insieme I delle istanze del problema e l'insieme S delle soluzioni del problema.

Nello specifico, un *problema di decisione astratto* è una funzione che fa corrispondere all'insieme delle istanze I all'insieme delle soluzioni $\{0,1\}$

AD ESEMPIO: Pensiamo al problema del MST, che richiede di trovare il cammino più corto tra due vertici dati, u e v , di un grafo pesato e orientato $G = (V,E)$.

In tal caso, una **istanza** del problema del MST è *una tripla formata dal grafo G e i due vertici u e v* , mentre una **soluzione** del problema è *una sequenza di vertici del grafo G* , dove la sequenza vuota indica il problema.

Il nostro scopo adesso è, dati: un grafo $G = (V,E)$, due vertici u e $v \in V$, ed un numero k intero positivo; esiste un cammino tra i due vertici la cui lunghezza sia al più k ?

Ebbene, se K-CAMMINO-MINIMO è la nostra funzione, ed i è un'istanza del problema, allora se $K\text{-CAMMINO-MINIMO}(i) = 1$, significa che esiste un cammino minimo con lunghezza al più k ; se $K\text{-CAMMINO-MINIMO}(i) = 0$, allora non esiste.

Da notar bene che molti dei problemi astratti non sono problemi di decisione, ma problemi di ottimizzazione, in cui alcuni valori devono essere minimizzati o massimizzati

Per applicare la teoria dei problemi NP-completi è necessario trasformare i problemi di ottimizzazione in problemi di decisione, ciò viene realizzato aggiungendo una limitazione al valore da ottimizzare (ad es. non più grande di k come nell'esempio precedente).

PROBLEMI CONCRETI

Un **problema concreto** è un problema in cui **insieme di istanze è codificato in un insieme di stringhe binarie**.

Un problema è concreto se è risolvibile in tempo **polinomiale**, ovvero se esiste un algoritmo per risolverlo in tempo $O(n^k)$, qualunque sia k .

NB: Per far corrispondere problemi astratti di decisione a problemi concreti di decisione si possono usare le codifiche.

Può capitare che a qualche stringa binaria non corrisponda alcuna istanza del problema, in tal caso possiamo pensare di far corrispondere a tali stringhe il valore di decisione 0. In questo modo il problema concreto di decisione produce le stesse soluzioni del problema astratto di decisione.

PROBLEMI DI DECISIONE

I **problemi di decisione** sono quei problemi che hanno la caratteristica di ammettere soluzioni (risposte) del tipo SI o NO.

Ad esempio, il problema relativo al grafo ammetteva come domanda “esiste in un grafo un cammino che passa attraverso tutti i suoi vertici una ed una sola volta?” e, di conseguenza, le possibili risposte potevano essere SI oppure NO.

I problemi di decisione possono essere *astratti* o *concreti*.

INTRODUZIONE A P e NP

Quando si sviluppa una soluzione, questa non solo deve essere corretta, ma deve essere anche efficiente, ovvero si vuole che la complessità delle operazioni da svolgere non aumenti troppo in funzione dei dati iniziali.

In linea generale si considerano algoritmi che possono essere risolti in tempo *al più* polinomiale, cioè $O(n^k)$ qualunque sia k . Tuttavia non tutti i problemi possono essere risolti entro tale tempo. Infatti ci sono:

- problemi che non possono essere risolti da un calcolatore (problemi indecidibili);
- problemi che possono essere risolti, ma non in tempo $O(n^k)$.

Se dei problemi possono essere risolti in tempo polinomiale, sono detti **problemi trattabili**, altrimenti sono detti **problemi intrattabili**.

nb: da non confondere i problemi indecidibili con quelli intrattabili. I primi sono problemi che non possono proprio essere risolti, i secondi sono problemi che possono essere risolti ma richiedono una quantità enorme di tempo (es. commesso viaggiatore).

CLASSE P (polinomiale)

I **problemi P** sono quei problemi la cui soluzione può essere **calcolata** in un tempo polinomiale, cioè $O(n^k)$ (es. algoritmi di ordinamento e di moltiplicazione delle matrici). In pratica la classe P si riferisce a tutti quei problemi che possono essere risolti in una ragionevole quantità di tempo.

Tali problemi sono considerati **trattabili**, per due motivi:

1. ci sono pochi problemi pratici che richiedono un tempo polinomiale per k molto elevati;
2. un problema che può essere risolto in tempo polinomiale con un modello, può essere risolto ancora in tempo polinomiale con un altro modello;

CLASSE NP (polinomiale non deterministico)

I **problemi NP** sono quei problemi la cui soluzione può essere **verificata** in un tempo polinomiale, ma potrebbero aver bisogno di un tempo esponenziale per essere risolti (es. commesso viaggiatore e SAT).

(es. un algoritmo per risolvere il sudoku richiede un tempo $O(n^n)$ (dove $n=9$), ma una volta risolto ci mettiamo molto poco per verificare se la soluzione sia corretta).

CLASSE P (continuo)

Funzione calcolabile in tempo polinomiale

Si dice che una funzione $f: \{0,1\}^* \rightarrow \{0,1\}^*$ è calcolabile in tempo polinomiale se esiste un algoritmo A con tempo polinomiale che, dato come input un qualsiasi $x \in \{0, 1\}^*$ produce come output $f(x)$.

Codifiche correlate polinomialmente

Per un dato insieme I di istanze del problema, si dice che **due codifiche e_1 ed e_2 sono correlate polinomialmente** se esistono due funzioni calcolabili in tempo polinomiale f_{12} ed f_{21} tali che per ogni $i \in I$, si abbia:

$$f_{12}(e_1(i)) = e_2(i) \quad e \quad f_{21}(e_2(i)) = e_1(i)$$

(È possibile trasformare in in tempo polinomiale in una nell'altra).

Lemma

Sia Q un problema di decisione astratto su un insieme I di istante, e siano e_1 e e_2 due codifiche di I correlative polinomialmente.

Allora $e_1(Q) \in P$ se e solo se $e_2(Q) \in P$.

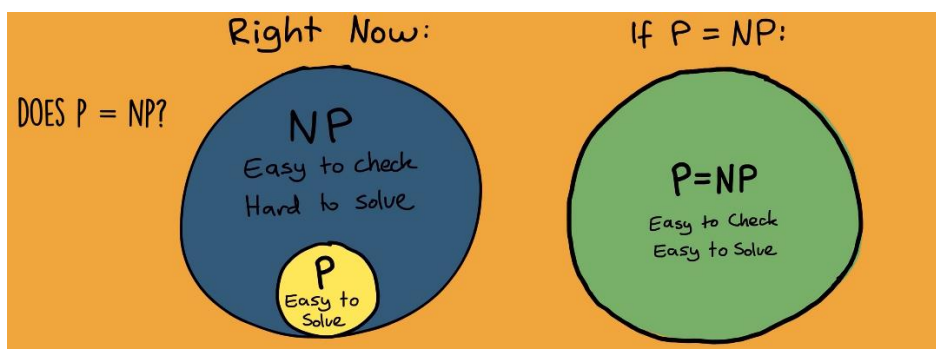
(La conseguenza è che se sappiamo che un problema è risolvibile in tempo polinomiale e la codifica di quel problema può essere trasformata polinomialmente in un altro problema, anche il problema che ne deriva sarà risolvibile in tempo polinomiale).

RELAZIONI TRA P & NP

La *classe* P ci dice che la soluzione di tutti i problemi appartenenti a questa classe può essere **trovata** in un tempo polinomiale, mentre la *classe* NP ci dice che, fornita una possibile soluzione, è possibile **verificarla** in un tempo polinomiale.

Da queste due definizioni dovrebbe essere immediato ed intuitivo capire che $P \subseteq NP$. Infatti, se posso trovare una soluzione di un problema in un tempo polinomiale allora sicuramente potrò anche verificarne la correttezza in un tempo polinomiale.

Qualcosa che spesso ci si chiede, però, è se $P = NP$ (e non un suo sottoinsieme). Infatti ciò comporterebbe che tutti i problemi NP possano in realtà essere risolti in tempo polinomiale.



Il motivo per cui si pensa che ciò possa essere possibile è perché finora non è stato ancora dimostrato che $P \neq NP$.

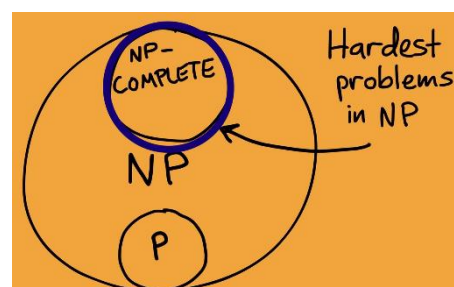
nb: tutti i problemi NP possono essere risolti in un tempo esponenziale o minore, e tutti i problemi P possono essere risolti in un tempo polinomiale o minore (e dunque sono compresi in NP). Alcuni dei problemi NP si sono dimostrati essere in realtà problemi P , tuttavia ce ne sono alcuni che sappiamo solo risolvere con algoritmi di complessità esponenziale, e questi sono detti **NP-COMPLETI**.

PROBLEMI NP-COMPLETI

I **problemi NP-completi** sono quei problemi che sappiamo risolvere solo con algoritmi di complessità esponenziale, e sono i più “difficili” problemi nella classe NP .

Con “difficile” intendiamo che nessuno di questi problemi è stato dimostrato essere in realtà un problema P , e anzi possiamo affermare che se mai un qualsiasi problema NP -completo fosse dimostrato essere in realtà di classe P , allora TUTTI i problemi NP -completi sarebbero in realtà di classe P .

Il motivo principale per cui si pensa che la congettura $P \neq NP$ sia valida è proprio l'esistenza dei problemi NP -completi.



ALGORITMO DI VERIFICA

Si definisce algoritmo di verifica un algoritmo A a due argomenti, dove uno dei due argomenti è una comune stringa x di input e l'altra è una stringa binaria y chiamata **certificato**.

Si dice che un algoritmo A a due argomenti verifica una stringa x di input se esiste un certificato y tale che $A(x,y) = 1$.

Il **linguaggio** verificato da un algoritmo di verifica sarà:

$$L = \{x \in \{0, 1\}^* \exists y \in \{0, 1\}^* : A(x, y) = 1\}$$

Intuitivamente, un algoritmo verifica un **linguaggio** L se per ogni stringa x esiste un certificato y che A può usare per dimostrare che $x \in L$.

Per qualunque stringa $x \notin L$ non esiste alcun certificato y che dimostrino che $x \in L$.

Inoltre, diremo che un linguaggio $L \in NP$ se e solo se esiste un algoritmo a due argomenti A , di tempo polinomiale, ed una costante c tale che:

$$L = \{x \in \{0, 1\}^* : \exists y : |y| = O(|x|^c) : A(x, y) = 1\}$$

In tal caso si dice che l'algoritmo A verifica il linguaggio L in tempo polinomiale.

PROBLEMI DI DECISIONE E DI RICERCA

I **problemi di decisione** sono quei problemi che hanno la caratteristica di ammettere soluzioni (risposte) del tipo SI o NO.

Ad esempio, il problema relativo al grafo ammetteva come domanda “esiste in un grafo un cammino che passa attraverso tutti i suoi vertici una ed una sola volta?” e, di conseguenza, le possibili risposte potevano essere SI oppure NO.

I problemi di decisione possono essere *astratti* o *concreti*.

I **problemi di ricerca** sono quei problemi che necessitano come risposta della formulazione della soluzione. In pratica non occorre solo determinare se esiste o meno una soluzione ad un dato problema, ma occorre anche formularla.

Ad esempio, un problema di ricerca potrebbe essere il seguente “dato un grafo G , e due vertici u e v , trovare il più lungo cammino che porta da u a v senza passare due volte per lo stesso vertice”.

È ovvio che i problemi di ricerca sono più difficili da risolvere rispetto ai problemi di decisione. In particolare, se si riesce a trovare una soluzione al problema di ricerca, si riesce sicuramente a trovare una soluzione al problema di decisione in un tempo polinomiale.

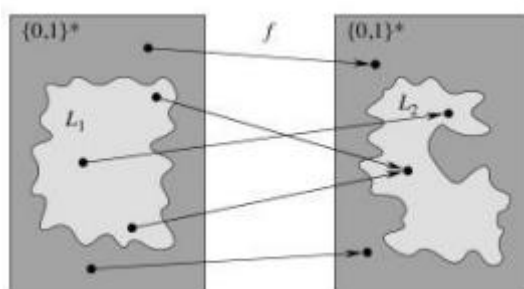
RIDUCIBILITÀ

Si dice che un linguaggio L_1 è riducibile in tempo polinomiale ad un linguaggio L_2 ($L_1 \leq_P L_2$) se esiste una funzione calcolabile in tempo polinomiale $f: \{0,1\}^* \rightarrow \{0,1\}^*$ tale che per ogni $x \in L_1$, se e solo se $f(x) \in L_2$.

La funzione f si dice **funzione di riduzione** e l'algoritmo F di tempo polinomiale che calcola $f(x)$ è detto **algoritmo di riduzione**.

Ciascun linguaggio è un sottoinsieme di $\{0,1\}^*$. La funzione di riduzione f fornisce, in tempo polinomiale, un'applicazione per cui, se $x \in L_1$, allora $f(x) \in L_2$.

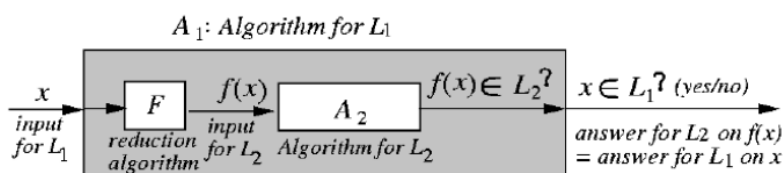
Analogamente, se $x \notin L_1$, allora $f(x) \notin L_2$.



LEMMA

Se $L_1, L_2 \subseteq \{0,1\}^*$ sono linguaggi tali che $L_1 \leq_P L_2$ allora $L_2 \in P \Rightarrow L_1 \in P$.

dimostrazione:



Sia L_2 un linguaggio deciso da un algoritmo A_2 , e sia F un algoritmo di riduzione che calcola la funzione di riduzione f . Si vuol costruire quindi un algoritmo di decisione.

Per un dato input $x \in \{0,1\}^*$, l'algoritmo A_1 usa F per trasformare x in $f(x)$, e quindi usa A_2 per verificare se $f(x) \in L_2$. L'output di A_2 è il valore fornito come output da A_1 .

La correttezza dell'algoritmo segue dalla definizione di riducibilità polinomiale. L'algoritmo è eseguito in tempo polinomiale poichè sia F che A_1 sono eseguiti in tempo polinomiale.

LINGUAGGIO NP-COMPLETO

Un linguaggio è NP-COMPLETO se:

1. $L \in NP$;
2. $L' \leq_p L \quad \forall L' \in NP$

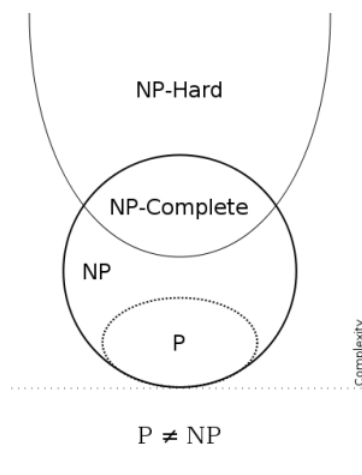
Se un linguaggio soddisfa la 2^a proprietà ma non la prima, viene detto **NP-arduo** (NP-hard).

CLASSE NPC

Si definisce NPC la classe dei **linguaggi NP-completi**.

RELAZIONE

Relazione tra NP-HARD e NP-Completi (Secondo l'ipotesi che $P \neq NP$)



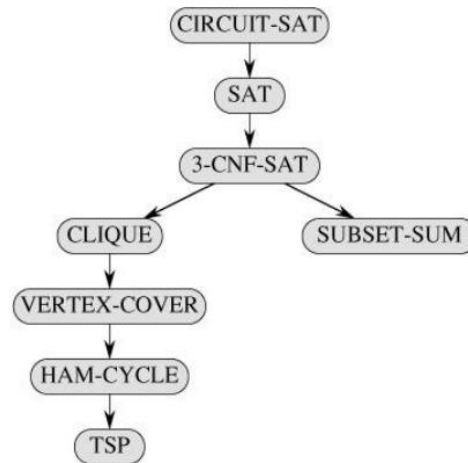
METODO PER DIMOSTRARE CHE UN LINGUAGGIO L è NPC

Per dimostrare che un linguaggio $L \in NP$, dobbiamo svolgere una serie di passaggi:

- Si deve selezionare un linguaggio L' noto come linguaggio NPC.
- Se deve descrivere un algoritmo che calcoli una funzione f che fa corrispondere ad ogni istanza di L' un'istanza di L .
- Si deve dimostrare che la funzione f è tale per cui $x \in L'$ se e solo se $f(x) \in L$, per tutti gli $x \in \{0,1\}^*$.
- Si deve dimostrare, infine, che l'algoritmo che calcola f è eseguito in tempo polinomiale.

“GERARCHIA” DEI PROBLEMI NP COMPLETI

Possiamo creare una “gerarchia” dei problemi NP completi, dove in cima c’è il problema soddisfattibilità dei circuiti, da cui possono essere ricavati tutti gli altri problemi.



CIRCUIT SAT: Si dice che un circuito combinatorio booleano ad un solo output è soddisfattibile se ha un **assegnamento di soddisfattibilità**, ossia un assegnamento di verità che comporta il valore 1 come output del circuito.

Un **assegnamento di verità** per un circuito combinatorio booleano è un insieme di valori booleani di input.

SAT: Consiste nel determinare se una formula booleana è soddisfattibile. Un'istanza del problema soddisfattibilità è una formula booleana ϕ composta da:

- Variabili booleane;
- Connettivi logici (AND, OR, NOT,...);
- parentesi;

3-FNC: È un sottinsieme di NPC, ed è la 3-soddisfattibilità di una forma normale congiuntiva (*3-soddisfattibilità-NFC*).

Il linguaggio delle formule booleane non risulta essere comodo da utilizzare per le dimostrazioni di NP-completezza in quanto l' algoritmo di riduzione deve gestire qualsiasi formula di input conducendo ad un enorme numero dei casi che devono essere considerati.

È pertanto utile considerarne un sottinsieme, che sia sempre NPC, in modo da poter considerare meno casi. In particolar modo, tale **formula** è espressa come un AND di clausole, ciascuna delle quali è una OR di una o più lettere, ove possiamo distinguere 3 lettere distinte.

CRICCA: Una **cricca** in un grafo non orientato $G = (V, E)$ è un sottoinsieme $V' \subseteq V$ di vertici tale che ogni coppia di suoi vertici è connessa da un arco in E .
Ossia una cricca (clique) è un sottografo completo di G .
La dimensione della cricca è il numero dei vertici della cricca.

Il **problema della cricca** è trovare, in un grafo orientato, una cricca di dimensione massima.

VERTEX COVER:

Si dice **copertura dei vertici**, o copertura tramite vertici (vertex cover), un sottoinsieme S dei nodi di un grafo $G=(V,E)$ tale che tutti gli archi in E abbiano almeno un estremo in S .
Il problema di determinare la più piccola copertura tramite vertici di un grafo (detto problema di copertura dei vertici) è un problema NP-completo.

HAM-CYCLE: Il problema del ciclo hamiltoniano è NP-completo.

Ricordiamo che: un **percorso Hamiltoniano** in un grafo non orientato è un percorso che visita ogni vertice esattamente una volta.

Un **ciclo Hamiltoniano** è un percorso Hamiltoniano tale che esiste un arco tra il primo e l'ultimo vertice del percorso.

TSP: Un piazzista deve visitare n città. Modellando il problema come un grafo completo con n vertici, il piazzista desidera fare un ciclo hamiltoniano.

È associato un costo $c(i,j)$ per viaggiare dalla città i alla città j ed il piazzista decide fare il giro il cui costo totale sia minimo, dove il costo totale è la somma dei singoli costi lungo gli archi del giro.

Il problema del commesso viaggiatore è NP-completo.

SUBSET-SUM: Dato un insieme finito S di numeri interi e un ulteriore intero $t \in \mathbb{N}$, il SUBSET-SUM consiste nel rispondere alla domanda: "Esiste un sottoinsieme $S' \subseteq S$ la cui somma è pari a t ?".

È un problema NP-completo.